Overview

LEVEL 1

One request, from harness to backend and back to the ledger

Token Economics sits between harnesses and backends. The gateway is the source of truth; hooks and telemetry only add detail. Pick a stage to go one level down.

1

Ingress and adapters

Every harness reaches Token Economics through a gateway URL first. Hooks, OpenTelemetry and transcripts add detail but never carry enforcement.

4 Ingress · 7 Adapter · 4 Client signal

Open stage (3 groups, 15 components) →

2

Backends

API providers and CLI agents sit behind one Usage and Result interface, with health tracking and fallback.

11 Backend

Open stage (3 groups, 11 components) →

3

Accounting

A versioned price book, a bucket normaliser and an append-only ledger turn raw usage into a cost with a stated source.

4 Store · 6 Calculator

Open stage (3 groups, 10 components) →

4

Policy engine

Budgets, turn caps, loop guard, routing, cache planning and tier choice decide each request before it is sent.

1 Store · 14 Policy

Open stage (4 groups, 15 components) →

5

Context guard

Controls that cut what goes into the prompt: schema deferral, output caps, masking, clearing, compaction.

8 Context guard

Open stage (1 groups, 8 components) →

SUPPORTING STAGES

6

Insight

Metadata store, attribution, dashboards and anomaly checks. Metadata only by default.

1 Store · 3 Calculator · 1 Control plane

Open stage (2 groups, 5 components) →

7

Eval and rollout

Cost per successful task, paired statistics, shadow runs, canaries, kill-switches and rollback triggers.

8 Eval and rollout

Open stage (2 groups, 8 components) →

8

Control plane

Admin API, policy as code, tenants and keys, audit log, secrets, retention.

7 Control plane

Open stage (2 groups, 7 components) →

CROSS-CUTTING DESIGN

Security

Keys, fail-closed budgets, metadata-only logs.

Open →

Reliability

Failure modes, fallback, kill-switch.

Open →

Performance

Where latency can come from.

Open →

Cost

Where Token Economics saves money, and what it costs.

Open →

Quality

How we know a saving did not hurt.

Open →

What this plan does not know

Evidence limits, stated up front.

Open →

Research and references

174 sources behind the cards.

Open →

Level 2 shows one stage as a few groups. Level 3 shows one group with its components and the arrows between them. Level 4 is a spec card for one component. Total: 79 components.

LEVEL 2

1. Ingress and adapters

Every harness reaches Token Economics through a gateway URL first. Hooks, OpenTelemetry and transcripts add detail but never carry enforcement.

Gateway edge

Every request passes this gateway. It resolves who is calling, copies the stream and applies the failure mode.

4 Ingress

Edge gateway · Key and tenant resolver · Stream tee and observer · Failure-mode switch

Receives from: Harness adapters, Context controls (Context guard), Admin and config (Control plane)

Hands to: Budgets (Policy engine), Metering (Accounting), Client signals

Open group (4 components) →

Harness adapters

One adapter per harness, each with eight capability flags. Missing flags downgrade control to observation.

7 Adapter

Capability registry · Claude Code adapter · Codex adapter · OpenAI-compatible adapter · SDK wrapper adapter · Headless CLI adapter · Observe-only adapter

Hands to: Gateway edge, CLI backends (Backends), Client signals

Open group (7 components) →

Client signals

Hooks, OpenTelemetry and transcripts add detail. They are never the enforcement path.

4 Client signal

Hook receiver · OTel receiver · Transcript replayer · Session and prompt-id joiner

Receives from: Harness adapters, Stores and attribution (Insight), Gateway edge

Hands to: Stores and attribution (Insight), Run control (Policy engine)

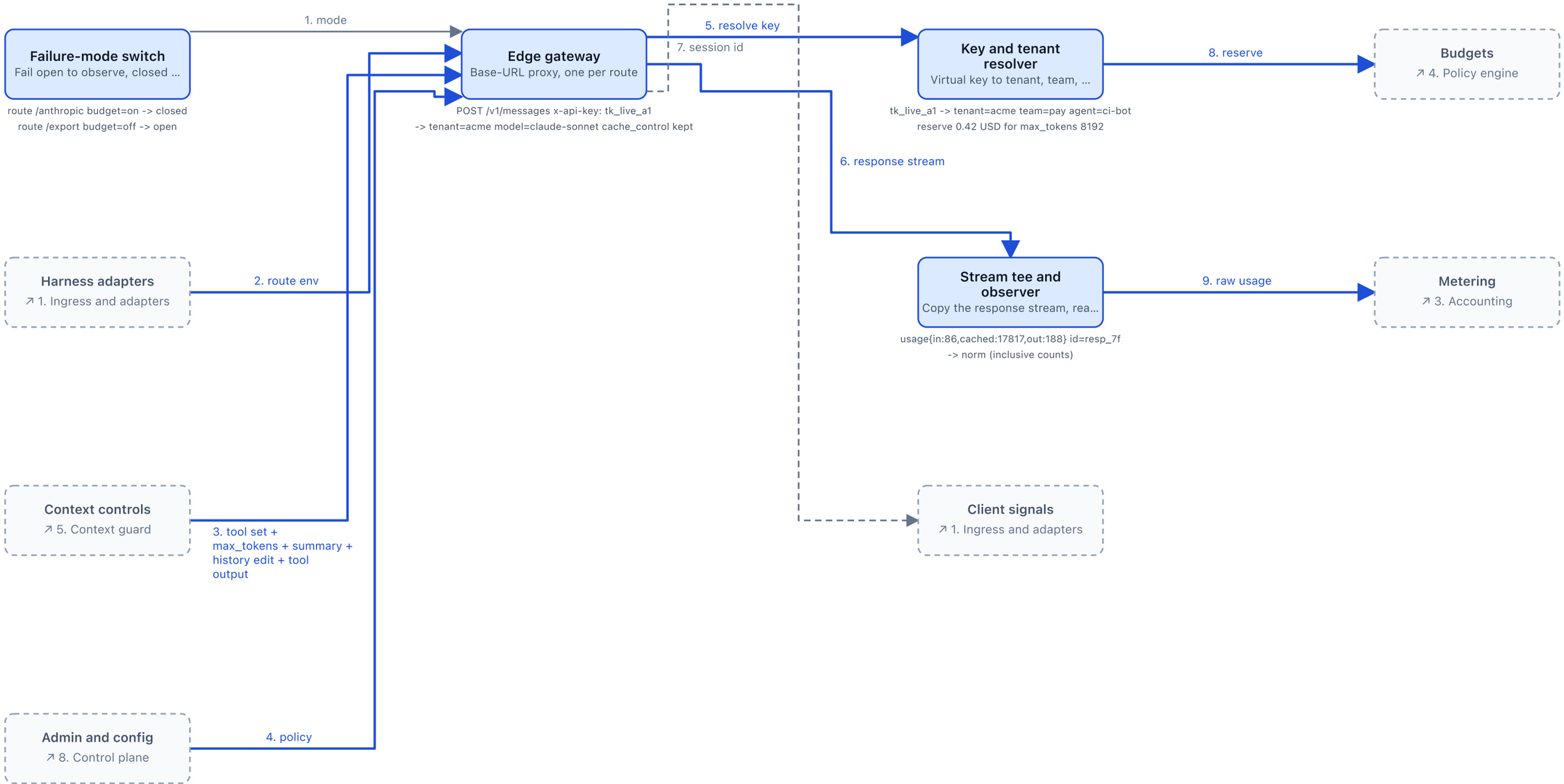
Open group (4 components) →

3 groups, 15 components. Hover a group for the plain-English story; click it to see its components and the arrows between them. Backends →

LEVEL 3

Gateway edge in Ingress and adapters

Every request passes this gateway. It resolves who is calling, copies the stream and applies the failure mode.



Ingress — main flow — read / call — scheduled / async — security — outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Failure-mode switch → Edge gateway

Per-route rule: observe paths fail open, budget paths fail closed.

2. Harness adapters (group) → Edge gateway

ANTHROPIC_BASE_URL and auth token set by managed settings.

3. Context controls (group) → Edge gateway

tool set + max_tokens + summary + history edit: tool set + max_tokens + summary: tool set + max_tokens: tool set: Deferred schemas and lookup path. + max_tokens: Hard cap set on every call. + summary: Compaction near the window, restart at 3. + history edit: Older observations replaced, only at a trigger. + tool output: Capped output with tracebacks kept.

4. Admin and config (group) → Edge gateway

Signed policy bundle pulled by the gateway.

5. Edge gateway → Key and tenant resolver

Bearer token or x-api-key in; tenant, team, agent id out.

6. Edge gateway → Stream tee and observer

Provider response bytes copied to the observer unchanged.

7. Edge gateway → Client signals (group)

Session ids shared so hook rows can join.

8. Key and tenant resolver → Budgets (group)

Tenant id and request size; asks for a worst-case hold.

9. Stream tee and observer → Metering (group)

Provider usage object, response id, model id, headers.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Ingress and adapters Harness adapters →

LEVEL 4

Edge gateway

Ingress | Stage 1: Ingress and adapters

Edge gateway

Base-URL proxy, one per route

POST /v1/messages x-api-key: tk_live_a1
-> tenant=acme model=claude-sonnet cache_control kept

IN PLAIN ENGLISH

> The gateway is a front door that every coding tool talks to first. It lets the call through, watches the answer, and says no when the money is gone.

Goes in	HTTP request from a harness: bearer token or x-api-key, anthropic-beta header, JSON body.
Comes out	Forwarded request to a provider connector, the response stream passed through, or a 402 or 429 error.
Reads / writes	Reads signed policy bundle, key records via authk, route table. Writes request metadata only.
If it fails	Fail open on observe routes (forward and mark usage unmetered). Fail closed on budget routes with 402, because an uncounted call breaks the cap.
Requirement source	RQ-A-001, RQ-A-002, RQ-A-008, RQ-A-009, RQ-A-010, RQ-A-011, RQ-A-012, RQ-A-014, RQ-A-015, RQ-A-017, RQ-A-025, RQ-A-035, RQ-A-041, RQ-A-045, RQ-A-048, RQ-A-053, RQ-A-054 Sources: A:193, C:088, C:001, C:060, A:234, C:052, C:054
Hardening, still open	Claude Code behaviour on a 402 is not in the corpus. Soft-budget overshoot is not quantified. Body rewrite of the model field is not documented as supported.
Not in client docs	**DevX proposal:** The 50 ms p95 overhead target is a DevX proposal.

WHAT IT IS FOR

One HTTP proxy per route so Claude Code, Codex and Aider reach their provider only through Token Economics. A developer cannot disable a gateway the way they can disable a client hook [A:234]. It is the control point and the source of truth for usage.

EXPECTED

- Accept Anthropic Messages, OpenAI Responses and OpenAI Chat Completions on separate route prefixes.
- Forward cache_control and every anthropic-beta value byte for byte [C:052].
- Resolve the key through authk before any upstream call.
- Return 402 when the key budget is spent and 429 when a rate cap fires.
- Attach a request id and session id to every upstream call and log row.
- Pull the signed policy bundle on a timer and keep the last good copy.

NOT EXPECTED

- It never edits prompt text, reorders blocks or strips headers on the pass-through path.
- It never stores prompt or response content by default.
- It never routes Claude Code to a non-Claude model [A:193].
- It never trusts a client-side cost figure.

ALGORITHM AND MATHS

Route, resolve, reserve, forward, tee

```
allow = key_valid AND policy_ok AND reserve_ok; else 401, 403, 402 or 429
```

STEPS

1. Match the route prefix to a wire protocol.
2. Call authk with the credential; reject with 401 if unknown.
3. Load the policy bundle and the fmode rule for this route.
4. Ask bres for a worst-case hold; on refusal return 402.
5. Open the upstream stream via the connector and start the tee.
6. Stream bytes to the client unchanged.
7. On stream end or error, hand usage to norm and release or true up the hold.

PARAMETERS

route_prefixes	/anthropic, /openai, /chat (DevX proposal)
policy_pull_interval_s	30 (DevX proposal)
forwarded_headers	anthropic-beta, anthropic-version, x-grok-conv-id, x-session-id [C:052, A:221, A:231]

SECURITY

- Admin and tenant tokens are separate from provider keys; provider keys come from the vault at call time.
- Redact Authorization, x-api-key and cookie headers from every log line.
- Terminate TLS (transport layer security) and refuse plain HTTP outside localhost.
- Reject bodies above a size cap to stop memory exhaustion (DevX proposal).

PERFORMANCE

Latency and throughput	Under 50 ms added at p95 before first byte (DevX proposal).
Complexity	O(1) per request plus the stream copy, which is O(bytes).
At 10x load	Stateless instances behind a load balancer; session stickiness is by key hash, not by instance.

EDGE CASES AND HOW EACH IS HANDLED

- Client sends anthropic-beta with a value Token Economics does not know: forward it unchanged.
- Upstream closes the stream mid-response: return what was sent, log usage_missing, bill the hold as an estimate.
- Policy bundle is stale or unsigned: keep the last good bundle, alert, never accept an unsigned one.
- Client disconnects after the upstream charged: the tee keeps reading usage to the end.
- Subscription OAuth token arrives: hand to b_oauth and mark auth_mode subscription.

TEST CASES

TEST	GIVEN	EXPECT
cache_control survives	Claude Code request with two cache_control breakpoints and anthropic-beta	Upstream body equals client body; cache_read_input_tokens above 0 on the second call
budget exhausted	Key with remaining 0.00 USD on a budget route	HTTP 402 with error code budget_exhausted and no upstream call
unknown key	x-api-key not in tenant table	HTTP 401 and no upstream call
observe route fail open	Observer crashes on an observe-only route	Client still gets the full response; row has usage_missing true
wrong protocol	Codex posts Chat Completions to the Responses route	HTTP 400 with unsupported_wire_api

WHERE IT SAVES COST

- Preserve prompt caching: reading at 0.1x versus 1x input on Anthropic [A:199].
- Enforce per-key budgets so runaway agents stop [C:136].
- Keep one provider endpoint per session to hold the cache [A:231].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Cache read tokens fall after a Claude Code upgrade.
- 402 rate rises for one key.
- Policy bundle pull fails twice in a row.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.gw.request`	request_id · session_id · tenant · route · status · latency_ms	On every completed request
`token_economics.gw.reject`	request_id · reason · tenant · key_id	On 401, 402, 403 or 429

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- LiteLLM proxy: Virtual keys, max_budget with budget_duration, cache_control_injection_points.
- Bifrost: HTTP 402 once a developer, team or customer budget is spent.
- Claude apps gateway: Forwards anthropic-beta values by construction.

HOW OTHERS BUILD IT

LiteLLM, Bifrost, Portkey and Anthropic's own gateway all sit on the base URL with virtual keys and budgets. Corpus sources mark all cost figures as estimates [A:241, C:088].

REFERENCES

- A:193 LLM gateway configuration
- C:088 Run Claude Code through a gateway
- C:001 Claude Code Token Usage: Monitor It with an AI Gateway
- C:060 5 ways to cut Claude Code costs with LiteLLM
- A:234 Track and optimize coding agent usage (Langfuse)
- C:052 Your cache hit rate is lying to you. It's the caller mix.
- C:054 The Coding Agent Economy

RECEIVES FROM

- Failure-mode switch
  mode
- Claude Code adapter
  route env
- Codex adapter
  route env
- OpenAI-compatible adapter
  route env
- SDK wrapper adapter
  route env
- Tool-schema deferral
  tool set, tool set
- Tool-output cap
  tool output, tool output
- Observation masking
  history edit, history edit
- Compaction guard
  summary, summary
- Max-tokens enforcer
  max_tokens, max_tokens
- Policy-as-code store
  policy, policy
- Context clearing
  history edit

HANDS TO

- Key and tenant resolver
  resolve key
- Stream tee and observer
  response stream
- Hook receiver
  session id

← Back to group: Gateway edge ← Stage 1

LEVEL 4

Key and tenant resolver

Ingress | Stage 1: Ingress and adapters

Key and tenant resolver

Virtual key to tenant, team, agent

tk_live_a1 -> tenant=acme team=pay agent=ci-bot
reserve 0.42 USD for max_tokens 8192

IN PLAIN ENGLISH

> This step reads the key on the request and works out who is paying. It then asks for a hold on the money before the call goes out.

Goes in	Credential from the request: bearer token or x-api-key, plus request size.
Comes out	Tenant id, team id, agent id, key id, and a reserve request sent to bres.
Reads / writes	Reads tenant and key records, budget window. Writes nothing except a last-used timestamp.
If it fails	Fail closed: an unresolved key gets 401, because spend with no owner cannot be budgeted.
Requirement source	RQ-A-007, RQ-A-041, RQ-A-042, RQ-A-043, RQ-A-054 Sources: C:060, C:001, A:245, A:193
Hardening, still open	BYOK spend was not counted by one gateway [A:245]. How Claude Code behaves when the key is revoked mid-session is not in the corpus.
Not in client docs	**DevX proposal:** Key prefix format and cache TTL are DevX proposals.

WHAT IT IS FOR

Map a virtual key to tenant, team and agent. A virtual key is a Token Economics key that stands in for the provider key. Every call is then budgeted and attributed.

EXPECTED

- Resolve a key in one lookup from a local cache with a short time to live (TTL).
- Return tenant, team, agent, budget window ids and scopes.
- Pass tenant id and request size to bres for a worst-case hold.
- Reject revoked, expired or unknown keys.
- Count spend made with a customer-supplied provider key as well [A:245].

NOT EXPECTED

- It never stores the provider key.
- It never reads prompt content.
- It never extends a budget; bst owns amounts.

ALGORITHM AND MATHS

Key lookup and scope check

```
scope_ok = request.route IN key.scopes AND now < key.expires_at AND NOT key.revoked
```

STEPS

1. Hash the presented key with a keyed hash and look it up.
2. Check revoked, expiry and route scope.
3. Load the tenant, team and agent ids.
4. Attach them to the request context.
5. Forward tenant id and request size to bres.

PARAMETERS

cache_ttl_s	10 (DevX proposal)
key_hash	HMAC-SHA256 with a vault secret (DevX proposal)

SECURITY

- Store only key hashes; show the key once at creation.
- Compare hashes in constant time.
- Rate-limit failed lookups per source address (DevX proposal).
- A revoked key must stop within one cache TTL.

PERFORMANCE

Latency and throughput	Under 2 ms per lookup on a cache hit (DevX proposal).
Complexity	O(1) hash lookup.
At 10x load	Local cache per gateway instance, invalidated by a revoke event.

EDGE CASES AND HOW EACH IS HANDLED

- Key valid but tenant suspended: 403 tenant_suspended.
- Two keys share one agent id: spend still rolls up by agent.
- Key cache holds a revoked key for up to one TTL: accept, and log the window.
- Request with no credential on a route that allows anonymous local use: only on localhost.

TEST CASES

TEST	GIVEN	EXPECT
known key	tk_live_a1 mapped to acme/pay/ci-bot	Context carries tenant acme, team pay, agent ci-bot
revoked key	Key revoked 5 s ago, TTL 10 s	Within TTL a revoke event purges the cache and the call gets 401
route scope	Key scoped to /anthropic used on /openai	HTTP 403 scope_denied
expired key	expires_at in the past	HTTP 401 key_expired

WHERE IT SAVES COST

- Per-key budgets stop runaway cost [C:136].
- Agent-level ids let attr show who spent [C:052].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Many 401 from one source.
- A key hits budget twice in a week.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.authk.resolve`	key_id · tenant · team · agent · cache_hit	On every lookup
`token_economics.authk.deny`	key_id · reason	On 401 or 403

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- LiteLLM: Virtual key model with budget and duration.
- Bifrost: Per-developer virtual keys.

HOW OTHERS BUILD IT

Gateways map virtual keys to developers, teams and customers and enforce a budget at each level [C:001, C:060].

REFERENCES

- C:060 5 ways to cut Claude Code costs with LiteLLM
- C:001 Claude Code Token Usage: Monitor It with an AI Gateway
- A:245 Cost-aware model routing with Vercel AI Gateway
- A:193 LLM gateway configuration

RECEIVES FROM

Edge gateway
resolve key

HANDS TO

Reserve and reconcile
reserve, reserve

← Back to group: Gateway edge ← Stage 1

LEVEL 4

Stream tee and observer

Ingress | Stage 1: Ingress and adapters

Stream tee and observer

Copy the response stream, read usage

usage{in:86,cached:17817,out:188} id=resp_7f
-> norm (inclusive counts)

IN PLAIN ENGLISH

> The tee holds a copy of the answer as it goes by and reads the usage numbers off it. The person using the tool sees nothing different.

Goes in	Provider response stream (server-sent events, SSE, or JSON) copied from the gateway.
Comes out	Raw usage object, response id, model id and headers, sent to norm.
Reads / writes	Reads response bytes in memory only. Writes the usage row and header subset, never content.
If it fails	Fail open: a tee error must not change the client stream. The row is marked usage_missing and cost falls back to the reservation.
Requirement source	RQ-A-003, RQ-A-004, RQ-A-009, RQ-A-012, RQ-A-013, RQ-A-017, RQ-A-018, RQ-A-019, RQ-A-048, RQ-A-054 Sources: A:233, A:199, A:216, C:052
Hardening, still open	Gateways can add extra usage keys that defeat conversion [A:233]. Gemini candidate-token split is not in the corpus.
Not in client docs	**DevX proposal:** Buffer size and the stall timeout are DevX proposals.

WHAT IT IS FOR

Read token usage from the response without touching what the client receives. Usage arrives at the end of a stream or in the final event.

EXPECTED

- Copy each chunk to a side buffer after it is sent to the client.
- Parse the final usage object per provider format.
- Pass response id, model id, service tier and rate-limit headers.
- Keep the raw usage object so unknown keys can be flagged [A:233].
- Emit one record per provider response id.

NOT EXPECTED

- It never alters, delays or reorders client bytes.
- It never keeps text content after the usage is parsed.
- It never computes cost; calc does that.

ALGORITHM AND MATHS

Side-copy usage parser

```
client_bytes == provider_bytes; usage = parse(final_event)
```

STEPS

1. Write the chunk to the client socket first.
2. Append a bounded copy to the parse buffer.
3. On message_delta, response.completed or the last SSE (server-sent events) chunk, parse usage.
4. Record headers: request id, rate-limit remaining, service tier.
5. Emit the raw usage object plus ids to norm.

PARAMETERS

parse_buffer_bytes	1 MiB, usage events only (DevX proposal)
stall_timeout_s	120 (DevX proposal)

SECURITY

- Drop content blocks from the buffer once usage is parsed.
- Redact headers that hold credentials.
- Bound the buffer to stop memory growth from a hostile stream.

PERFORMANCE

Latency and throughput	Under 1 ms added per chunk (DevX proposal).
Complexity	O(bytes) copy, O(1) parse.
At 10x load	Per-connection, no shared state.

EDGE CASES AND HOW EACH IS HANDLED

- Stream ends without a usage event: emit usage_missing and keep the response id.
- Provider sends placeholder per-step output_tokens: pass through, dedupe uses the final value [A:216].
- Usage has an unknown key: pass raw and set unknown_keys true.
- Client aborts early: continue reading to the end so usage is still captured.
- Non-streaming JSON response: parse the body directly.

TEST CASES

TEST	GIVEN	EXPECT
identical bytes	Provider stream of 40 chunks	Client receives the same 40 chunks in order
inclusive usage	prompt_tokens 17,903 with cached 17,817 [A:233]	Raw usage forwarded unchanged; norm computes fresh 86
missing usage	Stream cut after 10 chunks	usage_missing true, response id kept
unknown key	usage has extra field cache_tier_x	unknown_keys true, row flagged for review

WHERE IT SAVES COST

- Accurate usage is the base for every cost decision.
- Per-caller cache fields allow per-caller hit rate [C:052].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- usage_missing rate above 0.1% (DevX proposal).
- New unknown usage keys from a provider.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.tee.usage	response_id · model · provider · unknown_keys · usage_missing	When a response ends

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Langfuse: usage_details and cost_details; ingested cost overrides inferred.

HOW OTHERS BUILD IT

Gateway observers log usage from the stream. Langfuse prefers ingested cost over inferred cost [A:232].

REFERENCES

- A:233 LLM Cost Management: Track and Control Spend
- A:199 Claude prompt caching docs
- A:216 Track cost and usage (Claude Agent SDK)
- C:052 Your cache hit rate is lying to you. It's the caller mix.

RECEIVES FROM

Edge gateway
response stream

HANDS TO

Usage normaliser
raw usage, raw usage

← Back to group: Gateway edge ← Stage 1

LEVEL 4

Failure-mode switch

Ingress | Stage 1: Ingress and adapters

Failure-mode switch

Fail open to observe, closed for budgets

route /anthropic budget=on -> closed
route /export budget=off -> open

IN PLAIN ENGLISH

> This switch decides what happens when Token Economics itself breaks. Watching paths keep working, and spending paths stop.

Goes in	Route identifier and its failure-mode rule from the policy bundle.
Comes out	Mode flag (open or closed) returned to gw.
Reads / writes	Reads the per-route rule table from pol. Writes nothing.
If it fails	If the rule is missing, fail closed on any route with a budget. Fail open elsewhere (DevX proposal). A missing rule must not hide spend.
Requirement source	RQ-A-002, RQ-A-012, RQ-A-041 Sources: C:074, C:001, C:136
Hardening, still open	Claude Code behaviour on a 402 is not in the corpus; test it. Which routes are observe paths is a deployment choice.
Not in client docs	**DevX proposal:** Default for a missing rule is a DevX proposal.

WHAT IT IS FOR

Apply the R4 rule: fail open for observation, fail closed for budgets [C:074, C:001]. A timed-out hook yields no decision on most Claude Code events, so the gateway must hold the line.

EXPECTED

- Return one mode per route on every request.
- Return closed for any route with an active budget.
- Return open for observe-only routes.
- Log each time a failure triggered the mode.
- Reload the rule table with every policy bundle.

NOT EXPECTED

- It never changes budget amounts.
- It never fails open on a budget route during a store outage.
- It never depends on a client hook.

ALGORITHM AND MATHS

Per-route mode lookup

```
mode = rule[route] if present else (closed if route.has_budget else open)
```

STEPS

1. Read route id.
2. Look up the rule in the loaded bundle.
3. Apply the default if absent.
4. Return mode and reason.

PARAMETERS

default_budget_route	closed (DevX proposal)
default_observe_route	open (DevX proposal)

SECURITY

- Rule table is part of the signed policy bundle.
- A change to the table is an audited policy change.
- Closed mode must not leak internal error text to the client.

PERFORMANCE

Latency and throughput	In-memory lookup under 0.1 ms (DevX proposal).
Complexity	O(1).
At 10x load	Bundle held per instance.

EDGE CASES AND HOW EACH IS HANDLED

- Budget store unreachable: budget routes return 503 with retry-after.
- Observer down on observe route: pass through, flag the rows.
- Route has a budget only for some keys: closed for those keys.
- Policy bundle fails to load at boot: closed everywhere until a bundle loads.

TEST CASES

TEST	GIVEN	EXPECT
budget route	Store down, route /anthropic with budget	HTTP 503 and no upstream call
observe route	Observer down, route /export	Request passes, row flagged unmetered
missing rule	Route /new not in table, key has budget	mode closed
boot with no bundle	No bundle available	All routes closed, alert raised

WHERE IT SAVES COST

- Closed mode on budget routes prevents unmetered spend.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Mode closed fired more than once an hour.
- Open fired on a route that later showed spend.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.fmode.trigger`	route · mode · cause	When a failure forces a mode

REFERENCES

- C:074 Hooks reference
- C:001 Claude Code Token Usage: Monitor It with an AI Gateway
- C:136 Coding agents ignore spend (Ramp Labs)

RECEIVES FROM

None in this stage view.

HANDS TO

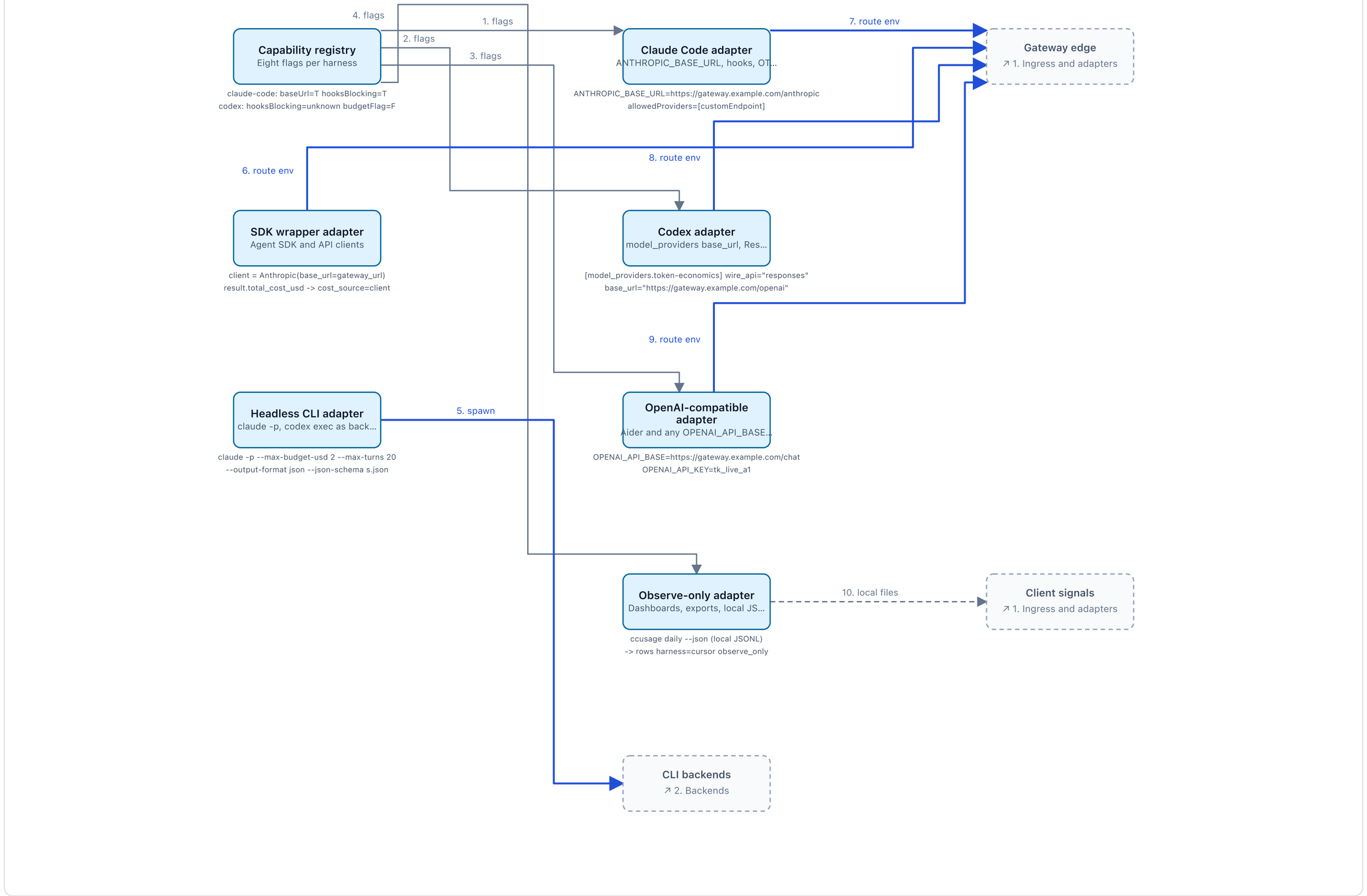
Edge gateway
mode

← Back to group: Gateway edge ← Stage 1

LEVEL 3

Harness adapters in Ingress and adapters

One adapter per harness, each with eight capability flags. Missing flags downgrade control to observation.



Adapter — main flow — read / call — scheduled / async — security — outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Capability registry → Claude Code adapter
Eight capability flags read by the Claude Code adapter.

2. Capability registry → Codex adapter
Capability flags for Codex.

3. Capability registry → OpenAI-compatible adapter
Capability flags for OpenAI-compatible clients.

4. Capability registry → Observe-only adapter
Observe-only harnesses have every control flag false.

5. Headless CLI adapter → CLI backends (group)
Headless run with budget, turn and schema flags.

6. SDK wrapper adapter → Gateway edge (group)
Base URL set in client constructor.

7. Claude Code adapter → Gateway edge (group)
ANTHROPIC_BASE_URL and auth token set by managed settings.

8. Codex adapter → Gateway edge (group)
model_providers base_url in user-level config, wire_api responses.

9. OpenAI-compatible adapter → Gateway edge (group)
OPENAI_API_BASE and OPENAI_API_KEY.

10. Observe-only adapter → Client signals (group)
JSONL, exports and vendor dashboards read on a schedule.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Ingress and adapters ← Gateway edge Client signals →

LEVEL 4

Capability registry

Adapter | Stage 1: Ingress and adapters

Capability registry

Eight flags per harness

claude-code: baseUrl=T hooksBlocking=T
codex: hooksBlocking=unknown budgetFlag=F

IN PLAIN ENGLISH

> This is a table that says what Token Economics can do with each coding tool. If it is not sure, it assumes it can only watch.

Goes in	Harness name and version.
Comes out	Eight capability flags, each true, false or unknown.
Reads / writes	Reads the registry table. Writes updates after a capability test.
If it fails	Fail closed to observe-only: an unknown flag is treated as false, so no control is assumed that has not been proven.
Requirement source	RQ-A-005, RQ-A-006, RQ-A-025, RQ-A-044, RQ-A-046 Sources: C:104, A:193, C:074, C:094, C:037, C:057
Hardening, still open	Codex hook schema, Codex ChatGPT sign-in through a custom provider and Codex budget flag are not in the corpus. modelRewrite for Claude Code is undocumented.
Not in client docs	**DevX proposal:** Tri-state values and the test-before-trust rule are DevX proposals.

WHAT IT IS FOR

Hold eight flags per harness: baseUrl, oauthPassthrough, hooksBlocking, otel, budgetFlag, headless, modelRewrite, transcriptPath. The policy engine reads them and degrades to observe-only when a flag is false.

EXPECTED

- Return all eight flags for every known harness.
- Mark Aider as baseUrl true and the rest false or unknown [C:037, C:127].
- Mark Codex budgetFlag false and hooksBlocking unknown [C:104].
- Mark Claude Code budgetFlag true via --max-budget-usd [C:080, C:094].
- Store a source id and a last-tested date on each flag.

NOT EXPECTED

- It never infers a flag from a marketing page.
- It never turns an unknown into true without a recorded test.

ALGORITHM AND MATHS

Flag lookup with degrade

```
effective_flag = (flag == true) ? true : false; mode = observe_only if NOT baseUrl
```

STEPS

1. Look up the harness and version.
2. Return the eight flags and their evidence.
3. Policy engine maps each false flag to a degraded behaviour.
4. A passing capability test updates the flag and its date.

PARAMETERS

flags	baseUrl, oauthPassthrough, hooksBlocking, otel, budgetFlag, headless, modelRewrite, transcriptPath [C:104]
unknown_handling	treat as false (DevX proposal)

SECURITY

- Only an admin role can edit a flag.
- A flag change is an audited event.
- Never mark oauthPassthrough true without a passthrough test.

PERFORMANCE

Latency and throughput	In-memory read (DevX proposal).
Complexity	O(1).
At 10x load	Tiny table, loaded per instance.

EDGE CASES AND HOW EACH IS HANDLED

- New harness version changes a flag: version key falls back to the last known flags with a warning.
- Harness has no entry: all flags unknown, observe-only.
- Managed Work Cloud ignores model_provider, so baseUrl is false there [C:104].
- Gemini CLI replaced by Antigravity CLI on 2026-06-18 for some users: entry needs a new row [C:057].

TEST CASES

TEST	GIVEN	EXPECT
aider flags	Lookup aider	baseUrl true, hooksBlocking unknown, otel unknown
codex budget	Lookup codex	budgetFlag false
unknown treated false	hooksBlocking unknown for codex	effective flag false and no hook-based rule applied
unlisted harness	Lookup cursor	baseUrl unknown, mode observe_only

WHERE IT SAVES COST

- Right flags stop the engine promising a cap the harness cannot take.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- A capability test fails after a harness upgrade.
- A new harness appears in gateway user agents.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.cap.lookup`	harness · version · degraded	When policy reads flags

REFERENCES

- C:104 Codex configuration reference
- A:193 LLM gateway configuration
- C:074 Hooks reference
- C:094 Run Claude Code programmatically
- C:037 How I Cut Aider's Token Bill 80%: Prompt Caching, MCP Code Mode, and Tier Routing
- C:057 Gemini CLI: Quotas and pricing

RECEIVES FROM

None in this stage view.

HANDS TO

- Claude Code adapter
  flags
- Codex adapter
  flags
- OpenAI-compatible adapter
  flags
- Observe-only adapter
  flags

← Back to group: Harness adapters ← Stage 1

LEVEL 4

Claude Code adapter

Adapter | Stage 1: Ingress and adapters

Claude Code adapter

ANTHROPIC_BASE_URL, hooks, OTel

ANTHROPIC_BASE_URL=https://gateway.example.com/anthropic
allowedProviders=[customEndpoint]

IN PLAIN ENGLISH

> This part points Claude Code at Token Economics and locks that setting so it cannot be skipped. It also reads the extra signals Claude Code gives out.

Goes in	Claude Code environment and settings: ANTHROPIC_BASE_URL, ANTHROPIC_AUTH_TOKEN or apiKeyHelper, managed settings.
Comes out	Routed Claude Code traffic to gw, UsageEvent rows, cap flags for the harness.
Reads / writes	Reads cap flags and managed settings template. Writes managed settings payload and adapter state.
If it fails	Fail open for observation, since hooks and OTel are optional. Fail closed for budgets at the gateway, because developers can disable hooks.
Requirement source	RQ-A-001, RQ-A-002, RQ-A-003, RQ-A-005, RQ-A-007, RQ-A-008, RQ-A-009, RQ-A-015, RQ-A-016, RQ-A-020, RQ-A-024, RQ-A-025, RQ-A-047 Sources: A:193, C:088, C:074, A:217, C:076, C:073, C:001
Hardening, still open	Anthropic does not support routing Claude Code to non-Claude models through a gateway [A:193]. Body rewrite of the model is undocumented. No hook rewrites the model of an ordinary request [C:074].

WHAT IT IS FOR

Implements the five adapter functions for Claude Code: route, identify, observe, cap, replay. Claude Code is the best-documented harness in the corpus: proxy, 30 hook events, OpenTelemetry (OTel), status line, headless mode [C:074, A:217, C:076].

EXPECTED

- route: emit ANTHROPIC_BASE_URL and ANTHROPIC_AUTH_TOKEN, or an apiKeyHelper, in managed settings [A:193, C:088].
- route: pin the route with allowedProviders customEndpoint in managed settings.
- identify: read session_id and prompt_id from hooks, and prompt.id from OTel [C:074].
- observe: take usage from the gateway first, OTel and status line second.
- cap: pass --max-budget-usd and --max-turns for headless runs [C:094, C:080].
- replay: hand transcript_path files to trans.

NOT EXPECTED

- It never relies on a hook for enforcement.
- It never reroutes to a non-Claude model [A:193].
- It never reads prompt text unless OTEL_LOG_USER_PROMPTS is set by the tenant.

ALGORITHM AND MATHS

Managed settings generator

```
settings = base_url + auth + allowedProviders + OTel env + hook config
```

STEPS

1. Read the harness flags from cap.
2. Write ANTHROPIC_BASE_URL and an auth token or apiKeyHelper.
3. Set CLAUDE_CODE_ENABLE_TELEMETRY=1 and OTEL_EXPORTER_OTLP_ENDPOINT to otelr.
4. Add an http hook handler pointing to hookr.
5. Set statusLine to forward rate_limits.* and cost.total_cost_usd.
6. Deploy as managed settings, which sit highest in the order [C:081, C:074].

PARAMETERS

settings_order	user, project, local, --settings, managed last and highest [C:081, C:074]
auth_modes	ANTHROPIC_AUTH_TOKEN, apiKeyHelper, subscription OAuth [A:193]

SECURITY

- Managed settings cannot be disabled by disableAllHooks set below managed level [C:074].
- The auth token is a virtual key, never the provider key.
- Do not set OTEL_LOG_* content opt-ins by default.
- allowedProviders pins the route so a developer cannot bypass the gateway [A:193].

PERFORMANCE

Latency and throughput	Settings applied at session start; no per-call cost (DevX proposal).
Complexity	O(1) per session.
At 10x load	One managed template per tenant.

EDGE CASES AND HOW EACH IS HANDLED

- Subscription OAuth with ANTHROPIC_BASE_URL only: gateway must forward the OAuth capability in anthropic-beta [A:193].
- Claude Code upgrade strips a beta header: cache reads fall; re-test [C:052].
- Developer sets a project-level settings override: managed settings win.
- Subagent inherits the session model unless its definition sets model [C:072, C:073].
- Traceparent is not sent to a gateway by default [A:217].

TEST CASES

TEST	GIVEN	EXPECT
managed route	Managed settings with base URL and allowedProviders	Claude Code calls reach gw; direct provider call is refused
hook disabled	Developer sets disableAllHooks in user settings	Managed http hook still fires; gateway budget still enforced
api key mode	apiKeyHelper returns virtual key	Gateway row has auth_mode api_key
oauth mode	Subscription token with anthropic-beta oauth value	Request forwarded; row auth_mode subscription, no dollar cost

WHERE IT SAVES COST

- Gateway budgets per virtual key [C:136].
- Keep model changes at session start to protect the cache [C:055].
- Status-line rate_limits give subscription window state [C:076].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Cache read share drops after upgrade.
- Hooks stop arriving for one developer.
- 402 reaches Claude Code and the session stalls.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.a_cc.session`	session_id · auth_mode · base_url_pinned · hooks_active	At session start
`token_economics.a_cc.cap_applied`	flags · budget_usd · max_turns	When a headless cap is set

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Claude apps gateway: Single sign-on, managed settings by group, OTLP metrics, anthropic-beta forwarded.
- ccusage: Local JSONL reports for replay.

HOW OTHERS BUILD IT

Anthropic ships a gateway inside the claude binary and documents managed settings and OTel [C:088, A:217]. Third-party gateways such as LiteLLM and Bifrost serve the same base URL [C:060, C:001].

REFERENCES

- A:193 LLM gateway configuration
- C:088 Run Claude Code through a gateway
- C:074 Hooks reference
- A:217 Monitoring usage (Claude Code OpenTelemetry)
- C:076 Customize your status line
- C:073 Model configuration and effort levels
- C:001 Claude Code Token Usage: Monitor It with an AI Gateway

RECEIVES FROM

Capability registry
flags

HANDS TO

Edge gateway
route env

← Back to group: Harness adapters ← Stage 1

LEVEL 4

Codex adapter

Adapter | Stage 1: Ingress and adapters

Codex adapter

model_providers base_url, Responses wire

[model_providers.token-economics] wire_api="responses"
base_url="https://gateway.example.com/openai"

IN PLAIN ENGLISH

> This part sets up Codex so its calls go through Token Economics. It must live in the user's own config file, or Codex ignores it.

Goes in	Codex CLI config: model_providers.<id>.base_url, env_key or auth.command, hooks table, otel table.
Comes out	Routed Codex traffic on the Responses wire, UsageEvent rows, cap flags.
Reads / writes	Reads cap flags. Writes a user-level config.toml snippet.
If it fails	Fail open for observation and fail closed for budgets at the gateway. Codex has no budget flag in the corpus, so only the gateway can enforce.
Requirement source	RQ-A-001, RQ-A-003, RQ-A-005, RQ-A-006, RQ-A-007, RQ-A-008, RQ-A-046, RQ-A-052 Sources: C:104, C:097, C:035
Hardening, still open	Not in the corpus: hook stdin and exit-code schema, OTel metric names, ChatGPT sign-in through a custom provider, and any budget flag. Managed Work Cloud ignores model_provider.

WHAT IT IS FOR

Adapter for Codex CLI. Gateway must speak the Responses API because wire_api accepts only responses [C:104].

EXPECTED

- route: write model_providers.token-economics with base_url, wire_api responses and auth.command for the bearer token [C:104].
- route: put model_provider in user-level config, never project config [C:104].
- identify: map Codex session and thread ids to session_id; hook rows if schema proves out.
- observe: take usage from the gateway; read OTel via the otel table.
- cap: set tool_output_token_limit and model_auto_compact_token_limit style keys where present [C:104].
- replay: read history.jsonl through trans.

NOT EXPECTED

- It never writes openai_base_url or otel to a project file, because Codex ignores them there [C:104].
- It never assumes hook payload fields before a test.
- It never assumes a ChatGPT sign-in can use a custom provider.

ALGORITHM AND MATHS

User-level config writer

```
config = model_provider + model_providers.token-economics + otel + hooks
```

STEPS

1. Read cap flags for the Codex version.
2. Write model_providers.token-economics with base_url and wire_api responses.
3. Set auth.command to print the virtual key.
4. Add [otel] exporter to otelr with log_user_prompt false.
5. Add a hooks table only after the schema test passes.
6. Verify the file is user-level, not project-level.

PARAMETERS

config_order	CLI flags, project, profile, user, cloud-managed, /etc, defaults [C:097]
ignored_in_project	openai_base_url, model_provider(s), notify, profile(s), otel [C:104]

SECURITY

- Keep bearer tokens out of the file by using auth.command.
- otel.log_user_prompt false by default.
- Do not enable features.network_proxy for model traffic; it governs tool traffic only [C:104].

PERFORMANCE

Latency and throughput	Config applied once per machine (DevX proposal).
Complexity	O(1).
At 10x load	One template per tenant.

EDGE CASES AND HOW EACH IS HANDLED

- ChatGPT sign-in (requires_openai_auth): custom provider behaviour unknown, mark observe-only.
- Cloud-orchestrated runs ignore local hooks and providers [C:104].
- Project config tries to set a provider: Codex ignores it; adapter reports drift.
- Wire mismatch (Chat Completions on a Responses route): gateway returns 400.

TEST CASES

TEST	GIVEN	EXPECT
user-level only	model_provider set in project .codex/config.toml	Adapter check reports ignored; route not pinned
responses wire	Codex call through gw	Gateway log wire_api responses, usage parsed
no budget flag	Policy asks for a Codex budget flag	Flag false; gateway virtual-key budget used instead
hook schema untested	Hook rule requested for Codex	Rule not applied; capability flag unknown

WHERE IT SAVES COST

- Gateway budgets because Codex has no budget flag in the corpus.
- Cap tool_output_token_limit style keys [C:104].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Codex version change alters config keys.
- Usage rows from Codex show no cache fields.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.a_cx.config`	version · provider_pinned · hooks_enabled	When config is generated

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Codex config reference: model_providers keys, hooks events, otel table.

HOW OTHERS BUILD IT

Codex's custom provider is the documented path to a gateway [C:104]. Hook and OTel details need a test in each deployment.

REFERENCES

- C:104 Codex configuration reference
- C:097 Codex local config (config.toml)
- C:035 Context Management Across Codex Surfaces

RECEIVES FROM

Capability registry
flags

HANDS TO

Edge gateway
route env

← Back to group: Harness adapters ← Stage 1

LEVEL 4

OpenAI-compatible adapter

Adapter | Stage 1: Ingress and adapters

OpenAI-compatible adapter

Aider and any OPENAI_API_BASE client

OPENAI_API_BASE=https://gateway.example.com/chat
OPENAI_API_KEY=tk_live_a1

IN PLAIN ENGLISH

> Aider only needs a web address and a key to talk to Token Economics. Token Economics then does all the counting, since Aider's own cost numbers can be wrong.

Goes in	Aider environment: OPENAI_API_BASE, OPENAI_API_KEY, --model, --map-tokens.
Comes out	Chat Completions traffic to gw and UsageEvent rows with gateway-computed cost.
Reads / writes	Reads cap flags. Writes env snippet.
If it fails	Fail closed on budgets at the gateway. Aider has no hook or cap surface, so the gateway is the only enforcement point.
Requirement source	RQ-A-001, RQ-A-003, RQ-A-005, RQ-A-006, RQ-A-008, RQ-A-045 Sources: C:037, C:040, C:127
Hardening, still open	Aider hooks, OTel, transcript and headless surfaces are not in the corpus. Aider's own cost display is unreliable behind a router [C:040].

WHAT IT IS FOR

Adapter for Aider and any OpenAI-compatible client. The harness does not know it is routed [C:037].

EXPECTED

- route: set OPENAI_API_BASE and OPENAI_API_KEY to the gateway and a virtual key.
- identify: use the key, a custom header or a model suffix for session; no client ids available.
- observe: gateway log only.
- cap: gateway budget; --map-tokens is a repo-map size, not a hard cap [C:127].
- replay: none; no transcript surface in the corpus.

NOT EXPECTED

- It never trusts Aider's cost line when the model is unknown [C:040].
- It never assumes hooks or OTel exist.

ALGORITHM AND MATHS

Env generator with model check

```
unknown_model => cost shown by Aider is 0; Token Economics cost from price book
```

STEPS

1. Write the two env variables.
2. Check every Aider --model id against cat.
3. If the id is unknown to Aider, warn that Aider assumes unlimited context and zero cost [C:040].
4. Rely on gateway cost for budgets.

PARAMETERS

protocol	Chat Completions [C:037]
map_tokens_default	1k [C:127]

SECURITY

- Virtual key only; never the provider key.
- No content logging at the gateway by default.

PERFORMANCE

Latency and throughput	None beyond the gateway (DevX proposal).
Complexity	O(1).
At 10x load	Stateless.

EDGE CASES AND HOW EACH IS HANDLED

- Aider shows zero cost for an unknown model: ignore; use gateway cost.
- No session id from the client: derive from key plus a time window (DevX proposal).
- Model id mismatch with the catalogue: reject with unknown_model, never partial match.
- Aider hits a 402: surface message text the user can read.

TEST CASES

TEST	GIVEN	EXPECT
zero cost display	Unknown --model in Aider	Gateway row has cost_usd above 0 from the price book
route env	OPENAI_API_BASE set to gw	Calls appear in the gateway log with harness aider
no hooks	Policy requests hook-based loop guard for aider	Rule skipped; gateway-side guard only
unknown model	--model foo-1	Gateway returns 400 unknown_model

WHERE IT SAVES COST

- Gateway budget and cat-priced cost for every Aider call [C:037].
- Map tokens kept at default unless measured [C:127].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Aider upgrade changes request shape.
- Gateway sees an id missing from cat.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.a_ai.request`	harness · model · session_hint	On each Aider call

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Lynkr: OpenAI-compatible translator in front of Aider.

HOW OTHERS BUILD IT

Aider routes through any OpenAI-compatible router; the corpus has one blog source for Lynkr [C:037].

REFERENCES

- C:037 How I Cut Aider's Token Bill 80%: Prompt Caching, MCP Code Mode, and Tier Routing
- C:040 How to Manage the Aider Context Window and Token Budgets
- C:127 How to Reduce Aider Token Usage

RECEIVES FROM

Capability registry
flags

HANDS TO

Edge gateway
route env

← Back to group: Harness adapters ← Stage 1

LEVEL 4

SDK wrapper adapter

Adapter | Stage 1: Ingress and adapters

SDK wrapper adapter

Agent SDK and API clients

client = Anthropic(base_url=gateway_url)
result.total_cost_usd -> cost_source=client

IN PLAIN ENGLISH

> This part wraps code that calls the models from inside an app. It makes sure each call is counted once, not twice.

Goes in	Software development kit (SDK) calls: Claude Agent SDK result.usage, modelUsage, total_cost_usd, or an API client with a base URL.
Comes out	Routed calls to gw and UsageEvent rows with cost_source client or gateway.
Reads / writes	Reads SDK result objects. Writes a wrapper config.
If it fails	Fail open for observation. Budgets are enforced at the gateway, because SDK totals are client-side estimates [A:216].
Requirement source	RQ-A-001, RQ-A-003, RQ-A-004, RQ-A-019, RQ-A-020, RQ-A-021 Sources: A:216, A:217, C:094
Hardening, still open	result.usage excludes subagents [A:216]. Resumed totals include earlier spend. Dual reporting between SDK telemetry and gateway is a trap.

WHAT IT IS FOR

Adapter for embedded use: the Agent SDK and plain API clients. SDK costs are client-side estimates; modelUsage has per-model cost and costBasis (list, managed, unknown) [A:216].

EXPECTED

- route: set base URL in the client constructor [C:088].
- observe: read modelUsage per model and costBasis.
- identify: add caller id through a header set by the wrapper.
- cap: pass max_tokens on every call; budget at gateway.
- dedupe: skip SDK rows whose response id is already in the gateway ledger.

NOT EXPECTED

- It never sums result.usage on a resumed session without subtracting earlier spend [A:216].
- It never treats total_cost_usd as a bill.

ALGORITHM AND MATHS

Single-count rule

```
row counted = gateway row if present else SDK row (cost_source client)
```

STEPS

1. Take the SDK result.
2. Look up the response id in the gateway rows.
3. If found, keep the gateway row and drop the SDK row.
4. If not found, store the SDK row labelled client.
5. Attach subagent spend separately via modelUsage and query_source.

PARAMETERS

cost_basis_values	list, managed, unknown [A:216]

SECURITY

- Wrapper adds only metadata headers.
- No prompt text is copied into telemetry.
- Virtual key supplied by environment, never source code.

PERFORMANCE

Latency and throughput	None beyond the gateway (DevX proposal).
Complexity	O(1) per call.
At 10x load	Library, scales with the host app.

EDGE CASES AND HOW EACH IS HANDLED

- Session resumed: totals include earlier spend; subtract the prior total.
- Subagent spend missing from result.usage: read from OTel query_source [A:217].
- Placeholder per-step output_tokens: use the final result [A:216].
- costBasis unknown: label cost_source client and flag.

TEST CASES

TEST	GIVEN	EXPECT
dual report	Same response id from gateway and SDK	One ledger row, cost_source gateway
resume	Resumed session with total 3.10 USD, prior 2.40	Row adds 0.70 USD
subagent	Parent result.usage 1.00 USD, subagent 0.40 USD via OTel	Two attributed rows
unknown basis	modelUsage costBasis unknown	cost_source client and review flag

WHERE IT SAVES COST

- Hard max_tokens on every call [C:025].
- One count per call avoids false overspend alarms.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- SDK and gateway totals diverge by more than 5% (DevX proposal).

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.a_sdk.dedupe`	response_id, kept, dropped	When SDK and gateway rows collide

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Claude Agent SDK: modelUsage, costBasis, total_cost_usd.

HOW OTHERS BUILD IT

Agent SDK users get per-turn usage and an estimated total; Langfuse and gateways add the bill-grade view [A:216, A:232].

REFERENCES

- A:216 Track cost and usage (Claude Agent SDK)
- A:217 Monitoring usage (Claude Code OpenTelemetry)
- C:094 Run Claude Code programmatically

RECEIVES FROM

None in this stage view.

HANDS TO

Edge gateway
route env

← Back to group: Harness adapters ← Stage 1

LEVEL 4

Headless CLI adapter

Adapter | Stage 1: Ingress and adapters

Headless CLI adapter

claude -p, codex exec as backends

claude -p --max-budget-usd 2 --max-turns 20
--output-format json --json-schema s.json

IN PLAIN ENGLISH

> This part lets Token Economics run a coding agent in the background as a worker. It sets the spend and turn limits before the run starts.

Goes in	Task spec and policy for a headless run: harness, model, budget, turn cap, JSON schema.
Comes out	A spawned claude -p or codex exec process, its JSON result and a UsageEvent row.
Reads / writes	Reads cap flags and policy. Writes run records.
If it fails	Fail closed on budget. If the cap flags cannot be set, the run does not start. An uncapped headless run can spend without limit.
Requirement source	RQ-A-003, RQ-A-004, RQ-A-014, RQ-A-021, RQ-A-050, RQ-A-052, RQ-A-053 Sources: C:094, C:035, C:104, C:136
Hardening, still open	--max-turns flag text is in one source only [C:080]. Codex has no budget flag in the corpus. Resumed runs report whole-conversation totals.

WHAT IT IS FOR

Adapter that treats headless agents as backends: spawn, cap, parse, replay. Used by b_cc and b_cx.

EXPECTED

- Spawn claude -p with --output-format json and the cap flags [C:094].
- Spawn codex exec, or codex exec resume --last, with user-level provider config [C:035, C:104].
- Parse session id, usage and structured_output [C:094].
- Treat exit 0 as success and non-zero as failure; record failed attempts as spend.
- Keep stdin under the 10MB pipe cap [C:094].

NOT EXPECTED

- It never starts a run with no cap.
- It never sums a resumed run's totals again.
- It never treats the JSON cost as a bill.

ALGORITHM AND MATHS

Capped spawn and parse

```
start iff budget_flag set OR gateway key has budget
```

STEPS

1. Read flags from cap for the harness.
2. Build the command with budget, turn and schema flags.
3. Spawn and stream stdout.
4. Parse the final JSON result.
5. Write a UsageEvent with cost_source client unless the gateway also saw the call.
6. Record success flag from exit code and schema validation.

PARAMETERS

stdin_cap	10MB [C:094]
turn_cap_start	P75 of own baseline, with reminder [C:025]

SECURITY

- Pass the virtual key by environment, never on the command line.
- Use a restrictive --permission-mode for unattended runs [C:095].
- Isolate working directory per run.

PERFORMANCE

Latency and throughput	Spawn overhead under 1 s (DevX proposal).
Complexity	O(1) setup, run time depends on the task.
At 10x load	Bounded by worker pool size.

EDGE CASES AND HOW EACH IS HANDLED

- --max-budget-usd stops remaining background work at the cap, so one more step may finish [C:094].
- Non-zero exit with partial JSON: record cost from partial usage and mark failed.
- Run resumed: report only the delta.
- --bare skips hooks, skills, MCP, memory and CLAUDE.md [C:094].

TEST CASES

TEST	GIVEN	EXPECT
cap set	claude -p task with budget 2 USD	Command line contains --max-budget-usd 2
exit failure	Process exits 1 after 0.30 USD	Row stored as failed attempt with cost 0.30
resume delta	Resumed run total 5.00 USD, prior 3.20	Row cost 1.80
no cap	Policy lacks budget for codex exec and key has no budget	Run refused with uncapped_run

WHERE IT SAVES COST

- Cap turns at P75 with a reminder [C:025].
- Count failed attempts in cost per success [C:028].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Cap fires on more than 20% of runs (DevX proposal).

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.a_cli.run	run_id, harness, budget_usd, turns, exit_code	When a run ends

REFERENCES

- C:094 Run Claude Code programmatically
- C:035 Context Management Across Codex Surfaces
- C:104 Codex configuration reference
- C:136 Coding agents ignore spend (Ramp Labs)

RECEIVES FROM

None in this stage view.

HANDS TO

- claude -p runner
  spawn, spawn
- codex exec runner
  spawn

← Back to group: Harness adapters ← Stage 1

LEVEL 4

Observe-only adapter

Adapter | Stage 1: Ingress and adapters

Observe-only adapter

Dashboards, exports, local JSONL

ccusage daily --json (local JSONL)
-> rows harness=cursor observe_only

IN PLAIN ENGLISH

> Some tools offer no way to plug Token Economics in. For these, Token Economics reads their reports and files and can only watch.

Goes in	Local JSONL files, vendor exports and dashboard pages for harnesses with no routing surface.
Comes out	UsageEvent rows labelled observe-only with cost_source client or bill.
Reads / writes	Reads files and exports on a schedule. Writes rows to the ledger path via trans.
If it fails	Fail open: a missed read only delays reporting. No control exists to fail closed.
Requirement source	RQ-A-003, RQ-A-004, RQ-A-005, RQ-A-013, RQ-A-016, RQ-A-044, RQ-A-049 Sources: C:003, A:056, C:057, C:117, C:119, A:263, C:121
Hardening, still open	No base-URL, hook or OTel surface is in the corpus for these tools [C:003]. Tools: Gemini CLI, Copilot CLI, Cursor, Amp, Kiro, OpenHands, Cline, Roo, Kilo, OpenCode.

WHAT IT IS FOR

Cover harnesses with no control surface. All control flags are false [C:003]. Vendor-side budgets are the only cap [A:263, C:119, A:056].

EXPECTED

- Read ccusage-style local JSONL for 15+ agent CLIs [C:003].
- Read Kiro credit usage through the AWS Kiro Cost Optimizer export [A:056].
- Read Copilot premium-request and Cursor pool reports as exports [A:263, C:119].
- Label every row observe-only with a cost_source of client or bill.
- Report plan quota in native units, never tokens [C:010].

NOT EXPECTED

- It never promises a cap.
- It never converts plan quota to tokens.
- It never scrapes pages the vendor blocks.

ALGORITHM AND MATHS

Scheduled read and label

```
row.control = none; row.cost_source = client|bill
```

STEPS

1. List configured sources and their read schedule.
2. Fetch or read each source.
3. Map fields into UsageEvent.
4. Store with harness, source id and read time.
5. Hand to trans for join and dedupe.

PARAMETERS

read_interval	hourly (DevX proposal)

SECURITY

- Read-only credentials for vendor APIs.
- Local files read under the user's consent; metadata only.
- No content fields are mapped.

PERFORMANCE

Latency and throughput	Data lag under 24 h (DevX proposal).
Complexity	O(rows).
At 10x load	One reader per source.

EDGE CASES AND HOW EACH IS HANDLED

- Gemini CLI token caching works for API key and Vertex only, not OAuth [C:121].
- Local transcript retention can be 30 days [C:003]: copy early.
- Vendor export format changes: reader fails and alerts.
- Gemini CLI replaced by Antigravity CLI for some users on 2026-06-18 [C:057].

TEST CASES

TEST	GIVEN	EXPECT
ccusage read	JSONL with 3 sessions	3 rows harness set, observe_only true
no control claim	Policy asks a cap for cursor	Response: unsupported, observe-only
native units	Copilot export in premium requests	Row keeps premium requests, no token value
vendor change	Export lacks expected column	Reader error raised, no partial rows stored

WHERE IT SAVES COST

- Visibility on harnesses Token Economics cannot route.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- A harness gains a base-URL surface and can move to a full adapter.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.a_obs.read`	source · rows · lag_h	After each scheduled read

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- ccusage: Local JSONL reports for 15+ agent CLIs.
- Kiro Cost Optimizer: Per-developer Kiro credit usage.

HOW OTHERS BUILD IT

ccusage and vendor dashboards are the only view into these harnesses [C:003].

REFERENCES

- C:003 How to Track Claude Code Usage in 2026: Built-in Commands, ccusage, and Desktop Dashboards Compared
- A:056 Gain per-developer visibility into Kiro Enterprise credit usage
- C:057 Gemini CLI: Quotas and pricing
- C:117 Understanding and managing requests in Copilot
- C:119 Cursor Models & Pricing
- A:263 Usage-based billing for organizations and enterprises (Copilot)
- C:121 Token caching and cost optimization (Gemini CLI)

RECEIVES FROM

Capability registry
flags

HANDS TO

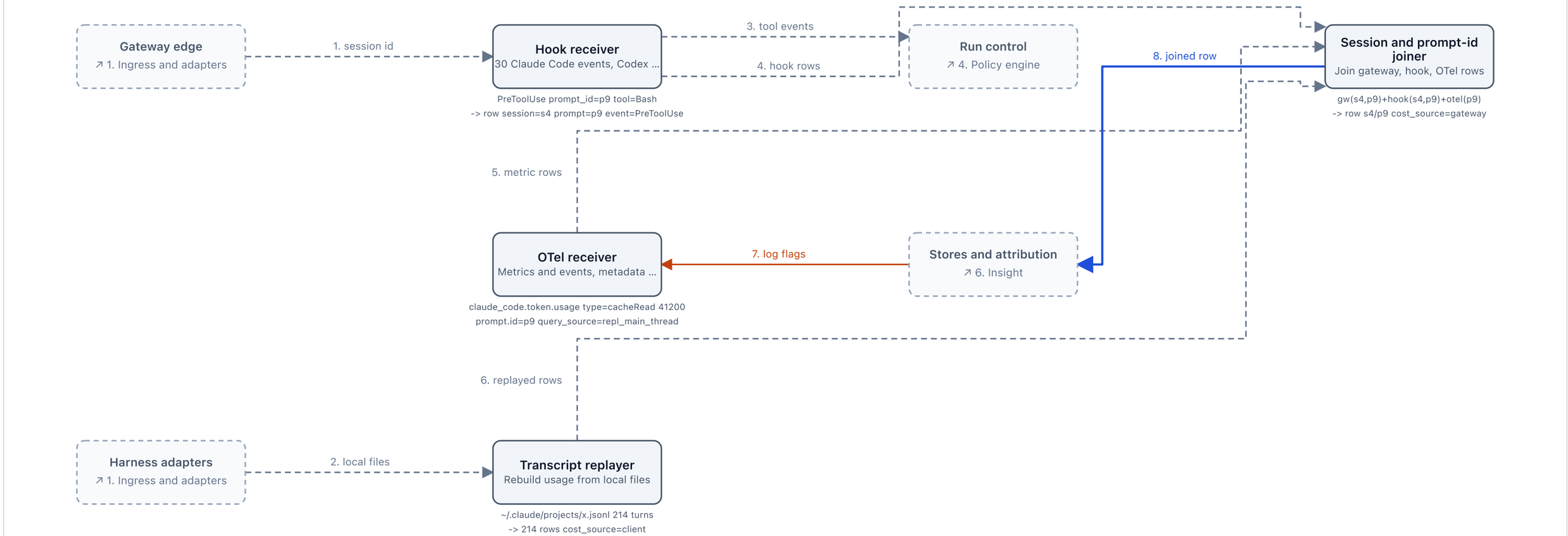
Transcript replayer
local files

← Back to group: Harness adapters ← Stage 1

LEVEL 3

Client signals in Ingress and adapters

Hooks, OpenTelemetry and transcripts add detail. They are never the enforcement path.



Client signal — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Gateway edge (group) → Hook receiver
Session ids shared so hook rows can join.

2. Harness adapters (group) → Transcript replayer
JSONL, exports and vendor dashboards read on a schedule.

3. Hook receiver → Run control (group)
Repeated reads and failures from hook events, when present.

4. Hook receiver → Session and prompt-id joiner
prompt_id, session_id, tool events; may be disabled by the developer.

5. OTel receiver → Session and prompt-id joiner
claude_code.token.usage and api_request events with prompt.id.

6. Transcript replayer → Session and prompt-id joiner
Usage rebuilt from transcripts, labelled client estimate.

7. Stores and attribution (group) → OTel receiver
Content logging stays off unless opted in.

8. Session and prompt-id joiner → Stores and attribution (group)
One row per session and prompt id, with cost_source.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Ingress and adapters ← Harness adapters

LEVEL 4

Hook receiver

Client signal | Stage 1: Ingress and adapters

Hook receiver

30 Claude Code events, Codex hooks

PreToolUse prompt_id=p9 tool=Bash
-> row session=s4 prompt=p9 event=PreToolUse

IN PLAIN ENGLISH

> The client tells Token Economics about each step it takes, such as running a tool. Token Economics keeps a short note of it and never depends on it for the rules.

Goes in	Hook event JSON posted by the client: session_id, prompt_id, transcript_path, hook_event_name, tool fields.
Comes out	Hook rows to the joiner, tool events to the loop guard, optional block decisions.
Reads / writes	Reads event JSON. Writes metadata rows; no tool content by default.
If it fails	Fail open. A hook timeout yields no decision on most Claude Code events [C:074]. Hooks can be disabled, so nothing critical depends on them.
Requirement source	RQ-A-002, RQ-A-011, RQ-A-012, RQ-A-013, RQ-A-022, RQ-A-023, RQ-A-024, RQ-A-046 Sources: C:074, C:090, C:104, A:234
Hardening, still open	Codex hook stdin and exit-code schema are not in the corpus. No Claude Code event rewrites the model of an ordinary request [C:074].

WHAT IT IS FOR

Receive Claude Code's 30 hook events and Codex's 14 events through an http handler [C:074, C:104]. Hooks add detail (prompt ids, tool events) but never carry enforcement.

EXPECTED

- Accept http handler POSTs with session_id, prompt_id, transcript_path, cwd, hook_event_name [C:074].
- Return exit 0 with JSON for structured control, or exit 2 to block where policy needs it [C:074].
- Use last_assistant_message on Stop and SubagentStop instead of transcript_path [C:074].
- Forward repeated-read and failure events to the loop guard.
- Cap each output string at 10,000 characters [C:074].

NOT EXPECTED

- It never stores tool_input or tool output by default.
- It never relies on hooks for budget enforcement [A:234].
- It never assumes a Codex field before a schema test.

ALGORITHM AND MATHS

Event intake and signal fan-out

```
row = {session_id, prompt_id, event, tool_name, ts}; block only on policy exit 2
```

STEPS

1. Authenticate the sender by a hook token.
2. Validate common fields.
3. Drop content fields unless the tenant opted in.
4. Write a hook row for the joiner.
5. If event is PreToolUse or PostToolUseFailure, emit a tool-event signal to loop.
6. If policy requires, return exit 2 with a reason.

PARAMETERS

claude_events	30 events listed in C:074
codex_events	14 events listed in C:104
string_cap	10,000 characters [C:074]

SECURITY

- Hook endpoint requires a per-tenant token.
- Accept only http and command handlers; mcp_tool handlers are out of scope until tested.
- onFailure block applies only to managed policy hooks [C:074].
- Tool content never logged by default.

PERFORMANCE

Latency and throughput	Respond under 200 ms at p95 so client timeouts are not hit (DevX proposal).
Complexity	O(1) per event.
At 10x load	Stateless receivers; volume is a few events per tool call.

EDGE CASES AND HOW EACH IS HANDLED

- Hook disabled by the developer: rows missing; gateway rows still complete.
- Timeout: Claude Code takes no decision on most events [C:074].
- Multiple hooks disagree: deny wins [C:074].
- transcript_path lags the turn [C:074]; use last_assistant_message.
- PermissionRequest ignores exit 2 [C:074].

TEST CASES

TEST	GIVEN	EXPECT
join ids	PostToolUse with prompt_id p9	Row stored with prompt_id p9 for the joiner
block	Policy rule deny on Bash(rm *), exit 2 returned	Tool call blocked, row event=blocked
content off	tool_input present, tenant not opted in	Stored row has no tool_input
timeout	Receiver slow beyond client timeout	Client proceeds; gateway rows still complete

WHERE IT SAVES COST

- Repeated-read signals feed the loop guard, since 51% of spend followed the first repeated failure [C:028].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Hook arrival rate drops for a tenant.
- Hook and gateway prompt counts diverge.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.hookr.event`	session_id · prompt_id · event · blocked	On each hook POST

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Claude Code hooks reference: Events, handlers, exit codes, field names.

HOW OTHERS BUILD IT

Hooks run in the client, so a developer can disable them; managed hooks and gateways cannot be disabled by them [A:234, C:074].

REFERENCES

- C:074 Hooks reference
- C:090 Automate actions with hooks (guide)
- C:104 Codex configuration reference
- A:234 Track and optimize coding agent usage (Langfuse)

RECEIVES FROM

Edge gateway
session id

HANDS TO

Session and prompt-id joiner
hook rows

Loop guard
tool events, tool events

← Back to group: Client signals ← Stage 1

LEVEL 4

OTel receiver

Client signal | Stage 1: Ingress and adapters

OTel receiver

Metrics and events, metadata only

claude_code.token.usage type=cacheRead 41200
prompt.id=p9 query_source=repl_main_thread

IN PLAIN ENGLISH

> The client sends usage meters to Token Economics. Token Economics keeps the numbers and drops anything that looks like private text.

Goes in	OpenTelemetry (OTel) metrics and log events over OTLP (OpenTelemetry protocol) from Claude Code or Codex.
Comes out	Metric rows and api_request rows to the joiner, metadata only.
Reads / writes	Reads OTLP payloads. Writes metadata rows; drops content attributes by default.
If it fails	Fail open: OTel is a supplement. Loss only reduces detail, because the gateway log is the source of truth.
Requirement source	RQ-A-011, RQ-A-013, RQ-A-019, RQ-A-020, RQ-A-046, RQ-A-047 Sources: A:217, C:002, C:104, C:076
Hardening, still open	Codex OTel metric and event names are not in the corpus. The gateway cannot join client traces by default.

WHAT IT IS FOR

Receive Claude Code OTel: metrics claude_code.token.usage and .cost.usage, events claude_code.api_request and others [A:217]. Supplies attribution (query_source, agent.name, skill.name, mcp_server.name).

EXPECTED

- Ingest claude_code.token.usage with types input, output, cacheRead, cacheCreation [A:217, C:002].
- Ingest claude_code.api_request with prompt.id and gen_ai.request.model.
- Store query_source and query_source_safe for subagent attribution [A:217].
- Drop prompt, tool-content and API-body events unless opt-in is set.
- Label cost.usage values as client estimate.

NOT EXPECTED

- It never stores OTEL_LOG_USER_PROMPTS content by default.
- It never sets the tenant's opt-ins.
- It never treats claude_code.cost.usage as a bill.

ALGORITHM AND MATHS

OTLP intake with content filter

```
keep = attribute in allowlist; drop = content attributes
```

STEPS

1. Accept OTLP over http protobuf or JSON.
2. Match metric and event names to the allowlist.
3. Remove content-bearing events.
4. Map to UsageEvent fields with cost_source client.
5. Send rows to the joiner.

PARAMETERS

enable	CLAUDE_CODE_ENABLE_TELEMETRY=1 [A:217]
content_optins	OTEL_LOG_USER_PROMPTS, _TOOL_DETAILS, _TOOL_CONTENT, _ASSISTANT_RESPONSES, _RAW_API_BODIES [A:217]
cardinality_options	OTEL_METRICS_INCLUDE_ACCOUNT_UUID, _ENTRYPOINT, _REPOSITORY, _RESOURCE_ATTRIBUTES [A:217]

SECURITY

- Reject OTLP without the tenant header.
- Strip content events at ingest even if sent.
- Cardinality options are set by policy, not left on by default.
- Codex otel.log_user_prompt and log_agent_responses stay false [C:104].

PERFORMANCE

Latency and throughput	Ingest lag under 60 s (DevX proposal).
Complexity	O(events).
At 10x load	Batch exporters send on OTEL_METRIC_EXPORT_INTERVAL [A:217].

EDGE CASES AND HOW EACH IS HANDLED

- Traceparent not propagated to the gateway by default [A:217].
- Metric points arrive out of order: dedupe by timestamp and key.
- Gateway and OTel both report one call: joiner keeps one [A:216].
- Codex event names unknown: store raw names, map later.

TEST CASES

TEST	GIVEN	EXPECT
content dropped	Event claude_code.user_prompt with prompt text	Stored row has no text field
usage type	token.usage type=cacheRead value 41,200	Row cache_read 41,200 with cost_source client
subagent attribution	query_source agent.custom.reviewer	Row attributed to agent reviewer
missing tenant header	OTLP with no header	HTTP 401, nothing stored

WHERE IT SAVES COST

- Per-agent attribution reveals which subagent spends [A:217].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- OTel and gateway totals diverge.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.otelr.ingest`	metric · points · dropped_content	Per OTLP batch

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Claude Code monitoring docs: Metric and event names, attribution attributes.

HOW OTHERS BUILD IT

Claude Code, Codex and Langfuse use OTel as a metadata channel [A:217, C:104, A:232].

REFERENCES

- A:217 Monitoring usage (Claude Code OpenTelemetry)
- C:002 Claude Code OpenTelemetry: Track Cost and Tool Use
- C:104 Codex configuration reference
- C:076 Customize your status line

RECEIVES FROM

Content-logging policy
log flags, log flags

HANDS TO

Session and prompt-id joiner
metric rows

← Back to group: Client signals ← Stage 1

LEVEL 4

Transcript replayer

Client signal | Stage 1: Ingress and adapters

Transcript replayer

Rebuild usage from local files

~/.claude/projects/x.jsonl 214 turns
-> 214 rows cost_source=client

IN PLAIN ENGLISH

> If the live feed missed anything, Token Economics can rebuild it from the files the tool keeps on disk. Those numbers are labelled as guesses.

Goes in	Local transcript JSONL (JSON Lines) files and history.jsonl.
Comes out	Replayed UsageEvent rows labelled client estimate.
Reads / writes	Reads local files. Writes rows and a copy of the usage fields only.
If it fails	Fail open: replay is a recovery path. A parse error skips that file and alerts.
Requirement source	RQ-A-004, RQ-A-013, RQ-A-021, RQ-A-022, RQ-A-049 Sources: C:003, C:074, C:104, C:076
Hardening, still open	Claude Code transcript retention is 30 days per one source [C:003]. Codex history.jsonl schema is not in the corpus.

WHAT IT IS FOR

Rebuild usage from local files when gateway or hook rows are missing. Persist copies, because the corpus notes 30-day retention [C:003].

EXPECTED

- Read transcript_path files and ccusage-style JSONL [C:074, C:003].
- Read Codex history.jsonl when history.persistence is on [C:104].
- Label rows cost_source client, never gateway or bill.
- Skip response ids already in the ledger.
- Copy files to storage before they expire.

NOT EXPECTED

- It never replaces a gateway row.
- It never stores message text.
- It never treats replayed cost as exact.

ALGORITHM AND MATHS

Replay with dedupe

```
emit row iff response_id NOT IN ledger
```

STEPS

1. List new files by mtime.
2. Parse lines for usage fields and ids.
3. Drop content fields.
4. Check each response id against the ledger.
5. Emit rows labelled client estimate.
6. Archive the file.

PARAMETERS

retention_seen	30 days [C:003]
codex_history_keys	history.persistence, history.max_bytes [C:104]

SECURITY

- Read only usage fields.
- Local read happens on the developer machine; upload is metadata only.
- Archive copies inherit the tenant retention rule.

PERFORMANCE

Latency and throughput	Replay of 10,000 lines under 10 s (DevX proposal).
Complexity	O(lines).
At 10x load	Run per machine; batch upload.

EDGE CASES AND HOW EACH IS HANDLED

- Transcript written asynchronously and lags the turn [C:074].
- Resumed session repeats earlier turns: dedupe by response id.
- File truncated mid-line: skip the partial line.
- Schema changes between versions: parser reports unknown shape.

TEST CASES

TEST	GIVEN	EXPECT
replay count	JSONL with 214 turns, none in ledger	214 rows, cost_source client
dedupe	Same response id already in ledger	Row skipped
partial line	Last line cut mid-JSON	Line skipped, file still processed
content strip	Line has message text	Stored row has no text

WHERE IT SAVES COST

- Recover spend that live feeds missed.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Replay fills more than 5% of rows (DevX proposal): live feed has a gap.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.trans.replay`	file · rows · skipped	After each file

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- ccusage: Local JSONL cost reports.

HOW OTHERS BUILD IT

ccusage reads local logs for 15+ agent CLIs [C:003].

REFERENCES

- C:003 How to Track Claude Code Usage in 2026: Built-in Commands, ccusage, and Desktop Dashboards Compared
- C:074 Hooks reference
- C:104 Codex configuration reference
- C:076 Customize your status line

RECEIVES FROM

Observe-only adapter
local files

HANDS TO

Session and prompt-id joiner
replayed rows

← Back to group: Client signals ← Stage 1

LEVEL 4

Session and prompt-id joiner

Client signal | Stage 1: Ingress and adapters

Session and prompt-id joiner

Join gateway, hook, OTel rows

gw(s4,p9)+hook(s4,p9)+otel(p9)
-> row s4/p9 cost_source=gateway

IN PLAIN ENGLISH

> Several reports describe the same step. This part matches them by id so each step is counted once.

Goes in	Gateway rows, hook rows, OTel rows and replayed rows with session_id and prompt_id.
Comes out	One joined row per session and prompt id with cost_source, sent to attr.
Reads / writes	Reads the four row streams. Writes joined rows.
If it fails	Fail open: an unjoined gateway row is still stored, so spend is never lost for lack of a match.
Requirement source	RQ-A-004, RQ-A-011, RQ-A-019, RQ-A-020, RQ-A-048 Sources: C:074, A:217, A:216, C:052
Hardening, still open	The gateway and client traces do not join by default [A:217]. Hooks can be disabled, so hook rows can be absent.
Not in client docs	**DevX proposal:** Join window length is a DevX proposal.

WHAT IT IS FOR

Join on session_id and prompt_id [C:074]. Hook prompt_id equals OTel prompt.id. Enforces single counting [A:216].

EXPECTED

- Key on (session_id, prompt_id).
- Prefer gateway usage; fill detail from hooks and OTel.
- Keep subagent rows separate by query_source [A:217].
- Mark source precedence: bill, gateway, client.
- Leave unmatched gateway rows as is.

NOT EXPECTED

- It never adds two sources for one response id.
- It never invents a prompt id.
- It never sums resumed totals again [A:216].

ALGORITHM AND MATHS

Keyed merge with precedence

```
joined.cost_source = max_precedence(bill > gateway > client)
```

STEPS

1. Index incoming rows by key.
2. Wait up to the join window for late rows.
3. Merge fields; take usage from the highest precedence.
4. Attach agent and skill names from OTel.
5. Emit to attr.

PARAMETERS

join_window	10 min (DevX proposal)
precedence	bill, gateway, client [C:076, A:232]

SECURITY

- Rows carry ids and counts only.
- Tenant id checked on every merge to prevent cross-tenant joins.

PERFORMANCE

Latency and throughput	Join lag under 5 min (DevX proposal).
Complexity	O(1) per row with a keyed store.
At 10x load	Partition by tenant and session.

EDGE CASES AND HOW EACH IS HANDLED

- Hook row arrives after the window: attach on late update.
- Two gateway rows with one response id: dedupe.
- No prompt id (Aider, SDK): key by session and response id.
- Clock skew between sources: order by id, not time.

TEST CASES

TEST	GIVEN	EXPECT
three sources	gw, hook and OTel rows for s4/p9	One joined row, cost_source gateway
late hook	Hook row 15 min after gw row	Row updated, no duplicate
no prompt id	Aider call	Keyed by session and response id
subagent	query_source agent.custom.x	Separate joined row with agent x

WHERE IT SAVES COST

- Correct attribution per caller and subagent [C:052].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Unjoined rate above 10% (DevX proposal).

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.joiner.join	session_id, prompt_id, sources, unjoined	On each emit

REFERENCES

- C:074 Hooks reference
- A:217 Monitoring usage (Claude Code OpenTelemetry)
- A:216 Track cost and usage (Claude Agent SDK)
- C:052 Your cache hit rate is lying to you. It's the caller mix.

RECEIVES FROM

- Hook receiver
  hook rows
- OTel receiver
  metric rows
- Transcript replayer
  replayed rows

HANDS TO

- Spend attribution
  joined row, joined row

← Back to group: Client signals ← Stage 1

LEVEL 2

2. Backends

API providers and CLI agents sit behind one Usage and Result interface, with health tracking and fallback.

API providers

Provider connectors that speak each vendor API and report usage in one normalised shape.

5 Backend

Anthropic connector · OpenAI connector · xAI connector · Gemini connector · Aggregator connector

Receives from: Budgets (Policy engine), Run control (Policy engine), Tier and cache (Policy engine), Security and data (Control plane)

Open group (5 components) →

CLI backends

CLI agents run headless as backends with budget, turn and schema flags.

3 Backend

claude -p runner · codex exec runner · OAuth passthrough guard

Receives from: Harness adapters (Ingress and adapters)

Open group (3 components) →

Backend health

Model catalogue, rate-limit tracking and a priced fallback chain.

3 Backend

Model catalogue · Rate-limit tracker · Fallback chain

Hands to: Routing (Policy engine), Tier and cache (Policy engine)

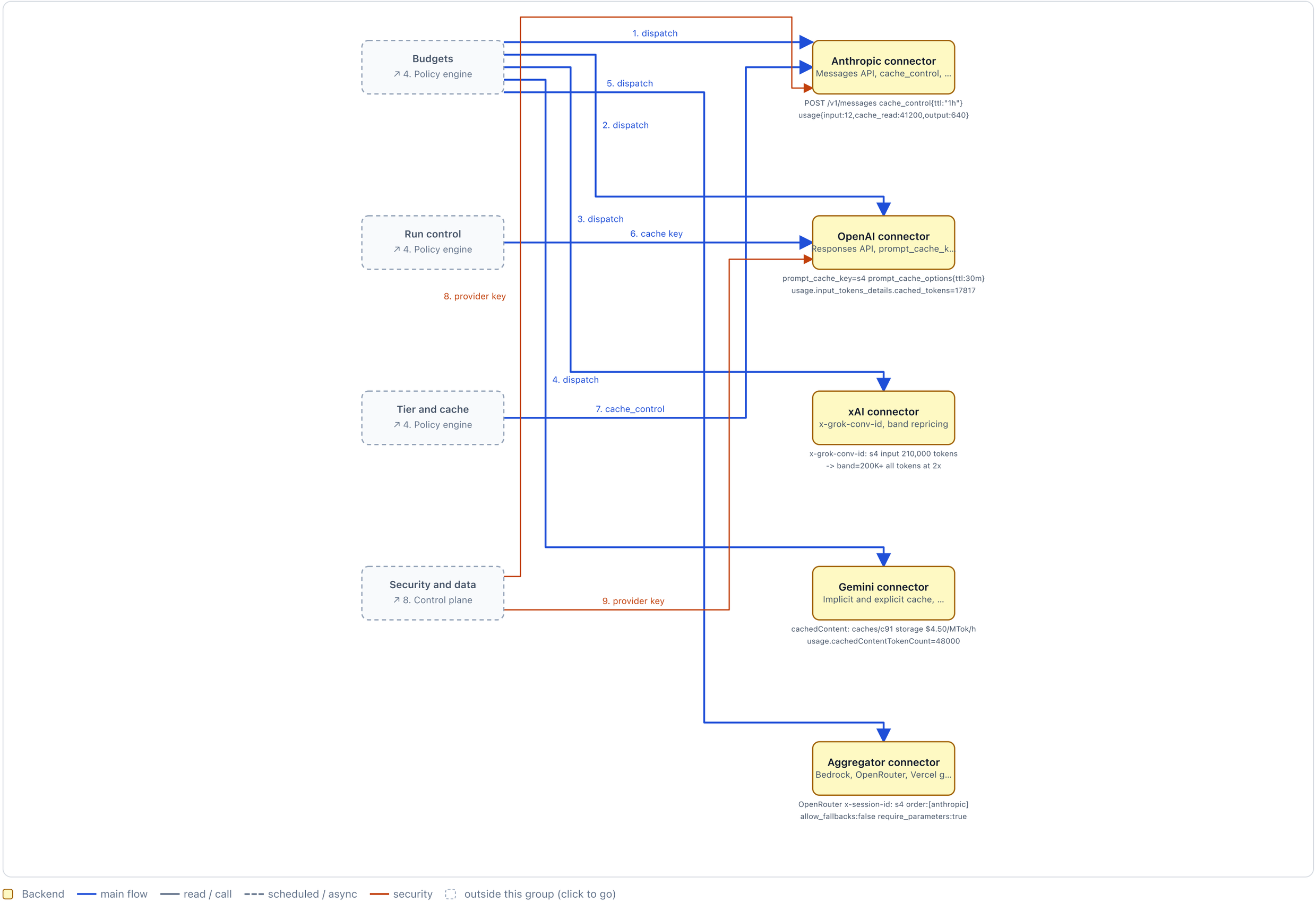
Open group (3 components) →

3 groups, 11 components. Hover a group for the plain-English story; click it to see its components and the arrows between them. ← Ingress and adapters Accounting →

LEVEL 3

API providers in Backends

Provider connectors that speak each vendor API and report usage in one normalised shape.



Backend — main flow — read / call - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Budgets (group) → Anthropic connector
Request after reservation, routing and cache planning.

2. Budgets (group) → OpenAI connector
Responses request with prompt_cache_key.

3. Budgets (group) → xAI connector
Chat request with x-grok-conv-id.

4. Budgets (group) → Gemini connector
Request with cache reference.

5. Budgets (group) → Aggregator connector
Aggregator request.

6. Run control (group) → OpenAI connector
prompt_cache_key per session, cap near 15 requests per minute.

7. Tier and cache (group) → Anthropic connector
Breakpoints, TTL, injection point.

8. Security and data (group) → Anthropic connector
Fetched at call time, never in config.

9. Security and data (group) → OpenAI connector
Fetched at call time.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Backends CLI backends →

LEVEL 4

Anthropic connector

Backend | Stage 2: Backends

Anthropic connector

Messages API, cache_control, tiers

POST /v1/messages cache_control{ttl:"1h"}
usage{input:12,cache_read:41200,output:640}

IN PLAIN ENGLISH

> This part talks to Anthropic's models. It passes along the cache hints so repeated text is cheaper.

Goes in	Dispatched Messages API request from bres, with cache_control plan from cplan.
Comes out	Provider response stream and the usage object to the tee.
Reads / writes	Reads provider key from the vault at call time. Writes nothing.
If it fails	Fail closed: on provider error the call returns the error or hands to fb. It never retries silently, because a retry can double the cost.
Requirement source	RQ-A-009, RQ-A-018, RQ-A-026, RQ-A-027, RQ-A-035, RQ-A-036, RQ-A-043 Sources: A:199, A:283, C:010, C:052, C:054, A:216, A:219
Hardening, still open	Minimum cacheable prefix conflicts: Sonnet 4.6 1,024 [A:199] vs 2,048 [C:051]; Opus 4.7 2,048 vs 4,096. Take the larger until tested. Prices are snapshots from the price book.

WHAT IT IS FOR

Messages API connector: cache_control, tiers, anthropic-beta, usage fields. Read multiplier 0.1x, 5m write 1.25x, 1h write 2x. Exceptions: 0.05x for Opus 5.5 and Sonnet 5.5, 0.025x for Fable 5.1 [A:199, A:283, C:010] (price-list snapshot).

EXPECTED

- Send cache_control {type ephemeral, ttl 5m or 1h} with at most 4 breakpoints [A:199].
- Put longer-TTL entries before shorter ones; send system as an array of blocks [A:199].
- Forward anthropic-beta unchanged [C:052].
- Return input_tokens, cache_creation_input_tokens, cache_read_input_tokens, cache_creation.ephemeral_5m and _1h, output_tokens, inference_geo [A:199].
- Support max_tokens 0 pre-warm [A:199].

NOT EXPECTED

- It never reorders tools, system or messages; prefix order is tools, system, messages [A:199].
- It never adds timestamps to the prefix.
- It never uses batch with streaming, speed or fallbacks [A:219].

ALGORITHM AND MATHS

Cache-preserving dispatch

```
total_input = input_tokens + cache_read + cache_creation [A:199]
```

STEPS

1. Load the provider key from the vault.
2. Apply the cplan breakpoints if present.
3. Check prefix size against the catalogue minimum; drop cache_control if below.
4. Send with anthropic-beta intact.
5. Stream back; pass the usage object to the tee.
6. Pass service tier and speed with the row.

PARAMETERS

max_breakpoints	4 [A:199]
lookback_blocks	20 [A:199]
min_prefix	512 to 4,096 by model [A:199]
ttl_values	5m, 1h [A:199]

SECURITY

- Provider key only from the vault.
- Cache is isolated per organisation, and per workspace on the Claude API [A:199].
- Never log request bodies by default.

PERFORMANCE

Latency and throughput	Adds under 5 ms over the gateway (DevX proposal).
Complexity	O(1) plus stream.
At 10x load	Connection pool per org key.

EDGE CASES AND HOW EACH IS HANDLED

- Prefix under the minimum: no cache_control sent.
- Fast mode toggle mid-session: fast and standard speed have separate caches [C:055].
- Gateway or proxy strips beta header: cache reads fall to 0; alert [C:052].
- Batch results kept 29 days; errored or expired not billed [A:219].
- Above 200K input some models reprice (Opus 4.6 $10/$37.50) [A:270].

TEST CASES

TEST	GIVEN	EXPECT
ttl order	Request has 5m block before 1h block	Connector rejects or reorders per rule with error ttl_order
below minimum	Prefix 300 tokens on a 1,024 model	cache_control removed, row cache_write 0
usage mapping	Response cache_read 41,200, input 12, creation 0	total_input 41,212
beta kept	anthropic-beta present	Header identical upstream

WHERE IT SAVES COST

- Cache read at 0.1x versus fresh input [A:199].
- TTL from reuse gap, not blanket 1h; blanket 1h made the bill 8.6% worse [C:052].
- Batch 50% off for non-urgent work [A:219].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- cache_read share falls below the caller's own baseline.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.p_anth.call	model · tier · cache_read · cache_write_5m · cache_write_1h · output	On each response

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- LiteLLM: cache_control_injection_points.

HOW OTHERS BUILD IT

Anthropic documents the cache rules; gateways such as LiteLLM inject cache_control [A:199, C:060].

REFERENCES

- A:199 Claude prompt caching docs
- A:283 What's new in Claude Opus 5
- C:010 Claude Code Billing & Usage Limits Explained (Oct 2026)
- C:052 Your cache hit rate is lying to you. It's the caller mix.
- C:054 The Coding Agent Economy
- A:216 Track cost and usage (Claude Agent SDK)
- A:219 Batch processing (Claude)

RECEIVES FROM

Reserve and reconcile
dispatch, dispatch

Cache planner
cache_control, cache_control

Secrets vault
provider key, provider key

HANDS TO

None in this stage view.

← Back to group: API providers ← Stage 2

LEVEL 4

OpenAI connector

Backend | Stage 2: Backends

OpenAI connector

Responses API, prompt_cache_key

prompt_cache_key=s4 prompt_cache_options{ttl:30m}
usage.input_tokens_details.cached_tokens=17817

IN PLAIN ENGLISH

> This part talks to OpenAI models. It sends a key that keeps a session's repeated text on the same cache.

Goes in	Dispatched Responses API request with prompt_cache_key from stick.
Comes out	Provider response stream and usage object to the tee.
Reads / writes	Reads provider key from the vault at call time. Writes nothing.
If it fails	Fail closed: return the provider error or hand to fb. No silent retry, because retries cost money.
Requirement source	RQ-A-010, RQ-A-018, RQ-A-026, RQ-A-028, RQ-A-029, RQ-A-030, RQ-A-035, RQ-A-036, RQ-A-043 Sources: A:201, A:202, A:203, A:204, A:148, A:233, A:197, A:195, A:150
Hardening, still open	Cached discount history is 50%, 75% and 90% by model [A:202]. Flex discount percentage is not stated in A:195. Azure 12h write charges start Nov 1 2026 [A:148].

WHAT IT IS FOR

Responses API connector: prompt_cache_key, prompt_cache_options, service_tier, usage detail fields. Routing hashes the first about 256 tokens plus the key. Above about 15 requests per minute per prefix and key, overflow misses [A:202, A:148].

EXPECTED

- Send prompt_cache_key per session [A:202].
- On GPT-5.6 and later send prompt_cache_options.mode implicit or explicit and ttl 30m or 12h [A:201, A:204].
- Before 5.6 use prompt_cache_retention in_memory or 24h with no write fee [A:201].
- Return usage.input_tokens_details.cached_tokens and cache_write_tokens [A:204].
- Pass service_tier flex when the tier selector chooses it [A:195].

NOT EXPECTED

- It never adds reasoning_tokens to output [A:197].
- It never sends more than about 15 requests a minute for one prefix and key.
- It never assumes prompt_tokens excludes cached tokens.

ALGORITHM AND MATHS

Keyed dispatch with rate cap

```
cap: requests per (prefix, key) per minute <= 15 [A:148]
```

STEPS

1. Build prompt_cache_key from the session id.
2. Check the per-key counter; shard the key if over the cap.
3. Set cache options by model generation.
4. Send; stream response.
5. Pass cached_tokens and cache_write_tokens to the tee.
6. Pass diagnostics (cache_hit, cache_miss) to hit.

PARAMETERS

min_prefix	1,024 tokens, hits in 128-token steps [A:201]
write_fee	1.25x for 30m, 2.5x for 12h on GPT-5.6+ [A:201]
read	0.1x, 0.05x on GPT-6.1 Sol and Terra [A:201]

SECURITY

- Provider key only from the vault.
- Do not put user ids in prompt_cache_key; use the session hash.
- Never log bodies.

PERFORMANCE

Latency and throughput	Adds under 5 ms (DevX proposal).
Complexity	O(1) plus stream.
At 10x load	Per-key counters kept in memory, sharded by key.

EDGE CASES AND HOW EACH IS HANDLED

- prompt_tokens includes cached: 17,903 with 17,817 cached means fresh 86 [A:233].
- Flex 429 is not charged, no auto-fallback [A:195].
- Overflow past 15 requests a minute causes misses [A:202].
- Prefix of 102 tokens never pays a write [A:201].
- Azure Global Batch completion_window 24h [A:152].

TEST CASES

TEST	GIVEN	EXPECT
inclusive usage	prompt_tokens 17,903, cached 17,817 [A:233]	Raw passed; norm gets fresh 86
reasoning inside output	output 1,186 of which reasoning 1,024 [A:197]	Output stays 1,186
key shard	20 requests a minute on one key	Key sharded so none exceeds 15
flex 429	service_tier flex returns 429	No charge; fb retries on standard

WHERE IT SAVES COST

- Cache read 0.1x on GPT-5.6+ [A:201].
- One write plus 9 reads costs 2.15x instead of 10x [A:201].
- Flex at batch rates for latency-tolerant work [A:195].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Cache hit falls near the 15 per minute cap.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.p_oai.call	model · tier · cached_tokens · cache_write_tokens · output	On each response

REFERENCES

- A:201 Prompt caching
- A:202 Prompt Caching 201
- A:203 Prompt Caching 101
- A:204 Prompt cache diagnostics
- A:148 Prompt caching with Azure OpenAI in Azure AI Foundry Models
- A:233 LLM Cost Management: Track and Control Spend
- A:197 Reasoning models
- A:195 Flex processing
- A:150 Use Flex processing with Azure OpenAI in Foundry Models

RECEIVES FROM

- Reserve and reconcile
  dispatch
- Session stickiness
  cache key, cache key
- Secrets vault
  provider key

HANDS TO

None in this stage view.

LEVEL 4

xAI connector

Backend | Stage 2: Backends

xAI connector

x-grok-conv-id, band repricing

x-grok-conv-id: s4 input 210,000 tokens
-> band=200K+ all tokens at 2x

IN PLAIN ENGLISH

> This part talks to xAI models and sends a session tag for caching. Past 200,000 tokens, everything on that call costs double.

Goes in	Dispatched chat request with x-grok-conv-id from the session.
Comes out	Provider response stream and usage object to the tee, with the context band.
Reads / writes	Reads provider key from the vault. Writes nothing.
If it fails	Fail closed: return the error or hand to fb; no silent retry.
Requirement source	RQ-A-018, RQ-A-026, RQ-A-030, RQ-A-031, RQ-A-035, RQ-A-043 Sources: A:221, A:223, A:224, A:225
Hardening, still open	xAI cached-rate and band details come from price-list pages; Grok model names are quoted as in the corpus (Grok 4.7).

WHAT IT IS FOR

xAI connector: x-grok-conv-id header or prompt_cache_key; cached rate is one quarter of input; 128-token blocks. At 200K and above all tokens reprice at 2x, cached included. Priority is 2x all token types; batch is 20% off [A:221 to A:225] (price-list).

EXPECTED

- Send x-grok-conv-id per session [A:221].
- Report the total prompt size so calc can apply the 200K band [A:225].
- Pass priority tier requests with the 2x flag [A:225].
- Return input_tokens_details.cached_tokens [A:221].
- Keep volatile text out of the prefix [A:224].

NOT EXPECTED

- It never puts the time at the top of the system prompt. That gave 87% vs 3% cache and 2.2 to 2.9x cost [A:224].
- It never assumes the 200K band applies only to fresh tokens.

ALGORITHM AND MATHS

Conversation-id dispatch

```
band = total_prompt_tokens >= 200,000 ? 'over200k' : 'base' [A:225]
```

STEPS

1. Set x-grok-conv-id to the session hash.
2. Check prefix for volatile fields; warn if found.
3. Send; stream.
4. Pass cached_tokens, total prompt size and tier to the tee.

PARAMETERS

cache_rate	1/4 of input [A:221]
block	128 tokens [A:221]
batch_discount	20% [A:225]
priority	2x [A:225]
us_endpoint	1.1x [A:225]

SECURITY

- Provider key from the vault only.
- Conversation id is a hash, never a user name.

PERFORMANCE

Latency and throughput	Adds under 5 ms (DevX proposal).
Complexity	O(1) plus stream.
At 10x load	Stateless.

EDGE CASES AND HOW EACH IS HANDLED

- Prompt crosses 200K mid-session: next calls reprice all tokens; alert.
- Tool calls cost $5 per 1,000 [A:225].
- Priority tier with cache: cache applied first, then 2x.
- Missing conv id: cache hits fall.

TEST CASES

TEST	GIVEN	EXPECT
band	Prompt 210,000 tokens	Row band over200k, scope all_tokens
conv id	Session s4 call	Header x-grok-conv-id equals hash of s4
volatile prefix	Timestamp in system prompt	Warning volatile_prefix raised
priority	Priority tier request	Row tier priority, multiplier 2.0

WHERE IT SAVES COST

- Cached input at 1/4 rate [A:221].
- Stay under 200K where possible [A:225].
- Batch 20% off [A:225].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Prompt size approaches 200K.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.p_xai.call	model · tier · band · cached_tokens	On each response

REFERENCES

- A:221 Usage & Pricing (xAI prompt caching)
- A:223 How Prompt Caching Works (xAI)
- A:224 Prompt Caching: Cut the Cost of Long Chats with the Grok API
- A:225 xAI pricing

RECEIVES FROM

Reserve and reconcile
dispatch

HANDS TO

None in this stage view.

← Back to group: API providers ← Stage 2

LEVEL 4

Gemini connector

Backend | Stage 2: Backends

Gemini connector

Implicit and explicit cache, storage fee

cachedContent: caches/c91 storage $4.50/MTok/h
usage.cachedContentTokenCount=48000

IN PLAIN ENGLISH

> This part talks to Google's Gemini models. It can store a long prompt with Google for a small hourly fee.

Goes in	Dispatched Gemini request with an optional CachedContent reference.
Comes out	Provider response and usage object to the tee, with cachedContentTokenCount.
Reads / writes	Reads provider key from the vault. Writes explicit-cache resource ids and expiry.
If it fails	Fail closed: return the error or hand to fb. Explicit caches incur a storage fee, so an orphaned cache is deleted on failure.
Requirement source	RQ-A-018, RQ-A-026, RQ-A-030, RQ-A-032, RQ-A-035, RQ-A-043 Sources: A:121, A:143, A:231, A:237, A:280, A:281
Hardening, still open	Implicit discount 75% [A:121] vs 90% [A:280]. Minimum prefix conflicts: 1,024 Flash and 2,048 Pro [A:121, A:237] vs 4,096 and 6,144 for Gemini 3 and 4 [A:280]. Storage rate is not in the corpus except Gemini 2.5 Pro $4.50 per MTok per hour. Candidate-token split is not in the corpus.

WHAT IT IS FOR

Gemini connector: implicit cache is on by default; explicit CachedContent has a storage fee (2.5 Pro $4.50 per MTok per hour). Prices tier at 200K (2.5 Pro $1.25 to $2.50 input). Batch 50%, stacking with cache to 95% on Gemini 3 and later but not on 2.5 [A:143, A:237, A:280, A:281] (price-list).

EXPECTED

- Read cachedContentTokenCount (REST) or cached_content_token_count (SDK) [A:143, A:121].
- Create, reuse and delete CachedContent resources with a TTL.
- Record storage hours for the calculator.
- Report the 200K band.
- Pass batch flag so calc applies the stacking rule per model [A:281].

NOT EXPECTED

- It never assumes the 75% or 90% discount; the price book decides.
- It never leaves an explicit cache running without an owner.
- It never assumes batch stacks on Gemini 2.5.

ALGORITHM AND MATHS

Explicit cache lifecycle

```
keep explicit cache iff expected_reads x saving > storage_fee x hours
```

STEPS

1. Check prefix against the model minimum in the catalogue.
2. If explicit is chosen, create CachedContent with a TTL.
3. Reference it in calls.
4. On expiry or low reuse, delete it.
5. Log storage hours.

PARAMETERS

storage_example	2.5 Pro $4.50 per MTok per hour [A:143]
batch_stack	yes on Gemini 3+, no on 2.5 [A:281]

SECURITY

- Provider key from the vault only.
- Cached content holds prompt text at the provider; delete on tenant offboarding.
- No content stored by Token Economics.

PERFORMANCE

Latency and throughput	Adds under 5 ms for calls (DevX proposal).
Complexity	O(1) plus stream.
At 10x load	Cache resources tracked per tenant.

EDGE CASES AND HOW EACH IS HANDLED

- Gemini CLI caching works with API key and Vertex only, not OAuth [C:121].
- Prefix below the minimum: no explicit cache.
- Batch queue 72h outside SLA [A:281].
- Cache TTL expires mid-session: re-create costs a write.

TEST CASES

TEST	GIVEN	EXPECT
cached count	cachedContentTokenCount 48,000	Row cache_read 48,000
orphan cleanup	Call fails after cache creation	CachedContent deleted
batch stack gem 2.5	Batch flag on Gemini 2.5	stacks false in row
band	Prompt 250,000 tokens on a 200K-tier model	Row band over200k

WHERE IT SAVES COST

- Implicit cache at no storage fee.
- Explicit cache only past the storage break-even.
- Batch 50% [A:281].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Explicit cache read count stays below the break-even.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.p_gem.call`	model · cached_tokens · explicit_cache · band	On each response
`token_economics.p_gem.cache_storage`	cache_id · hours · tokens	On cache delete or expiry

REFERENCES

- A:121 Gemini 2.5 Models now support implicit caching
- A:143 Save costs and decrease latency while using Gemini with Vertex AI context caching
- A:231 Prompt caching best practices (OpenRouter)
- A:237 Prompt caching (Helicone gateway)
- A:280 Context caching overview (Vertex AI)
- A:281 Batch inference (Gemini Enterprise)

RECEIVES FROM

Reserve and reconcile
dispatch

HANDS TO

None in this stage view.

← Back to group: API providers ← Stage 2

LEVEL 4

Aggregator connector

Backend | Stage 2: Backends

Aggregator connector

Bedrock, OpenRouter, Vercel gateway

OpenRouter x-session-id: s4 order:[anthropic]
allow_fallbacks:false require_parameters:true

IN PLAIN ENGLISH

> This part reaches models through middlemen such as Bedrock and OpenRouter. It pins each session to one provider so the cache keeps working.

Goes in	Dispatched request for Bedrock, OpenRouter or Vercel AI Gateway, with session id and routing options.
Comes out	Provider response and usage object to the tee.
Reads / writes	Reads provider key from the vault. Writes nothing.
If it fails	Fail closed: return the error or hand to fb. Aggregator fallbacks are priced hops, never silent [A:227].
Requirement source	RQ-A-010, RQ-A-018, RQ-A-026, RQ-A-033, RQ-A-034, RQ-A-043 Sources: A:157, A:159, A:160, A:227, A:229, A:231, A:247, A:248, A:158
Hardening, still open	Vercel usage field names are not in the corpus [A:247]. Cloudflare AI Gateway is not in the corpus. OpenRouter default router costs 6.0% more per request (7.3% per MTok) [A:229].

WHAT IT IS FOR

Aggregator connector. Bedrock Converse uses cachePoint and fields cacheReadInputTokens, cacheWriteInputTokens, inputTokens; total input is the sum [A:157, A:159]. OpenRouter uses session_id or x-session-id (max 256 chars), sticky routing with 60-minute expiry [A:231].

EXPECTED

- Bedrock: send cachePoint and sum the three input fields [A:157, A:159].
- Bedrock InvokeModel: use cache_control [A:159].
- OpenRouter: send x-session-id and set order, allow_fallbacks, require_parameters, data_collection [A:227, A:231].
- OpenRouter: read prompt_tokens_details.cached_tokens, cache_write_tokens, cache_discount [A:231, A:227].
- Vercel: session-affinity header, caching auto, cache_ttl [A:247, A:248].

NOT EXPECTED

- It never sends ttl to 5-minute-only Bedrock models [A:160].
- It never relies on cache for Bedrock batch or Nova reads [A:160, A:157].
- It never allows silent provider fallback that breaks the cache [A:227].

ALGORITHM AND MATHS

Aggregator pinning

```
provider_for_session = first choice; switch only through fb with priced cache loss
```

STEPS

1. Detect backend family from the model id in cat.
2. Set session affinity (session_id, x-session-id or Vercel header).
3. Set routing options explicitly.
4. Send; stream.
5. Map usage per family and pass to the tee.

PARAMETERS

openrouter_session_expiry	60 minutes [A:231]
bedrock_tiers	Flex, Standard, Priority, Reserved [A:159]
bedrock_batch	50%, up to 168h, no caching [A:160]

SECURITY

- Provider keys from the vault.
- Set data_collection explicitly for privacy [A:227].
- Bring-your-own-key spend must still be counted [A:245].

PERFORMANCE

Latency and throughput	Adds under 5 ms (DevX proposal).
Complexity	O(1) plus stream.
At 10x load	Stateless.

EDGE CASES AND HOW EACH IS HANDLED

- Nova: implicit caching, not billed or reported as reads, 20K max cacheable [A:157].
- Default OpenRouter router adds 6.0% per request [A:229].
- Provider fallback changes endpoint and loses the cache [A:231].
- Usage lacks cache fields: record unknown, do not assume zero.

TEST CASES

TEST	GIVEN	EXPECT
bedrock sum	inputTokens 100, read 900, write 0	total_input 1,000
ttl guard	5-minute-only Bedrock model with ttl 1h	ttl removed
openrouter pin	Session s4	x-session-id s4 and allow_fallbacks false
session length	session id 300 chars	Rejected, max 256

WHERE IT SAVES COST

- Sticky routing keeps cache hits [A:231].
- Avoid the default router premium where latency allows [A:229].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Cache read share drops after a provider fallback.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.p_rtr.call`	family · model · cache_read · provider_chosen	On each response

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- OpenRouter: Sticky routing, session_id.
- Vercel AI Gateway: Session affinity header.

HOW OTHERS BUILD IT

OpenRouter and Vercel gateways both provide session affinity for cache hits [A:231, A:247].

REFERENCES

- A:157 Prompt caching for faster model inference
- A:159 Capacity and Performance
- A:160 Process multiple prompts with batch inference
- A:227 Provider routing (OpenRouter)
- A:229 How OpenRouter rebuilt provider routing for better, faster inference
- A:231 Prompt caching best practices (OpenRouter)
- A:247 AI Gateway Automatic Prompt Caching (Vercel)
- A:248 Provider Options (Vercel AI Gateway)
- A:158 Understanding intelligent prompt routing in Amazon Bedrock

RECEIVES FROM

Reserve and reconcile
dispatch

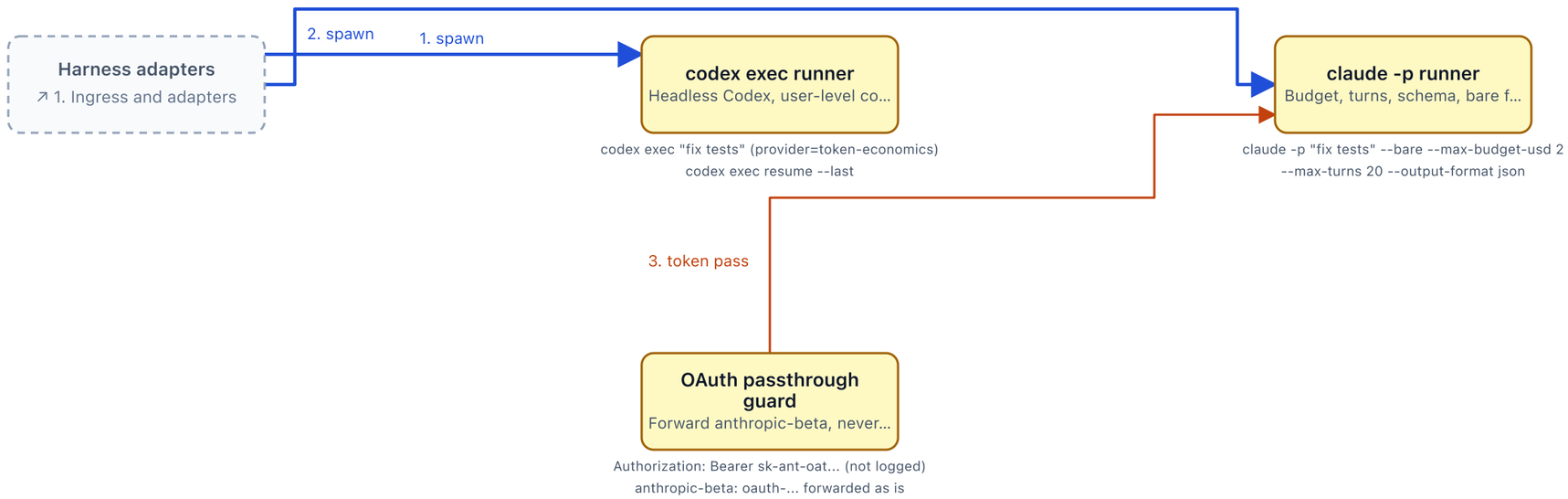
HANDS TO

None in this stage view.

← Back to group: API providers ← Stage 2

LEVEL 3 CLI backends in Backends

CLI agents run headless as backends with budget, turn and schema flags.



Backend — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Harness adapters (group) → codex exec runner
Headless run with user-level provider config.

2. Harness adapters (group) → claude -p runner
Headless run with budget, turn and schema flags.

3. OAuth passthrough guard → claude -p runner
Forwarded auth header and anthropic-beta; never stored.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Backends ← API providers Backend health →

LEVEL 4

claude -p runner

Backend | Stage 2: Backends

claude -p runner

Budget, turns, schema, bare flags

```
claude -p "fix tests" --bare --max-budget-usd 2
--max-turns 20 --output-format json
```

IN PLAIN ENGLISH

> This part runs Claude Code as a background worker with a spending limit. It collects the result and the usage numbers.

Goes in	Task, budget, turn cap and optional JSON schema from a_cli.
Comes out	A claude -p run result with session id, usage and structured_output.
Reads / writes	Reads policy and flags. Writes run records.
If it fails	Fail closed on budget: no run starts without --max-budget-usd. A failed run is recorded as spend against the task.
Requirement source	RQ-A-014, RQ-A-021, RQ-A-050, RQ-A-051, RQ-A-053 Sources: C:094, C:080, C:073, C:095, C:136
Hardening, still open	--max-budget-usd stops remaining background work at the cap, so a step in flight may finish. Figures are client-side estimates. --max-turns flag text appears only in C:080.

WHAT IT IS FOR

Backend runner for claude -p [C:094]. Flags: --output-format json, --json-schema, --bare, --permission-mode, --settings, --mcp-config, --agents, --allowedTools [C:094].

EXPECTED

- Run with --max-budget-usd and --max-turns on every run.
- Return session id, usage and structured_output; exit 0 on success [C:094].
- Use --bare for reproducible runs: it skips hooks, skills, MCP, plugins, memory and CLAUDE.md [C:094].
- Pass hooks explicitly with --settings when needed [C:094].
- Set model through --model or ANTHROPIC_MODEL [C:073].

NOT EXPECTED

- It never relies on the budget flag as a hard stop for in-flight work.
- It never reads CLAUDE.md in --bare mode.
- It never sums resumed totals again.

ALGORITHM AND MATHS

Capped headless run

```
stop when cost_estimate >= max_budget_usd OR turns >= max_turns
```

STEPS

1. Compose flags from policy.
2. Set ANTHROPIC_BASE_URL to gw so the gateway also meters.
3. Spawn and read JSON.
4. Validate structured_output against the schema.
5. Emit row; mark success from exit code and schema.

PARAMETERS

stdin_cap	10MB [C:094]
permission_mode	default, acceptEdits, plan, auto, dontAsk, bypassPermissions [C:095]

SECURITY

- Never use bypassPermissions in shared runners without a sandbox (DevX proposal).
- Virtual key via environment.
- Per-run working directory.

PERFORMANCE

Latency and throughput	Startup under 2 s (DevX proposal).
Complexity	O(1) setup.
At 10x load	Worker pool.

EDGE CASES AND HOW EACH IS HANDLED

- Budget reached: remaining background work stops, one step may finish [C:094].
- Resumed run reports whole-conversation totals [C:094].
- Schema invalid: mark failed attempt.
- Piped input above 10MB fails.

TEST CASES

TEST	GIVEN	EXPECT
flags present	Run request	Command has --max-budget-usd and --max-turns
bare hooks	--bare run	No hook rows from the client
schema fail	structured_output breaks schema	Row success false, cost kept
resume	Prior total 3.20, new total 5.00	Row cost 1.80

WHERE IT SAVES COST

- Turn cap at P75 [C:025].
- Count failed attempts in cost per success [C:028].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Budget cap hit rate high for a task class.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.b_cc.run`	run_id · budget_usd · turns · exit_code · schema_ok	When a run ends

REFERENCES

- C:094 Run Claude Code programmatically
- C:080 Best practices for Claude Code
- C:073 Model configuration and effort levels
- C:095 Choose a permission mode
- C:136 Coding agents ignore spend (Ramp Labs)

RECEIVES FROM

Headless CLI adapter
spawn, spawn

OAuth passthrough guard
token pass

HANDS TO

None in this stage view.

← Back to group: CLI backends ← Stage 2

LEVEL 4

codex exec runner

Backend | Stage 2: Backends

codex exec runner

Headless Codex, user-level config

codex exec "fix tests" (provider=token-economics)
codex exec resume --last

IN PLAIN ENGLISH

> This part runs Codex as a background worker through Token Economics. Since Codex has no spend flag, the money limit sits on Token Economics's key.

Goes in	Task, policy and user-level provider config for a Codex run.
Comes out	A codex exec run result.
Reads / writes	Reads policy and flags. Writes run records.
If it fails	Fail closed on budget via the gateway key, because Codex has no budget flag in the corpus.
Requirement source	RQ-A-014, RQ-A-052, RQ-A-053 Sources: C:035, C:104, C:097
Hardening, still open	Codex budget flag, hook schema and output JSON shape are not in the corpus. Command, local-config and plugin hooks are unsupported under cloud orchestration.

WHAT IT IS FOR

Backend runner for codex exec and codex exec resume --last [C:035], with model_provider in user-level config [C:104].

EXPECTED

- Run codex exec with the Token Economics provider selected in user-level config.
- Use a virtual key with a budget as the only hard cap.
- Set model_reasoning_effort and service_tier explicitly [C:104].
- Limit subagents with agents.max_threads [C:104].
- Record success from exit status.

NOT EXPECTED

- It never assumes a budget flag.
- It never writes provider config to the project file [C:104].
- It never assumes a usage JSON shape without a test.

ALGORITHM AND MATHS

Gateway-capped exec

```
hard cap = virtual key budget (no native flag)
```

STEPS

1. Check cap flags: budgetFlag false.
2. Require a budgeted key.
3. Spawn codex exec.
4. Read gateway rows for usage.
5. Mark success from exit status.

PARAMETERS

agents_max_threads	set per policy [C:104]
tool_output_token_limit	set per policy [C:104]

SECURITY

- Virtual key via auth.command.
- Sandbox mode set to the strictest that fits the task [C:104].
- No project-level provider override.

PERFORMANCE

Latency and throughput	Startup under 2 s (DevX proposal).
Complexity	O(1) setup.
At 10x load	Worker pool.

EDGE CASES AND HOW EACH IS HANDLED

- Managed Work Cloud ignores model_provider [C:104].
- ChatGPT sign-in cannot be assumed to work with a custom provider.
- Resume last session: totals may repeat; use gateway rows.
- Subagent fan-out multiplies spend; cap threads.

TEST CASES

TEST	GIVEN	EXPECT
budget key	Run with key budget 2.00 USD	Gateway returns 402 after 2.00 USD of spend
no flag	Policy requests a Codex budget flag	Flag false; key budget used
provider pinned	Run in repo with project config override	User-level provider still used
thread cap	agents.max_threads 2	No more than 2 subagent threads

WHERE IT SAVES COST

- Cap threads and tool output limits [C:104].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Gateway 402 mid-run.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.b_cx.run`	run_id · budget_usd · exit_code	When a run ends

REFERENCES

- C:035 Context Management Across Codex Surfaces
- C:104 Codex configuration reference
- C:097 Codex local config (config.toml)

RECEIVES FROM

Headless CLI adapter
spawn

HANDS TO

None in this stage view.

← Back to group: CLI backends ← Stage 2

LEVEL 4

OAuth passthrough guard

Backend | Stage 2: Backends

OAuth passthrough guard

Forward anthropic-beta, never store tokens

Authorization: Bearer sk-ant-oat... (not logged)
anthropic-beta: oauth-... forwarded as is

IN PLAIN ENGLISH

> Some people use a subscription login instead of a key. This part passes that login through untouched and never keeps a copy.

Goes in	Subscription OAuth token and the anthropic-beta header from the client.
Comes out	The same Authorization header and anthropic-beta values forwarded upstream.
Reads / writes	Reads headers in memory. Writes nothing: no token is stored or logged.
If it fails	Fail closed for the subscription route. If forwarding is unsafe, return 502 and skip the upstream call. A mangled token could fail or leak.
Requirement source	RQ-A-015, RQ-A-016 Sources: A:193, C:076, C:010
Hardening, still open	Whether a custom Codex provider can carry ChatGPT sign-in is not in the corpus. Subscription traffic has no dollar figure; only percentage windows exist.
Not in client docs	**DevX proposal:** The no-storage rule is a DevX proposal; the corpus states only that the capability must be forwarded.

WHAT IT IS FOR

Allow Claude Code on a subscription (claude.ai OAuth) to work through the gateway. The gateway must forward the OAuth capability in anthropic-beta [A:193]. Token Economics can route and observe; billing and limits stay with the subscription.

EXPECTED

- Forward Authorization and anthropic-beta untouched.
- Mark the row auth_mode subscription and cost_source client.
- Read rate_limits.five_hour and seven_day from the status-line channel [C:076].
- Show subscription spend as percentage windows, not dollars.
- Pass the token only to b_cc paths.

NOT EXPECTED

- It never stores, hashes for reuse or logs the token.
- It never converts subscription limits to tokens [C:010].
- It never applies a dollar budget to subscription traffic.

ALGORITHM AND MATHS

Header passthrough

```
upstream.headers = client.headers (auth, anthropic-beta) unchanged
```

STEPS

1. Detect OAuth by header shape.
2. Copy auth and anthropic-beta.
3. Strip nothing else.
4. Mark auth_mode.
5. Redact in every log line.

PARAMETERS

status_line_fields	rate_limits.five_hour.used_percentage, rate_limits.seven_day.used_percentage [C:076]

SECURITY

- Token never written to disk, log or metric.
- Memory-only handling with immediate release.
- Tokens never leave the Token Economics to Anthropic path.
- Alert if a token shape appears in any stored field.

PERFORMANCE

Latency and throughput	Zero added latency beyond header copy (DevX proposal).
Complexity	O(1).
At 10x load	Stateless.

EDGE CASES AND HOW EACH IS HANDLED

- Gateway drops the oauth beta value: calls fail with auth errors.
- Token refresh happens client-side; guard sees a new header.
- Subscription has no dollar meter; budget alerts use percentage windows.
- Anthropic does not support non-Claude routing for Claude Code [A:193].

TEST CASES

TEST	GIVEN	EXPECT
passthrough	Request with oauth beta and bearer	Upstream headers identical
no storage	Scan logs after 100 calls	Zero token strings found
mode label	OAuth call	auth_mode subscription, cost_source client
beta dropped	Test gateway build that strips anthropic-beta	Test fails before release

WHERE IT SAVES COST

- Window-aware routing on subscription avoids limit stalls.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Auth failure spike after Claude Code upgrade.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.b_oauth.pass	session_id, auth_mode, beta_forwarded	On each subscription call

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Claude apps gateway: Forwards anthropic-beta values.

HOW OTHERS BUILD IT

Anthropic's own gateway forwards anthropic-beta by construction [C:088].

REFERENCES

- A:193 LLM gateway configuration
- C:076 Customize your status line
- C:010 Claude Code Billing & Usage Limits Explained (Oct 2026)

RECEIVES FROM

None in this stage view.

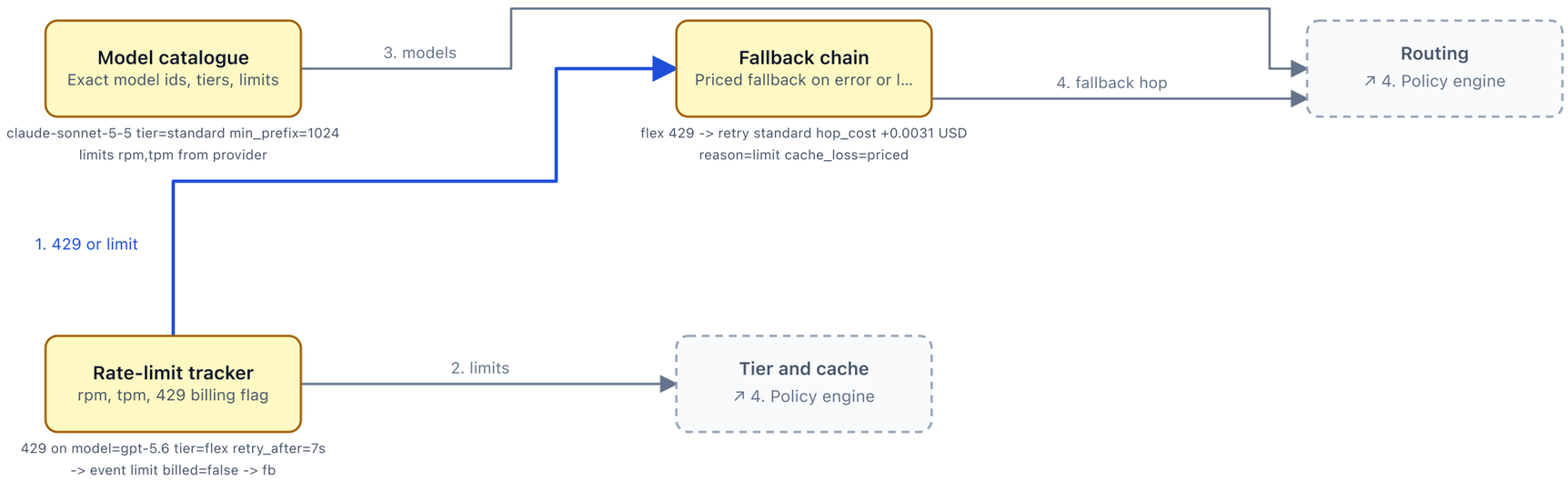
HANDS TO

claude -p runner
token pass

← Back to group: CLI backends ← Stage 2

LEVEL 3 Backend health in Backends

Model catalogue, rate-limit tracking and a priced fallback chain.



Backend — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Rate-limit tracker → Fallback chain
Limit events that trigger a priced fallback.

2. Rate-limit tracker → Tier and cache (group)
Provider limits inform tier choice.

3. Model catalogue → Routing (group)
Exact model ids, tiers and limits for the router.

4. Fallback chain → Routing (group)
Fallback is a priced hop, never silent.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Backends ← CLI backends

LEVEL 4

Model catalogue

Backend | Stage 2: Backends

Model catalogue

Exact model ids, tiers, limits

claude-sonnet-5-5 tier=standard min_prefix=1024
limits rpm,tpm from provider

IN PLAIN ENGLISH

> This is the list of exact model names Token Economics knows. If a name is not on it, Token Economics refuses rather than guessing.

Goes in	Model id from a request or policy.
Comes out	Exact model record: id, provider, tier, context limits, minimum prefix, rpm and tpm.
Reads / writes	Reads catalogue rows. Writes new rows after review.
If it fails	Fail closed on an unknown id: reject with unknown_model. A partial-name match gives a silent wrong price [A:232].
Requirement source	RQ-A-036, RQ-A-037, RQ-A-038, RQ-A-039 Sources: A:232, A:199, C:051, A:195, A:150, A:219, C:010
Hardening, still open	Anthropic minimum prefix conflicts: Sonnet 4.6 1,024 vs 2,048; Opus 4.7 2,048 vs 4,096 [A:199, C:051]; store the larger until tested. Model names (Fable 5.1, Opus 5.5, GPT-5.6, Grok 4.7) are quoted as in the corpus and unverifiable.

WHAT IT IS FOR

Exact model ids, tiers and limits for the router and connectors. Price rows live in pb; the catalogue holds facts such as minimum prefix, context window, tier support.

EXPECTED

- Match exact ids only.
- Hold per model: provider, tiers (standard, batch, flex, priority), min_prefix, max breakpoints, context band thresholds.
- Hold limits: rpm, tpm, rpd, batch_max_requests, batch_window [A:219].
- Hold flags: 429_billed, auto_fallback [A:195, A:150].
- Carry source id and verified date per row.

NOT EXPECTED

- It never prices a call; pb does.
- It never accepts aliases without an explicit alias row.
- It never edits rows without review.

ALGORITHM AND MATHS

Exact-id lookup

```
record = catalogue[provider][exact_id]; else error
```

STEPS

1. Normalise case only; keep dots and dashes.
2. Look up by (provider, id).
3. Return record and tier flags.
4. Log lookup misses.

PARAMETERS

min_prefix_rule	store the larger of conflicting values [C:051, A:199]
haiku_5_5_band	100K split [C:010]

SECURITY

- Catalogue edits are audited policy changes.
- No credentials stored.

PERFORMANCE

Latency and throughput	In-memory lookup (DevX proposal).
Complexity	O(1).
At 10x load	Loaded per instance.

EDGE CASES AND HOW EACH IS HANDLED

- Model retired: row expires; calls rejected with model_retired.
- Two providers expose one model: separate rows.
- Anthropic Haiku 5.5 tiered at 100K [C:010].
- Alias such as a latest tag: refuse unless mapped.

TEST CASES

TEST	GIVEN	EXPECT
exact	claude-sonnet-5-5	Record returned
partial	sonnet	unknown_model error
min prefix conflict	Sonnet 4.6	min_prefix 2,048
retired	Expired row	model_retired

WHERE IT SAVES COST

- Right min_prefix avoids useless cache writes [A:201].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Lookup miss rate above 0.5% (DevX proposal).

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.cat.miss	requested_id, provider	On an unknown model id

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Langfuse: Exact model match; partial-name matching gives wrong prices.

HOW OTHERS BUILD IT

Langfuse warns that partial-name matches silently price wrongly [A:232].

REFERENCES

- A:232 Token & Cost Tracking (Langfuse)
- A:199 Claude prompt caching docs
- C:051 How to Add Prompt Caching to an Anthropic SDK App and Measure the Hit Rate
- A:195 Flex processing
- A:150 Use Flex processing with Azure OpenAI in Foundry Models
- A:219 Batch processing (Claude)
- C:010 Claude Code Billing & Usage Limits Explained (Oct 2026)

RECEIVES FROM

None in this stage view.

HANDS TO

Rules and atomic classifier
models, models

← Back to group: Backend health ← Stage 2

LEVEL 4

Rate-limit tracker

Backend | Stage 2: Backends

Rate-limit tracker

rpm, tpm, 429 billing flag

429 on model=gpt-5.6 tier=flex retry_after=7s
-> event limit billed=false -> fb

IN PLAIN ENGLISH

> This part watches for signs that a provider is saying slow down. When it hears that, it tells the fallback step.

Goes in	Provider headers and status codes: rate-limit remaining, 429, retry-after.
Comes out	Limit events to fb, and limits to the tier selector.
Reads / writes	Reads response headers and catalogue limits. Writes counters per model, tier and key.
If it fails	Fail open on counter loss: calls proceed and the provider 429 is the backstop. Budget protection is in bres.
Requirement source	RQ-A-024, RQ-A-038, RQ-A-039 Sources: A:195, A:150, A:219, A:148, A:202
Hardening, still open	Header names per provider are not all in the corpus. Whether a 429 is billed varies; flex and Azure flex 429 are not charged [A:195, A:150].
Not in client docs	**DevX proposal:** Counter window length is a DevX proposal.

WHAT IT IS FOR

Track rpm (requests per minute), tpm (tokens per minute) and 429 responses, and record whether a 429 is billed [A:195, A:150].

EXPECTED

- Count requests and tokens per (provider, model, tier, key) per minute.
- Read provider rate-limit headers when present.
- Emit a limit event on 429 with the billing flag.
- Warn when a prefix and key nears 15 requests per minute on OpenAI [A:148].
- Feed limits to the tier selector.

NOT EXPECTED

- It never retries by itself.
- It never hides a billed 429.
- It never changes a model.

ALGORITHM AND MATHS

Sliding-window counter

```
near_limit = rpm_used / rpm_limit >= 0.8 (DevX proposal)
```

STEPS

1. Increment counters on each call.
2. Read remaining headers on each response.
3. On 429 emit event with retry-after and billed flag.
4. Expire old counts.

PARAMETERS

openai_prefix_cap	about 15 requests per minute [A:148, A:202]
flex_429	not charged [A:195, A:150]

SECURITY

- Counters hold counts only.
- Per-tenant isolation of counters.

PERFORMANCE

Latency and throughput	Update under 1 ms (DevX proposal).
Complexity	O(1) per call.
At 10x load	In-memory with periodic sync.

EDGE CASES AND HOW EACH IS HANDLED

- Batch jobs hit enqueued-token quota rather than rpm.
- 429 on a billed tier: cost recorded.
- Header missing: rely on counters.
- Azure flex has no auto-fallback [A:150].

TEST CASES

TEST	GIVEN	EXPECT
flex 429	Flex call returns 429	Event billed false, sent to fb
near limit	rpm 85 of 100	near_limit true
missing headers	No rate-limit headers	Counters used
prefix cap	16th request in a minute on one key	Warning prefix_cap

WHERE IT SAVES COST

- Avoid billed 429 and cache overflow [A:148].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- 429 rate rises for one key.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.rl.limit	provider · model · tier · billed · retry_after_s	On a 429

REFERENCES

- A:195 Flex processing
- A:150 Use Flex processing with Azure OpenAI in Foundry Models
- A:219 Batch processing (Claude)
- A:148 Prompt caching with Azure OpenAI in Azure AI Foundry Models
- A:202 Prompt Caching 201

RECEIVES FROM

None in this stage view.

HANDS TO

Fallback chain
429 or limit

Tier selector
limits, limits

← Back to group: Backend health ← Stage 2

LEVEL 4

Fallback chain

Backend | Stage 2: Backends

Fallback chain

Priced fallback on error or limit

flex 429 -> retry standard hop_cost +0.0031 USD
reason=limit cache_loss=priced

IN PLAIN ENGLISH

> When a provider says no, this part picks the next option. It always works out the cost of switching first and writes down why.

Goes in	Limit or error events from rl and provider failures, plus the fallback chain from policy.
Comes out	A priced fallback hop to casc or the next backend, with a logged reason.
Reads / writes	Reads chain config and price rows. Writes hop records.
If it fails	Fail closed: when no priced fallback exists, return the error. A silent switch could lose the cache and raise cost.
Requirement source	RQ-A-010, RQ-A-024, RQ-A-039, RQ-A-040 Sources: A:195, A:281, A:219, A:192, A:204, C:030, A:008, C:055
Hardening, still open	Cache loss at a model switch is unmeasured in the corpus (R1 largest unknown). OpenRouter fallbacks can break cache [A:231]. Azure flex has no auto-fallback [A:150].

WHAT IT IS FOR

Ordered fallback with pricing: flex to standard, batch to standard after a 429, primary to secondary model. Never silent [A:195, A:281, A:219].

EXPECTED

- Price each hop including cache loss on the incoming provider [C:030, A:204].
- Log reason: limit, error, timeout.
- Retry batch or flex work on standard after a 429 [A:195, A:281, A:219].
- Default to the same model after an error unless the saving beats cache-rewrite cost [A:008].
- Stop after the chain ends and return the error.

NOT EXPECTED

- It never switches model without a priced reason.
- It never retries a billed failure in a loop.
- It never changes the session's provider mid-session without bres approval.

ALGORITHM AND MATHS

Priced hop

```
hop iff budget_ok AND expected_saving > cache_rewrite_cost(incoming) [C:030]
```

STEPS

1. Receive the failure event.
2. Pick the next entry in the chain.
3. Ask swp for cache-loss price.
4. Ask bres for a hold.
5. Dispatch or return the error.
6. Log hop, reason and price.

PARAMETERS

max_hops	2 (DevX proposal)
same_model_retention_after_error	90.2% and 80.9% in A:008

SECURITY

- Fallback never crosses a data-residency boundary set by the tenant.
- Hop log has ids only.

PERFORMANCE

Latency and throughput	Decision under 10 ms (DevX proposal).
Complexity	O(chain length).
At 10x load	Stateless.

EDGE CASES AND HOW EACH IS HANDLED

- Both providers rate limited: return the first error.
- Fallback target is more expensive: needs explicit policy.
- Streaming already started: cannot hop; return partial and error.
- Batch error not billed [A:219].

TEST CASES

TEST	GIVEN	EXPECT
flex to standard	Flex 429	Standard dispatch, hop logged reason limit
no priced fallback	Chain empty	Original error returned
max hops	Three consecutive failures, max 2	Error after 2 hops
mid-stream	Stream failed after 10 chunks	No hop, partial returned

WHERE IT SAVES COST

- Avoid cache loss by staying on the same model after an error [A:008].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Hop rate above 5% for a route (DevX proposal).

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.fb.hop`	from · to · reason · cache_loss_usd	On each hop

RESEARCH BEHIND IT

A:008 MTRouter: Cost-Aware Multi-Turn LLM Routing with History–Model Joint Embeddings
Result we rely on: MTRouter retains the model after an error 90.2% and 80.9% of the time.
Used here: Default stay-put after an error.

HOW OTHERS BUILD IT

LiteLLM budget fallbacks and OpenRouter provider fallbacks are the common patterns [C:060, A:227].

REFERENCES

- A:195 Flex processing
- A:281 Batch inference (Gemini Enterprise)
- A:219 Batch processing (Claude)
- A:192 Auto Router
- A:204 Prompt cache diagnostics
- C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
- A:008 MTRouter: Cost-Aware Multi-Turn LLM Routing with History–Model Joint Embeddings
- C:055 How Claude Code uses prompt caching

RECEIVES FROM

Rate-limit tracker
429 or limit

HANDS TO

Cascade with fallback
fallback hop, fallback hop

← Back to group: Backend health ← Stage 2

LEVEL 2

3. Accounting

A versioned price book, a bucket normaliser and an append-only ledger turn raw usage into a cost with a stated source.

Price book

Append-only price rows with a lifecycle, plus a separate plan-units table.

3 Store

Price book · Price-row lifecycle · Plan-units table

Receives from: Reconciliation
Hands to: Metering, Reconciliation

Open group (3 components) →

Metering

Raw usage becomes five exclusive token buckets, then one cost row per provider response.

3 Calculator 1 Store

Usage normaliser · Dedupe and resume guard · Cost calculator · Append-only ledger

Receives from: Gateway edge (Ingress and adapters), Price book, Reconciliation, Budgets (Policy engine)
Hands to: Reconciliation, Stores and attribution (Insight), Budgets (Policy engine), Run control (Policy engine), Tier and cache (Policy engine), Evaluation (Eval and rollout)

Open group (4 components) →

Reconciliation

Estimated cost is checked against invoices weekly, and plan quota is tracked in native units.

3 Calculator

Invoice reconciler · Weekly price canary · Plan-quota tracker

Receives from: Price book, Metering
Hands to: Price book, Metering

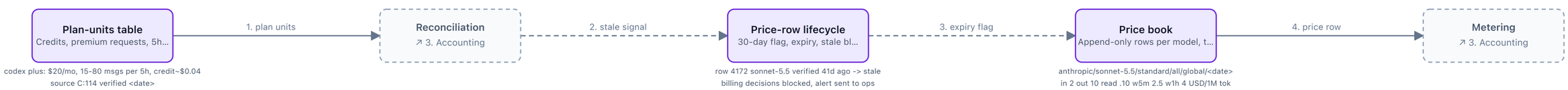
Open group (3 components) →

3 groups, 10 components. Hover a group for the plain-English story; click it to see its components and the arrows between them. ← Backends Policy engine →

LEVEL 3

Price book in Accounting

Append-only price rows with a lifecycle, plus a separate plan-units table.



Store — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Plan-units table → Reconciliation (group)

Credits and message limits per plan.

2. Reconciliation (group) → Price-row lifecycle

Drift above 1 percent marks the row stale.

3. Price-row lifecycle → Price book

Rows flagged at 30 days or expires_at; billing blocked on stale rows.

4. Price book → Metering (group)

Row chosen by provider, model, tier, band, region, effective_from.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Accounting Metering →

LEVEL 4

Price book

Store | Stage 3: Accounting

Price book

Append-only rows per model, tier, band

anthropic/sonnet-5.5/standard/all/global/<date>
in 2 out 10 read .10 w5m 2.5 w1h 4 USD/1M tok

IN PLAIN ENGLISH

> The price book is a list of what each model costs, with a start date on every line. Old lines stay forever, so any past request can be re-priced exactly.

Goes in	Price rows from vendor price pages, entered or imported by an operator with a source id and snapshot date.
Comes out	One price row per provider, model, service tier, context band, region and effective_from, with rates, multipliers and flags.
Reads / writes	Writes append-only price rows. Reads vendor snapshots. Never edits or deletes a row.
If it fails	Fail closed for billing decisions: a missing row returns PRICE_MISSING and the calculator marks the cost unpriced. A wrong price silently misstates spend.
Requirement source	RQ-B-012, RQ-B-013, RQ-B-014, RQ-B-015, RQ-B-016, RQ-B-017, RQ-B-004, RQ-B-018, RQ-B-010, RQ-B-025, RQ-B-027, RQ-B-026 Sources: A:199, A:201, A:225, A:270, A:214, A:069, A:176, A:258, A:159, A:281, A:252, A:255, A:177, A:232, A:233, A:148, A:121, A:280, A:182, C:010
Hardening, still open	Open: Gemini implicit cache 75 percent [A:121] against 90 percent [A:280], stored per model and date. Anthropic minimum prefix conflicts [A:199 against C:051], larger value kept. Together H100 price $3 per hour [A:277] against $5.49 [A:278] does not touch rows. Azure PTU to tokens per minute is not in the corpus. Model names such as Fable 5.1 are quoted as written and cannot be verified.
Not in client docs	**DevX proposal:** The insert-time check of cache rates against vendor multipliers and the tolerance of 0.0001 USD per million tokens are DevX proposals.

WHAT IT IS FOR

Hold every price the calculator needs as immutable, versioned rows, so each request is costed against the row that was valid when it ran. Prices are snapshots from vendor pages, never constants in code.

EXPECTED

- Row key is (provider, model_id, service_tier, context_band, region, effective_from). Model id is the exact string the provider returns.
- Rates per million tokens: input, output, cache_read, cache_write_5m, cache_write_1h, reasoning (default equals output).
- Multipliers carry applies_to (input, output, cached, all) and an order flag that says whether they apply before or after the cache discount. Kinds: batch, flex, priority, fast, region, residency.
- Flags per row: batch_stacks_with_cache (true, false, unknown), band_scope (marginal or all_tokens), input_includes_cached, reasoning_in_output, billed_units_source.
- Cache fields: min_prefix, TTLs, max_breakpoints, storage_rate. Limit fields: rpm, tpm, rpd, batch_max_requests, batch_window, 429_billed, auto_fallback.
- Lifecycle fields: effective_from, expires_at, source_id, snapshot_date, last_verified_at, confidence.

NOT EXPECTED

- No in-place edits. A price change adds a row with a new effective_from.
- No partial-name model matching. An unknown id returns PRICE_MISSING.
- No plan or subscription rows here. Those live in the plan-units table (pplan).
- No cost computation. The calculator (calc) reads rows and does the arithmetic.

ALGORITHM AND MATHS

Append-only versioned lookup with effective-date selection

```
row = argmax(effective_from) over rows where key matches and effective_from <= request_time and (expires_at is null or request_time < expires_at)
```

STEPS

1. On insert, validate required fields and that source_id and snapshot_date are present.
2. Check cache rates against vendor multipliers (read, 5m write 1.25x, 1h write 2x). Flag mismatches for review.
3. Assign a monotonic price_row_id and write the row. Never update.
4. On lookup, filter by provider, exact model_id, service_tier, region, then choose the context band that contains total prompt tokens.
5. Among candidates pick the greatest effective_from not after request time. Return the row and price_row_id.
6. If the row is stale (plife flag) return it with price_status=stale so budget logic can refuse to act on it.

PARAMETERS

stale_after_days	30 [A:214, R5 section 2]
expiry_warning_days	30 [R5 section 2]
haiku_5.5_band_split	100K tokens [A:199]
long_context_split_anthropic_gemini_xai	200K tokens [A:270, A:225]
price_unit	USD per 1M tokens

THE MATHS IN FULL

SYMBOLS

- base_input = list input rate of the row in USD per million tokens.
- m_read, m_w5, m_w1h = vendor cache multipliers on base_input.
- stored_rate = rate saved in the row.
- derived_rate = base_input x multiplier.
- band = context band chosen from total prompt tokens.

FORMULA

```
derived_read = base_input x m_read; derived_w5 = base_input x 1.25; derived_w1h = base_input x 2; flag if |stored − derived| > 0.0001
```

WHY IT IS RIGHT

Anthropic documents read 0.1x, 5m write 1.25x and 1h write 2x. Read is 0.05x on Opus 5.5 and Sonnet 5.5, and 0.025x on Fable 5.1 and Mythos 5.1 [A:199, A:283, C:010]. A stored rate that breaks its own multiplier is a typing error or a vendor change, so the insert check catches both.

WORKED EXAMPLE

```
Sonnet 5.5: input $2, read 0.05x gives $0.10, 5m write 1.25x gives $2.50, 1h write 2x gives $4 [A:283, A:199, C:010]. All three match the stored row. Sonnet 4.5: input $3, read 0.1x gives
$0.30, 5m $3.75, 1h $6. Haiku 5.5 needs two rows: $0.10 in and $0.50 out up to 100K, $0.50 in and $2.50 out above [C:010].
```

COST OF COMPUTING IT

Insert O(1). Lookup O(log n) on an index of the row key, then O(b) over b bands, where b is small.

HOW IT CAN GO WRONG, AND THE GUARD

- Vendor changes a multiplier and the check flags every row of that family.
- Two rows share one effective_from: the insert is rejected.
- Band missing for a prompt size: PRICE_MISSING, never a neighbouring band.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

The weekly canary (canary) compares computed cost with invoices. A drift above 1 percent sends a stale signal back to plife.

SECURITY

- Only an operator role from the authorisation component can insert rows. Each insert is written to the audit log.
- Rows are immutable at the storage layer, so a compromised service cannot rewrite history.
- Price pages are fetched by a separate job. The job stores source_id and a hash of the page, so tampering is visible.

PERFORMANCE

Latency and throughput	Lookup under 5 ms at the 95th percentile (P95) from an in-memory cache (DevX proposal, no corpus source).
Complexity	O(log n) per lookup.
At 10x load	Rows are small and read-heavy. Cache the active rows per gateway instance and refresh on a version counter.

EDGE CASES AND HOW EACH IS HANDLED

- Anthropic Haiku 5.5 has two bands by prompt size, so one model has two active rows.
- xAI above 200K reprices every token including cached ones, so band_scope is all_tokens for that row [A:225].
- Models with no write premium (pre-5.6 OpenAI, DeepSeek) store cache_write at 1x of input.
- A promo that expires leaves a gap. The row for the list price must already exist, or lookup returns PRICE_MISSING.
- Aggregators such as OpenRouter add a premium on top of the model row, so they need their own provider rows.

TEST CASES

TEST	GIVEN	EXPECT
derive_match	Sonnet 5.5 row with input 2.00 and read 0.10	insert accepted, no mismatch flag
derive_mismatch	Sonnet 5.5 row with input 2.00 and read 0.20	insert flagged PRICE_MULTIPLIER_MISMATCH
effective_date	rows A from Jan 1 and B from Mar 1, request on Feb 10	row A returned
exact_model	request model id 'claude-sonnet-5' with only 'claude-sonnet-5-5' in the book	PRICE_MISSING, no partial match
band_select	Haiku 5.5, 120,000 prompt tokens	row with context_band >100000, input $0.50
immutable	attempt to update an inserted row	write rejected, audit event written

WHERE IT SAVES COST

- Correct rows let the cache planner see real write premiums and skip caching that cannot pay back.
- Exact tier and band rows prevent over-billing estimates that lead to needless budget stops.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- pb.row_inserted
- pb.row_flagged_mismatch
- pb.lookup_missing

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.pb.row_inserted	price_row_id · provider · model_id · service_tier · context_band · effective_from · source_id	A new row is written.
token_economics.pb.lookup_missing	provider · model_id · service_tier · tokens_total	Lookup finds no row.
token_economics.pb.multiplier_mismatch	price_row_id · field · stored · derived	Insert check fails.

RESEARCH BEHIND IT

A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
Result we rely on: Sticker price reversed the ranking in 106 of 336 comparisons (32 percent), up to 28x.
Used here: Justifies per-row tier and band data and cost per task over list price.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Langfuse: Model-based pricing with tier conditions such as input>200000. No retroactive recalculation of history.
- LiteLLM: Provider normalisation and price maps to study. Token Economics keeps its own versioned book.

HOW OTHERS BUILD IT

Langfuse and LiteLLM keep model price tables with usage tiers. Langfuse stores ingested cost ahead of inferred cost and does not recalculate history. Token Economics adds expiry, flags for stacking and band scope, and an append-only rule.

REFERENCES

- A:199 Claude prompt caching docs
- A:201 Prompt caching
- A:225 xAI pricing
- A:270 Claude Opus 4.6
- A:214 Models and pricing for GitHub Copilot
- A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
- A:176 Priority Tier
- A:258 GLM 5.2 Fast: Faster Serverless Inference, Same Quality
- A:159 Capacity and Performance
- A:281 Batch inference (Gemini Enterprise)
- A:252 Groq Batch API
- A:255 Batch inference (Fireworks)
- A:177 How Does Cohere's Pricing Work?
- A:232 Token & Cost Tracking (Langfuse)
- A:233 LLM Cost Management: Track and Control Spend
- A:148 Prompt caching with Azure OpenAI in Azure AI Foundry Models
- A:121 Gemini 2.5 Models now support implicit caching
- A:280 Context caching overview (Vertex AI)
- A:182 Context Caching
- C:010 Claude Code Billing & Usage Limits Explained (Oct 2026)

RECEIVES FROM

Price-row lifecycle
expiry flag

HANDS TO

Cost calculator
price row

← Back to group: Price book ← Stage 3

LEVEL 4

Price-row lifecycle

Store | Stage 3: Accounting

Price-row lifecycle

30-day flag, expiry, stale block

row 4172 sonnet-5.5 verified 41d ago -> stale
billing decisions blocked, alert sent to ops

IN PLAIN ENGLISH

> Prices go out of date. This part checks every day, and when a price is too old it stops the system from acting on it and tells a person.

Goes in	Price rows and plan rows with expires_at, last_verified_at and the canary stale signal.
Comes out	A flag on each row (ok, expiring, stale, expired) and an alert to operators.
Reads / writes	Reads pb and pplan rows. Writes lifecycle flags in a separate flag table, never inside the immutable rows.
If it fails	Fail closed for billing decisions: a stale row blocks budget actions and the system alerts. Observation continues on the last good price.
Requirement source	RQ-B-026, RQ-B-027, RQ-B-029, RQ-B-036 Sources: A:214, A:121, A:280, A:148, A:069, C:011, C:115
Hardening, still open	Open: the right staleness threshold is unknown. R5 experiment 9 (monthly price-list diff) should calibrate it. Plan promos change on fixed dates that need manual entry.
Not in client docs	**DevX proposal:** Flag names, the alert channel and the rule that stale rows still allow observation are DevX proposals. The 30 day thresholds are from R5.

WHAT IT IS FOR

Stop old or expired prices from driving budget decisions. Vendors change prices, cache discounts and promos on their own schedule, so a row can be wrong without any code change.

EXPECTED

- A daily job flags rows within 30 days of expires_at as expiring.
- The same job flags rows with last_verified_at older than 30 days as stale.
- Rows past expires_at are expired. Expired promos fall back to the list price row and raise an alert.
- A stale signal from the canary (drift above 1 percent) marks the row stale at once.
- Budget logic reads the flag and refuses to act on a stale or expired row. Observation continues and labels the cost.

NOT EXPECTED

- No rewriting of price rows. Flags live outside the immutable rows.
- No automatic price scraping decisions. A person verifies and inserts the new row.
- No blocking of requests for stale rows on observe-only routes.

ALGORITHM AND MATHS

Daily sweep with threshold flags

```
flag = expired if now >= expires_at; expiring if expires_at - now <= 30d; stale if now - last_verified_at > 30d or canary_stale
```

STEPS

1. Run daily at a fixed hour and scan active rows.
2. Compute days_to_expiry and days_since_verified for each row.
3. Write the flag to the flag table with a reason code.
4. Emit one alert per row per state change, not per day.
5. On expiry of a promo row, confirm a list-price row exists. If not, alert as severity high.

PARAMETERS

stale_days	30 [R5 section 2]
expiry_warning_days	30 [R5 section 2]
canary_drift_threshold	1 percent [R5 experiment 1, DevX proposal as a trigger]

SECURITY

- Flag writes need a service role. Operators cannot clear a stale flag without a new verified row.
- Alerts carry row ids and no content.

PERFORMANCE

Latency and throughput	Sweep finishes within 5 minutes and the alert fires within 15 minutes of a state change (DevX proposal).
Complexity	O(n) rows per sweep.
At 10x load	Only active rows are scanned. Cost is negligible next to request traffic.

EDGE CASES AND HOW EACH IS HANDLED

- Copilot promos end Dec 1 and Dec 31 2026. The sweep must warn 30 days before each date [A:214].
- Azure charges for 12h cache writes on GPT-5.6+ from Nov 1 2026, so that row needs a dated successor [A:148].
- A row with no expires_at is only governed by the 30 day verification rule.
- Gemini cache discount differs by source, 75 against 90 percent. Low confidence rows should reach stale sooner [A:121, A:280].

TEST CASES

TEST	GIVEN	EXPECT
expiring	row expires_at in 20 days	flag expiring, one alert
stale_by_age	last_verified_at 31 days ago	flag stale, billing blocked
promo_fallback	promo row expired, list row present	calc uses list row, alert sent
promo_no_fallback	promo row expired, no list row	severity high alert, PRICE_MISSING on lookup
canary_signal	canary drift 1.4 percent on a row	flag stale at once

WHERE IT SAVES COST

- Prevents decisions on wrong prices, such as a tier choice based on a promo that ended.
- Early alerts avoid weeks of silent cost error.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- plife.row_expiring
- plife.row_stale
- plife.promo_expired

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.plife.flag_changed	price_row_id · old_flag · new_flag · reason	A flag changes state.
token_economics.plife.sweep_done	rows_scanned · rows_flagged · duration_ms	Daily sweep ends.

RESEARCH BEHIND IT

A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
Result we rely on: Price rankings reversed in 32 percent of 336 comparisons.
Used here: Shows why prices must be re-verified and not treated as constants.

HOW OTHERS BUILD IT

Langfuse does not recalculate historical cost after a price change. Token Economics follows that rule and adds expiry and stale flags so that current decisions use fresh rows.

REFERENCES

- A:214 Models and pricing for GitHub Copilot
- A:121 Gemini 2.5 Models now support implicit caching
- A:280 Context caching overview (Vertex AI)
- A:148 Prompt caching with Azure OpenAI in Azure AI Foundry Models
- A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
- C:011 Claude Code Limits: 5-Hour Caps Doubled in May 2026
- C:115 Codex CLI usage rate limits

RECEIVES FROM

Weekly price canary
stale signal

HANDS TO

Price book
expiry flag

← Back to group: Price book ← Stage 3

LEVEL 4

Plan-units table

Store | Stage 3: Accounting

Plan-units table

Credits, premium requests, 5h messages

codex plus: $20/mo, 15-80 msgs per 5h, credit~$0.04
source C:114 verified <date>

IN PLAIN ENGLISH

> Subscriptions do not count tokens. They count messages, credits or requests, so this table records each plan in its own unit.

Goes in	Plan facts from vendor pages: fee, unit, included quantity, credit value, reset rule, overage, model weighting.
Comes out	One plan row per plan with its native unit and limits.
Reads / writes	Writes append-only plan rows. Reads vendor pages. Read by the quota tracker (planq).
If it fails	Fail open for observation: a missing plan row means the tracker shows native units only, with no shadow cost. Plan figures never gate requests.
Requirement source	RQ-B-028, RQ-B-030, RQ-B-031, RQ-B-029 Sources: C:009, C:010, C:114, C:115, C:119, C:120, C:015, C:116, C:117, A:214, C:057, A:161, A:146, C:011
Hardening, still open	Open: Anthropic publishes no limit-to-token conversion [C:010]. Plan figures are approximate and short-lived: caps doubled May 2026 and a 50 percent weekly boost expired 2026-07-13 [C:011]. Copilot legacy premium-request billing against new credit billing [C:116, A:214]. Kiro credits per token not stated [A:161].
Not in client docs	**DevX proposal:** None for the row fields. Plan row updates by hand each month are a DevX proposal.

WHAT IT IS FOR

Describe subscription and credit plans in their native units, so spend on a plan is tracked without converting to tokens, which no vendor allows.

EXPECTED

- Fields: plan, fee, unit, included_qty (nullable), credit_usd, reset_rule, overage_rate, model_weighting (nullable), expires_at, source_id.
- Unit values: api-rate tokens, credits, premium requests, requests per day, messages per 5 hours, pool multiple.
- Rows for Claude: Pro $20, Max $100 and $200, Team $20 to $125 per seat. Codex: Plus $20, Pro $100 and $200. Cursor: Plus $8 base with 4x, 15x and 50x pools. Copilot: credits at $0.01. Gemini CLI: 1,000 to 2,000 requests per day.
- Each row has a source id and a snapshot date. Rows are append-only like price rows.

NOT EXPECTED

- No token-per-plan conversion for Claude. It is not published.
- No dollar figure for subscription traffic. Only the shadow API cost is computed elsewhere.
- No enforcement. Plan rows inform dashboards and the quota tracker.

ALGORITHM AND MATHS

Typed plan record with native unit

```
remaining_units = included_qty − used_units (null included_qty means unmetered or unknown)
```

STEPS

1. Each plan row names one unit. The tracker stores usage in that unit only.
2. credit_usd converts credits to dollars where the vendor states it (Copilot 1 credit = $0.01 [A:214]; Codex 1 credit about $0.04 [C:114]).
3. model_weighting holds per-model weights, such as Copilot reasoning 5x or 20x and review 13x [C:117].
4. reset_rule holds the window (5 hours, weekly, monthly).
5. Rows expire with promos and are re-verified by plife.

PARAMETERS

copilot_credit_usd	0.01 [A:214]
codex_credit_usd	about 0.04 [C:114]
gemini_gsu_output_weight	5, cached 0.1, 675 tokens per GSU [A:146]
cursor_pool_multiples	4x, 15x, 50x on the $8 base [C:119]

SECURITY

- Plan rows hold no user data. They are public vendor facts.
- Edits need the operator role and are audited.

PERFORMANCE

Latency and throughput	Reads from cache, same as the price book (DevX proposal).
Complexity	O(1) lookup by plan id.
At 10x load	A few dozen rows. No scaling concern.

EDGE CASES AND HOW EACH IS HANDLED

- Plan has included_qty null (Anthropic does not publish limits). The tracker shows usage only.
- Copilot moved from premium requests to credits, so two versions of the plan coexist in history [C:116, A:214].
- Cursor Pro moved from 500 requests to a usage pool [C:015].
- Gemini CLI quota differs by auth method, from 250 per day on a free API key to 2,000 [C:057].

TEST CASES

TEST	GIVEN	EXPECT
native_unit	Claude Max row	unit is messages per 5 hours, no token conversion field
credit_value	Copilot row with 250 credits used	shadow value $2.50 at $0.01 per credit
expired_promo	Copilot promo row past Dec 31 2026	plife flags expired, falls back to list row
null_included	Anthropic plan row	included_qty null, remaining_units null

WHERE IT SAVES COST

- Lets teams see whether a subscription beats API billing for each developer using shadow cost.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- pplan.row_inserted

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.pplan.row_inserted	plan, unit, fee, source_id	A new plan row is written.

REFERENCES

- C:009 Claude Code Rate Limits & Usage Quotas Explained (2026)
- C:010 Claude Code Billing & Usage Limits Explained (Oct 2026)
- C:114 Codex pricing: current costs, usage limits, and realistic budgets (2026)
- C:115 Codex CLI usage rate limits
- C:119 Cursor Models & Pricing
- C:120 Cursor account pricing
- C:015 Clarifying our pricing
- C:116 GitHub Copilot premium requests (billing concepts)
- C:117 Understanding and managing requests in Copilot
- A:214 Models and pricing for GitHub Copilot
- C:057 Gemini CLI: Quotas and pricing
- A:161 Billing
- A:146 Calculate Provisioned Throughput requirements
- C:011 Claude Code Limits: 5-Hour Caps Doubled in May 2026

RECEIVES FROM

None in this stage view.

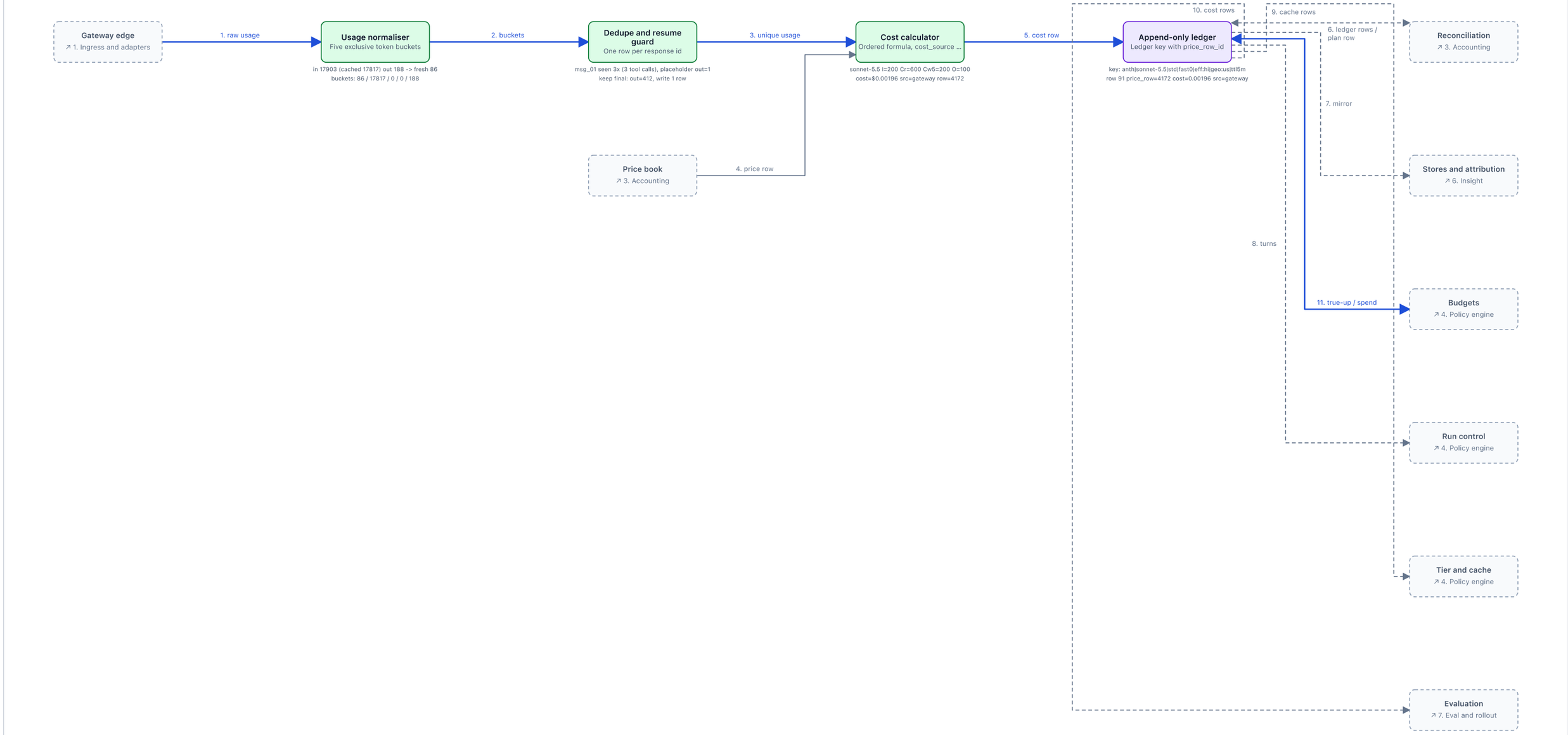
HANDS TO

Plan-quota tracker
plan units

← Back to group: Price book ← Stage 3

LEVEL 3 **Metering** in Accounting

Raw usage becomes five exclusive token buckets, then one cost row per provider response.



Calculator · Store · main flow · read / call · scheduled / async · security · outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Gateway edge (group) → Usage normaliser
Provider usage object, response id, model id, headers.

2. Usage normaliser → Dedupe and resume guard
Fresh input, cache read, cache write 5m and 1h, output.

3. Dedupe and resume guard → Cost calculator
One row per provider response id.

4. Price book (group) → Cost calculator
Row chosen by provider, model, tier, band, region, effective_from.

5. Cost calculator → Append-only ledger

Cost, price_row_id, cost_source, auth_mode.

6. Append-only ledger → Reconciliation (group)

ledger rows: Estimated cost per key and day. | back: plan row: Native units plus shadow API cost.

7. Append-only ledger → Stores and attribution (group)

Metadata-only copy for dashboards.

8. Append-only ledger → Run control (group)

Turns per task feed the per-model baseline.

9. Append-only ledger → Tier and cache (group)

read over fresh plus read plus write.

10. Append-only ledger → Evaluation (group)

Cost including failed attempts.

11. Budgets (group) → Append-only ledger

true-up: Actual cost replaces the reservation. | back: spend: Spend increments the budget window.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Accounting ← Price book Reconciliation →

LEVEL 4

Usage normaliser

Calculator | Stage 3: Accounting

Usage normaliser

Five exclusive token buckets

in 17903 (cached 17817) out 188 -> fresh 86
buckets: 86 / 17817 / 0 / 0 / 188

IN PLAIN ENGLISH

> Providers count tokens in different ways. This part rewrites every count into the same five boxes so nothing is counted twice.

Goes in	Raw provider usage object, response id, model id and headers from the stream observer (tee).
Comes out	One normalised record with five mutually exclusive token buckets and the provider's billed units when reported.
Reads / writes	Reads the price-row flags input_includes_cached and reasoning_in_output. Writes nothing durable; passes the record on.
If it fails	Fail closed on the record: unknown keys or impossible counts go to quarantine with a code. The user response is never touched, so the request path fails open.
Requirement source	RQ-B-001, RQ-B-002, RQ-B-003, RQ-B-004, RQ-B-019, RQ-B-015 Sources: A:199, A:233, A:190, A:197, A:251, A:157, A:159, A:182, A:177, A:232, A:216, A:204, A:221, A:231
Hardening, still open	Open: Gemini candidate-token split against total is not in the corpus. Mistral and Groq cached-token field names are not in the corpus. Bedrock Nova caching is implicit and not reported as reads [A:157]. DeepSeek native field names were seen in corpus text only [A:182].
Not in client docs	**DevX proposal:** Quarantine codes and the 0 token tolerance for bucket sums are DevX proposals.

WHAT IT IS FOR

Turn each provider's usage object into five exclusive buckets: fresh_input, cache_read, cache_write_5m, cache_write_1h, output. Reasoning tokens stay inside output.

EXPECTED

- Anthropic: fresh = input_tokens (already excludes cache); cache_read = cache_read_input_tokens; writes from cache_creation.ephemeral_5m and ephemeral_1h; total_input = read + creation + input_tokens [A:199].
- OpenAI Chat and Responses, xAI, OpenRouter: prompt_tokens or input_tokens include cached, so fresh = prompt - cached_tokens [A:233, A:202, A:221].
- Bedrock Converse: total input = inputTokens + cacheRead + cacheWrite, so buckets map directly [A:157, A:159].
- Gemini: cachedContentTokenCount (REST) or cached_content_token_count (SDK) is subtracted from prompt tokens [A:143, A:121].
- Reasoning tokens are never added to output because output_tokens already includes them (1,186 output holds 1,024 reasoning) [A:197, A:251].
- Where the vendor reports billed units, they are carried next to the token buckets (Cohere 6,772 billed against 7,596 tokens) [A:177].

NOT EXPECTED

- No cost computation. The calculator does that.
- No deduplication. The dedupe guard follows.
- No silent conversion of unknown fields. An unknown key quarantines the record.
- No content storage. Only counts and ids pass through.

ALGORITHM AND MATHS

Per-provider bucket mapping with conservation check

```
fresh = prompt_total − cached_read − cache_write (when input_includes_cached); output includes reasoning
```

STEPS

1. Identify the provider and wire format from the route and response shape.
2. Check the usage object keys against the allow-list for that wire format. Unknown key: quarantine with USAGE_UNKNOWN_KEY.
3. Map fields to buckets using the price-row flag input_includes_cached.
4. Check that all buckets are zero or more. Cached above prompt total gives USAGE_BUCKET_OVERFLOW.
5. Check conservation: fresh + read + write_5m + write_1h equals the provider's total input. Mismatch gives USAGE_SUM_MISMATCH.
6. Attach inference_geo, service_tier, speed and billed units if present, and pass on.

PARAMETERS

unknown_key_policy	reject, never convert [A:233]
reasoning_handling	inside output, never added [A:197]
sum_tolerance_tokens	0 (DevX proposal)

THE MATHS IN FULL

SYMBOLS

- P = prompt_tokens as the provider reports them.
- K = cached read tokens.
- W5, W1h = cache write tokens by TTL class.
- F = fresh_input after normalisation.
- O = output tokens including reasoning.

FORMULA

```
inclusive providers: F = P − K − W; exclusive providers (Anthropic): F = input_tokens, total_input = F + K + W5 + W1h; always O = output_tokens
```

WHY IT IS RIGHT

Anthropic reports the buckets separately [A:199]. OpenAI, xAI and OpenRouter include cached tokens inside the prompt count [A:233, A:221, A:231]. Summing the inclusive count with the cached count double-counts K. Subtraction gives the exclusive form.

WORKED EXAMPLE

```
OpenAI: prompt_tokens 17,903 with cached 17,817 and 188 output gives F = 86, K = 17,817, O = 188, total 18,091 [A:233]. Cloudflare: 1,000 input with 600 read and 200 write gives F = 200
[A:190]. Reasoning: output 1,186 already holds 1,024 reasoning, total 1,261 with input 75 [A:197, A:251].
```

COST OF COMPUTING IT

O(1) per response. Allow-list lookup is constant time.

HOW IT CAN GO WRONG, AND THE GUARD

- A gateway in the path adds extra usage keys and defeats conversion [A:233].
- Cached above prompt total after a provider bug.
- A provider changes a field name and every record quarantines.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

The weekly canary compares the normalised totals with vendor usage reports. A systematic gap points to a mapping fault.

SECURITY

- Records hold counts and ids only. No prompt text reaches this component.
- Quarantined raw usage objects are kept without content and expire with metadata retention.
- The allow-list is code reviewed. A new provider field never passes by default.

PERFORMANCE

Latency and throughput	Under 1 ms per record at the 95th percentile (P95) and off the response path (DevX proposal).
Complexity	O(1) per response.
At 10x load	Stateless. Scales with gateway instances.

EDGE CASES AND HOW EACH IS HANDLED

- Streaming responses report usage only in the last event. The normaliser waits for it.
- A response with cached tokens and no cache write field gives W = 0, not an error.
- Bedrock Nova caching is implicit and not reported as reads [A:157]. Reads show 0 and cost is overstated.
- DeepSeek reports hit and miss tokens (prompt_cache_hit_tokens, prompt_cache_miss_tokens), so cache_read = hit and F = miss [A:182].
- Provider returns a prompt cache miss reason (OpenAI diagnostics). It is stored as metadata only [A:204].

TEST CASES

TEST	GIVEN	EXPECT
openai_inclusive	prompt 17903, cached 17817, output 188	F=86, K=17817, O=188
anthropic_exclusive	input 200, read 600, creation 200 (5m), output 100	F=200, K=600, W5=200, O=100, total_input 1000
reasoning_once	output 1186 with reasoning 1024	O=1186, not 2210
unknown_key	usage object with key 'gateway_extra'	quarantine USAGE_UNKNOWN_KEY, no row
overflow	prompt 100, cached 150	quarantine USAGE_BUCKET_OVERFLOW
deepseek	hit 900, miss 100, output 50	K=900, F=100, O=50

WHERE IT SAVES COST

- Correct buckets are the base for every saving claim, since a double-counted cache read hides real spend.
- Exposes caller cache hit rates for the cache planner.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- norm.record_ok
- norm.record_quarantined

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.norm.record_ok	provider · model_id · fresh · cache_read · cache_write_5m · cache_write_1h · output	A record is normalised.
token_economics.norm.quarantined	provider · code · key	A record fails a check.

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Langfuse: Usage buckets that are mutually exclusive, with ingested cost winning over inferred cost.
- LiteLLM: Provider normalisation reference.

HOW OTHERS BUILD IT

Langfuse requires mutually exclusive usage buckets, and OpenAI's inclusive counts are converted by subtraction. LiteLLM maps over 100 providers to one shape. Token Economics adds a conservation check and a reject rule for unknown keys.

REFERENCES

- A:199 Claude prompt caching docs
- A:233 LLM Cost Management: Track and Control Spend
- A:190 Custom costs
- A:197 Reasoning models
- A:251 Reasoning models (OpenAI platform)
- A:157 Prompt caching for faster model inference
- A:159 Capacity and Performance
- A:182 Context Caching
- A:177 How Does Cohere's Pricing Work?
- A:232 Token & Cost Tracking (Langfuse)
- A:216 Track cost and usage (Claude Agent SDK)
- A:204 Prompt cache diagnostics
- A:221 Usage & Pricing (xAI prompt caching)
- A:231 Prompt caching best practices (OpenRouter)

RECEIVES FROM

Stream tee and observer
raw usage, raw usage

HANDS TO

Dedupe and resume guard
buckets

← Back to group: Metering ← Stage 3

LEVEL 4

Dedupe and resume guard

Calculator | Stage 3: Accounting

Dedupe and resume guard

One row per response id

msg_01 seen 3x (3 tool calls), placeholder out=1
keep final: out=412, write 1 row

IN PLAIN ENGLISH

> Sometimes the same answer is reported twice, or a resumed chat repeats old totals. This part keeps exactly one row per answer.

Goes in	Normalised usage records from norm, plus session and resume signals from the joiner.
Comes out	At most one record per provider response id, with the final output token count.
Reads / writes	Reads and writes a seen-id set (response id to ledger key) with a time-limited expiry. Reads the last cumulative total per session.
If it fails	Fail open on the response path. On store outage the guard passes records and marks them dedupe_unchecked, and the ledger unique key catches repeats later.
Requirement source	RQ-B-005, RQ-B-006, RQ-B-007 Sources: A:216, A:217, C:002
Hardening, still open	Open: how Codex and non-Anthropic harnesses report message ids and resumes is not in the corpus. Exact rules for SDK placeholder tokens beyond 'use the final result' are not in the corpus [A:216].
Not in client docs	**DevX proposal:** The seen-id expiry of 24 hours and the delta method for resumed totals are DevX proposals. The corpus only lists the pitfalls.

WHAT IT IS FOR

Make sure each provider response is costed once. It removes repeats from parallel tool calls, dual reporting paths, placeholder token counts and resumed session totals.

EXPECTED

- Key is the provider response id (for Anthropic, the message id). One row per key.
- Parallel tool calls that share a message id collapse to one row [A:216].
- Placeholder per-step output_tokens are ignored. The final result carries the real output count [A:216].
- A call seen through both SDK telemetry and the gateway is counted once. The gateway row wins, and the telemetry row is stored as a cross-check [A:216, A:217].
- On session resume, only the new spend is booked. Earlier spend is already in the resumed total [A:216].

NOT EXPECTED

- No cost computation.
- No dropping of distinct calls with equal token counts. Only the id decides.
- No guessing a missing id. A record with no id is stored with a synthetic key and flagged.

ALGORITHM AND MATHS

Idempotent upsert by response id with final-result precedence

```
row(key) = final record if final seen else latest partial; resumed_delta = new_total − last_total(session)
```

STEPS

1. Compute key = provider + response id.
2. If key is unseen, hold the record as partial until a final usage event arrives or a timeout passes.
3. If key is seen and the new record is final, replace the partial. If both are final and equal, drop the repeat. If both are final and differ, keep the first and raise DEDUPE_CONFLICT.
4. Collapse sibling tool-call records with the same message id into one record.
5. For resumed sessions, subtract the last stored cumulative total for the session before booking.
6. Pass the single record to the calculator.

PARAMETERS

seen_id_expiry	24 hours (DevX proposal)
partial_wait	30 seconds (DevX proposal)
key	provider + response id [A:216]

THE MATHS IN FULL

SYMBOLS

- n = records received for one response id.
- f = number of final records among them.
- T_new = cumulative session total on resume.
- T_last = last stored cumulative total.

FORMULA

```
rows = 1 for every distinct key; booked_spend_on_resume = T_new − T_last
```

WHY IT IS RIGHT

A resumed session reports a total that already includes earlier spend [A:216]. Adding the new total to the stored sum counts the earlier spend twice. Subtracting the last stored total books only the new part.

WORKED EXAMPLE

```
Illustrative arithmetic: three parallel tool calls share message id msg_01 and arrive as 3 records, so n = 3 and the guard writes 1 row. A session stored at $4.00 resumes and reports $4.50.
The booked delta is $0.50, not $4.50.
```

COST OF COMPUTING IT

O(1) expected per record with a hash set. Memory grows with ids seen in the expiry window.

HOW IT CAN GO WRONG, AND THE GUARD

- Store outage lets repeats through. The unique ledger key stops them later.
- Expiry too short lets a late replay create a second row.
- A resume with a reset total gives a negative delta. It is booked as zero and flagged.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Compare ledger row count with provider request counts in invoices and OTel api_request events each week.

SECURITY

- The seen-id set holds ids and hashes only, no content.
- A replayed transcript cannot overwrite a gateway row, since gateway rows win.

PERFORMANCE

Latency and throughput	Decision under 2 ms at the 95th percentile (P95) (DevX proposal).
Complexity	O(1) expected per record.
At 10x load	Partition the seen-id set by tenant. Expiry bounds memory.

EDGE CASES AND HOW EACH IS HANDLED

- Anthropic parallel tool calls share one message id across several assistant records.
- SDK sends placeholder output_tokens per step, then the true count in the final result [A:216].
- Gateway and SDK telemetry both report the same call [A:216, A:217].
- Resume after a crash can replay the last response, so the same id arrives twice.
- Subagent calls have their own response ids and must not be merged with the parent.

TEST CASES

TEST	GIVEN	EXPECT
parallel_calls	3 records share msg_01, last has output 412	1 row, output 412
placeholder	partial output 1 then final output 412	row output 412
dual_path	same id from gateway and OTel	1 ledger row, source gateway, OTel kept as cross-check
resume_delta	stored total $4.00, resumed total $4.50	book $0.50
conflict	two finals for one id with 400 and 412 output	keep first, flag DEDUPE_CONFLICT

WHERE IT SAVES COST

- Removes phantom spend that would make budgets stop work early.
- Keeps cache hit rates honest, since a repeat read inflates the ratio.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- dedupe.dropped
- dedupe.conflict

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.dedupe.dropped	response_id · reason · path	A repeat record is dropped.
token_economics.dedupe.conflict	response_id · tokens_a · tokens_b	Two finals disagree.

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- ccusage: Reads local JSONL cost data for many agent CLIs. Study how it handles repeated message records.

HOW OTHERS BUILD IT

Agent SDK cost data is a client-side estimate with known traps: double counting, placeholder tokens, subagent exclusion and resumed totals. Token Economics treats each trap as a rule on the ledger path.

REFERENCES

- A:216 Track cost and usage (Claude Agent SDK)
- A:217 Monitoring usage (Claude Code OpenTelemetry)
- C:002 Claude Code OpenTelemetry: Track Cost and Tool Use

RECEIVES FROM

Usage normaliser
buckets

HANDS TO

Cost calculator
unique usage

← Back to group: Metering ← Stage 3

LEVEL 4

Cost calculator

Calculator | Stage 3: Accounting

Cost calculator

Ordered formula, cost_source label

sonnet-5.5 I=200 Cr=600 Cw5=200 O=100
cost=$0.00196 src=gateway row=4172

IN PLAIN ENGLISH

> This part multiplies the counted tokens by the right price and adds up the bill for one request. It always notes which price it used and whether the number is a guess or a real bill.

Goes in	Deduplicated bucket record, the chosen price row (pb), request tier and region, tool fees.
Comes out	Cost in USD with price_row_id, cost_source and auth_mode, ready for the ledger.
Reads / writes	Reads price rows. Writes nothing itself; emits a cost row to the ledger.
If it fails	Fail open on the request path (response already sent). Pricing failure writes an unpriced row and keeps the reservation as the spend bound, because a zero price would hide real spend.
Requirement source	RQ-B-012, RQ-B-002, RQ-B-009, RQ-B-018, RQ-B-016, RQ-B-019, RQ-B-020, RQ-B-021, RQ-B-022, RQ-B-023, RQ-B-024, RQ-B-013, RQ-B-014, RQ-B-031 Sources: A:199, A:225, A:252, A:219, A:195, A:177, A:214, A:201, A:176, A:270, A:224, A:237, C:010, A:232, A:233, A:197, A:242, A:234, A:216, C:002
Hardening, still open	Open: stacking of batch and cache for Mistral is not stated [A:174]. OpenAI flex percentage is not stated [A:195]. Together batch is 'up to 50 percent' [A:275]. Priority multipliers differ: Mistral 1.75x after cache discount [A:176], xAI 2x on all token types with cache applied first [A:225], OpenAI fast 2x [A:209]. Claude Code average input per call is 84K in one source and above 200,000 in another [C:054], so band tests need real totals.
Not in client docs	**DevX proposal:** The unpriced status, the rule that calc runs off the request path, and rounding to 1e-9 USD are DevX proposals. The six-step order is from R5.

WHAT IT IS FOR

Compute the cost of one request with an ordered formula, so the same usage always gives the same dollars, and label where the figure came from.

EXPECTED

- Follows the six-step order from R5 exactly: normalise, band, base, tier multipliers, extras, plan billing.
- Stores price_row_id, cost_source (client, gateway or bill) and auth_mode on every cost row.
- Prefers provider-reported cost when present. Computed cost is stored beside it [A:232, A:233].
- Applies long-context repricing to every bucket when band_scope is all_tokens (xAI above 200K) [A:225, A:270].
- Applies Priority and regional multipliers after the cache discount [A:176, A:225].
- Unknown price gives price_status=unpriced, never zero [A:242].
- Failed or expired batch and flex requests cost 0 [A:219, A:195].

NOT EXPECTED

- No hard-coded prices. Every rate comes from a price row.
- No token conversion for plan billing. Claude plan quota has no published token conversion [C:010].
- No budget decisions. Budgets read the ledger.
- No cost for reasoning on top of output. Reasoning is part of output and billed at the output rate.

ALGORITHM AND MATHS

Ordered per-request cost formula (R5 section 2)

```
Base = (I*in + C_r*read + C_w5*w5 + C_w1h*w1h + (O+R)*out) / 1e6
```

STEPS

1. 1. Normalise buckets to the vendor's accounting (Anthropic reports them separately [A:199]; others include cached in input). Use vendor billed units [A:177].
2. 2. Band from total prompt tokens; if band_scope = all_tokens reprice every bucket [A:225].
3. 3. Base = (I*in + C_r*read + C_w5*w5 + C_w1h*w1h + (O+R)*out) / 1e6.
4. 4. Apply tier multipliers per applies_to and order. Batch cache component only if stacking flag is true; else bill reads at the batch input rate [A:252]. Use expected, not ideal, hit rate in batch [A:219].
5. 5. Add per-call tool fees, storage, per-review fees [A:225, A:214]. Failed batch or flex = 0 [A:219, A:195].
6. 6. Plan billing: marginal $ = 0 inside the pool, overage after. Track vendor units (credits, requests x multiplier). Report shadow API cost plus amortised fee. No token conversion for Claude [C:010].

PARAMETERS

I	uncached input tokens (fresh_input)
C_r	cache read tokens
C_w5	cache write tokens, 5 minute TTL
C_w1h	cache write tokens, 1 hour TTL
O	output tokens
R	reasoning tokens (billed as output; zero when already inside O)
in/read/w5/w1h/out	USD per 1M tokens from the price row [A:199]
band_scope	marginal or all_tokens, per row [A:225]
stacking_flag	true, false or unknown, per row [A:252, A:255, A:281]
long_context_threshold	200K (Anthropic, Gemini, xAI), 100K (Haiku 5.5) [A:270, C:010]

THE MATHS IN FULL

SYMBOLS

- I, C_r, C_w5, C_w1h, O, R: token counts per bucket for one request.
- in, read, w5, w1h, out: USD per million tokens from the price row.
- M_k: tier multiplier k with applies_to and order.
- B = Base cost in USD.
- X = extras: tool fees, storage, review fees.

FORMULA

```
Base = (I*in + C_r*read + C_w5*w5 + C_w1h*w1h + (O+R)*out) / 1e6; cost = apply(M_1..M_k, Base) + X; if status is failed batch or flex, cost = 0; if plan billing, marginal = 0 inside pool
```

WHY IT IS RIGHT

Step 3 prices each exclusive bucket at its own rate. Reasoning tokens bill as output, so R joins O at the out rate. When the normaliser already counted reasoning inside O, R is zero, which avoids double counting [A:197]. Step 2 runs first because the band decides which rates apply to all buckets. Step 4 runs after the cache rates are set so Priority and regional multipliers apply after the cache discount [A:176, A:225]. Without stacking, batch applies to input and output but cache reads bill at the batch input rate [A:252].

WORKED EXAMPLE

```
Sonnet 5.5, rates $2 in, $10 out, $2.50 w5, $4 w1h, $0.10 read [A:283, A:199]. Buckets I=200, C_r=600, C_w5=200, O=100 (illustrative split from the Cloudflare example [A:190]). Base = (200*2 + 600*0.10 + 200*2.50 + 100*10) / 1e6 = (400 + 60 + 500 + 1000) / 1e6 = $0.00196. Batch with stacking: x0.5 gives $0.00098. Band case: Opus 4.6 with 210,000 fresh input and 1,000 output above 200K uses $10 and $37.50: (210000*10 + 1000*37.5) / 1e6 = $2.1375 against $1.075 at $5 and $25 [A:270].
```

COST OF COMPUTING IT

O(1) per request: five multiplications, a few multipliers.

HOW IT CAN GO WRONG, AND THE GUARD

- Wrong band flag gives a 2x error on long prompts.
- Unknown stacking flag: assume no stacking for the estimate and flag unknown.
- Missing price row: unpriced row, reservation stays as the spend bound.
- Rounding drift: use integer micro-dollars or decimal types, never binary floats.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Weekly canary compares computed cost with vendor usage or invoices. Target error under 1 percent (DevX proposal from R5).

SECURITY

- Computes only from counts and price rows. No content is read.
- Price rows are immutable, so a user cannot change a past cost by editing a price.
- Cost rows are signed by the ledger writer. Clients cannot submit costs that are stored as gateway cost.

PERFORMANCE

Latency and throughput	Under 1 ms per request at the 95th percentile (P95) (DevX proposal).
Complexity	O(1) per request.
At 10x load	Stateless. Scales with the ledger writers.

EDGE CASES AND HOW EACH IS HANDLED

- xAI above 200K reprices all tokens including cached ones [A:225].
- Mistral Priority multiplies after the cache discount [A:176].
- Groq does not stack batch and cache, so reads bill at the batch input rate [A:252].
- Failed batch or flex request costs zero [A:219, A:195].
- Provider reports its own cost. The reported value is stored and preferred [A:232].
- Fast mode toggled mid-conversation bills the whole context uncached, so the buckets must reflect it.
- Models with no cache write premium use write rate equal to input [A:201].

TEST CASES

TEST	GIVEN	EXPECT
base_formula	Sonnet 5.5 I=200 Cr=600 Cw5=200 O=100	cost $0.00196
batch_stack	same usage, batch, stacking true	cost $0.00098
long_context	Opus 4.6 210,000 fresh input, 1,000 output	cost $2.1375
no_price	model id missing from price book	price_status unpriced, cost null, reservation kept
failed_batch	expired batch request	cost 0
reasoning_once	output 1186 includes reasoning 1024, R passed as 0	charged on 1186 output tokens only
provider_cost	response carries cost 0.0021, computed 0.0020	stored cost 0.0021 with computed beside it

WHERE IT SAVES COST

- Accurate tier and band pricing exposes where batch, flex or caching save money.
- Cost per request feeds routing, tier choice and cache planning.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- calc.cost_row
- calc.unpriced
- calc.provider_cost_mismatch

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.calc.cost_row	ledger_key · price_row_id · cost_usd · cost_source · auth_mode · price_status	A cost row is emitted.
token_economics.calc.unpriced	provider · model_id · service_tier	No price row found.
token_economics.calc.provider_delta	ledger_key · computed_usd · reported_usd	Provider cost differs from computed.

RESEARCH BEHIND IT

A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
Result we rely on: Sticker price misleads: total cost reversed the ranking in 32 percent of comparisons.
Used here: Cost is computed per request from usage, not taken from list price.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Langfuse: Tier conditions on price rows and ingested cost beating inferred cost.
- LiteLLM: Cost maps per provider to compare against.

HOW OTHERS BUILD IT

Langfuse computes cost from usage buckets and price tiers, and prefers cost sent by the client. LiteLLM computes cost per provider. Token Economics adds an explicit step order, per-row flags for stacking and band scope, and a cost_source label.

REFERENCES

- A:199 Claude prompt caching docs
- A:225 xAI pricing
- A:252 Groq Batch API
- A:219 Batch processing (Claude)
- A:195 Flex processing
- A:177 How Does Cohere's Pricing Work?
- A:214 Models and pricing for GitHub Copilot
- A:201 Prompt caching
- A:176 Priority Tier
- A:270 Claude Opus 4.6
- A:224 Prompt Caching: Cut the Cost of Long Chats with the Grok API
- A:237 Prompt caching (Helicone gateway)
- C:010 Claude Code Billing & Usage Limits Explained (Oct 2026)
- A:232 Token & Cost Tracking (Langfuse)
- A:233 LLM Cost Management: Track and Control Spend
- A:197 Reasoning models
- A:242 Budget limits (Portkey)
- A:234 Track and optimize coding agent usage (Langfuse)
- A:216 Track cost and usage (Claude Agent SDK)
- C:002 Claude Code OpenTelemetry: Track Cost and Tool Use

RECEIVES FROM

- Price book
  price row
- Dedupe and resume guard
  unique usage

HANDS TO

- Append-only ledger
  cost row

← Back to group: Metering ← Stage 3

LEVEL 4

Append-only ledger

Store | Stage 3: Accounting

Append-only ledger

Ledger key with price_row_id

key: anth|sonnet-5.5|std|fast0|eff:hi|geo:us|ttl5m
row 91 price_row=4172 cost=0.00196 src=gateway

IN PLAIN ENGLISH

> The ledger is a notebook where every request is written once and never erased. Fixes are added as new lines, so the history always stays readable.

Goes in	Cost rows from the calculator, true-ups from the reserve step (bres), and plan rows from the quota tracker (planq).
Comes out	Append-only rows for budgets, reconciliation, hit-rate monitoring, turn caps, evaluation and the metadata mirror.
Reads / writes	Writes ledger rows. Reads nothing in the write path. Never updates or deletes.
If it fails	Fail closed for budgets: if the ledger cannot write, the gateway keeps the reservation as the spend bound and retries from a local queue. The user response is not blocked.
Requirement source	RQ-B-009, RQ-B-010, RQ-B-011, RQ-B-007, RQ-B-012, RQ-B-013, RQ-B-023 Sources: A:232, A:233, A:055, A:217, A:216, C:002, A:242, C:028
Hardening, still open	Open: the error bound of ledger estimates against invoices is unknown for every provider (R1 needs experiment 8). Reconciliation tolerance is not in the corpus. Ledger key fields for non-Anthropic harnesses are not fully confirmed.
Not in client docs	**DevX proposal:** Row schema beyond the key, the supersede link and the local retry queue are DevX proposals.

WHAT IT IS FOR

Keep the single record of what each request cost, with the exact price row and cost source, so budgets, reports and audits use the same numbers.

EXPECTED

- Ledger key includes exact model id, service tier, speed, effort, inference geo, caller id, session id, TTL class [A:232, A:055, A:217].
- Row holds the five token buckets, cost_usd, price_row_id, cost_source, auth_mode, provider cost if reported, price_status.
- Append-only. A correction is a new row that names the row it supersedes.
- A unique constraint on provider response id stops repeats that slip past the dedupe guard.
- Plan rows (native units plus shadow API cost) share the table and a row_type field.
- Failed and retried attempts are rows too, tagged with task id and attempt number [C:028].

NOT EXPECTED

- No updates or deletes.
- No prompt or response content.
- No budget logic. The budget store reads spend from the ledger.
- No re-pricing of history when prices change [A:232].

ALGORITHM AND MATHS

Append-only keyed log with supersede links

```
row_id = monotonic; unique(provider, response_id, row_type); superseded_by = null until corrected
```

STEPS

1. Receive a cost row. Validate required fields and the key parts.
2. Insert with a unique check on provider and response id. A duplicate returns the existing row and counts a dedupe hit.
3. Emit change events to budgets (spend), hit monitor (cache rows), turn cap (turns), canary and reconciler.
4. Mirror a metadata-only copy to the metadata store.
5. For a correction, insert a new row with supersedes = old row_id. Readers use the latest row in each chain.

PARAMETERS

key_fields	model_id, service_tier, speed, effort, inference_geo, caller_id, session_id, ttl_class [A:232, A:055, A:217]
row_types	request, true_up, plan, correction (DevX proposal)
partition	by tenant and day (DevX proposal)

SECURITY

- Storage forbids updates and deletes at the database role level.
- Rows hold ids and counts, no content. Retention is enforced by the retention component.
- Tenant id is part of every query path to prevent cross-tenant reads.

PERFORMANCE

Latency and throughput	Write under 10 ms at the 95th percentile (P95) and 99.9 percent durable before acknowledging (DevX proposal).
Complexity	O(1) per insert with indexed keys.
At 10x load	Partition by tenant and day. Archive old partitions per retention policy.

EDGE CASES AND HOW EACH IS HANDLED

- Same call reported by gateway and SDK telemetry: one row, gateway source [A:216, A:217].
- Resumed session totals must not be re-booked [A:216].
- Reservation true-up replaces the reservation. The reservation row stays, the true-up row supersedes it.
- Subagent rows carry the subagent caller id and the parent session id [A:217].
- A price correction never changes old rows. A new row can supersede a mispriced one with a note.

TEST CASES

TEST	GIVEN	EXPECT
unique_response	insert two rows with the same provider and response id	second insert returns first row, no new row
no_update	update a stored row	rejected by database role
supersede	correction row with supersedes=91	readers see the new row, old row stays
key_fields	row missing inference_geo	rejected with LEDGER_KEY_INCOMPLETE
failed_attempt	retry attempt 2 of task t9	row with task_id t9, attempt 2, cost included in task total

WHERE IT SAVES COST

- One trustworthy cost trail lets budgets, routing and cache planning act on real spend.
- Failed attempts in the ledger expose waste such as 32 percent of spend in failed runs [C:028].

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- led.row_written
- led.duplicate
- led.correction

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.led.row_written	ledger_key, row_type, cost_usd, price_row_id, cost_source	A row is appended.
token_economics.led.duplicate	provider, response_id	A unique check hits.
token_economics.led.write_failed	reason, queue_depth	Write fails and the local queue grows.

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Langfuse: Usage and cost fields on observations, with history left unchanged after price updates.
- ccusage: Local JSONL cost estimates for Claude Code, useful for transcript replay checks.

HOW OTHERS BUILD IT

Gateways such as LiteLLM and Langfuse log per-request usage and cost. Token Economics adds the price row id, the cost source label and an append-only rule so an auditor can recompute any row.

REFERENCES

- A:232 Token & Cost Tracking (Langfuse)
- A:233 LLM Cost Management: Track and Control Spend
- A:055 Introducing upgrades to Codex
- A:217 Monitoring usage (Claude Code OpenTelemetry)
- A:216 Track cost and usage (Claude Agent SDK)
- C:002 Claude Code OpenTelemetry: Track Cost and Tool Use
- A:242 Budget limits (Portkey)
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)

RECEIVES FROM

- Cost calculator
  cost row
- Plan-quota tracker
  plan row
- Reserve and reconcile
  true-up, true-up

HANDS TO

- Invoice reconciler
  ledger rows
- Metadata store
  mirror, mirror
- Budget store
  spend, spend
- Turn cap
  turns, turns
- Cache-hit monitor
  cache rows, cache rows
- CPS calculator
  cost rows, cost rows
- Budget alerts
  spend

← Back to group: Metering ← Stage 3

LEVEL 3 Reconciliation in Accounting

Estimated cost is checked against invoices weekly, and plan quota is tracked in native units.



Calculator — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Invoice reconciler → Weekly price canary
Billed over estimated per provider.

2. Metering (group) → Invoice reconciler
Estimated cost per key and day.

3. Weekly price canary → Price book (group)
Drift above 1 percent marks the row stale.

4. Price book (group) → Plan-quota tracker
Credits and message limits per plan.

5. Plan-quota tracker → Metering (group)
Native units plus shadow API cost.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Accounting ← Metering

LEVEL 4

Invoice reconciler

Calculator | Stage 3: Accounting

Invoice reconciler

Billed against estimated, per key

anthropic 2026-10 est=$98.60 bill=$100.00
ratio=0.986 verdict=within 2%

IN PLAIN ENGLISH

> Our own numbers are estimates. This part compares them with the real bill from each vendor and shows how far apart they are.

Goes in	Ledger estimates per provider key and day, and provider invoices or usage exports.
Comes out	A reconciliation ratio per provider, key and period, with a verdict and the largest contributing rows.
Reads / writes	Reads ledger rows and invoice data. Writes reconciliation records and ratios to the metadata store.
If it fails	Fail open: a failed run only delays the report. It never blocks traffic. Missing invoice data gives status no_invoice, not a pass.
Requirement source	RQ-B-033, RQ-B-034 Sources: A:216, C:002, C:004, A:233, A:242, A:234
Hardening, still open	Open: reconciliation tolerance is not in the corpus. The error bound of ledger estimates is unknown for every provider. Bedrock, Vertex and Foundry usage does not appear in Anthropic dashboards [C:005 in R4], so those need their own invoice feeds. Gateway budgets are soft caps that can overshoot [A:234].
Not in client docs	**DevX proposal:** Daily and per-key grain, the tolerance and the contributor ranking are DevX proposals. The corpus gives the monthly ratio and says tolerance is not in it.

WHAT IT IS FOR

Compare ledger estimates with provider invoices so a pricing, bucket or tier error shows up as a ratio. Only the invoice is a bill, and client or gateway figures are estimates.

EXPECTED

- Ratio = ledger estimate / provider invoice, per provider and per period, at least monthly [A:216].
- Daily comparison against usage exports where a provider offers them (DevX proposal).
- Splits the gap by model, tier, key and bucket to find the cause.
- Marks the period no_invoice when invoice data is missing.
- Handles providers whose usage is billed outside the Anthropic console, such as Bedrock and Vertex, through their own feeds.

NOT EXPECTED

- No correction of ledger rows. A fix goes through a correction row and a price update.
- No payment or invoice approval.
- No blocking of traffic on a bad ratio.

ALGORITHM AND MATHS

Ratio per provider and key with contributor ranking

```
ratio = sum(ledger_cost) / invoice_amount; gap_usd = invoice_amount − sum(ledger_cost)
```

STEPS

1. Pull ledger rows for the invoice period, grouped by provider, key, model and tier.
2. Load the invoice or usage export for the same period and key.
3. Sum ledger cost, using the cost recorded at request time (price rows are frozen).
4. Compute ratio and gap.
5. Rank contributors by absolute gap in USD and by share of tokens.
6. Send the ratio to the canary and store the record.

PARAMETERS

period	monthly minimum [A:216]; daily proposed
tolerance	not in corpus, set from pilot (DevX proposal)
initial_review_band	1 percent, shared with the canary target (DevX proposal)

THE MATHS IN FULL

SYMBOLS

- E = sum of ledger cost estimates for the period.
- B = invoice amount for the period.
- r = E / B.
- g = B - E.

FORMULA

```
r = E / B; g = B − E; flag if |r − 1| > tolerance
```

WHY IT IS RIGHT

The ledger holds costs computed at request time from price rows [A:232]. The invoice is the bill. Their ratio is a direct measure of pricing and bucket error. A ratio below 1 means under-estimated spend.

WORKED EXAMPLE

```
Illustrative arithmetic, not corpus data: E = $98.60 and B = $100.00 give r = 0.986 and g = $1.40. With a 1 percent review band the period is flagged. With E = $99.50, r = 0.995 and it passes.
```

COST OF COMPUTING IT

O(n) rows per period, grouped in the database. Reports run in batch.

HOW IT CAN GO WRONG, AND THE GUARD

- Invoice covers more than gateway traffic (direct calls by developers), so r looks low.
- Invoice includes credits or taxes that the ledger does not model.
- Timezone and cut-off differences shift rows between periods.
- BYOK traffic is not counted by some gateways [A:245 in R2].

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Run for two pilot months, set tolerance from observed ratios, then compare against the canary.

SECURITY

- Invoice data is financial. Access needs the finance role and is logged.
- Only totals and ids are stored. No invoice PDFs are kept in the metadata store.
- Reports exclude content by design.

PERFORMANCE

Latency and throughput	Monthly report within one hour of invoice arrival (DevX proposal).
Complexity	O(n) rows per period.
At 10x load	Batch grouped queries on ledger partitions.

EDGE CASES AND HOW EACH IS HANDLED

- Invoice currency or credits differ from ledger USD.
- A promo discount applies only on the invoice.
- Batch cost is booked when the batch completes, so a month boundary splits a batch.
- A key was deleted during the month. Rows keep the key id.
- Provider reports cost in billed units, not tokens [A:177].

TEST CASES

TEST	GIVEN	EXPECT
within_band	E=99.50, B=100.00, band 1 percent	ratio 0.995, verdict within
outside_band	E=98.60, B=100.00, band 1 percent	ratio 0.986, flagged, contributors ranked
no_invoice	no invoice data for the period	status no_invoice, no verdict
grain	two keys with different ratios	separate records per key

WHERE IT SAVES COST

- Finds wrong tier or band pricing that hides real spend.
- Confirms the savings claims in dashboards against bills.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- rec.ratio_computed
- rec.flagged

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.rec.ratio`	provider, key_id, period, estimate_usd, invoice_usd, ratio	A reconciliation record is written.
`token_economics.rec.flagged`	provider, period, ratio, top_contributor	Ratio is outside the review band.

RESEARCH BEHIND IT

A:216 Track cost and usage (Claude Agent SDK)
Result we rely on: Client-side cost is an estimate, and the reconciliation ratio is monthly.
Used here: Defines the ratio and the need to reconcile to invoices.

HOW OTHERS BUILD IT

Teams compare gateway or client cost estimates with vendor invoices to find drift. The corpus has no tolerance, so Token Economics sets it from a two-month pilot.

REFERENCES

- A:216 Track cost and usage (Claude Agent SDK)
- C:002 Claude Code OpenTelemetry: Track Cost and Tool Use
- C:004 Monitoring Claude Code Usage with OpenObserve and MCP
- A:233 LLM Cost Management: Track and Control Spend
- A:242 Budget limits (Portkey)
- A:234 Track and optimize coding agent usage (Langfuse)

RECEIVES FROM

Append-only ledger
ledger rows

HANDS TO

Weekly price canary
ratio

← Back to group: Reconciliation ← Stage 3

LEVEL 4

Weekly price canary

Calculator | Stage 3: Accounting

Weekly price canary

Computed cost against invoice, 1% target

week 44 anth batch drift +1.4% (target 1.0%)
row 4172 flagged stale -> plife

IN PLAIN ENGLISH

> Each week this part checks that our price math matches what the vendors actually report. If the gap is bigger than one percent, it tells the price book to distrust that price.

Goes in	Computed cost per provider and tier, plus vendor usage reports or invoices, and reconciliation ratios from rec.
Comes out	Weekly drift value per provider and tier, and a stale signal to the price-row lifecycle when drift exceeds 1 percent.
Reads / writes	Reads ledger and rec records. Writes canary results to the metadata store.
If it fails	Fail open: a missed canary delays detection but never blocks traffic. A missed run raises its own alert after one skipped week.
Requirement source	RQ-B-035, RQ-B-036 Sources: A:216, C:002, A:069, A:214, A:148, A:121, A:280
Hardening, still open	Open: the corpus has no tolerance for reconciliation. The 1 percent target is a design aim from R5 experiment 1 (two weeks, computed against invoiced cost per provider and tier). Sample size per tier is not defined.
Not in client docs	**DevX proposal:** The weekly cadence, the 1 percent trigger and the stale signal path are DevX proposals taken from the R5 design. They are not corpus measurements.

WHAT IT IS FOR

Catch price-book errors early. Prices change, discounts differ by source and tiers are easy to get wrong, so a weekly comparison finds drift before a bill does.

EXPECTED

- Runs weekly per provider and per service tier.
- Computes drift = (computed - vendor reported) / vendor reported over the week.
- Emits a stale signal to plife when absolute drift is above 1 percent.
- Attaches the likely cause: price row, band, stacking flag or bucket mapping.
- Reports skipped weeks as a failure.

NOT EXPECTED

- No automatic price changes. A person verifies and inserts a new row.
- No traffic blocking.
- No use of content data.

ALGORITHM AND MATHS

Weekly computed-versus-reported drift check

```
drift = (computed_cost − vendor_cost) / vendor_cost
```

STEPS

1. Select the week window and the (provider, tier) groups with traffic.
2. Sum computed cost per group from ledger rows.
3. Fetch vendor usage or invoice cost for the same window.
4. Compute drift and compare with the trigger.
5. If drift exceeds the trigger, send a stale signal for the rows used in that group.
6. Write the result and the contributing price_row_ids.

PARAMETERS

cadence	weekly (R5 design, DevX proposal)
trigger	1 percent (R5 experiment 1 target, DevX proposal)
pilot_length	2 weeks [R5 experiment 1]

THE MATHS IN FULL

SYMBOLS

- C = computed cost for the group and week.
- V = vendor reported cost for the same group and week.
- d = drift.
- T = trigger, 0.01.

FORMULA

```
d = (C − V) / V; stale if |d| > T
```

WHY IT IS RIGHT

If every bucket and rate are right, C equals V. Any rate, band, multiplier or bucket error moves C away from V. A relative measure lets one threshold cover providers with different spend.

WORKED EXAMPLE

```
Illustrative arithmetic: C = $101.40 and V = $100.00 give d = +0.014, or 1.4 percent, so the group is flagged stale. With C = $100.60, d = 0.6 percent and it passes.
```

COST OF COMPUTING IT

O(g) groups per week. Trivial.

HOW IT CAN GO WRONG, AND THE GUARD

- Vendor report lags, so the week has partial data.
- Tiny groups give noisy ratios.
- Mixed tiers hide an error in one tier.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

After the two-week pilot, set the trigger from observed drift per provider.

SECURITY

- Vendor report credentials live in the secrets vault and are read-only.
- Results hold no content.
- A stale signal needs the canary service identity so a user cannot fake one.

PERFORMANCE

Latency and throughput	Weekly run completes within 1 hour (DevX proposal).
Complexity	O(g) groups.
At 10x load	Runs as a scheduled batch.

EDGE CASES AND HOW EACH IS HANDLED

- Promo or discount shows on the vendor report and not in the price row.
- Gemini cache discount 75 against 90 percent shows up as drift on cached tokens [A:121, A:280].
- Azure 12h write charge from Nov 1 2026 appears as drift on that date [A:148].
- A week with very little traffic produces no verdict.
- Vendor report in billed units, not tokens, needs the units flag.

TEST CASES

TEST	GIVEN	EXPECT
flag_stale	C=101.40, V=100.00	drift 1.4 percent, stale signal sent
pass	C=100.60, V=100.00	drift 0.6 percent, no signal
missed_week	scheduler skipped week 44	alert canary_missed
small_group	group with 3 requests	no verdict, status low_volume

WHERE IT SAVES COST

- Early detection avoids weeks of mispriced decisions on tier choice and cache planning.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- canary.drift
- canary.stale_signal

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.canary.drift`	provider · tier · week · computed_usd · vendor_usd · drift	Weekly check ends.
`token_economics.canary.stale_signal`	price_row_id · drift	Drift exceeds the trigger.

RESEARCH BEHIND IT

A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
Result we rely on: Price rankings reversed in 32 percent of 336 comparisons.
Used here: Shows prices move and list price alone is unreliable.

HOW OTHERS BUILD IT

No corpus source describes a price canary. This is a DevX design from R5 that follows the usual drift check between computed and vendor-reported cost.

REFERENCES

- A:216 Track cost and usage (Claude Agent SDK)
- C:002 Claude Code OpenTelemetry: Track Cost and Tool Use
- A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
- A:214 Models and pricing for GitHub Copilot
- A:148 Prompt caching with Azure OpenAI in Azure AI Foundry Models
- A:121 Gemini 2.5 Models now support implicit caching
- A:280 Context caching overview (Vertex AI)

RECEIVES FROM

Invoice reconciler
ratio

HANDS TO

Price-row lifecycle
stale signal

← Back to group: Reconciliation ← Stage 3

LEVEL 4

Plan-quota tracker

Calculator | Stage 3: Accounting

Plan-quota tracker

Native units, shadow API cost

claude max: 42/50 msgs 5h window, resets 17:40
shadow API $11.20, fee share $6.67/wk

IN PLAIN ENGLISH

> Subscription plans count messages or credits, not tokens. This part tracks those units, and also shows what the same use would have cost on the pay-per-token API.

Goes in	Plan rows from pplan, usage in native units from adapters and local transcripts, and ledger shadow costs.
Comes out	Per user and plan: units used, units left, reset time, shadow API cost and amortised fee.
Reads / writes	Reads pplan and ledger. Writes plan rows to the ledger with row_type plan.
If it fails	Fail open: plan tracking never blocks requests. Missing units show as unknown.
Requirement source	RQ-B-030, RQ-B-031, RQ-B-032 Sources: C:010, C:119, C:057, C:116, C:114, C:115, A:146, A:161, C:011, C:120, C:117
Hardening, still open	Open: Anthropic does not publish a limit-to-token conversion [C:010]. Codex weekly caps hit after about 2 million tokens is user-reported [C:115]. Plan figures change fast: caps doubled May 2026 and a weekly boost expired 2026-07-13 [C:011]. Kiro credits per token is not in the corpus.
Not in client docs	**DevX proposal:** The calibration method and the confidence band are DevX proposals from R5 experiment 4. The corpus has no Claude token conversion.

WHAT IT IS FOR

Track plan spend in native units without converting to tokens, and show a shadow API cost so users can compare a plan with API billing.

EXPECTED

- Stores usage in the plan's unit: messages per 5 hours, credits, premium requests, requests per day, pool multiples.
- Marginal dollars are zero inside the included pool and the overage rate applies after it [R5 formula step 6].
- Shadow API cost is computed by the calculator from tokens at list rates. The tracker stores it beside the units.
- Amortised fee is the plan fee spread over the period.
- Weights per model apply where the plan states them, such as Copilot review 13x or reasoning 5x and 20x [C:117].
- Reset windows follow the plan reset_rule.

NOT EXPECTED

- No conversion of Claude plan limits to tokens [C:010].
- No blocking when units run out. The plan provider enforces its own limits.
- No dollar bill for subscription traffic.

ALGORITHM AND MATHS

Native-unit tracker with shadow cost

```
marginal_usd = 0 while used_units <= included_qty; else (used_units − included_qty) x overage_rate
```

STEPS

1. Read the user's plan row and reset window.
2. Add usage in native units, weighted by model_weighting where it exists.
3. Compute remaining units and the next reset time.
4. Ask the calculator for shadow API cost on the same traffic.
5. Compute amortised fee = fee x (days elapsed / days in period).
6. Write a plan row to the ledger and update the dashboard feed.

PARAMETERS

copilot_credit_usd	0.01 [A:214]
codex_credit_usd	about 0.04 [C:114]
claude_window	5 hours, prompt counts approximate [C:010]
gemini_cli_daily_requests	1,000 to 2,000, 250 for unpaid Flash [C:057]

THE MATHS IN FULL

SYMBOLS

- Q = included_qty in native units (nullable).
- U = used units.
- o = overage_rate in USD per unit.
- S = shadow API cost for the same traffic.
- F = plan fee.
- d = days elapsed, D = days in period.

FORMULA

```
marginal = 0 if U <= Q else (U − Q) x o; amortised_fee = F x d / D; plan_value = S − amortised_fee
```

WHY IT IS RIGHT

A plan is a fixed fee that buys a pool of units. Inside the pool the extra dollar cost of one more request is zero. The shadow cost S answers what the same tokens would cost on the API, so the difference to the amortised fee shows whether the plan pays.

WORKED EXAMPLE

```
Illustrative arithmetic with corpus unit values: Copilot 250 credits used at $0.01 per credit is $2.50 at API-list rates [A:214]. Codex 100 credits at about $0.04 is about $4 [C:114]. Plan
Pro fee $20 per month at day 15 of 30 gives an amortised fee of $10. If S = $14, plan_value = $4.
```

COST OF COMPUTING IT

O(1) per update per user.

HOW IT CAN GO WRONG, AND THE GUARD

- Unit weights wrong after a vendor change.
- Plan row expired and not replaced.
- Usage from other devices not visible to the tracker.
- Shadow cost uses a stale price row.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

One developer-week per plan: log quota used against shadow API cost and fit dollars per unit with a band (R5 experiment 4, DevX proposal).

SECURITY

- Usage per user is personal data. It is stored per tenant, and dashboards show aggregates by default.
- No credentials for vendor accounts are stored here. Vendor dashboards are read through exports.
- Plan rows carry ids, not content.

PERFORMANCE

Latency and throughput	Update within 60 seconds of a usage event (DevX proposal).
Complexity	O(1) per update.
At 10x load	Per user state. Partition by tenant.

EDGE CASES AND HOW EACH IS HANDLED

- Plan with null included_qty: show usage only.
- Pool resets mid-window when a plan changes.
- Cursor pool multiples of 4x, 15x and 50x on the $8 base [C:119].
- Gemini GSU weights output 5 and cached 0.1 [A:146].
- User on two plans at once (personal and team).

TEST CASES

TEST	GIVEN	EXPECT
inside_pool	used 42 of 50 messages	marginal 0, remaining 8
overage	used 60 of 50 with overage rate $0.04	marginal $0.40
amortise	fee $20, day 15 of 30	amortised $10
null_pool	Anthropic plan with null included_qty	usage shown, remaining unknown
weighted	Copilot review request with 13x weight	13 units booked

WHERE IT SAVES COST

- Shows when a plan beats API billing and when heavy users should move.
- Highlights teams paying overage that a different plan would cover.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- planq.units_updated
- planq.near_limit

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.planq.update`	user_id · plan · unit · used · remaining · shadow_usd	Usage is recorded.
`token_economics.planq.overage`	user_id · plan · overage_units	Usage passes the pool.

REFERENCES

- C:010 Claude Code Billing & Usage Limits Explained (Oct 2026)
- C:119 Cursor Models & Pricing
- C:057 Gemini CLI: Quotas and pricing
- C:116 GitHub Copilot premium requests (billing concepts)
- C:114 Codex pricing: current costs, usage limits, and realistic budgets (2026)
- C:115 Codex CLI usage rate limits
- A:146 Calculate Provisioned Throughput requirements
- A:161 Billing
- C:011 Claude Code Limits: 5-Hour Caps Doubled in May 2026
- C:120 Cursor account pricing
- C:117 Understanding and managing requests in Copilot

RECEIVES FROM

Plan-units table
plan units

HANDS TO

Append-only ledger
plan row

← Back to group: Reconciliation ← Stage 3

LEVEL 2

4. Policy engine

Budgets, turn caps, loop guard, routing, cache planning and tier choice decide each request before it is sent.

Budgets

Virtual-key budgets with a worst-case reservation before dispatch and a true-up after.

1 Store · 2 Policy

Budget store · Reserve and reconcile · Budget alerts

Receives from: Gateway edge (Ingress and adapters), Metering (Accounting), Run control, Routing, Tier and cache, Admin and config (Control plane)

Hands to: API providers (Backends), Metering (Accounting)

Open group (3 components) →

Run control

Turn cap, loop guard, session stickiness and switch pricing keep a run from burning money.

4 Policy

Turn cap · Loop guard · Session stickiness · Switch pricer

Receives from: Client signals (Ingress and adapters), Metering (Accounting)

Hands to: API providers (Backends), Budgets, Routing

Open group (4 components) →

Routing

Rules first, a static router second, cascade only with measured acceptance.

5 Policy

Rules and atomic classifier · Static router · Cascade with fallback · Probe cost predictor · Strong-share ceiling

Receives from: Backend health (Backends), Run control

Hands to: Budgets

Open group (5 components) →

Tier and cache

Cache planner, tier selector and cache-hit monitor.

3 Policy

Cache planner · Tier selector · Cache-hit monitor

Receives from: Backend health (Backends), Metering (Accounting)

Hands to: API providers (Backends), Budgets

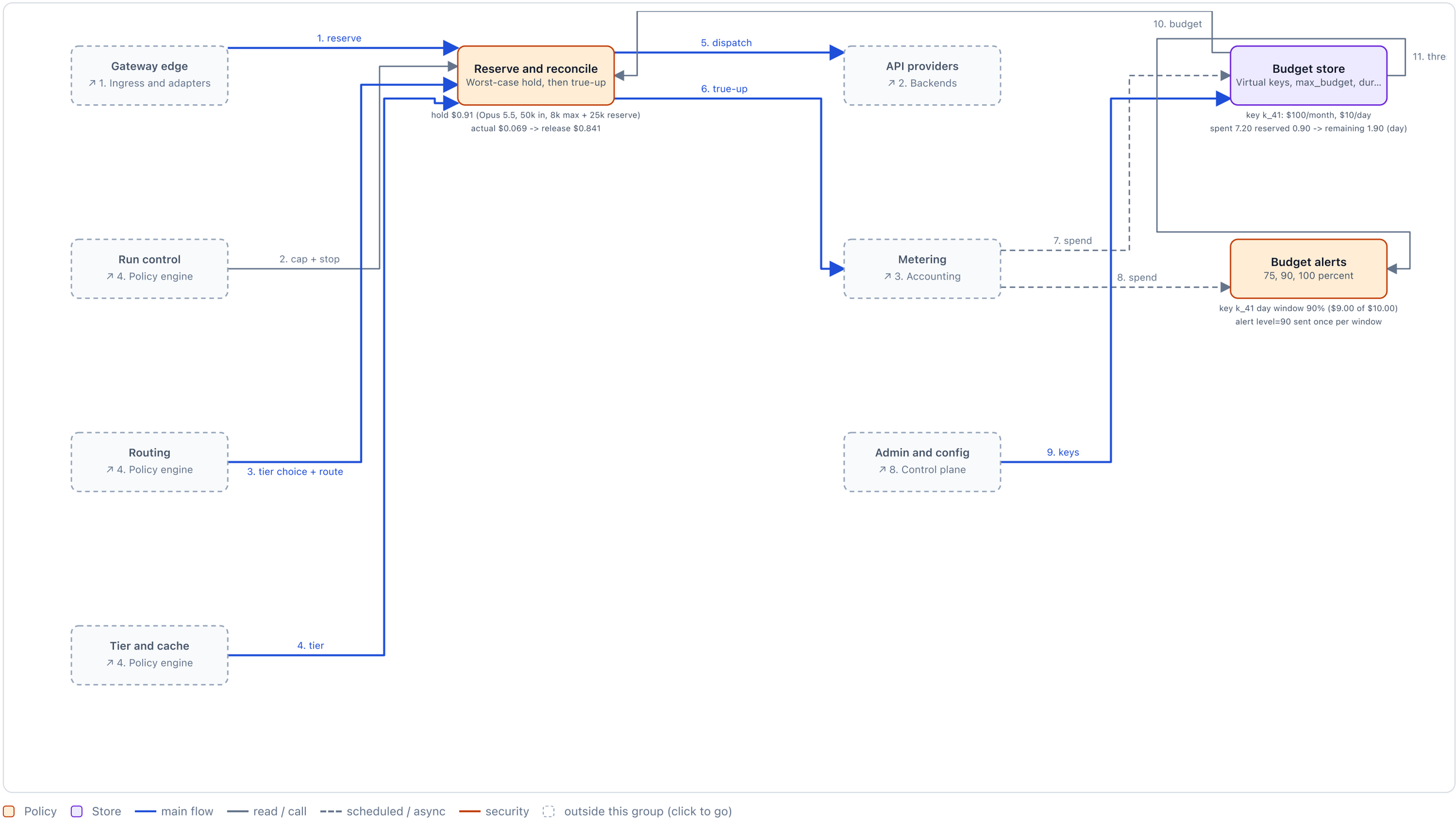
Open group (3 components) →

4 groups, 15 components. Hover a group for the plain-English story; click it to see its components and the arrows between them. ← Accounting Context guard →

LEVEL 3

Budgets in Policy engine

Virtual-key budgets with a worst-case reservation before dispatch and a true-up after.



Policy · Store · main flow · read / call · scheduled / async · security · outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Gateway edge (group) → Reserve and reconcile
Tenant id and request size; asks for a worst-case hold.

2. Run control (group) → Reserve and reconcile
cap: Turn cap and remaining-turns reminder. + stop: Loop signal ends or interrupts the run.

3. Routing (group) → Reserve and reconcile
tier choice: Model chosen by rules when no router is active. + route: Chosen model and escalation path.

4. Tier and cache (group) → Reserve and reconcile
Batch, flex, priority or standard.

5. Reserve and reconcile → API providers (group)
Request after reservation, routing and cache planning.

6. Reserve and reconcile → Metering (group)
Actual cost replaces the reservation.

7. Metering (group) → Budget store
Spend increments the budget window.

8. Metering (group) → Budget alerts
Spend triggers threshold checks.

9. Admin and config (group) → Budget store
Key records with budget and duration.

10. Budget store → Reserve and reconcile
Remaining budget for the key and window.

11. Budget store → Budget alerts
Crossing 75, 90 or 100 percent raises an alert.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Policy engine Run control →

LEVEL 4

Budget store

Store | Stage 4: Policy engine

Budget store

Virtual keys, max_budget, duration

key k_41: $100/month, $10/day
spent 7.20 reserved 0.90 -> remaining 1.90 (day)

IN PLAIN ENGLISH

> This is the account book for each key. It says how much money a key may spend per day or month and how much is left.

Goes in	Virtual key records from the tenant manager and spend increments from the ledger.
Comes out	Remaining budget per key and window, plus threshold state for the alert component.
Reads / writes	reads: key records (tenant), ledger spend | writes: max_budget, budget_duration, window_start, spent_usd, reserved_usd
If it fails	Fail closed for budget paths: if the store cannot be read, reject spend-bearing calls with 503, because an unmetered call is an uncapped call. Observe-only routes fail open.
Requirement source	RQ-C-001, RQ-C-002, RQ-C-004, RQ-C-005, RQ-C-006, RQ-C-007, RQ-C-008, RQ-C-011, RQ-C-050 Sources: C:060, C:061, C:136, A:234, A:191, C:054, C:074
Hardening, still open	Overshoot size is not quantified in the corpus. LiteLLM stacked windows are sourced; their 402 (payment required) behaviour is not measured. Cloudflare limits are eventually consistent, so bursts overshoot (A:191).
Not in client docs	**DevX proposal:** The default overshoot margin and the per-user sizing rule at P95 (95th percentile) are DevX proposals built on cited facts.

WHAT IT IS FOR

Hold the hard spend limit for every virtual key and answer one question fast: how much may this key still spend in each window.

EXPECTED

- Stores max_budget and budget_duration per key, with stackable windows such as $10 per day and $100 per month (C:060, C:061).
- Returns remaining = max_budget - spent - reserved for each window and the minimum across windows.
- Rolls a window when window_start + budget_duration passes and resets spent_usd to 0.
- Applies ledger spend after the true-up, so reserved and spent never double count.
- Exposes threshold state at 75, 90 and 100 percent to the alert component.

NOT EXPECTED

- Does not trust agent self-limits. The agent never reads the budget (C:136).
- Does not count an unpriced model as zero cost (A:242).
- Does not hold prompt or response content. It holds numbers and ids only.
- Does not decide routing or model choice.

ALGORITHM AND MATHS

Stacked window counters

```
remaining(key) = min over windows w of (max_budget_w − spent_w − reserved_w)
```

STEPS

1. Load the key record and all its windows.
2. Roll any window whose end time has passed, in one transaction.
3. Compute remaining per window and take the minimum.
4. On reserve, add to reserved_usd under a row lock. On true-up, move the amount to spent_usd.
5. Publish the new percent used to the alert component.

PARAMETERS

windows	stackable, e.g. $10/day and $100/month (C:060)
alert_thresholds	75, 90, 100 percent (A:213)
overshoot_margin	DevX proposal: reserved amount of in-flight calls

SECURITY

- Keys are virtual. The provider key never reaches the client and lives in the secrets vault.
- Writes to max_budget need an admin role and produce an audit row with before and after values.
- A key that cannot be found is rejected. There is no default unlimited key.

PERFORMANCE

Latency and throughput	DevX proposal: p99 read under 5 ms, write under 10 ms, on the request path.
Complexity	O(w) per request for w windows, usually 1 to 3.
At 10x load	Shard by key id. Use one row lock per key so concurrent reserves serialise without a global lock.

EDGE CASES AND HOW EACH IS HANDLED

- Two requests reserve at the same instant and only one fits: the second is rejected, not queued.
- A window rolls over while a reservation is in flight: the true-up lands in the new window by response time.
- Budget lowered below current spend: remaining goes negative and every call is rejected until the window resets.
- Clock skew between gateway nodes: use the store clock for window boundaries.
- Provider cost arrives late: the key shows reserved until the true-up writes.

TEST CASES

TEST	GIVEN	EXPECT
daily cap rejects	key $10/day, spent 9.95, reserve 0.10	402 returned, spent stays 9.95
stack takes minimum	day left $2, month left $50	remaining = $2
window rollover	day window ended 1 s ago, spent 9.95	spent resets to 0 and reserve succeeds
store down	store timeout on a spend route	503 returned and no call reaches the provider
concurrent reserve	two reserves of $1.00, remaining $1.50	one succeeds, one is rejected

WHERE IT SAVES COST

- Hard cap per key stops runaway spend, where 17 percent of tasks were failed runs and 32 percent of spend (C:028).
- Windows sized near P95 spend rather than the mean keep honest users unblocked (C:054).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Threshold crossing sent to alert.
- Cap hit sent to dashboards as a blocked-spend metric.
- Admin budget change written to the audit log.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.bst.window_rolled`	key_id · window · spent_usd	A window ends and resets.
`token_economics.bst.cap_hit`	key_id · window · remaining_usd	A reserve is rejected for budget.

RESEARCH BEHIND IT

C:136 Coding agents ignore spend (Ramp Labs)
Result we rely on: Agents ignored budgets across 14,000+ messages.
Used here: Limits are enforced in the store, never in the prompt.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- LiteLLM: Virtual keys with max_budget, budget_duration and stackable windows.
- Bifrost: HTTP 402 once developer, team or customer budget is spent (vendor-reported).

HOW OTHERS BUILD IT

LiteLLM, Bifrost and Portkey keep per-key budgets at the proxy. Cloudflare and Vercel limits are soft caps, so Token Economics treats them as such and adds a margin.

REFERENCES

- C:060 5 ways to cut Claude Code costs with LiteLLM
- C:061 5 Open-Source Tools to Control Your AI API Costs at the Code Level
- C:136 Coding agents ignore spend (Ramp Labs)
- A:234 Track and optimize coding agent usage (Langfuse)
- A:191 Spend limits
- C:054 The Coding Agent Economy
- C:074 Hooks reference

RECEIVES FROM

Append-only ledger
spend, spend

Tenant and key manager
keys, keys

HANDS TO

Reserve and reconcile
budget

Budget alerts
threshold

← Back to group: Budgets ← Stage 4

LEVEL 4

Reserve and reconcile

Policy | Stage 4: Policy engine

Reserve and reconcile

Worst-case hold, then true-up

hold $0.91 (Opus 5.5, 50k in, 8k max + 25k reserve)
actual $0.069 -> release $0.841

IN PLAIN ENGLISH

> Before a call goes out, this holds back the most it could possibly cost. After the answer returns, it swaps the hold for the real cost.

Goes in	Tenant id, request size, model and max_tokens, plus the route chosen by rules, router, tier selector and turn cap.
Comes out	A reservation or a rejection, the dispatch to the provider connector, and a true-up row to the ledger.
Reads / writes	reads: bst remaining, price book row, tcap cap, loop signal | writes: reserved_usd in bst, true-up row in the ledger
If it fails	Fail closed: if the reservation cannot be made, the call is rejected with 402 or 503. A budget path that fails open is an uncapped path.
Requirement source	RQ-C-001, RQ-C-003, RQ-C-004, RQ-C-005, RQ-C-006, RQ-C-032, RQ-C-050 Sources: A:197, C:136, A:191, A:242, A:245, C:027, C:110, C:074
Hardening, still open	The 25,000 token reasoning reserve is an OpenAI figure (A:197). Reasoning reserves for other providers are not in the corpus. Overshoot size is not quantified.
Not in client docs	**DevX proposal:** The overshoot margin and the choice to count max_tokens plus the reasoning reserve as the worst-case output are DevX proposals on the cited pattern.

WHAT IT IS FOR

Make sure no call starts unless its worst-case cost fits in the remaining budget, then correct the books with the real cost.

EXPECTED

- Computes worst-case cost = worst input cost + (max_tokens + reasoning reserve) x output price (A:197).
- Rejects the call when worst-case cost exceeds remaining budget, before any provider traffic.
- Dispatches to the connector chosen by the routing components and forwards cache_control and anthropic-beta unchanged.
- Writes a true-up row so the actual cost replaces the hold (edge bres to led).
- Prices every model and path. An unpriced model is rejected (A:242).

NOT EXPECTED

- Does not estimate cost from the agent's own claim.
- Does not pick the model. It receives the route from rules, srt, casc and tsel.
- Does not store content.
- Does not count a failed batch or flex call as spend (A:219, A:195).

ALGORITHM AND MATHS

Worst-case hold then true-up

```
hold = P_in_max x n_in + P_out x (max_tokens + R); actual = priced from the response; release = hold − actual
```

STEPS

1. Read the price row for provider, model, tier and band.
2. Compute the hold from the largest input price among write, fresh and read rates.
3. Atomically add the hold to reserved_usd if remaining allows. Otherwise reject.
4. Dispatch. On response, the calculator prices actual usage.
5. Subtract the hold from reserved_usd and add actual to spent_usd in one transaction.
6. On error or timeout, release the hold unless the provider bills the failure (429 billing flag).

PARAMETERS

reasoning_reserve	at least 25,000 tokens (A:197)
price_row	chosen by provider, model, tier, band, region, effective_from
overshoot_margin	DevX proposal

THE MATHS IN FULL

SYMBOLS

- n_in: input tokens estimated at the gateway (count or provider token-count call).
- P_in_max: highest per-token input rate in the row, usually the cache write rate.
- P_out: output price per token, with reasoning billed at the output rate (A:197).
- max_tokens: the cap set by the max-tokens enforcer.
- R: reasoning reserve in tokens, at least 25,000 (A:197).
- remaining: bst remaining for the tightest window.

FORMULA

```
hold = (n_in x P_in_max + (max_tokens + R) x P_out) / 1e6 with prices per million tokens; admit iff hold <= remaining - margin; after response release = hold - actual
```

WHY IT IS RIGHT

Output cost has no upper bound beyond max_tokens plus reasoning, and input cost is bounded by n_in at the dearest rate. The sum is therefore the largest bill the call can produce. Holding it guarantees spent plus reserved never exceeds the cap.

WORKED EXAMPLE

```
Opus 5.5: input $4, output $20, 5m write $5.00, read $0.20 per MTok (R1). n_in 50,000, max_tokens 8,000, R 25,000. Hold = 50,000 x 5.00/1e6 + 33,000 x 20/1e6 = 0.25 + 0.66 = $0.91. Actual:
5,000 fresh x 4/1e6 = 0.02, 45,000 read x 0.20/1e6 = 0.009, 2,000 output x 20/1e6 = 0.04, total $0.069. Release $0.841.
```

COST OF COMPUTING IT

O(1) per request: one price row read and one row-locked update.

HOW IT CAN GO WRONG, AND THE GUARD

- Underestimated n_in lets the real cost exceed the hold. The true-up then drives remaining negative and later calls are rejected.
- A reserve that is too loose blocks calls that would have fit. Many in-flight calls multiply this effect.
- Hold never released after a crash: a sweeper releases holds older than the provider timeout.
- Reserve cost for a model with no reasoning field is over-counted by R.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Compare hold to actual per provider and model. Track the ratio hold/actual at p50 and p99. Tighten R only where p99 actual reasoning stays well below 25,000.

SECURITY

- Hold and true-up writes are atomic so a client cannot race two calls past the cap.
- The provider key is fetched from the vault at call time and never logged.
- Rejections return a fixed error body with no price or budget detail beyond the remaining amount for that key.

PERFORMANCE

Latency and throughput	DevX proposal: under 3 ms added at p99 for reserve plus true-up, excluding provider time.
Complexity	O(1) per call.
At 10x load	Stateless workers. All contention sits on the per-key row in bst.

EDGE CASES AND HOW EACH IS HANDLED

- Streaming call is cut halfway: the true-up uses the usage seen by the tee, and the hold is released for the rest.
- Provider returns no usage object: the ledger row is labelled estimate and the hold is kept until the reconciler corrects it.
- Request for an unpriced model: rejected with a clear error and an alert, not counted as zero.
- max_tokens absent: the max-tokens enforcer sets it first. Without it no hold can be computed.
- Batch or flex call: hold uses the discounted rate. A failed batch row bills 0 (A:219).

TEST CASES

TEST	GIVEN	EXPECT
hold fits	Opus 5.5, remaining $5.00, hold $0.91	call admitted, reserved_usd +0.91
hold rejects	remaining $0.50, hold $0.91	402, no provider call
true-up	hold 0.91, actual 0.069	reserved -0.91, spent +0.069
unpriced model	model id absent from price book	rejected, alert raised, spend unchanged
crash release	hold older than provider timeout	sweeper releases the hold

WHERE IT SAVES COST

- Stops runaway loops before they bill.
- Lets the router and tier selector see an exact price per call.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Actual cost and hold ratio sent to calibration.
- Rejections sent to alerts and dashboards.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.bres.reserved`	key_id · hold_usd · model	A hold is taken.
`token_economics.bres.trued_up`	key_id · hold_usd · actual_usd	The response is priced.
`token_economics.bres.rejected`	key_id · hold_usd · remaining_usd · reason	A call is refused.

RESEARCH BEHIND IT

A:197 Reasoning models
Result we rely on: Reasoning tokens are billed as output and sit inside output_tokens.
Used here: Sets the reasoning reserve in the hold formula.

A:191 Spend limits
Result we rely on: Spend limits are eventually consistent so bursts overshoot.
Used here: Token Economics holds before dispatch instead of counting after.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- thealirazadev/llm-gateway: Worst-case cost reservation before dispatch, then reconcile (named in R1).

HOW OTHERS BUILD IT

Cloudflare and Vercel count spend after the fact and overshoot. Reservation gateways hold a worst case first. Token Economics follows the second pattern.

REFERENCES

- A:197 Reasoning models
- C:136 Coding agents ignore spend (Ramp Labs)
- A:191 Spend limits
- A:242 Budget limits (Portkey)
- A:245 Cost-aware model routing with Vercel AI Gateway
- C:027 Can your AI agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks
- C:110 Where Do Your Tokens Go: Empirical Token Consumption in Coding Agents
- C:074 Hooks reference

RECEIVES FROM

- Key and tenant resolver
  reserve, reserve
- Budget store
  budget
- Turn cap
  cap
- Loop guard
  stop
- Rules and atomic classifier
  tier choice
- Cascade with fallback
  route
- Tier selector
  tier

HANDS TO

- Anthropic connector
  dispatch, dispatch
- OpenAI connector
  dispatch
- xAI connector
  dispatch
- Gemini connector
  dispatch
- Aggregator connector
  dispatch
- Append-only ledger
  true-up, true-up

← Back to group: Budgets ← Stage 4

LEVEL 4

Budget alerts

Policy | Stage 4: Policy engine

Budget alerts

75, 90, 100 percent

key k_41 day window 90% ($9.00 of $10.00)
alert level=90 sent once per window

IN PLAIN ENGLISH

> This sends a warning when a key has used 75, 90 or 100 percent of its budget. Each warning is sent once per window.

Goes in	Threshold state from the budget store and spend events from the ledger.
Comes out	Alert events at 75, 90 and 100 percent to the admin channel and the dashboards.
Reads / writes	reads: bst percent used, ledger spend | writes: alert state per key, window and threshold
If it fails	Fail open: a lost alert never blocks a call, because the hard cap in the budget store still applies. A missed alert is retried on the next spend event.
Requirement source	RQ-C-007, RQ-C-008, RQ-C-050 Sources: A:213, C:116, C:054, C:101
Hardening, still open	Budget alerts at 75, 90 and 100 percent are sourced. Anomaly alerts have no corpus method and belong to the anomaly detector.
Not in client docs	**DevX proposal:** Alert fan-out channels and the repeat suppression window are DevX proposals.

WHAT IT IS FOR

Warn owners before a key reaches its hard cap, so they can raise the cap or stop the work.

EXPECTED

- Raises one alert per key, window and threshold at 75, 90 and 100 percent (A:213, C:116).
- Triggers on each ledger spend event, within seconds of the true-up.
- Includes key id, window, percent used, remaining dollars and the top three spenders by caller.
- Suppresses repeats of the same threshold inside the same window.
- Re-arms all thresholds when the window rolls.

NOT EXPECTED

- Does not block or throttle calls. The budget store does that.
- Does not detect odd spend patterns. The anomaly detector owns that.
- Does not carry prompt content.

ALGORITHM AND MATHS

Edge-triggered threshold check

```
fire(level) iff pct_used >= level and last_fired(window, level) is empty
```

STEPS

1. On each spend event, read pct_used from the budget store.
2. For each level in 75, 90, 100 find the highest crossed and not yet fired.
3. Write fired state, then emit the alert.
4. On window roll, clear fired state.

PARAMETERS

levels	75, 90, 100 percent (A:213, C:116)
suppression	once per window per level (DevX proposal)

SECURITY

- Alert bodies hold ids and dollar amounts only.
- Webhook targets are set by an admin role and stored in the secrets vault.
- Alert state is written before the send so a crash cannot cause a second send.

PERFORMANCE

Latency and throughput	DevX proposal: alert emitted within 30 seconds of the crossing event.
Complexity	O(1) per spend event.
At 10x load	Consume the spend stream partitioned by key id.

EDGE CASES AND HOW EACH IS HANDLED

- Spend jumps from 70 to 100 percent in one call: fire the 75, 90 and 100 alerts together.
- Budget is raised after the 90 alert: levels are recomputed against the new cap and re-armed if below.
- Webhook down: retry with backoff and keep the alert in the dashboard.
- Two windows cross at once: send one alert per window.
- Key deleted: pending alerts are dropped.

TEST CASES

TEST	GIVEN	EXPECT
fires at 75	cap $10, spend 7.50	one alert level=75
no repeat	spend moves 7.50 to 7.80	no new alert
jump	spend moves 6.00 to 10.00	alerts 75, 90, 100 together
window roll	new day window starts	fired state cleared, 75 can fire again
raise cap	cap raised $10 to $20 at 9.00 spend	pct_used 45, 90 and 100 re-armed

WHERE IT SAVES COST

- Early warning lets owners act before the hard cap stops work.
- Budgets sized near P95 per user raise few false alerts (C:054).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Alert acknowledgements reduce repeat nagging in the dashboard.
- Cap raises after alerts feed budget sizing reviews.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.alert.fired`	key_id · window · level · pct_used	A threshold is crossed.

REFERENCES

- A:213 Overview of request-based billing (legacy)
- C:116 GitHub Copilot premium requests (billing concepts)
- C:054 The Coding Agent Economy
- C:101 ChatGPT Work: usage and cost

RECEIVES FROM

Budget store
threshold

Append-only ledger
spend

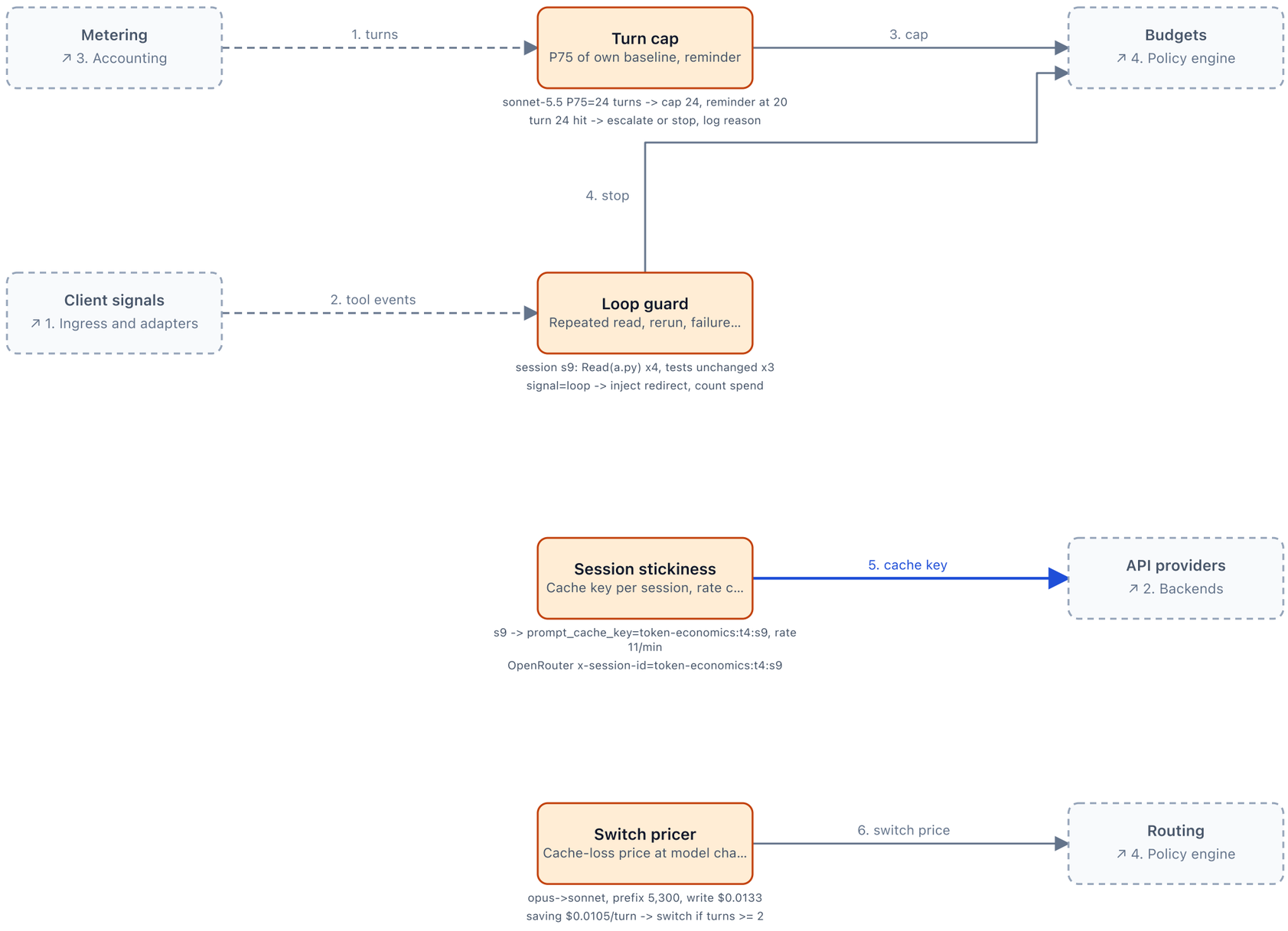
HANDS TO

None in this stage view.

← Back to group: Budgets ← Stage 4

LEVEL 3 **Run control** in Policy engine

Turn cap, loop guard, session stickiness and switch pricing keep a run from burning money.



Policy — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Metering (group) → Turn cap
Turns per task feed the per-model baseline.

2. Client signals (group) → Loop guard
Repeated reads and failures from hook events, when present.

3. Turn cap → Budgets (group)
Turn cap and remaining-turns reminder.

4. Loop guard → Budgets (group)
Loop signal ends or interrupts the run.

5. Session stickiness → API providers (group)
prompt_cache_key per session, cap near 15 requests per minute.

6. Switch pricer → Routing (group)
Cache-loss price before any mid-session model change.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Policy engine ← Budgets Routing →

LEVEL 4

Turn cap

Policy | Stage 4: Policy engine

Turn cap

P75 of own baseline, reminder

sonnet-5.5 P75=24 turns -> cap 24, reminder at 20
turn 24 hit -> escalate or stop, log reason

IN PLAIN ENGLISH

> This stops a task from running too many turns. The limit comes from how long past tasks on the same model took, and the agent gets a warning before it hits it.

Goes in	Turn counts per finished task from the ledger, the model id, and the current turn index of a live task.
Comes out	A per-model turn cap, a remaining-turns reminder, and a cap-hit signal to the reserve component.
Reads / writes	reads: ledger turns per task and model | writes: per-model turn baseline and percentile table
If it fails	Fail open: if the baseline is missing the cap is skipped and the budget cap still applies. A wrong turn cap can cut valid work, so a failure must not guess a low value.
Requirement source	RQ-C-009, RQ-C-010, RQ-C-011, RQ-C-051 Sources: C:025, C:028, C:131, C:132, C:080, C:094, C:136
Hardening, still open	C:025 ran with caching disabled, one run, 100 SWE-bench Verified tasks. The effect with caching on is not in the corpus (experiment E6). C:132 shows a larger cap does not help a capability-limited agent.
Not in client docs	**DevX proposal:** The minimum sample size per model before a cap activates, and the refresh interval, are DevX proposals.

WHAT IT IS FOR

Bound per-task turns at a percentile of the model's own history, so runaway tasks stop while normal tasks finish.

EXPECTED

- Starts the cap at the 75th percentile (P75) of the model's own baseline turns, never at P25 (C:025).
- Keeps one baseline per model, because baselines differ across models (C:025).
- Sends a remaining-turns reminder to the agent. C:025 reports solve rate fell to 24 percent on Sonnet at P25 without it.
- On cap hit, runs the fallback: escalate to the strong tier or stop with a reason.
- Passes --max-turns on headless Claude runs (C:080, C:094).

NOT EXPECTED

- Does not use P25 or any cap below P50 as a default.
- Does not rely on the agent to honour the cap. The gateway counts turns (C:136).
- Does not apply one global cap across models.
- Does not read prompt content.

ALGORITHM AND MATHS

Per-model percentile cap with reminder

```
cap(model) = P75(turns of completed tasks for model); reminder when turn >= cap − k
```

STEPS

1. Collect turns per completed task per model from the ledger.
2. Compute P75 once at least n tasks exist for the model.
3. At turn index i inject a remaining-turns reminder when i >= cap - k.
4. At i = cap run the fallback and log the cap, model and reason.
5. Recompute the baseline on a schedule and compare to the last value.

PARAMETERS

percentile	75th, never 25th (C:025)
reminder	required (C:025)
k	DevX proposal: 4 turns before the cap
min_sample	DevX proposal: 30 tasks per model

THE MATHS IN FULL

SYMBOLS

- T_m: set of turns-per-task for completed tasks on model m.
- P75_m: 75th percentile of T_m.
- k: reminder lead in turns.
- cost_base: mean task cost with no cap.
- solve_base: solve rate with no cap.

FORMULA

```
cap_m = P75(T_m); cost_saving = (cost_base − cost_capped)/cost_base; accept iff solve_capped >= solve_base − margin
```

WHY IT IS RIGHT

A percentile cap cuts the right tail of the turn distribution, where failed runs sit (17 percent of tasks, 32 percent of spend, C:028). At P75 about one task in four hits the cap. Lower caps cut more valid work, so solve rate falls.

WORKED EXAMPLE

```
C:025 at P75 with reminder: Sonnet cost −23.62 percent, solve 75 to 71 percent; Gemini cost −68.28 percent, solve 63 to 65 percent; GPT-4.1 cost −38.43 percent, solve 62 to 57 percent. On a $100 baseline Sonnet falls to $76.38. At P25 Gemini solve fell to 26 percent with 63 empty patches. A dynamic cap of 25 to 50 turns saved 5.95 to 24.30 percent versus a fixed 50.
```

COST OF COMPUTING IT

O(n log n) to compute the percentile over n tasks, once per refresh. O(1) per turn.

HOW IT CAN GO WRONG, AND THE GUARD

- Small sample gives a noisy P75, so the cap is too tight. The minimum sample gate prevents activation.
- Model change in the baseline period shifts turns. Key the baseline on the exact model id.
- A harder task mix than the baseline hits the cap often. Watch the cap-hit rate.
- Agent ignores the reminder. The hard stop still applies.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Compare cap-hit rate to 25 percent expected at P75. Run the E6 A/B with caching on: no cap, P75 with reminder, prompt-only cap. Track solve rate, cost and empty-patch rate.

SECURITY

- Turn counts are metadata. No content is stored.
- Reminder text is a fixed template from the policy store, so a tenant cannot inject text into the agent.
- Cap changes go through the policy-as-code store with review and an audit row.

PERFORMANCE

Latency and throughput	DevX proposal: under 1 ms per turn check.
Complexity	O(1) per turn after the baseline is cached.
At 10x load	Baseline table is small, one row per model, cached in memory per worker.

EDGE CASES AND HOW EACH IS HANDLED

- Task resumed from a previous session: count turns from the original task id, not the resume.
- Subagent turns: count under the parent task unless the policy sets a separate cap.
- Model changes mid-task: the cap of the model that started the task applies.
- No baseline yet for a new model: skip the cap and rely on budget.
- Tasks with a legitimate long run: tenant override through policy with an audit row.

TEST CASES

TEST	GIVEN	EXPECT
P75 compute	turns [8,10,12,14,16,20,24,30] for a model	cap equals the 75th percentile of that list
reminder	cap 24, k 4, turn 20	reminder injected
cap hit	turn 24 reached	fallback runs and the reason is logged
no baseline	new model, 5 tasks, min_sample 30	no cap applied
reject P25	policy sets percentile 25	policy validation rejects it

WHERE IT SAVES COST

- Cuts the failed-run tail: Sonnet -23.62 percent cost at 75 to 71 percent solve (C:025).
- Dynamic caps from 25 to 50 turns saved 5.95 to 24.30 percent versus fixed 50 (C:025).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Cap-hit rate and solve rate after cap feed the rollback triggers.
- Empty-patch rate after cap is a guard metric.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.tcap.cap_set`	model · percentile · cap · sample_n	A baseline is refreshed.
`token_economics.tcap.hit`	task_id · model · cap · outcome	A task reaches its cap.

RESEARCH BEHIND IT

C:025 An Empirical Study of Turn-Control Strategies for Efficient Coding Agents
Result we rely on: Turn cap at the 75th percentile with a reminder cuts cost with a small solve loss.
Used here: Sets percentile, reminder and per-model baseline.

C:132 Credit-Budgeted ICPC-Style Coding: When Agents Must Pay for Every Decision
Result we rely on: More budget does not help a capability-limited agent.
Used here: Supports a cap and an escalate fallback.

HOW OTHERS BUILD IT

Claude Code offers --max-turns for headless runs. Token Economics adds a data-driven cap per model and enforces it at the gateway for harnesses with no flag.

REFERENCES

- C:025 An Empirical Study of Turn-Control Strategies for Efficient Coding Agents
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:131 Budget-Aware Tool-Use Enables Effective Agent Scaling
- C:132 Credit-Budgeted ICPC-Style Coding: When Agents Must Pay for Every Decision
- C:080 Best practices for Claude Code
- C:094 Run Claude Code programmatically
- C:136 Coding agents ignore spend (Ramp Labs)

RECEIVES FROM

Append-only ledger
turns, turns

HANDS TO

Reserve and reconcile
cap

← Back to group: Run control ← Stage 4

LEVEL 4

Loop guard

Policy | Stage 4: Policy engine

Loop guard

Repeated read, rerun, failure signature

session s9: Read(a.py) x4, tests unchanged x3
signal=loop -> inject redirect, count spend

IN PLAIN ENGLISH

> This spots an agent that keeps doing the same thing, like reading one file again and again. It tells the agent to change course or stops the run.

Goes in	Request and tool-result signatures from the gateway, and tool events from the hook receiver when present.
Comes out	A stop or redirect signal to the reserve component, and a spend-after-first-failure metric.
Reads / writes	reads: recent signatures per session, hook events | writes: loop counters per session, flagged-run log
If it fails	Fail open: a missed detection leaves the budget cap and turn cap in place. A false stop of good work is the greater harm, so the guard redirects before it terminates.
Requirement source	RQ-C-012, RQ-C-013, RQ-C-014, RQ-C-051 Sources: C:028, A:059, C:062, A:246, C:074, C:059
Hardening, still open	R6 cites the 79 to 98 percent waste study as C059 and R3 cites it as R:059. The corpus file A:059 is that paper, so A:059 is used. Thresholds are not in the corpus.
Not in client docs	**DevX proposal:** Signature definitions, repeat counts and window sizes are DevX proposals. The corpus names the three signals but not the thresholds.

WHAT IT IS FOR

Cut spend on runs stuck in repeated reads, unchanged test reruns and repeated failures.

EXPECTED

- Detects repeated reads of the same target, unchanged test reruns and repeated failure signatures (A:059, C:028, C:062).
- Redirects first with an injected hint, then stops if the pattern continues (C:028).
- Logs spend after the first repeated failure. Baseline: 51 percent of spend (C:028).
- Works from gateway-level request signatures, so it holds when hooks are disabled (A:234).
- Takes extra signal from hook events when they exist (C:074).

NOT EXPECTED

- Does not depend on hooks alone, because developers can disable them.
- Does not stop a task that is making progress. Progress resets the counters.
- Does not read or store file content, only hashes.

ALGORITHM AND MATHS

Signature counters with redirect then stop

```
count(sig) over a window; redirect when count >= r1; stop when count >= r2 after a redirect
```

STEPS

1. Hash each tool call and result into a signature: (tool, target, result_hash).
2. Keep a sliding window of signatures per session.
3. Mark a repeated failure when the same failure signature recurs.
4. Mark an unchanged rerun when the test result hash is equal to the previous one.
5. At r1 repeats inject a redirect hint. At r2 repeats after that, return a stop signal.
6. Record the spend since the first repeated failure.

PARAMETERS

r1	DevX proposal: 3 repeats
r2	DevX proposal: 5 repeats
window	DevX proposal: last 20 calls
signals	repeated read, unchanged test rerun, repeated failure (A:059, C:028)

SECURITY

- Only hashes of content are kept, never the content.
- The redirect hint is a fixed template. Tool output is never copied into it.
- Loop state is keyed by tenant and session so one tenant cannot read another's counters.

PERFORMANCE

Latency and throughput	DevX proposal: under 1 ms per request, in-memory counters.
Complexity	O(1) per request with a bounded window.
At 10x load	Counters live per session in a small cache with expiry.

EDGE CASES AND HOW EACH IS HANDLED

- A legitimate retry after a transient error: a changed result hash or a time gap resets the counter.
- The same file is read after an edit: the edit changes the file hash, so it is not a repeat.
- Harness has no tool metadata at the gateway: use request-body hash only and a higher threshold.
- Hooks disabled: gateway signatures carry the load.
- Parallel tool calls with one message id: count once.

TEST CASES

TEST	GIVEN	EXPECT
repeat read	same Read target 4 times, no edits	redirect at r1=3
unchanged rerun	test result hash equal 3 times	loop flagged
progress resets	read x2 then edit then read	counter reset to 1
stop after redirect	5 repeats after a redirect	stop signal sent to bres
hooks off	no hook events	gateway signatures still detect

WHERE IT SAVES COST

- Targets failed runs: 17 percent of tasks and 32 percent of spend, with 51 percent of spend after the first repeated failure (C:028).
- TokenEconomizer reports -18.4 to -30.9 percent cost including controller overhead (C:028, self-reported).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Redirect outcomes (run recovered or stopped) feed threshold review.
- False stops raised by users feed the rollback triggers.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.loop.redirected`	session_id · signal · count	A redirect is injected.
`token_economics.loop.stopped`	session_id · signal · spend_after_first_failure_usd	A run is stopped.

RESEARCH BEHIND IT

A:059 Analyzing and Mitigating Cost-Inefficient Behaviors in Coding Agents
Result we rely on: Repeated retrieval and unchanged reruns occur in 79 to 98 percent of tasks and cost 6.86 to 22.75 percent.
Used here: Defines the three signals.

C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
Result we rely on: 51 percent of spend comes after the first repeated failure; redirect before terminating.
Used here: Sets redirect-then-stop.

HOW OTHERS BUILD IT

Gateways such as the one in C:062 describe a rate-based loop breaker. Token Economics adds a redirect step before the stop.

REFERENCES

- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- A:059 Analyzing and Mitigating Cost-Inefficient Behaviors in Coding Agents
- C:062 AI Agent Cost Optimization: Token FinOps for the Age of Runaway Agent Spend
- A:246 AI Gateway vs. OpenAI API (Vercel)
- C:074 Hooks reference
- C:059 Gemini API Pricing and Quotas: Complete 2026 Guide

RECEIVES FROM

Hook receiver
tool events, tool events

HANDS TO

Reserve and reconcile
stop

← Back to group: Run control ← Stage 4

LEVEL 4

Session stickiness

Policy | Stage 4: Policy engine

Session stickiness

Cache key per session, rate cap

s9 -> prompt_cache_key=token-economics:t4:s9, rate 11/min
OpenRouter x-session-id=token-economics:t4:s9

IN PLAIN ENGLISH

> This gives every session one fixed label so the provider sends its requests to the same cache. It also slows a session that sends too many requests per minute.

Goes in	Session id and provider from the gateway, plus the request rate per prefix.
Comes out	A per-session affinity key (prompt_cache_key, x-grok-conv-id or session_id) and a rate-cap decision.
Reads / writes	reads: session table, rate counters | writes: session to key and endpoint map, rate counters
If it fails	Fail open: with no key the request still goes through, only the cache hit rate may fall. Blocking a call over a missing affinity key would cost more than a miss.
Requirement source	RQ-C-015, RQ-C-016, RQ-C-017 Sources: A:202, A:148, A:221, A:231, A:247, A:149, C:055, A:204
Hardening, still open	The 15 requests per minute figure is self-reported by OpenAI (A:202, A:148). Azure session affinity does not guarantee hits (A:149). Whether our callers reach the overflow is not known (R1 section 5).
Not in client docs	**DevX proposal:** The key format, the session timeout and the overflow action are DevX proposals.

WHAT IT IS FOR

Keep each session on one provider endpoint and one cache key so repeated prefixes hit the cache.

EXPECTED

- Sets prompt_cache_key on OpenAI Responses, x-grok-conv-id on xAI and session_id or x-session-id on OpenRouter (A:202, A:221, A:231).
- Keeps one provider endpoint per session. Switching provider loses the cache (A:231, A:247).
- Caps each key near 15 requests per minute per prefix on OpenAI, then spreads over a second key suffix (A:202, A:148).
- Truncates session_id to at most 256 characters on OpenRouter (A:231).
- Expires the session map after 60 minutes of silence, matching OpenRouter sticky expiry (A:231).

NOT EXPECTED

- Does not change the model, effort or tool set. The switch pricer guards those.
- Does not guarantee a hit. Azure affinity may still miss (A:149).
- Does not put user content into the key.

ALGORITHM AND MATHS

Session-scoped affinity key with rate cap

```
key = hash(tenant, session_id, prefix_id); route to endpoint(key); if rate(key) > 15/min then key' = key + shard
```

STEPS

1. Look up the session in the map. Create it with a stable key if absent.
2. Pin the endpoint chosen for the first call.
3. Attach the provider-specific header or field.
4. Count requests per key per minute.
5. Over the cap, add a shard suffix so overflow does not land on the same machine.
6. Expire the entry after the idle timeout.

PARAMETERS

rate_cap	about 15 requests per minute per prefix and key on OpenAI (A:202, A:148)
session_id_max	256 characters (A:231)
idle_expiry	60 minutes (A:231)

SECURITY

- The key is a hash of tenant and session, so one tenant cannot guess or reuse another's cache key.
- Caches are isolated per org and workspace at the provider (A:199). The key never crosses tenants.
- No user text goes into any header value.

PERFORMANCE

Latency and throughput	DevX proposal: under 1 ms added per request.
Complexity	O(1) map lookup and counter update.
At 10x load	Session map in a shared store with time to live (TTL) equal to the idle expiry.

EDGE CASES AND HOW EACH IS HANDLED

- Two harness windows share one session id: they share one key and may hit the rate cap.
- Provider endpoint is down: fail over through the priced fallback chain and accept the cache loss.
- Session id longer than 256 characters: hash it down.
- Provider ignores the key (Azure): metrics show a low hit rate and the cache hit monitor flags it.
- Rate over 15 per minute: sharding lowers per-machine overflow but splits the cache.

TEST CASES

TEST	GIVEN	EXPECT
openai key	session s9, tenant t4, OpenAI route	prompt_cache_key set and stable across calls
xai header	xAI route	x-grok-conv-id equals the session key
rate cap	20 requests in one minute on one key	requests above 15 get a shard suffix
long id	session id of 400 characters	key hashed to 256 characters or fewer
idle expiry	no call for 61 minutes	map entry removed, new key on next call

WHERE IT SAVES COST

- Hit rates reported with a key: 60 to 87 percent on OpenAI (A:202).
- Time at the top of a prompt gave 87 versus 3 percent hits and 2.2 to 2.9x cost (A:224).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Per-session hit rate from the hit monitor.
- Overflow count per key.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.stick.key_set`	session_id · provider · key_hash	A key is attached.
`token_economics.stick.rate_capped`	key_hash · rate_per_min	A key exceeds the rate cap.

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- OpenRouter: session_id sticky routing with a 60-minute expiry.

HOW OTHERS BUILD IT

OpenRouter and Vercel route by session. OpenAI hashes the first 256 tokens plus prompt_cache_key. Token Economics sets a stable key per session.

REFERENCES

- A:202 Prompt Caching 201
- A:148 Prompt caching with Azure OpenAI in Azure AI Foundry Models
- A:221 Usage & Pricing (xAI prompt caching)
- A:231 Prompt caching best practices (OpenRouter)
- A:247 AI Gateway Automatic Prompt Caching (Vercel)
- A:149 Model router for Microsoft Foundry concepts
- C:055 How Claude Code uses prompt caching
- A:204 Prompt cache diagnostics

RECEIVES FROM

None in this stage view.

HANDS TO

OpenAI connector
cache key, cache key

← Back to group: Run control ← Stage 4

LEVEL 4

Switch pricer

Policy | Stage 4: Policy engine

Switch pricer

Cache-loss price at model change

opus->sonnet, prefix 5,300, write $0.0133
saving $0.0105/turn -> switch if turns >= 2

IN PLAIN ENGLISH

> Changing models throws away the saved prompt cache, which costs money. This component adds up that cost and allows a switch only when the saving is larger.

Goes in	Current model, candidate model, cached prefix size, time since last cache write, remaining turns estimate, and prices.
Comes out	A switch price in dollars and a go or stay verdict with a logged reason.
Reads / writes	reads: price book rows, session prefix size and write time | writes: switch decision log
If it fails	Fail closed to stay: if the price cannot be computed, the session stays on its current model. A blind switch risks a cold-cache bill for the whole context.
Requirement source	RQ-C-016, RQ-C-017, RQ-C-018, RQ-C-019, RQ-C-020, RQ-C-028, RQ-C-045 Sources: A:192, C:030, A:204, C:055, C:086, A:282, A:008, A:228, C:052
Hardening, still open	The measured cost of a model switch on a real agent loop is not in the corpus. Whether effort change keeps the cache differs by model: kept on Opus 5.5, Sonnet 5.5, Haiku 5.5 and Fable 5.1, lost on most others (R1).
Not in client docs	**DevX proposal:** The remaining-turns estimate is a DevX proposal. The corpus gives no measured cost of a real switch (experiment E2).

WHAT IT IS FOR

Price the cache loss of any mid-session change of model, effort, speed or tool set, and block changes that do not pay back.

EXPECTED

- Treats model, effort, fast mode and tool set as cache keys (C:055, C:086, A:204, A:282).
- Computes the incoming model's cache write cost on the current prefix (C:030).
- Allows a switch only when expected saving over remaining turns exceeds that cost, and logs the reason (A:192).
- Prefers an effort change to a model switch where the cache survives it (A:228, A:282).
- Stays on the current model after an error by default (A:008).

NOT EXPECTED

- Does not switch on a hunch or on a single failed turn.
- Does not treat fast mode as free: turning it on mid-conversation bills the whole context uncached (C:086).
- Does not price subagents as if they read the parent cache. They do not unless forked (C:052).

ALGORITHM AND MATHS

Cache-loss break-even

```
switch iff T x (dRead + dOut) > P x W_in
```

STEPS

1. Read P, the cached prefix tokens, and the time since the last write.
2. If the cache is already cold (past TTL), the loss is zero. Compare fresh costs.
3. Else cost_switch = P x W_in, the incoming write rate on the prefix.
4. Per-turn saving = P x (R_out - R_in) + O x (price_out_cur - price_out_new), with O expected output tokens per turn.
5. Estimate remaining turns T. Switch iff T x saving > cost_switch.
6. Log reason code and inputs.

PARAMETERS

prefix_median	about 5,300 tokens (C:030 median)
write_rate	1.25x input for 5m, 2x for 1h (C:052)
effort_cache	kept on Opus 5.5, Sonnet 5.5, Haiku 5.5, Fable 5.1 (R1)

THE MATHS IN FULL

SYMBOLS

- P: cached prefix size in tokens.
- W_in: incoming model write rate per MTok for the chosen TTL.
- R_cur, R_in: cache read rates of current and incoming model.
- O: expected output tokens per turn.
- T: expected remaining turns (DevX proposal estimate).
- price_out_cur, price_out_in: output rates per MTok.

FORMULA

```
cost_switch = P x W_in / 1e6; saving_per_turn = (P x (R_cur − R_in) + O x (price_out_cur − price_out_in)) / 1e6; switch iff T x saving_per_turn > cost_switch and cache is warm; if cache is cold cost_switch = 0
```

WHY IT IS RIGHT

A switch pays one write of the whole prefix at the incoming rate. After that each turn reads at the incoming rate. Staying pays the current read rate each turn. The difference per turn, times turns left, must beat the one-time write.

WORKED EXAMPLE

```
Opus 5.5 to Sonnet 5.5 (prices from R1): Opus read $0.20, output $20; Sonnet read $0.10, 5m write $2.50, output $10 per MTok. P 5,300 (C:030 median). O 1,000 (illustrative). cost_switch = 5,300 x 2.50/1e6 = $0.01325. Per-turn saving = 5,300 x 0.10/1e6 + 1,000 x 10/1e6 = 0.00053 + 0.01 = $0.01053. Break-even T = 0.01325 / 0.01053 = 1.26, so a switch pays from turn 2.
```

COST OF COMPUTING IT

O(1) per decision.

HOW IT CAN GO WRONG, AND THE GUARD

- Wrong T: a short task makes the switch a loss. Use a conservative T.
- Prefix size drifts as the session grows, so the loss grows. Recompute at each decision.
- Provider silently changes cache behaviour on effort. Verify per provider with a probe.
- Output tokens per turn are underestimated and the saving is understated.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Run E2: force a mid-session switch at several prefix sizes and time gaps and compare billed cost to this estimate. Fit T from per-model turn baselines.

SECURITY

- Decision log holds model ids, token counts and dollars only.
- Switch overrides by a user need an admin role and are audited.
- Fast mode and effort headers are validated against an allow list before forwarding.

PERFORMANCE

Latency and throughput	DevX proposal: under 1 ms per decision.
Complexity	O(1).
At 10x load	Stateless. Reads price rows from a cached copy of the price book.

EDGE CASES AND HOW EACH IS HANDLED

- Cache already expired: switch cost is zero, so the switch is allowed on price alone.
- Switch at a session start: no cache to lose.
- Tool set changes because an MCP server was added: priced like a model switch.
- Fork or subagent: new cache anyway, so cost is a fresh write on either model.
- Incoming model has a higher minimum prefix than P: no caching, so compare fresh input costs.

TEST CASES

TEST	GIVEN	EXPECT
pays back	P 5,300, Opus 5.5 to Sonnet 5.5, O 1,000, T 3	switch allowed, reason logged
too short	same inputs, T 1	stay, saving 0.01053 under cost 0.01325
cold cache	last write older than TTL	cost_switch 0, switch decided on price
fast mode	fast on mid-conversation, P 100,000	priced as a full uncached input, likely stay
error retry	tool error on turn 4	stay on the same model by default

WHERE IT SAVES COST

- Avoids full-context uncached bills after a switch (C:086).
- Effort change over model switch where the cache survives (A:228, A:282).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Realised cost after each switch for comparison with the estimate.
- Switch count per session.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.swp.priced`	session_id · from_model · to_model · cost_switch_usd · saving_per_turn_usd · verdict	A switch is evaluated.

RESEARCH BEHIND IT

C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
Result we rely on: Mid-trajectory switches must be priced at the incoming tier's cache-write rate.
Used here: Defines cost_switch.

A:008 MTRouter: Cost-Aware Multi-Turn LLM Routing with History–Model Joint Embeddings
Result we rely on: Agents retained the same model after an error 90.2 and 80.9 percent of the time.
Used here: Default is to stay after an error.

HOW OTHERS BUILD IT

Cloudflare Auto Router switches only when the benefit exceeds the cache-loss cost and keys by session id (A:192). OpenRouter prefers effort change to model switch (A:228).

REFERENCES

- A:192 Auto Router
- C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
- A:204 Prompt cache diagnostics
- C:055 How Claude Code uses prompt caching
- C:086 Speed up responses with fast mode
- A:282 Effort parameter (Claude)
- A:008 MTRouter: Cost-Aware Multi-Turn LLM Routing with History–Model Joint Embeddings
- A:228 Auto Router: Smart AI Model Selection
- C:052 Your cache hit rate is lying to you. It's the caller mix.

RECEIVES FROM

None in this stage view.

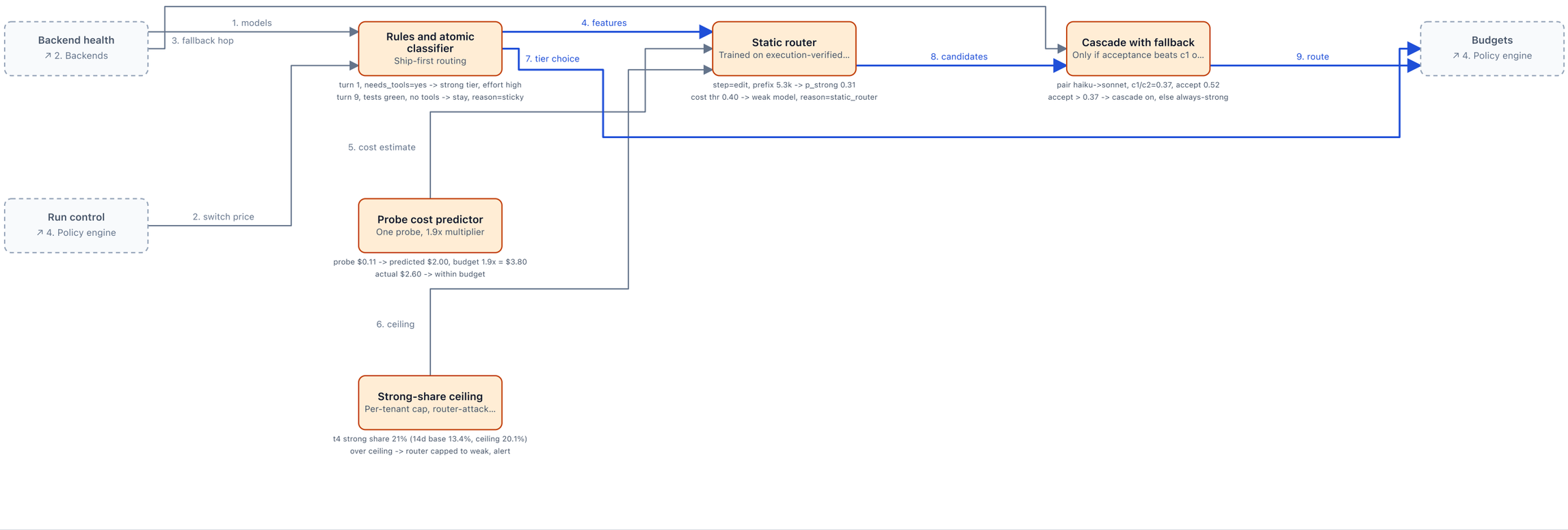
HANDS TO

Rules and atomic classifier
switch price

← Back to group: Run control ← Stage 4

LEVEL 3 **Routing** in Policy engine

Rules first, a static router second, cascade only with measured acceptance.



Policy — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Backend health (group) → Rules and atomic classifier
Exact model ids, tiers and limits for the router.

2. Run control (group) → Rules and atomic classifier
Cache-loss price before any mid-session model change.

3. Backend health (group) → Cascade with fallback
Fallback is a priced hop, never silent.

4. Rules and atomic classifier → Static router
Atomic yes or no signals and request features.

5. Probe cost predictor → Static router
One probe call, 1.9x multiplier.

6. Strong-share ceiling → Static router
Strong-model share limit per tenant.

7. Rules and atomic classifier → Budgets (group)
Model chosen by rules when no router is active.

8. Static router → Cascade with fallback
Router scores and cost threshold.

9. Cascade with fallback → Budgets (group)
Chosen model and escalation path.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Policy engine ← Run control Tier and cache →

LEVEL 4

Rules and atomic classifier

Policy | Stage 4: Policy engine

Rules and atomic classifier

Ship-first routing

turn 1, needs_tools=yes -> strong tier, effort high
turn 9, tests green, no tools -> stay, reason=sticky

IN PLAIN ENGLISH

> This picks the model with simple rules and a few yes or no questions, such as whether the task needs tools. It starts with one good default model and only moves on clear signs.

Goes in	Request features, session id, turn index, current model, tool flags, remaining budget, previous outcome signals, and the model catalogue.
Comes out	A model and effort choice for the request, atomic yes or no signals for the router, and a logged reason.
Reads / writes	reads: catalogue (cat), switch price (swp), budget remaining | writes: routing reason log
If it fails	Fail open to the configured default model: if the rules fail, the request goes to the Best Single model. A router fault must not stop work, and the budget cap still applies.
Requirement source	RQ-C-018, RQ-C-019, RQ-C-020, RQ-C-021, RQ-C-022, RQ-C-023, RQ-C-024, RQ-C-026, RQ-C-027, RQ-C-028, RQ-C-047, RQ-C-049, RQ-C-051 Sources: A:052, A:018, A:245, C:031, A:228, A:282, A:006, A:002, A:001, C:026, C:030, A:019, A:008, C:055
Hardening, still open	Atomic classifier agreement fell from 0.742 on its own test set to 0.60 on blind Opus labels (A:018, n=100). A:052 used 50 tasks and one run. The feature set on interactive CLI sessions is not in the corpus. Ladder from R2: ship first = this component; ship second = srt, casc, probe; third = online budgeted routing and bandits; never = prompt-only budget limits, frontier model as router, one-model logs, preference-only labels, sum log-probability score, switching without cache-loss pricing, large pools, unpriced spend.
Not in client docs	**DevX proposal:** The rule order and the atomic question list are DevX proposals shaped by A:018 and A:052.

WHAT IT IS FOR

Choose the model and effort for each request with plain rules and atomic questions. This is the ship-first step of the R2 ladder and needs no training data.

EXPECTED

- Ship first (R2): a best-single-model baseline, then rules plus atomic yes or no questions at session start or tier boundaries (A:052, A:018, A:245, C:031).
- Routes tool-bearing requests and failed verifiable checks to the strong tier (C:031, A:245).
- Prefers a change of effort to a change of model, after checking the provider's cache rule (A:228, A:282, C:055).
- Asks the switch pricer before any mid-session model change and stays after an error by default (A:008).
- Logs reason, features, router latency and router cost on every request (C:026).

NOT EXPECTED

- Never uses a broad single-question classifier or a regex-only rule: regex lost 23.2 points held out (A:052).
- Never prompts a frontier model as the router. A prompted Opus 4.6 router failed all 40 trajectories (C:030).
- Never reports a saving against a random router (A:001). It reports against Best Single and an oracle (A:006, A:002).
- Never relies on prompt-only budget limits (C:136).

ALGORITHM AND MATHS

Rule list with atomic questions

```
model = first rule in ordered list whose conditions hold; default = Best Single
```

STEPS

1. Read the catalogue for allowed models and tiers.
2. Rule 1: session already has a model and no trigger, so stay (stickiness).
3. Rule 2: tool-bearing or failed verifiable check, so strong tier.
4. Rule 3: atomic questions at a boundary (needs_tools, needs_vision, needs_long_context, stakes, difficulty, follow_up from A:018) each yield yes or no.
5. Rule 4: pick the cheapest model that satisfies all yes answers.
6. Rule 5: ask the switch pricer. If it says stay, keep the current model.
7. Log the reason code. Pass features to srt when a router is active.

PARAMETERS

atomic_questions	needs_tools, needs_vision, needs_long_context, stakes, difficulty, follow_up (A:018)
default	Best Single model per tenant (A:006)
blind_label_check	re-measure against blind labels (A:018 fell 0.742 to 0.60)

SECURITY

- Rules read request features, never raw prompt text, unless the tenant policy allows a content-based question.
- A user prompt cannot set a model name in the rule input. Model names come only from the catalogue.
- Rule changes go through the policy-as-code store with review and audit.

PERFORMANCE

Latency and throughput	DevX proposal: under 2 ms for rules, under 20 ms with atomic classifier calls. A:018 reports a 14 ms CPU classifier head.
Complexity	O(r) for r rules.
At 10x load	Stateless. Rule sets cached per tenant.

EDGE CASES AND HOW EACH IS HANDLED

- Catalogue lists a model with no price row: excluded.
- Two rules disagree: first match wins and the order is versioned.
- Classifier unavailable: fall back to the rule-only path and then to Best Single.
- Session already mid-task on a model with a warm cache: stay unless rule 2 fires.
- Harness cannot rewrite the model field (no modelRewrite flag): rules log a recommendation only.

TEST CASES

TEST	GIVEN	EXPECT
tools to strong	needs_tools=yes at turn 1	strong tier chosen, reason=tool_bearing
sticky	turn 9, no trigger	current model kept, reason=sticky
effort first	provider keeps cache on effort change	effort raised, model unchanged
regex only	config uses a regex-only classifier	policy validation rejects it
no price row	candidate model lacks a price row	candidate excluded

WHERE IT SAVES COST

- Task routing on 21 tasks gave -44.7 percent cost and -4.8 percent mean quality, with a quality floor of 65 (A:051, self-reported).
- 70 to 80 percent of Claude Code turns are tool selection, so only 20 to 30 percent are eligible for a cheaper tier (C:031).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Verified outcome per request (tests pass, schema valid) for label building.
- Blind-label agreement checks.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.rules.routed	request_id · model · effort · reason · router_latency_ms	A route is chosen.

RESEARCH BEHIND IT

A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels
Result we rely on: Atomic yes or no questions work; one broad question collapses. Regex lost 23.2 points held out.
Used here: Defines the classifier shape.

A:018 s1-llm-auto-router
Result we rely on: A 7-head classifier agreed 0.742 on its own set and 0.60 on blind labels.
Used here: Sets the question list and the blind check.

A:006 LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing
Result we rely on: Most routers tie; use a Best Single baseline.
Used here: Sets the default.

HOW OTHERS BUILD IT

Vercel and OpenRouter ship rule-like auto routing with session stickiness. Their accept threshold of 0.85 is a placeholder (A:245).

REFERENCES

- A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels
- A:018 s1-llm-auto-router
- A:245 Cost-aware model routing with Vercel AI Gateway
- C:031 Response-Level Speculative Decoding for Cost-Efficient LLM API Serving
- A:228 Auto Router: Smart AI Model Selection
- A:282 Effort parameter (Claude)
- A:006 LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing
- A:002 RouterBench: A Benchmark for Multi-LLM Routing System
- A:001 RouteLLM: Learning to Route LLMs with Preference Data
- C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
- C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
- A:019 Zen Router 0.6B
- A:008 MTRouter: Cost-Aware Multi-Turn LLM Routing with History–Model Joint Embeddings
- C:055 How Claude Code uses prompt caching

RECEIVES FROM

Model catalogue
models, models

Switch pricer
switch price

HANDS TO

Static router
features

Reserve and reconcile
tier choice

← Back to group: Routing ← Stage 4

LEVEL 4

Static router

Policy | Stage 4: Policy engine

Static router

Trained on execution-verified labels

step=edit, prefix 5.3k -> p_strong 0.31
cost thr 0.40 -> weak model, reason=static_router

IN PLAIN ENGLISH

> This is a small trained model that guesses, for each step, whether a cheap model is good enough. It learns from past runs where we checked the result by running the tests.

Goes in	Features from the rules component, the cost threshold, the strong-share ceiling and the probe estimate.
Comes out	Scores per candidate model and a ranked candidate list for the cascade.
Reads / writes	reads: trained router model, balanced label set, share ceiling | writes: router decision log, label records
If it fails	Fail open to the rules route: if the router errors or times out, the rules choice or Best Single is used. The router is an optimisation, so its fault must not block work.
Requirement source	RQ-C-021, RQ-C-023, RQ-C-025, RQ-C-026, RQ-C-027, RQ-C-028, RQ-C-051 Sources: C:030, A:019, A:013, A:001, A:006, A:002, A:046, C:032, C:029, A:010, A:018, A:052, A:008, C:031, A:007
Hardening, still open	C:030 (-53.1 percent, 75 vs 74 resolved) is self-reported on 100 locked SWE-bench instances with no confidence intervals. A:006 and A:046 show most routers tie or lose on public benchmarks. Savings in C:030 and C:031 use different baselines. Ladder from R2: this is ship second; third is online budgeted routing (A:007) and bandits (C:032).
Not in client docs	**DevX proposal:** Router latency budget, the label refresh cadence and the out-of-distribution (OOD) gate level are DevX proposals.

WHAT IT IS FOR

Pick the cheapest model per agent step with a static router trained on execution-verified labels. This is the ship-second step of the R2 ladder.

EXPECTED

- Ship second (R2): a step-level static router trained on execution-verified counterfactual labels (C:030, A:019).
- Needs class-balanced labels. One-model logs give a majority-class router (A:019).
- Is checked on blind labels and out-of-distribution tasks before rollout (A:018, C:032, A:052).
- Respects the per-tenant strong-share ceiling (A:010).
- Reports against Best Single and an oracle in dollars per solved task (A:006, A:002).

NOT EXPECTED

- Never uses a frontier model prompted as the router (C:030, A:228).
- Never trains on logs from one dominant model (A:019).
- Never trains on preference labels without 100 to 1,000 gold samples (A:013, A:001).
- Third rung, not here: online budgeted routing (A:007) and bandits (C:032) wait for ground truth and a drift case.
- Never uses a large model pool. Keep a small complementary set (A:006).

ALGORITHM AND MATHS

Static step router with cost threshold

```
choose weak if p_weak_ok(x) >= tau else strong; tau from the cost threshold and the error price
```

STEPS

1. Build x: step type, prefix size, previous outcome, turn index, tool flag, remaining budget (R2 section 3).
2. Score p_weak_ok with the trained classifier or k-means router (A:006 favours a cheap router).
3. Apply tau from the cost threshold and the ceiling from share.
4. Return ranked candidates to the cascade.
5. Log the decision with features and router cost.

PARAMETERS

training_labels	execution-verified, balanced; C:030 used 970 step labels
gold_samples	100 to 1,000 for preference labels (A:013)
router_cost	MF router $3.32 per M requests (A:001)
mid_switch	priced at the incoming write rate (C:030)

THE MATHS IN FULL

SYMBOLS

- x: feature vector for the step.
- p: predicted chance the weak model is good enough.
- tau: decision threshold.
- c_w, c_s: expected cost of weak and strong model on the step.
- c_r: router cost per request.

FORMULA

```
E[cost] = c_r + p_strong x c_s + (1 - p_strong) x c_w; choose weak iff p >= tau; report CPS = total spend / solved tasks
```

WHY IT IS RIGHT

The router pays its own cost plus the cost of the model it picks. It saves money only where weak is picked and still solves the step. Tau trades saving against failed steps, which cost an escalation.

WORKED EXAMPLE

```
C:030: routed $25.66 versus unrouted Opus $54.73, a cut of (54.73 - 25.66)/54.73 = 53.1 percent, with 75 of 100 resolved versus 74 of 100. Per resolved task: 25.66/75 = $0.342 versus
54.73/74 = $0.740. RouteLLM MF router cost $3.32 per M requests, at most 0.4 percent of GPT-4 generation cost (A:001).
```

COST OF COMPUTING IT

O(1) per step for a linear or small tree model. Router latency spans 11.1 to 546.8 ms across routers (A:046).

HOW IT CAN GO WRONG, AND THE GUARD

- Out-of-distribution tasks: static classifiers fell to 8.93 to 21.43 percent (C:032).
- Queries few models solve: accuracy 23 to 25 percent when at most 3 experts are right (A:006).
- Confounder gadget raises strong routing 73 to 100 percent (A:010). The share ceiling limits this.
- Label drift after a model update.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

E1 replay harness: same tasks through Best Single, each router and an oracle, with 95 percent confidence intervals over at least 5 seeds. E7 label audit and E8 OOD and adversarial tests.

SECURITY

- Router input is features and hashes, not raw prompts, unless the tenant opts in.
- Router weights are signed and loaded from the policy store.
- Share ceiling blocks router-attack cost inflation (A:010).

PERFORMANCE

Latency and throughput	DevX proposal: router p99 under 20 ms. TTFT (time to first token) impact is measured in E4 (C:031 saw 2.1x worse).
Complexity	O(1) per step.
At 10x load	Stateless inference, model cached per worker.

EDGE CASES AND HOW EACH IS HANDLED

- Router times out: use the rules route.
- Label set has one dominant class: training is blocked by the balance check.
- New model joins the pool: router retrains or the model is excluded until labelled.
- Mid-trajectory switch: priced by the switch pricer first.
- Router disagrees with a verifiable failed check: the failed check wins and goes to strong.

TEST CASES

TEST	GIVEN	EXPECT
balance gate	labels 95 percent one class	training refused
threshold	p 0.31, tau 0.40	strong model chosen
timeout	router exceeds its latency budget	rules route used, event logged
ceiling	tenant at share ceiling, p says strong	weak model plus alert
blind label	agreement on blind set falls under the gate	rollout blocked

WHERE IT SAVES COST

- -53.1 percent on locked instances at 75 vs 74 resolved (C:030, self-reported, no confidence intervals).
- LLMRouterBench: best router up to +4 percent accuracy and 31.7 percent cost cut versus best single model (A:006).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Verified outcome per step becomes a new label.
- Escalation events from the cascade.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.srt.scored`	request_id · p_weak_ok · tau · choice · router_ms	A step is scored.
`token_economics.srt.label_added`	step_id · verified_outcome	A verified label is stored.

RESEARCH BEHIND IT

C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
Result we rely on: Static router on 970 execution-verified labels cut cost 53.1 percent at equal solve.
Used here: Defines the training recipe.

A:019 Zen Router 0.6B
Result we rely on: One-model logs yield a majority-class router.
Used here: Class balance gate.

A:010 Rerouting LLM Routers
Result we rely on: A short gadget upgrades routing to the strong model.
Used here: Share ceiling.

A:006 LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing
Result we rely on: Most routers tie; use Best Single and a cheap k-means router.
Used here: Baseline and router choice.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- RouteLLM: Matrix factorisation router and gold-sample debiasing.

HOW OTHERS BUILD IT

Vendor routers (Copilot Auto about 10 percent, Kiro Auto about 23 percent) are vendor claims with no baseline (A:155, A:162).

REFERENCES

- C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
- A:019 Zen Router 0.6B
- A:013 Meta-Router: Bridging Gold-standard and Preference-based Evaluations in Large Language Model Routing
- A:001 RouteLLM: Learning to Route LLMs with Preference Data
- A:006 LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing
- A:002 RouterBench: A Benchmark for Multi-LLM Routing System
- A:046 RouterArena: An Open Platform for Comprehensive Comparison of LLM Routers
- C:032 Agentic Model Routing for Coding Tasks
- C:029 A Unified Approach to Routing and Cascading for LLMs
- A:010 Rerouting LLM Routers
- A:018 s1-llm-auto-router
- A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels
- A:008 MTRouter: Cost-Aware Multi-Turn LLM Routing with History–Model Joint Embeddings
- C:031 Response-Level Speculative Decoding for Cost-Efficient LLM API Serving
- A:007 Efficient Training-Free Online Routing for High-Volume Multi-LLM Serving

RECEIVES FROM

- Rules and atomic classifier
  features
- Probe cost predictor
  cost estimate

HANDS TO

- Cascade with fallback
  candidates

Strong-share ceiling
ceiling

← Back to group: Routing ← Stage 4

LEVEL 4

Cascade with fallback

Policy | Stage 4: Policy engine

Cascade with fallback

Only if acceptance beats c1 over c2

pair haiku->sonnet, c1/c2=0.37, accept 0.52
accept > 0.37 -> cascade on, else always-strong

IN PLAIN ENGLISH

> The cheap model tries first and the strong model steps in only if a check fails. We turn this on for a model pair only when the cheap model passes often enough to pay for itself.

Goes in	Ranked candidates and cost threshold from the router, a cheap post-hoc verifier result, and fallback hops from the fallback chain.
Comes out	A chosen model and an escalation path to the reserve component.
Reads / writes	reads: measured acceptance per pair, price rows | writes: cascade decision log, acceptance counters
If it fails	Fail open to Always-Strong: if acceptance or the verifier is unavailable, the pair is disabled and the strong model runs. This costs more but never lowers quality.
Requirement source	RQ-C-021, RQ-C-029, RQ-C-030, RQ-C-031, RQ-C-051 Sources: A:028, C:031, A:123, A:002, A:021, C:029, A:022, A:010, A:046
Hardening, still open	A:123 gives the error-price break-even as $0.01 in the abstract and $0.20 in the body; take neither and measure (E5). A:028 used multiple choice with 16 samples per tier and did not price sampling. C:029: threshold cascades fail with binary feedback; test pass or fail is post-hoc. Ladder from R2: ship second.
Not in client docs	**DevX proposal:** The acceptance window, the minimum sample before a pair is enabled, and the verifier choice per task type are DevX proposals.

WHAT IT IS FOR

Run the cheap model first and escalate on a failed post-hoc check, for pairs where measured acceptance beats the cost ratio. This is ship second in the R2 ladder.

EXPECTED

- Ship second (R2): a cascade where a cheap post-hoc verifier exists (tests, schema) and measured acceptance exceeds c1/c2 (C:029, C:031, A:028, A:123).
- Keeps an Always-Strong fallback for every pair (A:028).
- Prices every hop and judge call. Fallbacks are priced, never silent (A:021).
- Uses a length-normalised or post-hoc score, never a raw sum log-probability (A:022).
- Disables a pair when judge error exceeds 0.2, where quality collapses (A:002).

NOT EXPECTED

- Never uses a sum log-probability deferral score (A:022).
- Does not cascade above a $0.10 error price without a measured crossover (A:123).
- Does not cascade tool-bearing requests to the weak tier (C:031).
- Third rung, not here: online budgeted routing (A:007).

ALGORITHM AND MATHS

Acceptance-gated two-tier cascade

```
enable(pair) iff acceptance_measured > c1/c2; expected cost = c1 + (1 − a) x c2 + c_v
```

STEPS

1. Read the pair's measured acceptance a over a rolling window and sample size n.
2. Compute the ratio c1/c2 from the price book on the real prefix and cache state.
3. Enable only if a > c1/c2 and n >= n_min and judge error <= 0.2.
4. Send to tier 1. Run the post-hoc check (tests, schema).
5. On a failed check, escalate to tier 2 and price both hops.
6. Update the acceptance counter.

PARAMETERS

break_even	a > c1/c2; ratio 1:2.7 gives 0.37 (A:028)
judge_error_limit	0.2 (A:002)
n_min	DevX proposal; A:028 used n >= 200 for calibration
skip_break_even	13.3 percent in C:031

THE MATHS IN FULL

SYMBOLS

- c1, c2: expected cost of tier 1 and tier 2 on this task type.
- a: measured tier-1 acceptance, the share of tier-1 outputs that pass the check.
- c_v: verifier cost per attempt.
- e: error price in dollars per wrong answer (A:123).

FORMULA

```
cost_cascade = c1 + c_v + (1 − a) x c2; cost_strong = c2; cascade wins on cost iff a > (c1 + c_v)/c2; include error: add e x P(error) to both sides
```

WHY IT IS RIGHT

Every request pays tier 1. A share (1 - a) also pays tier 2. Setting cost_cascade below c2 gives a > c1/c2 when c_v is 0.

WORKED EXAMPLE

```
A:028 cost ratio 1:2.7 gives break-even acceptance 1/2.7 = 0.37. Take c1 = 1, c2 = 2.7, a = 0.52: cost = 1 + 0.48 x 2.7 = 2.30 versus 2.70, a saving of 14.8 percent. At a = 0.30: cost = 1 +
0.70 x 2.7 = 2.89, 7.0 percent worse than Always-Strong. C:031 reports a break-even skip rate of 13.3 percent and a draft accepted 88.8 percent.
```

COST OF COMPUTING IT

O(1) per request. O(n) to refresh the rolling acceptance.

HOW IT CAN GO WRONG, AND THE GUARD

- Judge error above 0.2 collapses cascade quality (A:002).
- Acceptance ranges 0 to 95.6 percent across tiers (A:028), so a pair can fall below break-even after a task mix change.
- Above a $0.10 error price one big model beats cascades (A:123).
- Sampling overhead is unpriced in A:028 and must be added.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

E5 error-price sweep from $0.01 to $1.00 per pair to find the crossover with a single strong model. Track measured acceptance weekly.

SECURITY

- Verifier runs inside the tenant sandbox. Test output is hashed, not stored.
- A user cannot force Always-Strong or cascade through the prompt. Pair state comes from policy.
- Escalation hops pass through the budget reserve like any call.

PERFORMANCE

Latency and throughput	DevX proposal: cascade overhead under 5 ms excluding provider time. Latency of two hops is reported (TTFT 2.1x worse in C:031).
Complexity	O(1) per request.
At 10x load	Acceptance counters are small and sharded by pair.

EDGE CASES AND HOW EACH IS HANDLED

- No cheap verifier for a task type: cascade stays off.
- Tier 1 passes the check but is wrong: the error price is paid. Track escape rate.
- Pair acceptance falls under c1/c2: auto-disable and alert.
- Fallback hop after a 429: priced as a hop with the provider's billing flag.
- Tier 1 output is empty: treat as a failed check.

TEST CASES

TEST	GIVEN	EXPECT
enable	c1/c2 0.37, measured a 0.52, n 300	cascade enabled
disable	a 0.30, c1/c2 0.37	pair disabled, Always-Strong used
judge error	judge error 0.25	cascade disabled for the pair
hop priced	tier 1 fails, tier 2 runs	two cost rows linked by one task id
log-prob	config uses sum log-probability score	policy validation rejects it

WHERE IT SAVES COST

- FrugalGPT reports 59.2 to 98.3 percent cheaper at matched accuracy (A:021, March 2023 prices, self-reported).
- RLM-Cascade proxy -45.8 percent on 125 requests (C:031, self-reported).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Acceptance and escape rate per pair.
- Escalation cost per task.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.casc.escalated`	task_id · pair · reason · extra_cost_usd	Tier 2 runs after a failed check.
`token_economics.casc.pair_state`	pair · acceptance · break_even · enabled	A pair is enabled or disabled.

RESEARCH BEHIND IT

A:028 Conformal Cascade: Distribution-Free Accuracy Guarantees for Multi-Tier LLM Inference
Result we rely on: Cost ratio 1:2.7 gives break-even acceptance 0.37; guard rail is Always-Strong below 1/r.
Used here: Enable rule.

A:123 Economic Evaluation of LLMs
Result we rely on: A single big model wins above a $0.10 error price.
Used here: E5 sweep.

A:002 RouterBench: A Benchmark for Multi-LLM Routing System
Result we rely on: Cascade quality collapses when judge error exceeds 0.2.
Used here: Disable limit.

A:022 Language Model Cascades: Token-level uncertainty and beyond
Result we rely on: Sum log-probability is length-biased.
Used here: Banned score.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- LiteLLM: budget_fallbacks as a model for priced fallbacks.

HOW OTHERS BUILD IT

Cloudflare and OpenRouter use fallback chains. Token Economics treats each hop as a priced step with an acceptance gate.

REFERENCES

- A:028 Conformal Cascade: Distribution-Free Accuracy Guarantees for Multi-Tier LLM Inference
- C:031 Response-Level Speculative Decoding for Cost-Efficient LLM API Serving
- A:123 Economic Evaluation of LLMs
- A:002 RouterBench: A Benchmark for Multi-LLM Routing System
- A:021 FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance
- C:029 A Unified Approach to Routing and Cascading for LLMs
- A:022 Language Model Cascades: Token-level uncertainty and beyond
- A:010 Rerouting LLM Routers
- A:046 RouterArena: An Open Platform for Comprehensive Comparison of LLM Routers

RECEIVES FROM

Fallback chain
fallback hop, fallback hop

Static router
candidates

HANDS TO

Reserve and reconcile
route

← Back to group: Routing ← Stage 4

LEVEL 4

Probe cost predictor

Policy | Stage 4: Policy engine

Probe cost predictor

One probe, 1.9x multiplier

probe $0.11 -> predicted $2.00, budget 1.9x = $3.80
actual $2.60 -> within budget

IN PLAIN ENGLISH

> Before a big task, we run one small trial call and use its cost to guess the whole task's cost. Then we set the budget at 1.9 times the guess, because guesses are often low.

Goes in	A new task description, the model and a one-call probe result with its cost.
Comes out	A predicted task cost and a budget size equal to 1.9 times the prediction, sent to the reserve component.
Reads / writes	reads: probe cost, per-model run history | writes: prediction and actual pairs for error tracking
If it fails	Fail open to the key budget: if no probe is available the task uses the default per-task budget and the hard cap. A missing probe must not block the task.
Requirement source	RQ-C-032 Sources: C:027, C:110, C:023, C:136, C:131, C:132
Hardening, still open	C:027 covers Kimi K3 only, on 5 tasks and 2,700 runs. The 1.9x multiplier is not verified for other models. Identical runs differ by up to 30x in tokens (C:023). Ladder from R2: ship second.
Not in client docs	**DevX proposal:** The mapping from probe cost to predicted full-task cost is a DevX proposal, since C:027 gives errors and the multiplier but not a fitted formula.

WHAT IT IS FOR

Size a per-task budget from one measured probe call, instead of letting the agent guess. This is ship second in the R2 ladder.

EXPECTED

- Runs one probe call and predicts the task cost from it (C:027).
- Sets the task budget at 1.9 times the prediction, which covered 90 percent of runs (C:027).
- Never asks the agent to estimate its own cost (C:136, C:110).
- Compares prediction to actual after each task and tracks the error.
- Passes the budget to the reserve component as the task cap.

NOT EXPECTED

- Does not use agent self-estimates or human labels.
- Does not use the budget as a prompt hint. Agents ignore it (C:136).
- Does not replace the hard per-key cap.
- Does not guarantee cost. Identical runs differ up to 30x (C:023).

ALGORITHM AND MATHS

One-probe prediction with 1.9x headroom

```
budget = 1.9 x predicted_cost; predicted_cost = f(probe_cost, model)
```

STEPS

1. Choose a probe: one short run of the task with a small step limit.
2. Record the probe cost from the ledger.
3. Predict full-task cost with the per-model fit.
4. Set budget = 1.9 x prediction and pass it to bres.
5. After the task, store prediction, actual and error.

PARAMETERS

probes	1 probe (more probes: 3 gives 29 percent, 10 gives 25 percent median error)
multiplier	1.9 covers 90 percent of runs (C:027)
probe_cost	$0.11 in C:027

THE MATHS IN FULL

SYMBOLS

- p: probe cost in dollars.
- c_hat: predicted task cost.
- A: actual task cost.
- m: headroom multiplier, 1.9.
- err = |c_hat - A| / A.
- sigma_g: geometric standard deviation of run cost, 1.34 in C:027.

FORMULA

```
budget = m x c_hat; err = |c_hat − A|/A; coverage = share of runs with A <= budget; target coverage 0.90 at m = 1.9
```

WHY IT IS RIGHT

Run-to-run cost is skewed, so a multiplier above 1 covers the right tail. C:027 found m = 1.9 gave 90 percent coverage after one probe. A lognormal check with sigma_g 1.34 gives exp(1.2816 x ln 1.34) = 1.46, so 1.9 also absorbs prediction error. The lognormal step is a DevX check, not a source claim.

WORKED EXAMPLE

```
C:027 median error: 161 percent with no probe, 36 percent with one probe ($0.11), 29 percent with three, 25 percent with ten. Per-run correlation r 0.08 without a probe and 0.72 with one
(R6). If the prediction is $2.00, budget = 1.9 x 2.00 = $3.80. An actual of $2.60 fits. An actual of $4.50 hits the cap and the cap-hit path runs.
```

COST OF COMPUTING IT

O(1) per task plus the probe call.

HOW IT CAN GO WRONG, AND THE GUARD

- Probe cost does not predict heavy tasks that run many turns.
- The multiplier is fitted on one model and may not hold on others.
- Output dominates cost at high cache hit (2.7 percent of tokens, 51.1 percent of dollars, C:027), so probes with short outputs underpredict.
- Probe cost can exceed the saving on small tasks.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Per model, track coverage at 1.9x and overrun rate. Refit the multiplier when coverage leaves 0.85 to 0.95 (DevX proposal band).

SECURITY

- Probe uses the same key and budget as the task. It cannot bypass the cap.
- Probe content is the real task, so it follows the tenant's content-logging policy.
- Predictions are metadata and hold no task text.

PERFORMANCE

Latency and throughput	DevX proposal: probe overhead under 5 percent of task cost.
Complexity	O(1).
At 10x load	Prediction table is small, keyed by model.

EDGE CASES AND HOW EACH IS HANDLED

- Task too small to probe: skip and use the default budget.
- Probe fails: use the default per-task budget.
- Model changes after the probe: the prediction is void, re-probe.
- Resumed session: do not count earlier spend in the new prediction (R1 rule 7).
- Prediction above remaining key budget: reject early with a clear message.

TEST CASES

TEST	GIVEN	EXPECT
budget size	predicted $2.00, m 1.9	task budget $3.80
cap hit	actual reaches $3.80	cap-hit path runs, spend logged
probe fails	probe call errors	default budget used, event logged
over key budget	budget $3.80, key remaining $2.00	rejected before the task starts
coverage	100 tasks, 88 within 1.9x	coverage 0.88, inside the DevX band

WHERE IT SAVES COST

- Cuts median prediction error from 161 percent to 36 percent for $0.11 (C:027).
- Keeps the budget tied to a measured cost, so caps are neither too tight nor too loose.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Prediction and actual pairs for refitting.
- Overrun rate per model.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.probe.predicted`	task_id · model · probe_usd · predicted_usd · budget_usd	A budget is set.
`token_economics.probe.scored`	task_id · predicted_usd · actual_usd · error	A task ends.

RESEARCH BEHIND IT

C:027 Can your AI agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks
Result we rely on: One probe cuts median error from 161 to 36 percent; 1.9x covers 90 percent.
Used here: Sets the probe and multiplier.

C:023 How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks
Result we rely on: Identical runs differ up to 30x in tokens.
Used here: Justifies headroom.

HOW OTHERS BUILD IT

Most gateways set one fixed per-key budget. A probe-based task budget is not described in the vendor sources in the corpus.

REFERENCES

- C:027 Can your AI agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks
- C:110 Where Do Your Tokens Go: Empirical Token Consumption in Coding Agents
- C:023 How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks
- C:136 Coding agents ignore spend (Ramp Labs)
- C:131 Budget-Aware Tool-Use Enables Effective Agent Scaling
- C:132 Credit-Budgeted ICPC-Style Coding: When Agents Must Pay for Every Decision

RECEIVES FROM

None in this stage view.

HANDS TO

Static router
cost estimate

← Back to group: Routing ← Stage 4

LEVEL 4

Strong-share ceiling

Policy | Stage 4: Policy engine

Strong-share ceiling

Per-tenant cap, router-attack defence

t4 strong share 21% (14d base 13.4%, ceiling 20.1%)
over ceiling -> router capped to weak, alert

IN PLAIN ENGLISH

> This limits how often one customer's requests may go to the most expensive model. It stops someone from tricking the router into always choosing the strong model.

Goes in	Router decisions per tenant and the tenant's strong-model share over a rolling window.
Comes out	A strong-share ceiling signal to the router, and a drift alert.
Reads / writes	reads: per-tenant routing counts and spend | writes: rolling share, ceiling, alert state
If it fails	Fail open to the rules route with the ceiling last known: if share data is missing the ceiling stays at its last value. A missing counter must not block calls, and budgets still apply.
Requirement source	RQ-C-033, RQ-C-034, RQ-C-051 Sources: A:010, A:001, A:019, A:006, C:054
Hardening, still open	A:010 names per-tenant quotas on strong-model use as the defence, with no numeric ceiling. No source gives a safe share. RouteLLM needed 13.4 percent strong calls for 50 percent of the gap on MT Bench (A:001), which is workload specific.
Not in client docs	**DevX proposal:** The ceiling value (1.5 times the tenant's 30-day baseline share), the window length and the alert method are DevX proposals.

WHAT IT IS FOR

Cap strong-model use per tenant so a router attack or drift cannot inflate cost.

EXPECTED

- Tracks strong-model share per tenant over a rolling window, by calls and by dollars.
- Sets a ceiling per tenant and signals the router to stop choosing strong above it (A:010).
- Raises a drift alert when share rises sharply from the tenant's own baseline.
- Lets an admin raise the ceiling with an audit row.
- Reports share next to cost per solved task so quality loss is visible.

NOT EXPECTED

- Does not block tool-bearing or failed-check escalations that the rules mark mandatory, which go through a separate budget.
- Does not judge prompt content.
- Does not replace the budget cap.

ALGORITHM AND MATHS

Rolling share ceiling

```
s = strong_calls / total_calls over window W; capped iff s >= theta
```

STEPS

1. Count strong and total routed calls per tenant over W.
2. Compare s with theta, set from the tenant's own baseline.
3. If s >= theta, signal the router: prefer weak unless a mandatory escalation applies.
4. If s exceeds baseline by the drift margin, alert.
5. Log ceiling state changes.

PARAMETERS

theta	DevX proposal: 1.5 x tenant 30-day baseline share
window	DevX proposal: 24 hours
attack_effect	73 to 100 percent strong routing in white-box tests (A:010)

THE MATHS IN FULL

SYMBOLS

- W: window length.
- s: strong-model share of calls in W.
- b: tenant's baseline share over 30 days.
- theta = k x b: ceiling, k = 1.5 (DevX proposal).
- c_s, c_w: cost per call on strong and weak model.

FORMULA

```
s = N_strong/N_total; theta = k x b; extra_cost_per_call = (s − b) x (c_s − c_w); alert iff s > theta
```

WHY IT IS RIGHT

Mean cost per call is s x c_s + (1 - s) x c_w. Raising s from the baseline b adds (s - b)(c_s - c_w) per call. Capping s at theta bounds this extra cost.

WORKED EXAMPLE

```
RouteLLM MF router used 13.4 percent strong calls on MT Bench (A:001). With k = 1.5 the ceiling is 20.1 percent. A router attack raised OpenRouter cost on 72 queries from $0.03 to $0.25, a factor of 8.3 (A:010). At a 20.1 percent ceiling the strong share cannot reach the 73 to 100 percent rates seen in attacks.
```

COST OF COMPUTING IT

O(1) per call with counters. O(W) to rebuild from the ledger.

HOW IT CAN GO WRONG, AND THE GUARD

- A ceiling set too low caps quality on a genuine hard-task week.
- A baseline taken during an attack hides it. Use a trailing baseline with attack screening.
- Perplexity filters are bypassable (A:010), so share limits must not depend on them.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Replay 30 days of routing per tenant and check false alert rate. Run E8 with confounder gadgets and measure the share shift.

SECURITY

- Main defence against a router attack by crafted prompts (A:010).
- Ceiling and baseline are tenant-scoped. One tenant's share never leaks into another's data.
- Overrides need an admin role and an audit row.

PERFORMANCE

Latency and throughput	DevX proposal: counter update under 1 ms.
Complexity	O(1).
At 10x load	Per-tenant counters sharded by tenant id.

EDGE CASES AND HOW EACH IS HANDLED

- New tenant with no baseline: use a fleet default until 30 days exist (DevX proposal).
- Genuine hard-task surge: alert first, cap after the grace window.
- Mandatory escalation (tool-bearing) while capped: allowed and counted.
- Clock window boundary: use sliding buckets.
- Baseline poisoned by an old attack: allow an admin to reset.

TEST CASES

TEST	GIVEN	EXPECT
ceiling	b 13.4 percent, k 1.5, s 21 percent	capped, alert raised
under ceiling	s 15 percent	no cap
mandatory	capped, tool-bearing request	escalation allowed and counted
new tenant	no history	fleet default ceiling used
override	admin raises ceiling	audit row written, cap lifted

WHERE IT SAVES COST

- Bounds router-attack cost inflation (8.3x on 72 queries in A:010).
- Keeps strong spend near the tenant's own baseline.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Cap hits and quality metrics per tenant.
- Alert outcomes (attack or real hard tasks).

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.share.capped`	tenant_id · share · ceiling	The ceiling blocks strong routing.
`token_economics.share.drift`	tenant_id · share · baseline	Share rises over the drift margin.

RESEARCH BEHIND IT

A:010 Rerouting LLM Routers
Result we rely on: A short gadget upgrades routing to the strong model 73 to 100 percent; defence is per-tenant quotas.
Used here: Defines the ceiling.

HOW OTHERS BUILD IT

Router vendors do not publish per-tenant strong-share caps in the corpus. A:010 is the only source.

REFERENCES

- A:010 Rerouting LLM Routers
- A:001 RouteLLM: Learning to Route LLMs with Preference Data
- A:019 Zen Router 0.6B
- A:006 LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing
- C:054 The Coding Agent Economy

RECEIVES FROM

None in this stage view.

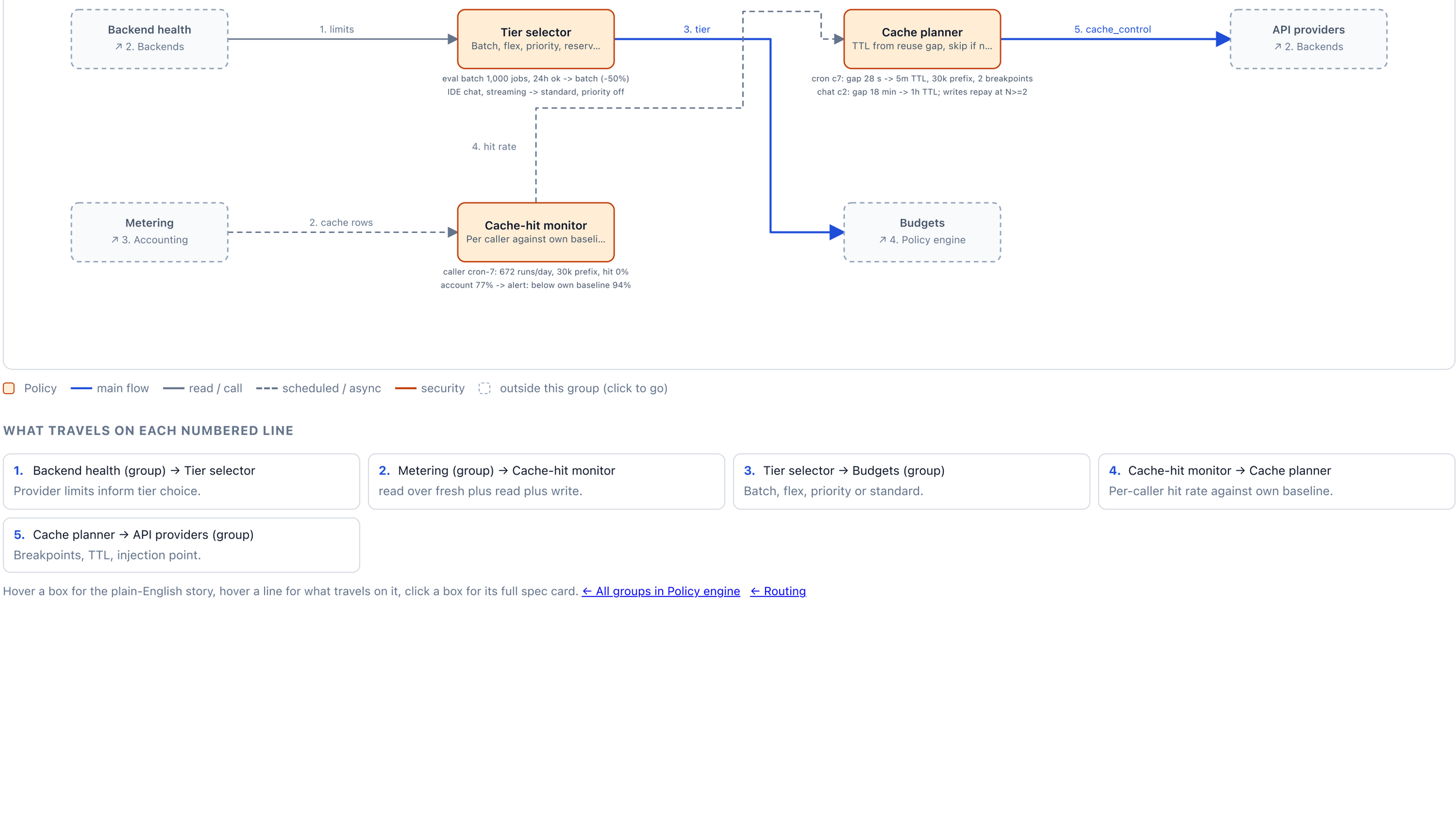
HANDS TO

Static router
ceiling

← Back to group: Routing ← Stage 4

LEVEL 3 **Tier and cache** in Policy engine

Cache planner, tier selector and cache-hit monitor.



Policy — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Backend health (group) → Tier selector
Provider limits inform tier choice.

2. Metering (group) → Cache-hit monitor
read over fresh plus read plus write.

3. Tier selector → Budgets (group)
Batch, flex, priority or standard.

4. Cache-hit monitor → Cache planner
Per-caller hit rate against own baseline.

5. Cache planner → API providers (group)
Breakpoints, TTL, injection point.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Policy engine ← Routing

LEVEL 4

Cache planner

Policy | Stage 4: Policy engine

Cache planner

TTL from reuse gap, skip if no payback

cron c7: gap 28 s -> 5m TTL, 30k prefix, 2 breakpoints
chat c2: gap 18 min -> 1h TTL; writes repay at N>=2

IN PLAIN ENGLISH

> This decides what to save in the provider's prompt cache and for how long. It picks the cheaper choice per caller from how soon each caller comes back, and skips caching when it cannot pay off.

Goes in	Prefix size, model, caller id, observed reuse gap per caller, hit rate from the hit monitor, and price rows.
Comes out	cache_control breakpoints, TTL choice and injection point for the provider connector, or a skip decision.
Reads / writes	reads: price book, per-caller reuse gap, hit monitor | writes: per-caller cache plan
If it fails	Fail open to no change: if planning fails the request goes out as sent and the client's own cache_control stays. A wrong plan can raise cost, but a missing plan only forgoes savings.
Requirement source	RQ-C-035, RQ-C-036, RQ-C-037, RQ-C-038, RQ-C-039, RQ-C-044, RQ-C-051 Sources: A:199, A:201, C:052, A:224, C:054, A:157, A:160, A:281, A:219, A:252, A:255, A:202, A:231, C:055
Hardening, still open	Anthropic minimum prefix conflicts: Sonnet 4.6 1,024 (A:199) vs 2,048 (C:051); Opus 4.7 2,048 vs 4,096. Take the larger until tested. Gemini implicit discount 75 vs 90 percent. Subagent TTL: docs say 5 minutes, observation differs (C:052). Break-even prefix at N=10 is 220.16 tokens by formula; R1 writes 221.
Not in client docs	**DevX proposal:** The reuse-gap percentile used to pick TTL and the keepalive rule are DevX proposals on the C:052 findings.

WHAT IT IS FOR

Choose cache breakpoints and TTL per caller, and skip caching when the write premium cannot be repaid.

EXPECTED

- Picks the TTL per caller from the observed reuse gap, never one global TTL (C:052).
- Skips caching when the prefix is under the model minimum, or the expected reads cannot repay the write (A:199, A:201).
- Keeps stable content first and volatile content such as timestamps last (A:224).
- Forwards cache_control and anthropic-beta unchanged and uses at most 4 breakpoints, longer TTL first (A:199).
- Decides batch and cache stacking per provider and model (A:252, A:255, A:281, A:219).

NOT EXPECTED

- Never sets a blanket 1-hour TTL. It made the bill 8.6 percent worse (C:052).
- Does not rely on cache for Bedrock batch or Nova reads (A:160, A:157).
- Does not send ttl to 5-minute-only Bedrock models (A:157).
- Does not move content that changes semantics. It reorders only fields the policy marks volatile.

ALGORITHM AND MATHS

Per-caller TTL and payback planner

```
TTL = 5m if gap_p98 <= 5m else 1h if N_reads x saving > premium_1h else none
```

STEPS

1. Read the model's minimum prefix. If P is below it, skip.
2. Estimate reads N in the TTL window from the caller's observed gap distribution.
3. Compute the write premium (5m 1.25x, 1h 2x of input) and the saving per read.
4. Choose 5m if almost all reuse falls within 5 minutes, 1h if N reads repay the 1h premium, else none.
5. Place breakpoints at the end of tools, system and the stable message block, at most 4.
6. Check the provider-specific rule (OpenAI key, Gemini storage fee) and hand off to stick.

PARAMETERS

anthropic_multipliers	5m write 1.25x, 1h write 2x, read 0.1x; 0.05x on Opus 5.5 and Sonnet 5.5; 0.025x on Fable 5.1 and Mythos 5.1 (A:199)
min_prefix	512 to 4,096 tokens by model (A:199); take the larger on conflict
reuse_window	98 percent of reuse within about 34 s (C:052)
keepalive_crossover	62.5 minutes (C:052)

THE MATHS IN FULL

SYMBOLS

- P: prefix tokens.
- I: input price per MTok. W5 = 1.25 I, W1 = 2 I, R = read price.
- N: number of reads within the TTL.
- g: observed gap between uses.
- X: write premium over fresh input.

FORMULA

```
cost_none = (N + 1) x P x I; cost_5m = P x (W5 + N x R) when g <= 5m; cost_1h = P x (W1 + N x R) when g <= 60m; cache iff cost_cached < cost_none and P >= min_prefix; break-even N_5m = (W5 - I)/(I - R)
```

WHY IT IS RIGHT

Without cache every call pays I. With cache the first call pays the write rate and each read pays R. Reads repay the premium (W - I) at a saving (I - R) per read, so N must exceed (W - I)/(I - R). OpenAI GPT-5.6+ gives a break-even prefix of 102.4 + 1,177.6/N tokens (A:201).

WORKED EXAMPLE

```
Sonnet 5.5 (R1): I $2.00, W5 $2.50, W1 $4.00, R $0.10 per MTok. N_5m = (2.50 - 2.00)/(2.00 - 0.10) = 0.263, so one read repays a 5-minute write. N_1h = (4.00 - 2.00)/1.90 = 1.05, so 2 reads repay a 1-hour write. For a 30,000 token prefix and 10 reads inside 5 minutes: none 11 x 0.03 x 2.00 = $0.66; 5m 0.03 x (2.50 + 1.0) = $0.105. OpenAI GPT-5.6 at N=10: break-even prefix 102.4 + 117.76 = 220.16 tokens. C:052: blanket 1h gave +8.6 percent, targeted fixes -6.0, -1.0, -7.6 and -13.6 percent.
```

COST OF COMPUTING IT

O(1) per request after per-caller gap stats are cached.

HOW IT CAN GO WRONG, AND THE GUARD

- Reuse gap estimated from too few calls: choose 5m by default and refresh.
- Subagent TTL differs from docs (C:052), so check realised hit rate.
- Provider ignores cache_control behind a gateway: hit monitor flags it (C:052, C:054).
- A volatile field left in the prefix drops hits from 87 to 3 percent (A:224).

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Experiment 2 of R5: caching off, 5m and 1h per workload, confirm the N-read rule and the blanket-1h regression.

SECURITY

- Cache is isolated per org and workspace at the provider (A:199). Plans never merge prefixes across tenants.
- Injected cache_control touches structure only, never message text.
- Plans are logged with ids and token counts, not content.

PERFORMANCE

Latency and throughput	DevX proposal: under 3 ms planning time per request.
Complexity	O(1).
At 10x load	Per-caller stats in a keyed store with expiry.

EDGE CASES AND HOW EACH IS HANDLED

- Prefix just under the minimum: skip. Take the larger minimum on conflicting sources.
- Caller sends its own cache_control: respect it and only add missing breakpoints.
- Compaction rewrites the prefix: expect a miss and do not alert on that turn.
- Haiku 5.5 crosses 100K and reprices: re-plan with the band (R1).
- Mistral or Groq cached-token field names are not in the corpus: skip planning and log.

TEST CASES

TEST	GIVEN	EXPECT
5m pays	Sonnet 5.5, P 30,000, gap 28 s, N 10	5m TTL chosen, cost 0.105 per the formula
below min	P 400 tokens, Haiku 4.5 min 4,096	caching skipped
1h choice	gap 18 min, N 3	1h TTL chosen
blanket 1h	policy sets 1h for all callers	policy validation warns, per-caller required
bedrock batch	Bedrock batch inference	no cache plan, reason logged

WHERE IT SAVES COST

- Effective input about $0.30 per M at a 92 percent hit rate versus $3.00 list (C:054).
- Targeted per-caller fixes gave -13.6 percent combined (C:052).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Hit rate per caller from the hit monitor.
- Realised write and read cost per plan.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.cplan.planned`	caller_id · model · prefix_tokens · ttl · breakpoints	A plan is set.
`token_economics.cplan.skipped`	caller_id · reason	Caching is skipped.

RESEARCH BEHIND IT

C:052 Your cache hit rate is lying to you. It's the caller mix.
Result we rely on: Blanket 1h TTL cost 8.6 percent more; 98 percent of reuse within about 34 seconds.
Used here: Per-caller TTL.

A:201 Prompt caching
Result we rely on: Break-even prefix is 102.4 + 1,177.6/N tokens.
Used here: Payback check.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- LiteLLM: cache_control_injection_points.

HOW OTHERS BUILD IT

LiteLLM injects cache_control at set points. Anthropic offers a top-level automatic mode. Token Economics adds per-caller TTL and a payback check.

REFERENCES

- A:199 Claude prompt caching docs
- A:201 Prompt caching
- C:052 Your cache hit rate is lying to you. It's the caller mix.
- A:224 Prompt Caching: Cut the Cost of Long Chats with the Grok API
- C:054 The Coding Agent Economy
- A:157 Prompt caching for faster model inference
- A:160 Process multiple prompts with batch inference
- A:281 Batch inference (Gemini Enterprise)
- A:219 Batch processing (Claude)
- A:252 Groq Batch API
- A:255 Batch inference (Fireworks)
- A:202 Prompt Caching 201
- A:231 Prompt caching best practices (OpenRouter)
- C:055 How Claude Code uses prompt caching

RECEIVES FROM

Cache-hit monitor
hit rate

HANDS TO

Anthropic connector
cache_control, cache_control

← Back to group: Tier and cache ← Stage 4

LEVEL 4

Tier selector

Policy | Stage 4: Policy engine

Tier selector

Batch, flex, priority, reserved, small

eval batch 1,000 jobs, 24h ok -> batch (-50%)
IDE chat, streaming -> standard, priority off

IN PLAIN ENGLISH

> This picks how a job is bought: batch for slow bulk work, standard for normal work, priority only when speed matters. It also checks whether a small model can do a narrow job safely.

Goes in	Task latency tolerance, streaming need, SLA need, size, provider limits from the rate-limit tracker, price book tier rows, and the model catalogue.
Comes out	A tier choice (batch, flex, priority, reserved, standard) and, where the job allows it, a small-model option, sent to the reserve component.
Reads / writes	reads: price book tier rows and flags, rate limits | writes: tier decision log
If it fails	Fail open to standard tier: if selection fails the request runs at the standard rate. Standard is correct but may cost more, and the budget still applies.
Requirement source	RQ-C-039, RQ-C-043, RQ-C-044, RQ-C-045, RQ-C-046, RQ-C-047, RQ-C-048, RQ-C-049, RQ-C-051 Sources: A:111, A:152, A:219, A:174, A:252, A:255, A:159, A:281, A:195, A:150, A:176, A:225, A:209, A:129, A:272, C:050, A:041, A:186, A:157, A:069, C:086
Hardening, still open	OpenAI flex percentage is not stated (A:195). Together batch is 'up to 50 percent' on one model (A:275). Stacking differs by provider and model. Small-model failure rates have no default in the corpus. Gemini cache discount 75 vs 90 percent changes cache-adjusted tier cost.
Not in client docs	**DevX proposal:** The mapping from workload class to tier and the 24-hour tolerance rule are DevX proposals on the cited tier facts.

WHAT IT IS FOR

Pick the cheapest tier that meets the job's latency, streaming and SLA needs, and gate small-model use on measured parity.

EXPECTED

- Sends latency-tolerant, non-streaming, non-SLA work to batch or flex (A:195, A:281, A:219).
- Retries on standard after a flex 429. Flex has no automatic fallback at OpenAI or Azure (A:195, A:150).
- Decides cache and batch stacking per provider and model (A:252, A:255, A:281, A:219).
- Applies priority multipliers only when an SLA binds: Mistral 1.75x after the cache discount (A:176), OpenAI fast 2x (A:209), xAI priority 2x (A:225).
- Recommends reserved capacity only above about 40 percent of on-demand cost (A:159).

NOT EXPECTED

- Does not change tier mid-conversation without costing a cold cache. Fast mode mid-conversation bills the whole context uncached (C:086).
- Does not rely on cache in Bedrock batch (A:157).
- Does not route to a small model without per-task eval parity (A:129, C:050).
- Does not move streaming or SLA work to batch (A:219).

ALGORITHM AND MATHS

Constraint filter then cheapest tier

```
tier = argmin over allowed tiers of cost(tier); allowed = tiers meeting latency, streaming and SLA constraints
```

STEPS

1. Read job flags: streaming, deadline, SLA, size, repeat prefix.
2. Drop tiers that fail a constraint (batch fails streaming, windows 24h at OpenAI and Anthropic, 72h queue at Gemini).
3. Cost each remaining tier with stacking flags and cache state.
4. Pick the lowest cost. For a narrow, verifiable, high-volume job also evaluate the small-model rule.
5. Set the fallback: retry on standard after a flex 429.
6. Log the choice and its alternatives.

PARAMETERS

batch	50 percent off at OpenAI, Azure, Anthropic, Mistral, Groq, Fireworks, Bedrock, Gemini (A:111, A:152, A:219, A:174, A:252, A:255, A:159, A:281); xAI 20 percent (A:225)
priority	Mistral 1.75x (A:176); OpenAI fast 2x (A:209); xAI 2x (A:225)
reserved_trigger	steady load above about 40 percent of on-demand (A:159)
small_model	route iff cost_small + P(fail) x cost_escalation < cost_big

THE MATHS IN FULL

SYMBOLS

- B: base cost at standard tier.
- m_t: multiplier for tier t (batch 0.5, flex 0.5, priority 1.75 to 2).
- S: batch and cache stacking flag.
- c_small, c_big, c_esc: cost of small model, big model and escalation.
- p_f: failure rate of the small model on the labelled slice.

FORMULA

```
cost(t) = B x m_t (cache component only if S is true; else cache reads bill at the batch input rate); small model wins iff c_small + p_f x c_esc < c_big
```

WHY IT IS RIGHT

Tier multipliers scale the base. Stacking decides whether the cache discount survives the tier. A small model pays its own cost plus the expected escalation, and it wins only below the big model's cost.

WORKED EXAMPLE

```
A $100 standard-rate eval job: batch $50 (A:219); Mistral priority $175 (1.75x, A:176); OpenAI fast $200 (2x, A:209). Small model, illustrative arithmetic: c_big $1.00, c_small $0.10 (10x cheaper, low end of the 10 to 30x in A:041), c_esc $1.00. Small wins iff 0.10 + p_f x 1.00 < 1.00, so p_f < 0.90 on cost alone. Quality parity must hold first (C:050: 3B model 55.62 percent on multi-turn tool use).
```

COST OF COMPUTING IT

O(t) over a handful of tiers per request.

HOW IT CAN GO WRONG, AND THE GUARD

- Wrong stacking flag overstates batch savings (Groq no, Fireworks yes, Gemini 3 yes, 2.5 no).
- Batch hit rates are best effort, typically 30 to 98 percent (A:219), so use expected not ideal.
- Price rows stale after 30 days: billing blocked by the price-row lifecycle.
- Small-model failure rate taken from another workload.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

R5 experiment 3: batch, flex and standard on 1,000 fixed requests, measure price, latency, hit rate, error and 429 rates. Experiment 6: small-model parity on 1,000+ labelled samples.

SECURITY

- Tier flags come from the price book and policy, never from the request body alone.
- Batch payloads follow the same content-logging policy as live calls.
- Small-model routing needs an admin-set allow list per tenant and task type.

PERFORMANCE

Latency and throughput	DevX proposal: under 2 ms per decision.
Complexity	O(t).
At 10x load	Stateless. Price rows cached.

EDGE CASES AND HOW EACH IS HANDLED

- Batch window of 24h does not suit the deadline: choose flex or standard.
- Flex returns 429: retry on standard, not billed for the failed call (A:195, A:150).
- Bedrock batch with caching: no cache component (A:157).
- Gemini 2.5 batch: no stacking with cache.
- Azure 12h cache write charges from Nov 1 2026 change cache-adjusted cost (A:148).

TEST CASES

TEST	GIVEN	EXPECT
batch pick	1,000 eval jobs, 24h ok, non-streaming	batch tier, cost x0.5
streaming	chat with streaming	standard tier
flex 429	flex returns 429	retry on standard, failed call billed 0
no stacking	Groq batch with warm cache	cache reads priced at the batch input rate
small model	p_f 0.95, c_small 0.10, c_big 1.00, c_esc 1.00	rejected, cost rule fails

WHERE IT SAVES COST

- Batch halves cost on eight providers (A:111, A:152, A:219, A:174, A:252, A:255, A:159, A:281).
- Reserved capacity saves 30 to 50 percent above the load trigger (A:159).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Realised cost and error rate per tier.
- 429 and fallback counts.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.tsel.chosen`	request_id · tier · base_usd · tier_usd · constraints	A tier is chosen.
`token_economics.tsel.fallback`	request_id · from_tier · reason	A flex or batch call falls back to standard.

RESEARCH BEHIND IT

A:129 Investing in Performance: Fine-tune small models with LLM insights
Result we rely on: 900k labels for about $70; a small model about 80x cheaper per instance-hour on a 105-sample test.
Used here: Supports the small-model rule, with a larger parity set.

HOW OTHERS BUILD IT

Providers publish batch, flex, priority and reserved tiers. Gateways rarely choose among them per job. Token Economics selects the tier from job flags and the versioned price book.

REFERENCES

- A:111 Batch API guide
- A:152 How to use global batch processing with Azure OpenAI
- A:219 Batch processing (Claude)
- A:174 Batch Processing
- A:252 Groq Batch API
- A:255 Batch inference (Fireworks)
- A:159 Capacity and Performance
- A:281 Batch inference (Gemini Enterprise)
- A:195 Flex processing
- A:150 Use Flex processing with Azure OpenAI in Foundry Models
- A:176 Priority Tier
- A:225 xAI pricing
- A:209 Fast mode for API Customers
- A:129 Investing in Performance: Fine-tune small models with LLM insights
- A:272 Introducing pplx-api
- C:050 Evaluation and Optimization of Small Language Models for Agentic Tasks on Edge Devices
- A:041 Small Language Models are the Future of Agentic AI
- A:186 The new economics of AI cost management
- A:157 Prompt caching for faster model inference
- A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
- C:086 Speed up responses with fast mode

RECEIVES FROM

Rate-limit tracker
limits, limits

HANDS TO

Reserve and reconcile
tier

← Back to group: Tier and cache ← Stage 4

LEVEL 4

Cache-hit monitor

Policy | Stage 4: Policy engine

Cache-hit monitor

Per caller against own baseline

caller cron-7: 672 runs/day, 30k prefix, hit 0%
account 77% -> alert: below own baseline 94%

IN PLAIN ENGLISH

> This measures how often each caller reuses the provider's saved prompt, one caller at a time. The total hides bad callers, so it warns when a caller is worse than its own normal.

Goes in	Cache rows from the ledger: fresh input, cache read and cache write tokens per call, with caller id, model and prefix hash.
Comes out	A hit rate per caller, model and prefix hash, a miss cause token-economics, and an alert when a caller is below its own baseline.
Reads / writes	reads: ledger cache rows, miss causes | writes: per-caller hit rate series and baselines
If it fails	Fail open: a missing metric never blocks a call. The monitor only observes and alerts, so a lost sample delays an alert and costs no more.
Requirement source	RQ-C-040, RQ-C-041, RQ-C-042 Sources: C:052, C:054, C:076, A:204, C:055, A:224
Hardening, still open	Account-level and per-caller rates differ widely: account 77 percent hid a cron near 0 percent (C:052). Gateway rates by agent, from C:054, show selection bias. Whether OpenAI per-key overflow near 15 requests per minute affects our callers is unknown.
Not in client docs	**DevX proposal:** The alert rule (3 standard deviations below the 14-day mean) and the minimum call count are DevX proposals. The corpus asks for per-caller baselines but gives no alert method.

WHAT IT IS FOR

Measure cache hit rate per caller against its own baseline and name the miss cause, so a failing caller is found even when the account rate looks fine.

EXPECTED

- Computes hit = cache_read / (fresh + cache_read + cache_write) per caller, model and prefix hash (C:052).
- Alerts when a caller is below its own baseline even if the account rate is high (C:052, C:054).
- Flags a miss as more than 5 percent and 2,000 or more tokens re-processed, and records the cause (C:076).
- Tags causes such as model_changed, tools_changed and reasoning_effort_changed where the provider reports them (A:204, C:055).
- Feeds the hit rate back to the cache planner (edge hit to cplan).

NOT EXPECTED

- Does not alert on the account average alone.
- Does not mix response-level cache rates such as Helicone or Portkey with provider prompt caching (R1 section 4).
- Does not change requests. It only observes.

ALGORITHM AND MATHS

Per-caller hit rate against baseline

```
hit = read / (fresh + read + write); alert iff hit < baseline − z x sd for n >= n_min calls
```

STEPS

1. Read each ledger cache row. Join caller id, model and prefix hash.
2. Add tokens to the caller's rolling window.
3. Compute hit. Compare with the caller's 14-day baseline mean and standard deviation.
4. On breach, attach the top miss cause from the provider diagnostics or the status-line fields.
5. Send an alert and a signal to the cache planner.

PARAMETERS

hit_formula	read / (fresh + read + write) (R1 rule 18)
miss_definition	more than 5 percent and 2,000+ tokens re-processed (C:076)
baseline_window	DevX proposal: 14 days
z	DevX proposal: 3
n_min	DevX proposal: 50 calls

THE MATHS IN FULL

SYMBOLS

- fresh, read, write: token counts in the window (write is 5m plus 1h).
- hit: cache hit rate.
- mu_c, sd_c: caller's baseline mean and standard deviation.
- z: alert multiplier.
- L: lost cost per day from misses.

FORMULA

```
hit = read/(fresh + read + write); alert iff hit < mu_c − z x sd_c; L = N x P x (W − R)/1e6 per day if every call misses and re-writes instead of reading
```

WHY IT IS RIGHT

A miss re-bills the prefix at the write rate instead of the read rate. The lost cost per call is P x (W - R). Multiplying by calls per day gives the daily loss from a caller at 0 percent hit.

WORKED EXAMPLE

```
C:052: account hit 77 percent hid a cron job at about 0 percent: 672 runs per day, prefix about 30,000 tokens. Using Sonnet 5.5 rates from R1 (W5 $2.50, R $0.10 per MTok): all misses = 672 x 30,000 x 2.50/1e6 = $50.40 per day; all hits = 672 x 30,000 x 0.10/1e6 = $2.02 per day; loss about $48.38 per day. C:054: at a 92 percent hit rate effective input is about $0.30 per M versus $3.00 list.
```

COST OF COMPUTING IT

O(1) per ledger row. O(c) memory for c active callers.

HOW IT CAN GO WRONG, AND THE GUARD

- Low call counts give noisy rates, so n_min gates alerts.
- A deliberate compaction or tool change causes an expected miss. Cause tags suppress those alerts.
- Selection bias in gateway data (C:054) can mislead baselines for new callers.
- Provider does not report cache fields (Nova reads are not reported, A:157 via R1), so hit shows 0.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Compare alerts to confirmed causes weekly. Start with a 30-day baseline (R5 experiment 7) and tune z against false alerts.

SECURITY

- Stores counts and hashes only. Prefix hash is a one-way hash, so content cannot be recovered.
- Per-caller views respect tenant scope.
- Alerts show caller ids to authorised roles only.

PERFORMANCE

Latency and throughput	DevX proposal: alert within 5 minutes of the window breach.
Complexity	O(1) per row.
At 10x load	Stream the ledger and partition by caller id.

EDGE CASES AND HOW EACH IS HANDLED

- Caller with a rate-capped OpenAI key: low hit with the cause overflow.
- Subagent callers do not read the parent cache (C:052), so their hit is low by design. Baseline them separately.
- First call of a session: a write by design, so exclude from the miss count.
- Model changed: miss is expected and labelled model_changed.
- Provider reports no cache fields: mark unavailable, do not report 0.

TEST CASES

TEST	GIVEN	EXPECT
hit formula	fresh 500, read 4,000, write 500	hit 0.80
below baseline	caller baseline 0.94, sd 0.02, hit 0.60, n 100	alert raised
account hides	account 0.77, cron 0.00 over 672 calls	alert for the cron caller
model change	model_changed miss after a planned switch	no alert, cause tagged
low volume	hit 0.0 over 5 calls	no alert, below n_min

WHERE IT SAVES COST

- Finds callers that pay write rates instead of read rates: about $48.38 per day in the C:052 cron example at Sonnet 5.5 rates.
- Gateway-reported hit by agent runs from 92 percent (Claude Code) to 46 percent (Kilo) (C:054).

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Hit rate and miss cause to the cache planner.
- Alert outcomes to threshold tuning.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.hit.measured`	caller_id · model · prefix_hash · hit · calls	A window closes.
`token_economics.hit.alert`	caller_id · hit · baseline · top_cause	A caller falls below its baseline.

RESEARCH BEHIND IT

C:052 Your cache hit rate is lying to you. It's the caller mix.
Result we rely on: Account hit 77 percent hid a cron job near 0 percent across 672 runs per day.
Used here: Per-caller baseline and alert.

C:076 Customize your status line
Result we rely on: A miss is more than 5 percent and 2,000+ tokens re-processed.
Used here: Miss definition.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- OpenRouter: Reports cached_tokens and cache_discount in usage.

HOW OTHERS BUILD IT

Claude Code's status-line exposes warm, hit_ratio and last_miss_cause (C:076). OpenAI Responses returns cache diagnostics (A:204). Token Economics joins these by caller.

REFERENCES

- C:052 Your cache hit rate is lying to you. It's the caller mix.
- C:054 The Coding Agent Economy
- C:076 Customize your status line
- A:204 Prompt cache diagnostics
- C:055 How Claude Code uses prompt caching
- A:224 Prompt Caching: Cut the Cost of Long Chats with the Grok API

RECEIVES FROM

Append-only ledger
cache rows, cache rows

HANDS TO

Cache planner
hit rate

← Back to group: Tier and cache ← Stage 4

LEVEL 2

5. Context guard

Controls that cut what goes into the prompt: schema deferral, output caps, masking, clearing, compaction.

Context controls

Eight controls on what enters the prompt.

8 Context guard

Tool-schema deferral · Tool-output cap · Observation masking · Context clearing · Compaction guard · Max-tokens enforcer · Instruction-file linter · Prompt-wording lint

Hands to: Gateway edge (Ingress and adapters), Admin and config (Control plane)

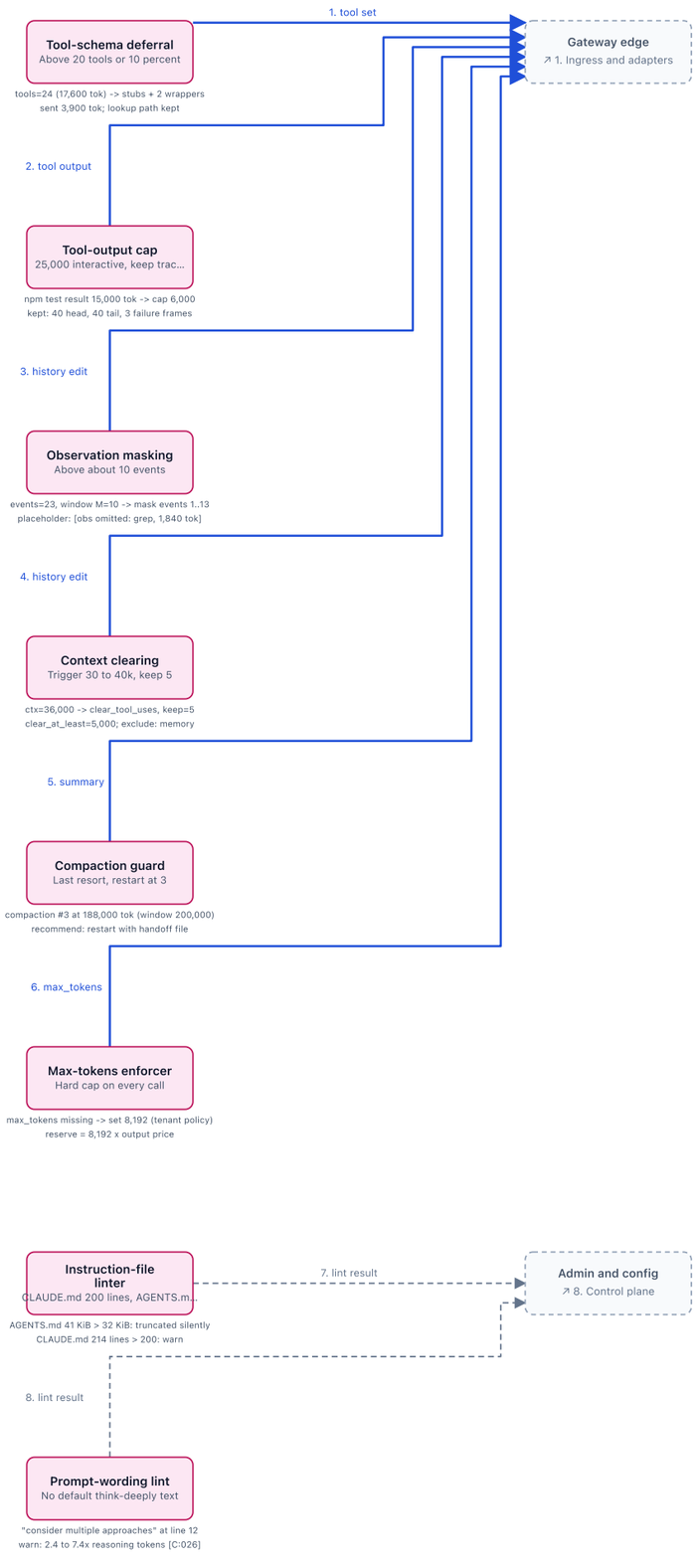
Open group (8 components) →

1 groups, 8 components. Hover a group for the plain-English story; click it to see its components and the arrows between them. ← Policy engine Insight →

LEVEL 3

Context controls in Context guard

Eight controls on what enters the prompt.



Context guard — main flow — read / call — scheduled / async — security — outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Tool-schema deferral → Gateway edge (group)
Deferred schemas and lookup path.

2. Tool-output cap → Gateway edge (group)
Capped output with tracebacks kept.

3. Observation masking → Gateway edge (group)
Older observations replaced, only at a trigger.

4. Context clearing → Gateway edge (group)
Tool results cleared at trigger.

5. Compaction guard → Gateway edge (group)
Compaction near the window, restart at 3.

6. Max-tokens enforcer → Gateway edge (group)
Hard cap set on every call.

7. Instruction-file linter → Admin and config (group)
Instruction-file size findings.

8. Prompt-wording lint → Admin and config (group)
Flags default think-deeply or multi-approach text.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Context guard

LEVEL 4

Tool-schema deferral

Context guard | Stage 5: Context guard

Tool-schema deferral

Above 20 tools or 10 percent

tools=24 (17,600 tok) -> stubs + 2 wrappers
sent 3,900 tok; lookup path kept

IN PLAIN ENGLISH

> Big tool lists are sent on every turn and cost money each time. Token Economics replaces rarely used tool descriptions with short stubs. The model asks for the full text when it needs it.

Goes in	Request tools array, tool count, schema tokens, context window size, session turn index.
Comes out	Original tools array, or stub entries plus get_tool_schema and invoke_tool wrapper tools.
Reads / writes	Reads policy thresholds and session tool-set hash. Writes tool-set hash per session and one decision row to the metadata store.
If it fails	Fail open. Send the original tool set unchanged. A missing tool breaks the task, while an extra schema only costs tokens.
Requirement source	RQ-D-001, RQ-D-002, RQ-D-003 Sources: C:019, C:021, C:022, C:055, C:111, C:104, C:028, C:020, C:026, C:060, A:199
Hardening, still open	Quoted overhead for one 93-tool server ranges 8,000 [C:019], about 17,600 [C:022], about 42,000 [C:020], about 55,000 [C:107] tokens per turn. Chosen: use the measured prefix of each tenant, not a quote. Quality effect of deferral has no numbers [C:022].
Not in client docs	**DevX proposal:** Session-start pinning, the keep-set size of 3 and the wrapper tool names follow cited sources. The combined design is a DevX proposal. The billed saving is not in corpus.

WHAT IT IS FOR

Cut the fixed per-turn tool overhead. Tool definitions are part of the 74.7 percent harness base [C:024]. Every tool stays reachable. The Model Context Protocol (MCP) servers add the largest schemas.

EXPECTED

- Fires only at turn 0 of a session. Trigger: more than 20 tools, or schema tokens above 10 percent of the window.
- Keeps the 3 most-used tools fully defined, based on per-tenant call history (3 tools gave 98 percent of calls [C:028]).
- Adds get_tool_schema and invoke_tool so every deferred tool stays callable [C:022].
- Leaves the tool set byte-identical for the rest of the session.

NOT EXPECTED

- Does not prune tools with no recovery path [C:021].
- Does not change the tool list mid-session [C:055].
- Does not double-defer when the client already runs its own tool search (auto-on above 10 percent [C:019]).

ALGORITHM AND MATHS

Session-start schema deferral

```
defer = (N_tools > 20) or (T_schema / window > 0.10); keep K most-called tools
```

STEPS

1. On the first request of a session, read tools[] and compute N_tools and T_schema with the tenant tokenizer estimate.
2. If the client already sends a tool-search or stub pattern, record decision skip_client_defers and stop.
3. If defer is true, keep the top K tools by 30-day call count for this tenant and agent. Replace the rest with stubs (name plus one-line description).
4. Append the two wrapper tools. Hash the final tools array and pin the hash for the session.
5. On later turns, compare the tools hash with the pinned hash. If it differs, pass the request through and emit a drift event.

PARAMETERS

tool_count_threshold	20 [C:019]
context_share_threshold	0.10 [C:019]
min_session_turns_for_payback	10 to 15 [C:019]
keep_set_K	3 (DevX proposal, from C:028 98 percent of calls)
codex_tool_input_schema_max_bytes	5,000 [C:104]

THE MATHS IN FULL

SYMBOLS

- dT = schema tokens removed per turn (T_before minus T_after).
- r = cache-read price per token; w = cache-write price per token (5-minute).
- n = turns in the session; L = lookup calls; c_L = cost of one lookup turn.

FORMULA

```
net = dT*w + n*dT*r − L*c_L  (cached case); pays when net > 0 and n >= 10
```

WHY IT IS RIGHT

Deferred schemas are written once to the cache, saving dT*w on the first turn. They are read on every later turn, saving dT*r. Each lookup adds a turn that costs a prefix read plus output. Net saving is the sum of the first two terms minus lookup cost.

WORKED EXAMPLE

```
dT = 17,600 − 3,900 = 13,700 tokens [C:022]. Sonnet 5.5: write $2.50, read $0.10, output $10 per MTok [A:199]. First turn 13,700*2.50/1e6 = $0.03425. 100 turns 100*13,700*0.10/1e6 = $0.137. Assume L=10 lookups. Each is a 20,000-token prefix read ($0.002) plus 500 output tokens ($0.005), so c_L = $0.007. The prefix size is from C:026. The output length is a DevX assumption. Net = 0.03425 + 0.137 − 0.07 = $0.101 per 100-turn session.
```

COST OF COMPUTING IT

O(N_tools) per session start, O(1) per later turn (hash compare).

HOW IT CAN GO WRONG, AND THE GUARD

- Lookup turns outnumber saved turns when most stubbed tools are called.
- Client changes tools mid-session and breaks the pinned hash.
- The stub is too short and the model picks the wrong tool. A 95 vs 71 percent accuracy claim is a product pitch [C:020].

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Replay each tenant tool-call log; choose K to cover 95 percent of calls. Run the 20, 50, 90 tool A/B from R3 section 6 and compare billed cost per successful task.

SECURITY

- Stubs and wrapper calls use the same tenant authorisation as the original tools. invoke_tool must not reach a tool the key may not use.
- Tool descriptions are untrusted input; the guard must not execute or follow instructions inside them.
- Decision rows hold counts and hashes only, never schema text.

PERFORMANCE

Latency and throughput	Added gateway latency under 5 ms p95 at turn 0, under 1 ms later (DevX proposal).
Complexity	Linear in tool count at session start.
At 10x load	State is one hash per session; stateless otherwise.

EDGE CASES AND HOW EACH IS HANDLED

- Tool count is exactly 20: no defer (threshold is strictly above).
- Session resumes with a different tool set: treat as a new session and re-decide.
- Client sends a tool-search stub already: record skip_client_defers.
- Tool schema bytes exceed 5,000 on Codex: let the client truncate; do not double-edit.

TEST CASES

TEST	GIVEN	EXPECT
above_count	24 tools, 17,600 tokens, window 200,000	defer true, K=3 kept, 2 wrappers added
share_trigger	8 tools, 25,000 schema tokens, window 200,000	share 0.125 above 0.10, defer true
below_both	12 tools, 9,000 tokens, window 200,000	pass through unchanged
mid_session_drift	pinned hash H1, turn 7 sends hash H2	request unmodified, event token_economics.schema.drift
client_defers	request already contains tool-search stub	skip_client_defers, no edit

WHERE IT SAVES COST

- Lower fixed prefix tokens per turn.
- Higher cache hit by holding a stable tools array.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Lookup rate per deferred tool.
- Wrong-tool-call rate before and after.
- Billed cost per successful task A/B.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.schema.decision`	session_id · n_tools · t_schema · decision · k_kept	turn 0 of each session
`token_economics.schema.lookup`	session_id · tool_name · turn	each get_tool_schema call
`token_economics.schema.drift`	session_id · pinned_hash · new_hash	tool hash change mid-session

RESEARCH BEHIND IT

C:022 MCP Compression: Preventing tool bloat in AI agents
Result we rely on: Wrapper tools cut 17,600 tokens to 3,900 or fewer; self-reported.
Used here: Wrapper design and token ladder.

C:021 Dynamic Tool Gating and Lazy Schema Loading for Eliminating the MCP/Tools Tax in Scalable Agentic Workflows
Result we rely on: Static pruning without recovery hurt success; simulation only.
Used here: Rule: always keep a recovery path.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- mcp-compressor: get_tool_schema and invoke_tool wrapper pattern.
- LiteLLM: mcp_tool_search_enabled option as a reference for gateway-side deferral.

HOW OTHERS BUILD IT

Codex defers tools by default since v0.142.2 [C:111]. Claude Code turns tool search on above 10 percent of context [C:019]. Gateways add the same wrapper idea for clients without it.

REFERENCES

- C:019 How MCP Tool Definitions Inflate Your AI Agent Token Costs
- C:021 Dynamic Tool Gating and Lazy Schema Loading for Eliminating the MCP/Tools Tax in Scalable Agentic Workflows
- C:022 MCP Compression: Preventing tool bloat in AI agents
- C:055 How Claude Code uses prompt caching
- C:111 The Context Window Gap: Codex CLI caps GPT-5.6 at 272K
- C:104 Codex configuration reference
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:020 MCP Tool Overload: Why More Tools Make Your Agent Worse
- C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
- C:060 5 ways to cut Claude Code costs with LiteLLM
- A:199 Claude prompt caching docs

RECEIVES FROM

None in this stage view.

HANDS TO

Edge gateway
tool set, tool set

← Back to group: Context controls ← Stage 5

LEVEL 4

Tool-output cap

Context guard | Stage 5: Context guard

Tool-output cap

25,000 interactive, keep tracebacks

npm test result 15,000 tok -> cap 6,000
kept: 40 head, 40 tail, 3 failure frames

IN PLAIN ENGLISH

> Some tool results are huge, like a full test log. Token Economics trims them to a safe size and always keeps the error lines.

Goes in	Tool result block, tool name, route class (interactive or batch), token estimate.
Comes out	Same block, or a capped block with head, tail and preserved failure anchors plus a pagination hint.
Reads / writes	Reads cap values from policy. Writes one metadata row per capped result (tokens before and after).
If it fails	Fail open. Forward the result unchanged. Cap errors must never drop a traceback or break the tool-result pairing.
Requirement source	RQ-D-004, RQ-D-005, RQ-D-006, RQ-D-007 Sources: A:284, C:106, C:036, C:024, C:128, C:028, C:090, C:074
Hardening, still open	Conflict: tool outputs are 3.3 percent of billed cost [C:024] versus observations 84 percent of a turn in raw tokens [C:130]. Chosen: billed cost, so expected saving is small (ceiling about 5 percent [C:024]). Cap for batch is a range 6,000 to 12,000.
Not in client docs	**DevX proposal:** Head, tail and anchor-preserving split is a DevX design. Cap values are sourced.

WHAT IT IS FOR

Stop outlier tool results from entering and staying in context. The payoff is on outliers, such as a 610 KB spec reread [C:036]. The average share is low (3.3 percent of billed cost [C:024]).

EXPECTED

- Caps interactive results at 25,000 tokens and batch results at 6,000 to 12,000 [A:284, C:106].
- Keeps tracebacks, failing test names and edit anchors in full.
- Adds a pagination hint so the agent can fetch the next page.
- Never filters or reformats content inside the cap.

NOT EXPECTED

- No default output compressor [C:024].
- No rewriting of search or count output piped to a shell.
- No cap applied to results below the limit.

ALGORITHM AND MATHS

Cap with anchors

```
if tokens > cap: keep head H, tail Tl, all anchor spans; drop middle
```

STEPS

1. Estimate tokens of the result block.
2. If at or below cap for the route class, forward unchanged.
3. Find anchors: lines matching traceback frames, failure markers, and edit-anchor lines referenced in the last 5 tool calls.
4. Keep head and tail windows plus anchors until cap minus 200 tokens. Then insert a marker: "[N tokens omitted, page 2 at offset X]".
5. Emit the capped block and a metadata row.

PARAMETERS

interactive_cap_tokens	25,000 [A:284]
batch_cap_tokens	6,000 to 12,000 [C:106]
codex_tool_output_token_limit_default	12,000 [C:106]
hook_output_char_cap	10,000 [C:090]
anchor_marker_recall_floor	0.99 [C:024]

THE MATHS IN FULL

SYMBOLS

- s_out = share of billed cost from tool outputs (0.033 [C:024]).
- q = fraction of that share removed by the cap.
- R = marker recall of the cap on a labelled set.

FORMULA

```
saving_max = s_out * q;  ship only if R >= 0.99 and billed cost per successful task (CPS) does not rise
```

WHY IT IS RIGHT

Tool outputs are 3.3 percent of billed cost. Even removing all of them saves 3.3 percent. Tool-call arguments add 1.4 percent. Realistic saving is the share times the removed fraction.

WORKED EXAMPLE

```
q = 0.5: 0.033 * 0.5 = 1.65 percent of the bill. One failing npm test log is about 15,000 tokens, and about 50 are needed [C:128]. A cap at 6,000 removes 9,000 tokens. Sonnet 5.5 cache
write: 9,000*2.50/1e6 = $0.0225 once. Cache read: 9,000*0.10/1e6 = $0.0009 per turn [A:199].
```

COST OF COMPUTING IT

O(L) in result length.

HOW IT CAN GO WRONG, AND THE GUARD

- Cap cuts a traceback: recall drops (pytest filter 56.1 percent recall at 31.9 percent savings [C:024]).
- Cap pushes the agent into repeated searches (compression delayed first edit 0.94 turns [C:024]).
- Anchor list misses a needed line.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Sweep 6k, 12k, 25k per R3 experiment 6; measure re-read loops, first-edit turn and recall on a labelled failure set.

SECURITY

- Never log result content; the metadata row has token counts only.
- A marker string is generated by Token Economics; untrusted result text must not be able to forge a marker that hides content.

PERFORMANCE

Latency and throughput	Under 3 ms p95 for results up to 100,000 tokens (DevX proposal).
Complexity	Linear in result length.
At 10x load	Stateless; scales with gateway replicas.

EDGE CASES AND HOW EACH IS HANDLED

- Result is exactly at the cap: forward unchanged.
- Result is binary or base64: cap by bytes and mark as binary.
- Result contains no anchors and exceeds the cap: head and tail only.
- Result arrives in a streamed tool call: cap on the completed block only.

TEST CASES

TEST	GIVEN	EXPECT
under_cap	interactive result of 24,999 tokens	forwarded unchanged
over_cap_anchor	30,000-token result with traceback at line 29,000	traceback kept in full, marker inserted
batch_cap	batch route, 9,000-token result, cap 6,000	capped, hint present
shell_pipe	result flagged as input to a count command	no rewrite, event skip_shell_pipe
recall_gate	labelled set recall 0.97	policy refuses to enable, reason recall_below_0.99

WHERE IT SAVES COST

- Fewer tokens re-read on every later turn.
- Fewer outlier rereads of the same large file.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Re-read rate after a cap.
- First-edit turn.
- Marker recall on labelled failures.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.ocap.applied	session_id, tool, tokens_in, tokens_out, anchors_kept	each capped result
token_economics.ocap.skip	session_id, reason	result skipped
token_economics.ocap.reread	session_id, tool, offset	agent requests another page

RESEARCH BEHIND IT

C:024 Token Reduction Is Not Cost Reduction
Result we rely on: 2,848-run study: compressors gave -2.7 percent to +48.4 percent cost; sign flips by harness.
Used here: Justifies cap-only default and the 99 percent recall gate.

HOW OTHERS BUILD IT

Claude Code caps tool responses at 25,000 tokens and Codex ships a tool_output_token_limit. Vendor compressors claim 60 to 95 percent savings without a method [C:060].

REFERENCES

- A:284 Writing effective tools for AI agents
- C:106 Mastering Context Compaction in Codex CLI
- C:036 Codex Context Management: Auto-Compact and /context
- C:024 Token Reduction Is Not Cost Reduction
- C:128 How to Measure AI Agent Token Usage
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:090 Automate actions with hooks (guide)
- C:074 Hooks reference

RECEIVES FROM

None in this stage view.

HANDS TO

Edge gateway
tool output, tool output

← Back to group: Context controls ← Stage 5

LEVEL 4

Observation masking

Context guard | Stage 5: Context guard

Observation masking

Above about 10 events

events=23, window M=10 -> mask events 1..13
placeholder: [obs omitted: grep, 1,840 tok]

IN PLAIN ENGLISH

> Old tool results pile up in the conversation. Token Economics swaps the oldest ones for a one-line note but only once the conversation is long enough.

Goes in	Message history, event count since task start, observation blocks with token sizes, trigger state.
Comes out	History with older observation bodies replaced by a short placeholder, or unchanged.
Reads / writes	Reads policy (window M, trigger). Writes last-applied event index per session and a metadata row.
If it fails	Fail open. Send the history unchanged. A skipped mask costs tokens only.
Requirement source	RQ-D-008, RQ-D-009, RQ-D-010 Sources: C:130, C:133, C:122, C:055, C:066
Hardening, still open	Masking vs summaries: C:130 and C:133 (masking wins) versus C:122 (condensation up to 2x lower per-turn cost, no threshold). Chosen: masking. Window needs re-tuning per scaffold [C:130]. Software-engineering tasks only; other tasks not in corpus.
Not in client docs	**DevX proposal:** Gateway-side masking and the batch-slide step are DevX proposals. The published results come from agent scaffolds with their own history control [C:130, C:133].

WHAT IT IS FOR

Cut the cost of carried-over tool observations on long runs. Masking was the only condenser that saved cost in the DiscoveryBench study [C:133].

EXPECTED

- Fires only above about 10 events.
- Keeps the newest M=10 observations in full.
- Replaces older observation bodies with a placeholder that names the tool and original size.
- Applies at a trigger and then waits, to limit cache breaks.

NOT EXPECTED

- No LLM summary call.
- No edit below 10 events (loses 45 to 149 percent at 7 events or fewer [C:133]).
- No masking of protected, non-re-fetchable results.

ALGORITHM AND MATHS

Windowed observation masking

```
if E >= 10 and trigger_met: mask observations older than the newest M
```

STEPS

1. Count observation events E in the history.
2. If E < 10, do nothing and emit skip_short.
3. If tokens since last edit are below the trigger, do nothing (protects the cache).
4. Mask every observation older than the newest M=10, except protected tools.
5. Record the new masked index. The prefix up to that index is stable on later turns.

PARAMETERS

window_M	10 [C:130]
min_events	about 10 [C:133]
trigger_tokens	30,000 (DevX proposal, align with clear trigger [C:066])
slide_step	5 events (DevX proposal)

THE MATHS IN FULL

SYMBOLS

- c0 = cost of the unmasked run; c1 = cost with masking.
- E = number of observation events; M = window.
- p = cache-write price per token of the new suffix after an edit.

FORMULA

```
saving = (c0 − c1)/c0;  edit cost = (tokens after first masked block) ∗ (w − r);  fire when E >= 10 and projected saving > edit cost
```

WHY IT IS RIGHT

Masking removes old observation tokens from every later turn. It also changes the history, so the cache must re-write the suffix after the first changed block once. Slide in steps so the re-write happens rarely.

WORKED EXAMPLE

```
Gemini 2.5 Flash, M=10: $0.41 to $0.18, (0.41−0.18)/0.41 = 56.1 percent saving; LLM summary cost $0.24 [C:130]. Qwen3-Coder solve rate: masking 54.8 percent, summary 53.8 percent, raw 53.4
percent [C:130]. At 7 events or fewer, masking cost 45 to 149 percent more [C:133].
```

COST OF COMPUTING IT

O(E) per trigger.

HOW IT CAN GO WRONG, AND THE GUARD

- Masked result was needed later and the agent re-fetches it.
- Window too small for the scaffold.
- Frequent edits repeatedly break the cache.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Sweep M in 5, 10, 20 and trigger in 30k, 60k, 100k per scaffold (R3 experiment 2). Use billed cost with caching on.

SECURITY

- Placeholders must not leak the omitted content, only tool name and size.
- Masking must not remove audit-relevant safety events or permission prompts from the history.

PERFORMANCE

Latency and throughput	Under 10 ms p95 for histories up to 500 messages (DevX proposal).
Complexity	Linear in event count at trigger time only.
At 10x load	Per-session state is one integer.

EDGE CASES AND HOW EACH IS HANDLED

- Event count is exactly 10: no masking.
- Observation was produced by a protected tool: kept.
- History already edited by the client: detect by hash change and skip one cycle.
- Short task ends at event 9: no edit ever fires.

TEST CASES

TEST	GIVEN	EXPECT
short_skip	7 events, window 10	no edit, skip_short
mask_fires	23 events, trigger met	events 1..13 masked, 10 kept
protected	tool in exclude list at event 3	event 3 kept
no_trigger	23 events, 4,000 tokens since last edit, trigger 30,000	no edit
idempotent	apply twice to same history	second pass changes nothing

WHERE IT SAVES COST

- Fewer input tokens per later turn.
- No summary call cost.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Re-fetch rate of masked results.
- Solve rate versus control arm.
- Cache re-write tokens after each edit.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.mask.applied	session_id, events, masked_n, tokens_before, tokens_after	edit applied
token_economics.mask.skip	session_id, reason	no edit
token_economics.mask.refetch	session_id, tool	masked result requested again

RESEARCH BEHIND IT

C:130 The Complexity Trap: Observation Masking vs LLM Summarization
Result we rely on: Window M=10 masking cuts cost 56.1 percent with equal solve rate; software-engineering only.
Used here: Default window and comparison to summaries.

C:133 Evaluating Memory Condensation Strategies for Coding Agents in Data-Driven Scientific Discovery
Result we rely on: Only masking saved cost among 8 condensers; negative at 7 events or fewer.
Used here: Minimum event rule.

HOW OTHERS BUILD IT

OpenHands ships a context condenser with masking and LLM variants [C:122]. Most coding harnesses summarise instead; Token Economics prefers masking where the evidence supports it.

REFERENCES

- C:130 The Complexity Trap: Observation Masking vs LLM Summarization
- C:133 Evaluating Memory Condensation Strategies for Coding Agents in Data-Driven Scientific Discovery
- C:122 OpenHands Context Condensation for More Efficient AI Agents
- C:055 How Claude Code uses prompt caching
- C:066 Memory & context management with Claude (cookbook)

RECEIVES FROM

None in this stage view.

HANDS TO

Edge gateway
history edit, history edit

← Back to group: Context controls ← Stage 5

LEVEL 4

Context clearing

Context guard | Stage 5: Context guard

Context clearing

Trigger 30 to 40k, keep 5

ctx=36,000 -> clear_tool_uses, keep=5
clear_at_least=5,000; exclude: memory

IN PLAIN ENGLISH

> When the conversation gets long, the provider can drop old tool results. Token Economics asks for that only when the saving is bigger than the cost of rebuilding the cache.

Goes in	Token count of context, tool-result list, exclude_tools policy, session cache state.
Comes out	A context_management block that asks the provider to clear old tool results, or no change.
Reads / writes	Reads policy and last-clear token mark. Writes clear events and cache re-write tokens to metadata.
If it fails	Fail open. Do not add the clearing block. The request runs with the original context.
Requirement source	RQ-D-011, RQ-D-012, RQ-D-013, RQ-D-008 Sources: C:065, C:066, C:067, C:035, C:006, A:271, A:268, C:130, C:055, A:199
Hardening, still open	Conflict: API defaults trigger 100,000 and keep 3 [C:065, C:066] versus production guidance trigger 30,000 to 40,000 and keep 5 [C:066]. Chosen: guidance. Cookbook peak 335,279 to 173,137 is a single synthetic run without dollars [C:006]. The context editing -84 percent claim has no stated baseline [A:268].
Not in client docs	**DevX proposal:** The break-even test before firing is a DevX derivation. The thresholds come from Anthropic guidance.

WHAT IT IS FOR

Use server-side tool-result clearing (strategy clear_tool_uses_20250919 [C:065]) to cut the context when re-fetchable results dominate. The cookbook case had file reads at 96.3 percent of context [C:006].

EXPECTED

- Adds context_management with trigger 30,000 to 40,000, keep 5, clear_at_least 3,000 to 5,000.
- Sets exclude_tools for non-re-fetchable tools [C:067].
- Fires only if the break-even test passes.
- Keeps thinking blocks for 1 turn so the thinking cache survives [C:065].

NOT EXPECTED

- No clearing of results the agent cannot re-fetch.
- No firing on small yield.
- No firing when the client already manages context editing.

ALGORITHM AND MATHS

Break-even clearing

```
fire iff n_remaining * Y * r > P_after * (w - r) and ctx >= trigger
```

STEPS

1. If ctx tokens are below the trigger, stop.
2. Estimate Y, the tokens removable above clear_at_least, from the tool-result sizes older than the newest 5.
3. Estimate P_after, the tokens at and after the first cleared block (these must be re-written).
4. Estimate remaining turns n_remaining from the task baseline for this tenant.
5. If the break-even test passes, add context_management with keep and exclude_tools; else emit skip_no_payback.

PARAMETERS

trigger_tokens	30,000 to 40,000 [C:066]
keep	5 [C:066]
clear_at_least	3,000 to 5,000 [C:066]
api_default_trigger	100,000 [C:065]
api_default_keep	3 [C:065]
thinking_keep_turns	1 [C:065]

THE MATHS IN FULL

SYMBOLS

- Y = tokens cleared; P_after = tokens re-written after the first cleared block.
- r, w = cache-read and cache-write price per token.
- n = turns left in the task.

FORMULA

```
fire iff n*Y*r > P_after*(w − r);  break-even turns n* = P_after*(w − r)/(Y*r)
```

WHY IT IS RIGHT

Clearing removes Y tokens from every later read, saving Y*r per turn. It breaks the cached prefix after the first cleared block. So P_after tokens are written once at w instead of read at r.

WORKED EXAMPLE

```
Sonnet 5.5 5-minute write $2.50 and read $0.10 per MTok [A:199]. P_after = 20,000, Y = 5,000: extra = 20,000*2.40/1e6 = $0.048; saving per turn 5,000*0.10/1e6 = $0.0005; n* = 96 turns. Y =
20,000: saving $0.002 per turn; n* = 24 turns. Small yield rarely pays on cost alone.
```

COST OF COMPUTING IT

O(R) in tool-result count per trigger check.

HOW IT CAN GO WRONG, AND THE GUARD

- Fires often with small yield and thrashes the cache [C:066].
- Clears a result the agent needs; content is unrecoverable [C:065].
- Cache time to live (TTL) expired anyway, so the re-write is free.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Sweep clear_at_least 3k, 5k, 10k against compaction at 150K; count cache re-writes (R3 experiment 3).

SECURITY

- exclude_tools must always include memory and secret-bearing tools so they are never cleared and re-fetched from a weaker source.
- Clear events log counts only, never cleared content.

PERFORMANCE

Latency and throughput	Decision under 2 ms p95 (DevX proposal).
Complexity	Linear in tool-result count.
At 10x load	One counter per session.

EDGE CASES AND HOW EACH IS HANDLED

- Context just under trigger: no clear.
- Client sets its own context_management: skip and record.
- Cache already expired: treat w - r as 0 in the test.
- All old results are protected: Y = 0, skip.

TEST CASES

TEST	GIVEN	EXPECT
below_trigger	ctx 28,000, trigger 30,000	no action
pays_back	Y=20,000, P_after=20,000, n_remaining=30, Sonnet 5.5	n*=24 below 30, fire
no_payback	Y=5,000, P_after=20,000, n_remaining=40	n*=96 above 40, skip_no_payback
exclude	memory tool results in range	memory results kept
client_owns	request already has context_management	skip_client_owns

WHERE IT SAVES COST

- Lower tokens read per turn.
- Avoids a compaction call later.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Cache re-write tokens after clear.
- Tasks that re-fetch cleared content.
- Clear frequency per session.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.clear.fired`	session_id · ctx_tokens · y_tokens · p_after · n_remaining	clear requested
`token_economics.clear.skip`	session_id · reason	no clear
`token_economics.clear.refetch`	session_id · tool	cleared content fetched again

RESEARCH BEHIND IT

C:006 Context engineering: memory, compaction, and tool clearing
Result we rely on: Cookbook peak 335,279 to 173,137 tokens; one synthetic task.
Used here: Illustrates yield only; no dollars.

HOW OTHERS BUILD IT

Anthropic documents clear_tool_uses_20250919 with default trigger 100,000 and keep 3, and guidance to start at 30,000 to 40,000 [C:065, C:066]. OpenAI and Google have no equivalent server-side strategy in corpus.

REFERENCES

- C:065 Context editing on the Claude Developer Platform
- C:066 Memory & context management with Claude (cookbook)
- C:067 Context Editing Looks Like a Feature. It's Actually a Garbage Collector Without Write Barriers
- C:035 Context Management Across Codex Surfaces
- C:006 Context engineering: memory, compaction, and tool clearing
- A:271 Context engineering: memory, compaction, and tool clearing (Claude cookbook)
- A:268 Managing context on the Claude Developer Platform
- C:130 The Complexity Trap: Observation Masking vs LLM Summarization
- C:055 How Claude Code uses prompt caching
- A:199 Claude prompt caching docs

RECEIVES FROM

None in this stage view.

HANDS TO

Edge gateway
history edit

← Back to group: Context controls ← Stage 5

LEVEL 4

Compaction guard

Context guard | Stage 5: Context guard

Compaction guard

Last resort, restart at 3

compaction #3 at 188,000 tok (window 200,000)
recommend: restart with handoff file

IN PLAIN ENGLISH

> Compaction asks the model to summarise the whole chat. It costs real money and loses detail. Token Economics counts them and suggests a fresh start after three.

Goes in	Context tokens, window size, compaction count in session, harness compaction setting.
Comes out	Advisory flag or hook response: allow, warn, or recommend restart. Optional summary instructions.
Reads / writes	Reads harness thresholds from adapters. Writes compaction count and cost per session.
If it fails	Fail open. Let the harness compact on its own. Token Economics only counts and warns.
Requirement source	RQ-D-014, RQ-D-009, RQ-D-008 Sources: C:112, C:106, C:007, C:089, C:006, A:224, C:055, C:074, C:090, C:130, C:133, C:122
Hardening, still open	Threshold conflict: about 50 percent [C:112] vs about 90 percent [C:106] vs window minus 33,000 [C:007]. Cache effect conflict: C:112 says compaction destroys the cache. C:055 says it reuses the parent prefix at a 5-minute time to live (TTL). A:224 shows a miss and a net loss. No head-to-head test. Token Economics measures, and does not set the threshold.
Not in client docs	**DevX proposal:** Counting and the restart recommendation are sourced rules. Using PreCompact hooks as an advisory channel is a DevX design; hooks can be disabled so nothing here enforces.

WHAT IT IS FOR

Treat compaction as a last resort. Each one is a model call, can miss the cache, and drops specifics [C:112, C:106, C:006].

EXPECTED

- Counts compactions per session and shows the count to the user.
- Warns before the harness compacts if the context is below the break-even point.
- Recommends a restart with a handoff file at the third compaction.
- Uses PreCompact only to add summary instructions or to block, when hooks exist [C:074].

NOT EXPECTED

- No Token Economics-run summary call.
- No compaction below about 10 events [C:133].
- No claim that the compaction threshold is tuned; no head-to-head evidence exists.

ALGORITHM AND MATHS

Late-compaction advisor

```
warn if n_compactions >= 2; recommend_restart if n_compactions >= 3
```

STEPS

1. Read context tokens from the usage row after each response.
2. Detect a compaction by a drop in input tokens above 50 percent with a summary-sized block.
3. Increment the per-session counter and store the cost of the call from the ledger.
4. Run the break-even test (see math) before an expected compaction and warn if it fails.
5. At count 3, emit recommend_restart with a handoff-file hint.

PARAMETERS

restart_at_compactions	3 [C:106, A:224, C:112]
harness_threshold_claude_code	about 89 percent [C:112]
harness_threshold_codex	about 90 percent [C:112]
harness_threshold_gemini_cli	about 50 percent [C:112]
claude_code_formula	window - min(max_output, 20,000) - 13,000 [C:007, estimate]

THE MATHS IN FULL

SYMBOLS

- H = history tokens before; S = summary tokens; c_call = cost of the compaction call.
- r, w = cache-read and cache-write price per token.
- n = turns left.

FORMULA

```
pays iff n*(H − S)*r > c_call + S*w;  n* = (c_call + S*w)/((H − S)*r)
```

WHY IT IS RIGHT

After compaction each later turn reads S tokens in place of H. The one-time cost is the compaction call plus writing the summary to the cache.

WORKED EXAMPLE

```
H = 125,000; one compaction costs $0.40, about 21 cached turns [C:112, secondary]; S = 2,783 [C:006]. Sonnet 5.5 write $2.50 and read $0.10 per MTok [A:199] (the prices behind C:112 are not stated, so this mixes sources). Saved per turn (122,217)*0.10/1e6 = $0.01222. One-time 0.40 + 2,783*2.50/1e6 = $0.40696. n* = 33.3 turns. Grok example: 3 compactions cost 1.4, 1.7 and 3.3 cents. They removed 600 to 900 tokens, and the next turn missed the cache [A:224].
```

COST OF COMPUTING IT

O(1) per response.

HOW IT CAN GO WRONG, AND THE GUARD

- Summary drops obscure specifics (kept 3 of 3 high-level facts, 0 of 3 obscure ones [C:006]).
- Thrashing when one output refills the context [C:089].
- Repeated compactions reduce accuracy [C:106].

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Break-even at 50K, 100K, 200K contexts and accuracy after 1, 2, 3 compactions (R3 experiment 4).

SECURITY

- Never store or forward the summary text; keep only token counts and cost.
- A hook response must not inject text that overrides user instructions; summary instructions are limited to an allow-listed template.

PERFORMANCE

Latency and throughput	Under 1 ms per response for counter update (DevX proposal).
Complexity	Constant per response.
At 10x load	One counter per session.

EDGE CASES AND HOW EACH IS HANDLED

- Harness compacts on its own at an unexpected point: detect by the token drop and count it.
- Session is resumed: reload the counter from metadata.
- Two compactions within 3 turns: flag as thrash.
- No hooks present: advisory shown only in the dashboard.

TEST CASES

TEST	GIVEN	EXPECT
count_three	three detected compactions	recommend_restart on the third
break_even_fail	H=40,000 S=2,783 n_remaining=5	pays false, warn
break_even_pass	H=125,000 S=2,783 n_remaining=40, Sonnet 5.5	pays true (n*=33.3)
thrash	two compactions 2 turns apart	event token_economics.compact.thrash
short_session	8 events at threshold	no warning needed, advisory off

WHERE IT SAVES COST

- Fewer summary calls.
- Fewer cache misses after compaction.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Compactions per session.
- Cost of each compaction.
- Post-compaction success rate.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.compact.detected`	session_id · h_tokens · s_tokens · cost_usd · count	compaction seen
`token_economics.compact.warn`	session_id · reason	advisory raised
`token_economics.compact.restart_rec`	session_id · count	third compaction

RESEARCH BEHIND IT

C:112 Context Compaction Showdown: Codex CLI, Claude Code and 5 Other Agents
Result we rely on: One compaction at 125,000 tokens costs $0.40, about 21 cached turns; secondary source.
Used here: Worked example input.

C:133 Evaluating Memory Condensation Strategies for Coding Agents in Data-Driven Scientific Discovery
Result we rely on: LLM condensers cost 24 to 94 percent more with no significant quality gain.
Used here: Reason to avoid summaries below 10 events.

HOW OTHERS BUILD IT

Compaction thresholds differ by harness. Gemini CLI is about 50 percent, Claude Code about 89 percent, Codex about 90 percent [C:112]. OpenHands condensers report 54 vs 53 percent solve [C:122].

REFERENCES

- C:112 Context Compaction Showdown: Codex CLI, Claude Code and 5 Other Agents
- C:106 Mastering Context Compaction in Codex CLI
- C:007 Shedding Heavy Memories: Context Compaction in Codex, Claude Code, and OpenCode
- C:089 How Claude Code works
- C:006 Context engineering: memory, compaction, and tool clearing
- A:224 Prompt Caching: Cut the Cost of Long Chats with the Grok API
- C:055 How Claude Code uses prompt caching
- C:074 Hooks reference
- C:090 Automate actions with hooks (guide)
- C:130 The Complexity Trap: Observation Masking vs LLM Summarization
- C:133 Evaluating Memory Condensation Strategies for Coding Agents in Data-Driven Scientific Discovery
- C:122 OpenHands Context Condensation for More Efficient AI Agents

RECEIVES FROM

None in this stage view.

HANDS TO

Edge gateway
summary, summary

← Back to group: Context controls ← Stage 5

LEVEL 4

Max-tokens enforcer

Context guard | Stage 5: Context guard

Max-tokens enforcer

Hard cap on every call

max_tokens missing -> set 8,192 (tenant policy)
reserve = 8,192 x output price

IN PLAIN ENGLISH

> Every call must say how long the answer may be. Token Economics adds a limit if the client forgot, so one runaway answer cannot burn the budget.

Goes in	Request body, route class, tenant policy, model id, worst-case reserve input.
Comes out	Request with a hard max_tokens value, or a rejection when none can be set.
Reads / writes	Reads per-tenant and per-model cap policy. Writes cap value into the reservation record in the budget path.
If it fails	Fail closed. Reject with 503 policy_unavailable if no cap can be resolved. An uncapped call makes the worst-case reserve unbounded.
Requirement source	RQ-D-015, RQ-D-016, RQ-D-017 Sources: C:073, C:093, A:197, A:052, C:027, C:136, A:104
Hardening, still open	Not in corpus: a safe default cap per task class. A 300-token cap broke 7 of 25 hard tasks [A:052]; OpenAI guidance reserves at least 25,000 tokens for reasoning [A:197]. Effort control does not cap tokens [C:073]. Hosted-API support for output-length control methods is not in corpus.
Not in client docs	**DevX proposal:** Cap values, floors and the 503 behaviour are DevX proposals. The corpus gives no numeric default.

WHAT IT IS FOR

Put an upper bound on output tokens. At a 96.3 percent cache hit rate they are 2.7 percent of tokens and 51.1 percent of dollars [C:027].

EXPECTED

- Sets max_tokens on every call that lacks one.
- Lowers an oversized client value to the policy cap only when policy says so.
- Applies a per-class floor so the cap cannot be set below the floor.
- Passes the final value to the reserve step.

NOT EXPECTED

- No reliance on effort or prompt text to limit output [C:073, C:136].
- No cap below the class floor.
- No silent change of a value the client set lower than policy.

ALGORITHM AND MATHS

Resolve and enforce cap

```
cap = clamp(client_value or policy_default, floor_class, ceiling_policy)
```

STEPS

1. Resolve class (interactive, batch, reasoning) from route and model.
2. Read policy default, floor and ceiling for tenant and class.
3. If client value is absent, use the default. If present, clamp to the ceiling.
4. If the resolved value is below the floor, raise to the floor and emit a warning event.
5. Write the value to the request and the reservation record.

PARAMETERS

default_cap	per tenant policy (DevX proposal, no corpus value)
floor_cap	per class (DevX proposal; 300 broke 7 of 25 tasks [A:052])
reasoning_reserve_tokens	at least 25,000 for OpenAI reasoning [A:197]

SECURITY

- The tenant cannot raise its own ceiling through a request header; only policy sets it.
- Rejections return a stable error code without echoing the request body.

PERFORMANCE

Latency and throughput	Under 0.5 ms added latency p99 (DevX proposal).
Complexity	Constant time.
At 10x load	Stateless.

EDGE CASES AND HOW EACH IS HANDLED

- Client sets max_tokens above the ceiling.
- Model uses reasoning tokens that count inside max_tokens; reserve must include them.
- Model id unknown to policy: reject, not default (unpriced models escape budgets [A:242]).
- Batch route with larger ceiling.

TEST CASES

TEST	GIVEN	EXPECT
missing	no max_tokens, default 8,192	request sent with 8,192
above_ceiling	client 64,000, ceiling 32,000	clamped to 32,000
below_floor	client 300, floor 1,024 for class hard	raised to 1,024, warning event
no_policy	policy unreadable	503 policy_unavailable, no dispatch
reasoning_reserve	OpenAI reasoning model, cap 8,000	reserve adds 25,000 reasoning tokens

WHERE IT SAVES COST

- Bounds output cost per call.
- Makes the reserve step exact.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Truncation rate (stop_reason max_tokens).
- Retry rate after truncation.
- Floor warnings per class.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.mtok.set`	tenant · model · class · value · source	cap applied
`token_economics.mtok.floor`	tenant · model · client_value · floor	value raised to floor
`token_economics.mtok.reject`	tenant · reason	no cap resolvable

RESEARCH BEHIND IT

A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels
Result we rely on: maxOutputTokens=300 broke 7 of 25 hard tasks.
Used here: Basis for the per-class floor.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- LiteLLM: Per-key budgets and budget fallbacks as the surrounding enforcement path.

HOW OTHERS BUILD IT

Gateways usually forward max_tokens untouched and rely on budgets after the fact. Token Economics sets the cap before dispatch so the reserve is bounded.

REFERENCES

- C:073 Model configuration and effort levels
- C:093 Choosing a Claude model and effort level in Claude Code
- A:197 Reasoning models
- A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels
- C:027 Can your AI agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks
- C:136 Coding agents ignore spend (Ramp Labs)
- A:104 The Benefits of a Concise Chain of Thought on Problem-Solving in Large Language Models

RECEIVES FROM

None in this stage view.

HANDS TO

Edge gateway
max_tokens, max_tokens

← Back to group: Context controls ← Stage 5

LEVEL 4

Instruction-file linter

Context guard | Stage 5: Context guard

Instruction-file linter

CLAUDE.md 200 lines, AGENTS.md 32 KiB

AGENTS.md 41 KiB > 32 KiB: truncated silently
CLAUDE.md 214 lines > 200: warn

IN PLAIN ENGLISH

> Rule files are read on every turn, so long ones cost money and some get cut off without warning. Token Economics checks their size and tells you before it hurts.

Goes in	Instruction files (CLAUDE.md, AGENTS.md, MEMORY.md), skill descriptions, file sizes.
Comes out	Lint findings with file path, size, limit and severity.
Reads / writes	Reads local files at scan time. Writes counts and findings (no file text) to the policy store.
If it fails	Fail open. Advisory only. A failed scan blocks nothing.
Requirement source	RQ-D-018, RQ-D-019 Sources: C:077, C:036, A:269, C:075, C:079, C:104, C:028, C:074
Hardening, still open	Saving from instruction-file caps is not in corpus. Stale skills are re-billed with each prefix [C:028], but no size is given. Caps differ by harness: 200 lines vs 32 KiB vs 25 KB.
Not in client docs	**DevX proposal:** Finding severity and the confidence interval (CI) and SessionStart scan placement are DevX proposals. Saving figure is not in corpus.

WHAT IT IS FOR

Keep always-loaded instruction files under harness limits so they do not bloat the prefix or get silently truncated.

EXPECTED

- Flags CLAUDE.md above 200 lines [C:077].
- Flags AGENTS.md above 32 KiB and warns of silent truncation [C:036].
- Flags MEMORY.md above 200 lines or 25 KB [A:269].
- Flags skill descriptions above 1,536 characters and Codex skills above 10,000 [C:075, C:104].

NOT EXPECTED

- No auto-edit of files.
- No upload of file content.
- No block of a session start.

ALGORITHM AND MATHS

Size and staleness lint

```
finding if size > limit(file_type)
```

STEPS

1. Locate instruction files from adapter paths and the InstructionsLoaded hook when present [C:074].
2. Measure lines, bytes and per-skill description length locally.
3. Compare to limit table; raise warn at 90 percent, error above limit (90 percent is a DevX proposal).
4. Mark skills not invoked in 30 days as stale candidates (DevX proposal).
5. Send counts to the policy store.

PARAMETERS

claude_md_lines	200 [C:077]
agents_md_bytes	32 KiB [C:036]
memory_md	200 lines or 25 KB [A:269]
skill_description_chars	1,536 [C:075]
skills_reattach_after_compaction	5,000 each, 25,000 total [C:075, C:079]
codex_skills_window_share	2 percent, cap 10,000 [C:104]

SECURITY

- The scan runs locally and sends sizes only; file text may hold secrets.
- Findings must not include file excerpts.

PERFORMANCE

Latency and throughput	Scan of a repo under 1 s (DevX proposal).
Complexity	Linear in number of instruction files.
At 10x load	Runs on the developer machine or CI, not the gateway.

EDGE CASES AND HOW EACH IS HANDLED

- File is a symlink to a shared file.
- Encoding with multi-byte characters changes byte count.
- No instruction file present.
- Skill directory with hundreds of skills.

TEST CASES

TEST	GIVEN	EXPECT
claude_long	CLAUDE.md 214 lines	warn, limit 200
agents_big	AGENTS.md 41 KiB	error, silent truncation warning
memory_ok	MEMORY.md 150 lines 12 KB	no finding
skill_desc	skill description 1,700 chars	warn, limit 1,536
no_content	any finding	contains no file text

WHERE IT SAVES COST

- Smaller always-loaded prefix.
- Fewer stale skills re-billed.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Findings fixed per week.
- Prefix tokens before and after fixes.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.ilint.finding	file_type, size, limit, severity	each finding
token_economics.ilint.scan	files, findings	each scan

REFERENCES

- C:077 How Claude remembers your project
- C:036 Codex Context Management: Auto-Compact and /context
- A:269 The components of agent memory
- C:075 Extend Claude with skills
- C:079 Explore the context window
- C:104 Codex configuration reference
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:074 Hooks reference

RECEIVES FROM

None in this stage view.

HANDS TO

Policy-as-code store
lint result, lint result

← Back to group: Context controls ← Stage 5

LEVEL 4

Prompt-wording lint

Context guard | Stage 5: Context guard

Prompt-wording lint

No default think-deeply text

"consider multiple approaches" at line 12
warn: 2.4 to 7.4x reasoning tokens [C:026]

IN PLAIN ENGLISH

> Some prompt phrases make the model think far more without being right more often. Token Economics flags those phrases and any timestamp at the top of a prompt.

Goes in	System prompt files, skill text, custom command text, first 40 lines of each.
Comes out	Lint findings that name risky wording and the measured cost range.
Reads / writes	Reads local prompt files. Writes finding counts to the policy store.
If it fails	Fail open. Advisory only.
Requirement source	RQ-D-020, RQ-D-021, RQ-D-022 Sources: C:026, A:224, A:204, C:055, A:104
Hardening, still open	Effort control saved 19 percent under one harness and 75 percent under another [C:026]. Findings are warnings, not proof. Conflict: "be concise" cut cost 22.67 percent but cost 27.69 percent accuracy on GPT-3.5 math [A:104].
Not in client docs	**DevX proposal:** Pattern lists and severity are DevX proposals. Cost ranges are sourced.

WHAT IT IS FOR

Catch prompt wording that raises reasoning cost without raising success, and volatile text that breaks cache reuse.

EXPECTED

- Flags default text asking for multiple approaches (2.4 to 7.4x reasoning tokens) [C:026].
- Flags default "think deeply" wording (1.6 to 2.2x) [C:026].
- Flags timestamps and per-request values in the first lines of a prompt (reuse fell 87 to 3 percent [A:224]).
- Reports how many findings exist per tenant.

NOT EXPECTED

- No rewrite of prompts.
- No default recommendation to add "be concise" [A:104].
- No claim that the wording changes success; the evidence shows no gain, not harm.

ALGORITHM AND MATHS

Pattern lint

```
finding if pattern matches in scope
```

STEPS

1. Load pattern list from policy (multi-approach, deep-thinking, stop-early, volatile-top).
2. Scan prompt files; for volatile-top, check the first 40 lines for date, time or uuid patterns (40 is a DevX proposal).
3. Attach the sourced cost range to each finding.
4. Send counts only.

PARAMETERS

multi_approach_cost	2.4 to 7.4x reasoning tokens [C:026]
deep_think_cost	1.6 to 2.2x [C:026]
volatile_top_effect	reuse 87 to 3 percent, cost 2.2 to 2.9x [A:224]
stop_early_effect	3 of 5 hidden-test failures [C:026]

SECURITY

- Scan locally; send no prompt text.
- Pattern list is policy-controlled so a tenant cannot hide findings by editing it locally.

PERFORMANCE

Latency and throughput	Under 500 ms per repo (DevX proposal).
Complexity	Linear in prompt file size.
At 10x load	Runs on the developer machine or confidence interval (CI).

EDGE CASES AND HOW EACH IS HANDLED

- Pattern appears inside a quoted example.
- Date appears in a footer, not the top.
- Non-English prompt text.
- Template variable that renders to a timestamp at runtime.

TEST CASES

TEST	GIVEN	EXPECT
multi	prompt contains "try multiple approaches"	finding with range 2.4 to 7.4x
timestamp_top	line 2 is "Today is 2026-11-09"	volatile-top finding
timestamp_footer	date on line 300	no finding
clean	plain prompt	zero findings
no_text	any finding	contains no prompt text

WHERE IT SAVES COST

- Lower reasoning tokens.
- Higher prefix reuse.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Findings resolved.
- Reasoning tokens per task before and after.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.pwl.finding`	rule · file · severity	each finding
`token_economics.pwl.scan`	files · findings	each scan

RESEARCH BEHIND IT

C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
Result we rely on: 4,644 preregistered runs: multiple-approaches wording cost 2.4 to 7.4x reasoning tokens with no success gain.
Used here: Pattern and cost ranges.

A:104 The Benefits of a Concise Chain of Thought on Problem-Solving in Large Language Models
Result we rely on: Be concise cut GPT-3.5 accuracy 27.69 percent.
Used here: Reason brevity wording is not a default.

HOW OTHERS BUILD IT

Prompt linters usually check style. Cost-aware wording lint is new in this corpus.

REFERENCES

- C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
- A:224 Prompt Caching: Cut the Cost of Long Chats with the Grok API
- A:204 Prompt cache diagnostics
- C:055 How Claude Code uses prompt caching
- A:104 The Benefits of a Concise Chain of Thought on Problem-Solving in Large Language Models

RECEIVES FROM

None in this stage view.

HANDS TO

Policy-as-code store
lint result

← Back to group: Context controls ← Stage 5

LEVEL 2

6. Insight

Metadata store, attribution, dashboards and anomaly checks. Metadata only by default.

Stores and attribution

Metadata-only store, spend attribution and the content-logging policy.

1 Store · 1 Calculator · 1 Control plane

Metadata store · Spend attribution · Content-logging policy

Receives from: Client signals (Ingress and adapters), Metering (Accounting), Security and data (Control plane)
Hands to: Client signals (Ingress and adapters), Views and alerts

Open group (3 components) →

Views and alerts

Dashboards and one flagged anomaly detector.

2 Calculator

Dashboards and reports · Anomaly detector

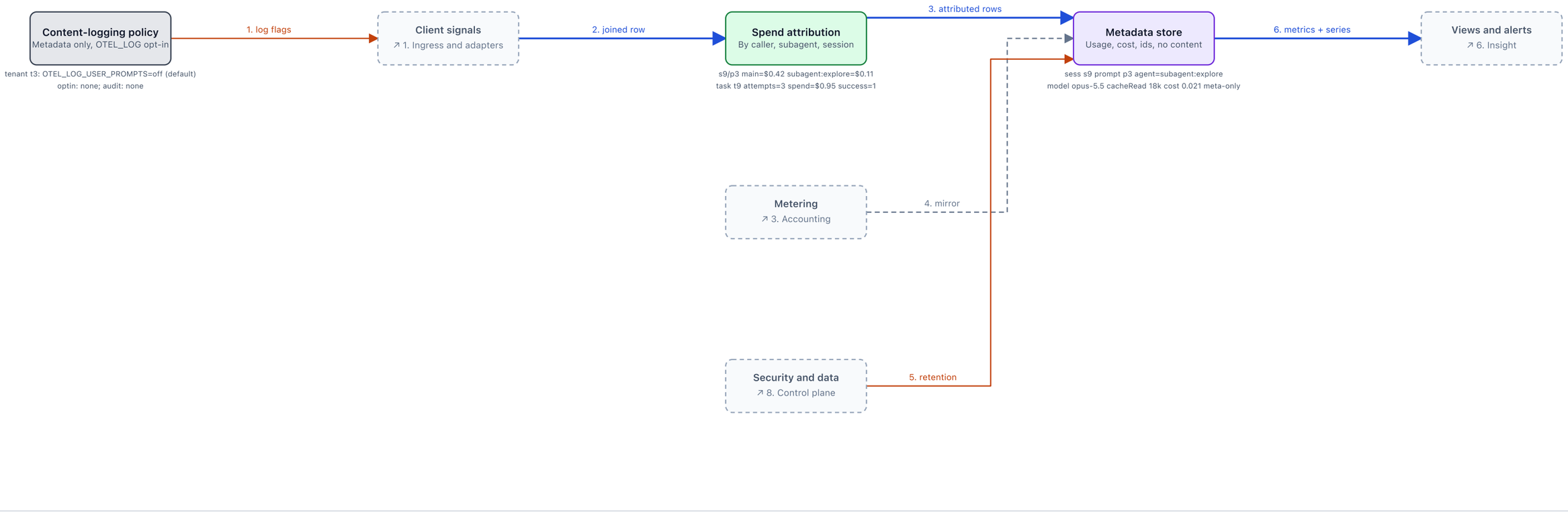
Receives from: Stores and attribution
Hands to: Rollout safety (Eval and rollout)

Open group (2 components) →

2 groups, 5 components. Hover a group for the plain-English story; click it to see its components and the arrows between them. ← Context guard Eval and rollout →

LEVEL 3 Stores and attribution in Insight

Metadata-only store, spend attribution and the content-logging policy.



Control plane · Calculator · Store · main flow · read / call · scheduled / async · security · outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Content-logging policy → Client signals (group)
Content logging stays off unless opted in.

2. Client signals (group) → Spend attribution
One row per session and prompt id, with cost_source.

3. Spend attribution → Metadata store
Spend by caller, subagent and session.

4. Metering (group) → Metadata store
Metadata-only copy for dashboards.

5. Security and data (group) → Metadata store
Expiry and redaction on stored rows.

6. Metadata store → Views and alerts (group)
metrics: Median and P95 per user, hit rate, waste share. + series: Cost and token series for outlier checks.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Insight Views and alerts →

LEVEL 4

Metadata store

Store | Stage 6: Insight

Metadata store

Usage, cost, ids, no content

sess s9 prompt p3 agent=subagent:explore
model opus-5.5 cacheRead 18k cost 0.021 meta-only

IN PLAIN ENGLISH

> This is the store behind the charts. It keeps counts, costs and ids, and never keeps what anyone typed.

Goes in	Metadata-only mirror rows from the ledger, attributed rows from attr, and retention rules.
Comes out	Query-ready tables for dashboards and anomaly checks. No prompt or response content.
Reads / writes	Writes and reads metadata tables (usage, cost, ids, hit rates, canary results). Retention expiry set by the retention component.
If it fails	Fail open: dashboards may lag, and requests are never blocked. The ledger remains the source of truth and can rebuild the store.
Requirement source	RQ-B-041, RQ-B-042, RQ-B-037, RQ-B-046, RQ-B-047, RQ-B-045 Sources: A:217, C:074, C:052, C:054, C:076, C:003
Hardening, still open	Open: retention duration is not in the corpus. Claude Code transcripts are kept about 30 days according to one source [C:003]. Cardinality limits on OTel attributes are listed but their effect on cost is not measured.
Not in client docs	**DevX proposal:** Expiry rules and the rebuild-from-ledger approach are DevX proposals.

WHAT IT IS FOR

Hold the data that dashboards and anomaly checks read. It keeps usage, cost and ids without content, so it is safe to share broadly.

EXPECTED

- Holds the UsageEvent fields: harness, session_id, prompt_id, agent, model, input, output, cache_read, cache_write, cost_estimate_usd, cost_source, auth_mode.
- Adds price_row_id, query_source, skill name, MCP server name, effort, speed and task id where known [A:217].
- Stores per-caller cache hit rates, cache miss events and reconciliation or canary results.
- Rows carry an expires_at set by the retention policy.
- Rebuildable from the ledger if lost.

NOT EXPECTED

- No prompts, tool inputs, tool outputs or responses.
- No secrets or provider keys.
- No source of truth role. The ledger holds that.

ALGORITHM AND MATHS

Columnar metadata store with retention

```
hit_rate = cache_read / (fresh + cache_read + cache_write) per caller, model and prefix hash
```

STEPS

1. Receive mirrored ledger rows and attributed rows.
2. Validate against the content-free schema. Reject rows with unknown text fields.
3. Store with tenant id, partition by day.
4. Maintain rollups: spend per caller, per model, per user (median and 95th percentile (P95)), hit rate.
5. Set expires_at and let the retention job delete expired partitions.
6. On request, rebuild rollups from the ledger.

PARAMETERS

hit_rate	cache_read / (fresh + read + write) [C:052]
cache_miss_event	more than 5 percent and at least 2,000 tokens reprocessed [C:076]
partition	tenant and day (DevX proposal)

SECURITY

- Schema validation rejects free-text fields, so content cannot enter by accident.
- Row access is scoped by tenant, and user-level views follow the authorisation roles.
- Content logging stays off by default [A:217].

PERFORMANCE

Latency and throughput	Rollup lag under 5 minutes (DevX proposal).
Complexity	O(1) per insert, rollups incremental.
At 10x load	Partition by tenant and day, archive old partitions.

EDGE CASES AND HOW EACH IS HANDLED

- Hook, OTel and gateway rows for one prompt must join on session_id and prompt_id [C:074].
- OTel may be missing when a developer turns it off, so rows carry a source field.
- A rollup has a high-cardinality dimension such as repository. Limit by config.
- Rows from a transcript replay are labelled client estimate [R4 edge: trans to joiner].
- Transcript retention is about 30 days, so replay data must be copied early [C:003].

TEST CASES

TEST	GIVEN	EXPECT
no_content	row with a field named prompt_text	rejected by schema
hit_rate	fresh 100, read 900, write 0	hit rate 0.90
expiry	row with expires_at yesterday	removed by retention job
rebuild	drop rollups then rebuild from ledger	rollups equal original

WHERE IT SAVES COST

- Fast, cheap queries on cost and hit rate point teams to the biggest savings.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- meta.rollup_updated
- meta.row_rejected

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.meta.row_rejected`	reason, field	Schema validation fails.
`token_economics.meta.rollup`	dimension, rows, lag_ms	A rollup refresh ends.

REFERENCES

- A:217 Monitoring usage (Claude Code OpenTelemetry)
- C:074 Hooks reference
- C:052 Your cache hit rate is lying to you. It's the caller mix.
- C:054 The Coding Agent Economy
- C:076 Customize your status line
- C:003 How to Track Claude Code Usage in 2026: Built-in Commands, ccusage, and Desktop Dashboards Compared

RECEIVES FROM

- Append-only ledger
  mirror, mirror
- Spend attribution
  attributed rows
- Retention and redaction
  retention, retention

HANDS TO

- Dashboards and reports
  metrics
- Anomaly detector
  series

← Back to group: Stores and attribution ← Stage 6

LEVEL 4

Spend attribution

Calculator | Stage 6: Insight

Spend attribution

By caller, subagent, session

s9/p3 main=$0.42 subagent:explore=$0.11
task t9 attempts=3 spend=$0.95 success=1

IN PLAIN ENGLISH

> This part works out who spent what, such as which person, which helper agent and which task. It also adds failed attempts to the task, so a task shows its true cost.

Goes in	Joined rows from the joiner: gateway, hook and OTel rows by session id and prompt id, with cost_source.
Comes out	Spend by caller, subagent, session, task, repository and model, written to the metadata store.
Reads / writes	Reads the ledger and joined rows. Writes attributed rows to meta.
If it fails	Fail open: if the join is incomplete, rows are stored as unattributed with the missing key. Requests are never blocked.
Requirement source	RQ-B-008, RQ-B-006, RQ-B-037, RQ-B-038, RQ-B-039, RQ-B-040 Sources: A:216, A:217, C:002, C:074, C:052, C:028, C:122, A:207, C:054, C:023
Hardening, still open	Open: Codex equivalents of prompt.id and query_source are not in the corpus. A measured agent-team token multiplier is not in the corpus; R6 says the system must measure it. Hooks can be disabled by developers, so hook-based attribution can have gaps [C:074].
Not in client docs	**DevX proposal:** The unattributed fallback and the task id rule for non-SDK harnesses are DevX proposals.

WHAT IT IS FOR

Give every dollar an owner: caller, subagent, session, task, repository and model. Without it a high account hit rate or a low mean can hide the real problem.

EXPECTED

- Joins gateway, hook and OTel rows on session_id and prompt_id. Hook prompt_id equals OTel prompt.id [C:074].
- Uses query_source (main, subagent, auxiliary), agent.name, skill.name and mcp_server.name to attribute subagents separately [A:217, A:216].
- Counts a call once even if SDK telemetry and the gateway both reported it [A:216].
- Adds failed and retried attempts to the task total, so cost per successful task = total over all attempts / successes [C:122, C:028].
- Computes failed-run waste share and spend after the first repeated failure signature [C:028].
- Stores unattributed rows with the missing join key.

NOT EXPECTED

- No content access.
- No blame scoring of users. Output is spend, not performance.
- No budget enforcement.

ALGORITHM AND MATHS

Key join with precedence and task roll-up

```
CPS(task) = sum(cost of all attempts) / successes
```

STEPS

1. Take a joined row. Prefer gateway cost; keep OTel and hook rows as cross-checks.
2. Resolve caller from the virtual key and from OTel query_source.
3. Attach task id and attempt number when the harness provides them, else derive a session-level task.
4. Roll up spend by dimension (caller, subagent, session, task, repository, model).
5. Compute failed-run waste share = failed spend / total spend.
6. Flag the point where the first repeated failure signature appears and sum spend after it.
7. Write attributed rows to meta.

PARAMETERS

join_keys	session_id, prompt_id [C:074]
caller_source	query_source values main, subagent, auxiliary [A:217]
baseline_failed_spend_share	32 percent of spend, 17 percent of tasks [C:028]
baseline_spend_after_first_repeat	51 percent [C:028]

SECURITY

- Attribution uses ids. It stores no prompts or code.
- User-level views follow the authorisation roles, and the default dashboard shows teams.
- Hook rows are untrusted input. The gateway row wins on conflict.

PERFORMANCE

Latency and throughput	Attributed rows within 5 minutes of the request (DevX proposal).
Complexity	O(1) per row for join, incremental roll-ups.
At 10x load	Partition by tenant and session.

EDGE CASES AND HOW EACH IS HANDLED

- Subagent calls have no cache from the parent [C:052].
- A resumed session must not add earlier spend again [A:216].
- Hooks disabled: attribution falls back to gateway ids.
- SDK reports placeholder output tokens per step, final result carries the count [A:216].
- The same task ran on two harnesses with different ids.

TEST CASES

TEST	GIVEN	EXPECT
join	hook p3 and OTel prompt.id p3 and gateway row	one attributed row with cost from gateway
subagent	query_source agent.builtin.general-purpose	spend credited to subagent, not main
cps	task with 3 attempts costing $0.30, $0.40, $0.25, one success	CPS $0.95
waste	failed spend $32 of $100	waste share 0.32
unattributed	row with no prompt_id	stored with missing_key prompt_id

WHERE IT SAVES COST

- Shows which callers, subagents and tasks drive spend, so the biggest lever is chosen first.
- Failed-run waste (32 percent of spend in C:028) is a direct target for the loop guard.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- attr.row_attributed
- attr.unattributed

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.attr.row	session_id · prompt_id · caller · query_source · model_id · cost_usd	A row is attributed.
token_economics.attr.unattributed	session_id · missing_key	A join key is missing.

RESEARCH BEHIND IT

C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
Result we rely on: Failed runs are 17 percent of tasks and 32 percent of spend. 51 percent of spend falls after the first repeated failure.
Used here: Sets the waste share metrics.

C:023 How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks
Result we rely on: Same-task runs differ up to 30x in tokens.
Used here: Task-level, not token-level, attribution.

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- ccusage: Local JSONL usage reports for many agent CLIs, a source for replay.

HOW OTHERS BUILD IT

Claude Code OTel attaches prompt.id, query_source, agent.name and skill.name to metrics and events. Token Economics joins them with gateway rows and prefers the gateway cost.

REFERENCES

- A:216 Track cost and usage (Claude Agent SDK)
- A:217 Monitoring usage (Claude Code OpenTelemetry)
- C:002 Claude Code OpenTelemetry: Track Cost and Tool Use
- C:074 Hooks reference
- C:052 Your cache hit rate is lying to you. It's the caller mix.
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:122 OpenHands Context Condensation for More Efficient AI Agents
- A:207 The builder's guide to GPT-5.6
- C:054 The Coding Agent Economy
- C:023 How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks

RECEIVES FROM

Session and prompt-id joiner
joined row, joined row

HANDS TO

Metadata store
attributed rows

← Back to group: Stores and attribution ← Stage 6

LEVEL 4

Content-logging policy

Control plane | Stage 6: Insight

Content-logging policy

Metadata only, OTEL_LOG opt-in

tenant t3: OTEL_LOG_USER_PROMPTS=off (default)
optin: none; audit: none

IN PLAIN ENGLISH

> By default Token Economics keeps counts and costs only, never what people wrote. A team can switch content logging on, but it is written down and expires.

Goes in	Tenant policy flags and client OTel settings (OTEL_LOG_USER_PROMPTS and related).
Comes out	The effective content-logging policy per tenant and harness, with an audited opt-in and expiry.
Reads / writes	Reads and writes policy records through the policy store. Writes opt-in events to the audit log.
If it fails	Fail closed for content: if the policy is unreadable, content logging stays off. Metadata collection continues.
Requirement source	RQ-B-042, RQ-B-043, RQ-B-044 Sources: A:217
Hardening, still open	Open: Codex OpenTelemetry (OTel) event names and logging flags are not in the corpus. Retention duration for opted-in content is not in the corpus. Developers can change local env vars unless managed settings pin them.
Not in client docs	**DevX proposal:** Per-tenant opt-in with expiry, the audit event and the enforcement through managed settings are DevX proposals. The corpus says content logging is opt-in.

WHAT IT IS FOR

Keep content out of the system by default. Prompt, tool and response logging are opt-in switches in Claude Code, and this component holds the policy for them.

EXPECTED

- Default state: OTEL_LOG_USER_PROMPTS, OTEL_LOG_TOOL_DETAILS, OTEL_LOG_TOOL_CONTENT, OTEL_LOG_ASSISTANT_RESPONSES and OTEL_LOG_RAW_API_BODIES are all off [A:217].
- Pins the values through managed settings so developers cannot turn content logging on locally.
- An opt-in names the tenant, the flags, the reason and an expiry date, and is written to the audit log.
- The OTel receiver drops content fields if a client sends them without an active opt-in.
- Policy is versioned in the policy store.

NOT EXPECTED

- No content storage by default.
- No silent opt-in by a client env var.
- No retention logic. The retention component owns expiry.

ALGORITHM AND MATHS

Default-off policy with expiring opt-in

```
effective_flag = opted_in(tenant, flag) and now < opt_in_expiry
```

STEPS

1. Load the tenant policy. Default every content flag to off.
2. On opt-in, require role, reason and expiry, then write the audit event.
3. Push the effective flags to the managed settings and to the OTel receiver.
4. The receiver strips content attributes whose flag is off.
5. A daily check expires opt-ins past their date and logs the change.

PARAMETERS

content_flags	OTEL_LOG_USER_PROMPTS, OTEL_LOG_TOOL_DETAILS, OTEL_LOG_TOOL_CONTENT, OTEL_LOG_ASSISTANT_RESPONSES, OTEL_LOG_RAW_API_BODIES [A:217]
default	off
max_opt_in_days	30 (DevX proposal)

SECURITY

- Opt-in needs an admin role and a recorded reason.
- The receiver strips content fields at ingress, so a client mistake cannot store content.
- Raw API bodies can hold secrets, so that flag needs a second approver (DevX proposal).

PERFORMANCE

Latency and throughput	Policy change reaches the receiver within 5 minutes (DevX proposal).
Complexity	O(1) per tenant lookup.
At 10x load	Small policy table.

EDGE CASES AND HOW EACH IS HANDLED

- A developer sets OTEL_LOG_RAW_API_BODIES locally. The receiver drops the bodies.
- Opt-in expires mid-session. Logging stops at the next flush.
- Harness has no content flags in the corpus, such as Codex. Policy marks it unknown and treats it as off.
- Two tenants share a gateway. Policy applies per tenant id.

TEST CASES

TEST	GIVEN	EXPECT
default_off	new tenant, no opt-in	all five flags effective off
optin_expiry	opt-in for 7 days, day 8	flags off, audit event written
strip_content	client sends user_prompt with flag off	content attribute removed, metadata kept
no_reason	opt-in without a reason	rejected

WHERE IT SAVES COST

- Metadata-only collection keeps storage and review cost low and removes the need for content redaction.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- logp.optin
- logp.optin_expired
- logp.content_stripped

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.logp.optin	tenant_id · flags · expiry · actor	An opt-in is created.
token_economics.logp.content_stripped	tenant_id · flag · source	Content is removed at ingress.

REFERENCES

- A:217 Monitoring usage (Claude Code OpenTelemetry)

RECEIVES FROM

None in this stage view.

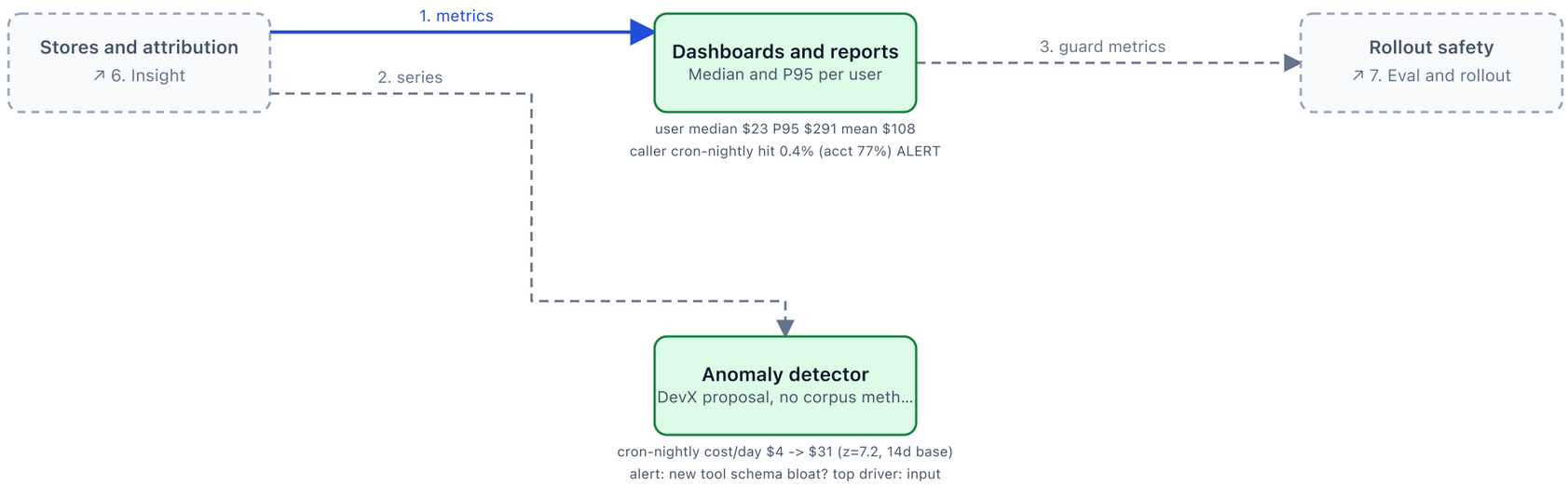
HANDS TO

OTel receiver
log flags, log flags

← Back to group: Stores and attribution ← Stage 6

LEVEL 3 Views and alerts in Insight

Dashboards and one flagged anomaly detector.



Calculator — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Stores and attribution (group) → Dashboards and reports
Median and P95 per user, hit rate, waste share.

2. Stores and attribution (group) → Anomaly detector
Cost and token series for outlier checks.

3. Dashboards and reports → Rollout safety (group)
Success floor, CPS, hit rate, empty patches, p95, divergence.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Insight ← Stores and attribution

LEVEL 4

Dashboards and reports

Calculator | Stage 6: Insight

Dashboards and reports

Median and P95 per user

user median $23 P95 $291 mean $108
caller cron-nightly hit 0.4% (acct 77%) ALERT

IN PLAIN ENGLISH

> This is the screen where people see what AI costs them. It shows the typical user and the heavy users, and it flags a job that is wasting money even when the overall numbers look fine.

Goes in	Rollups from the metadata store: cost, tokens, hit rate, waste share, per user and per caller.
Comes out	Dashboards and reports: median and 95th percentile (P95) per user, hit rate per caller, waste share, cost per successful task, reconciliation ratio.
Reads / writes	Reads meta rollups and rec, canary and planq records. Writes saved reports.
If it fails	Fail open: a stale or missing dashboard shows a last-updated stamp. Guard metrics sent to rollback triggers use cached values and alert on staleness.
Requirement source	RQ-B-048, RQ-B-049, RQ-B-039, RQ-B-040, RQ-B-046, RQ-B-047 Sources: C:054, C:101, A:213, C:116, C:052, C:076, C:028, C:027, C:024, A:069, C:122
Hardening, still open	Open: baselines in C:054 come from one gateway with selection bias (mean $108, median $23, P95 $291). A mean alone hides the tail, so P95 is mandatory. A measured agent-team multiplier is not in the corpus.
Not in client docs	**DevX proposal:** Layout, refresh rate and the choice of which guard metrics feed rollback are DevX proposals. Metric definitions are from R6.

WHAT IT IS FOR

Show spend and waste in forms that lead to action. Views: median and 95th percentile (P95) per user, hit rate per caller against its own baseline, failed-run waste, and cost per successful task.

EXPECTED

- Spend per user as mean, median and P95. Baseline reference: Claude Code active mean $108, median $23, P95 $291 [C:054].
- Cache hit rate per caller, model and prefix hash. Each caller is compared to its own baseline. An account rate of 77 percent hid a cron at about 0 percent [C:052].
- Cache miss events: more than 5 percent and at least 2,000 tokens reprocessed [C:076].
- Failed-run waste share and spend after the first repeated failure (baselines 32 percent and 51 percent [C:028]).
- Cost per successful task as the headline, never token savings alone [C:024, A:069].
- Reconciliation ratio and canary drift per provider. Plan units and shadow cost per user.
- Guard metrics for the rollback triggers: success floor, CPS, hit rate, empty patches, p95 latency, billed against estimate divergence.

NOT EXPECTED

- No content views.
- No ranking of individuals as performance scores.
- No savings claim without a baseline arm.

ALGORITHM AND MATHS

Rollup views with percentile and baseline comparison

```
P95 per user over a window; hit_rate(caller) vs baseline(caller)
```

STEPS

1. Read rollups from the metadata store for the chosen window and tenant.
2. Compute mean, median and P95 per user from per-user totals.
3. Compare each caller's hit rate with its own trailing baseline.
4. Compute waste share = failed spend / total and CPS = total spend / successes.
5. Show the reconciliation ratio, canary drift and plan units.
6. Publish the guard metrics to the rollback component every few minutes.

PARAMETERS

percentiles	median and P95 [C:054, C:101]
miss_event	more than 5 percent and at least 2,000 tokens [C:076]
baseline_window	trailing 14 days (DevX proposal)
budget_alert_thresholds	75, 90 and 100 percent [A:213, C:116]

SECURITY

- User-level views need a role. Default views show team aggregates.
- No content fields exist in the source data.
- Exports are logged in the audit log.

PERFORMANCE

Latency and throughput	Dashboard loads under 3 seconds at the 95th percentile (P95), data lag under 5 minutes (DevX proposal).
Complexity	Reads precomputed rollups.
At 10x load	Rollups by tenant and day.

EDGE CASES AND HOW EACH IS HANDLED

- A caller with few requests gives a noisy hit rate. Show the sample size.
- A new caller has no baseline. Show absolute value only.
- Gateway-derived baselines have selection bias [C:054].
- Spend of subagents must not roll into the parent without a label.

TEST CASES

TEST	GIVEN	EXPECT
percentile	user totals 10, 20, 30, 40, 100	median 30, P95 above 40, mean 40
own_baseline	caller baseline 0.90, now 0.40 while account is 0.77	alert on caller
waste_share	failed spend $32 of $100	0.32 shown
cps_headline	spend $0.95 over 1 success	CPS $0.95, no token-savings headline

WHERE IT SAVES COST

- Pointing at the tail (P95) and the worst caller finds the largest saving.
- Waste share points at loop guard and turn cap targets.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- dash.guard_metrics
- dash.caller_alert

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.dash.view`	view · tenant_id · role	A dashboard is opened.
`token_economics.dash.guard_metrics`	metric · value · baseline	Guard metrics are published.

RESEARCH BEHIND IT

C:052 Your cache hit rate is lying to you. It's the caller mix.
Result we rely on: Account hit rate 77 percent hid a cron near 0 percent; blanket 1h TTL made the bill 8.6 percent worse.
Used here: Hit rate is shown per caller against its own baseline.

A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
Result we rely on: Price rank reversed in 32 percent of comparisons.
Used here: Rank by cost per task, not list price.

HOW OTHERS BUILD IT

Gateway vendors publish per-agent hit rates and spend percentiles from their own traffic [C:054], with selection bias. Token Economics computes the same views from the customer's own ledger.

REFERENCES

- C:054 The Coding Agent Economy
- C:101 ChatGPT Work: usage and cost
- A:213 Overview of request-based billing (legacy)
- C:116 GitHub Copilot premium requests (billing concepts)
- C:052 Your cache hit rate is lying to you. It's the caller mix.
- C:076 Customize your status line
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:027 Can your AI agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks
- C:024 Token Reduction Is Not Cost Reduction
- A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
- C:122 OpenHands Context Condensation for More Efficient AI Agents

RECEIVES FROM

Metadata store
metrics

HANDS TO

Rollback triggers
guard metrics, guard metrics

← Back to group: Views and alerts ← Stage 6

LEVEL 4

Anomaly detector

Calculator | Stage 6: Insight

Anomaly detector

DevX proposal, no corpus method

cron-nightly cost/day $4 -> $31 (z=7.2, 14d base)
alert: new tool schema bloat? top driver: input

IN PLAIN ENGLISH

> This part watches the spend lines and shouts when something jumps far above normal. It only warns, it never stops anything.

Goes in	Cost and token series per caller, model and tenant from the metadata store.
Comes out	Alerts on outliers, with the series, the baseline and the likely driver.
Reads / writes	Reads meta series. Writes alert records and a per-series baseline state.
If it fails	Fail open: it only alerts. A broken detector never blocks traffic. A silent detector raises a heartbeat alert.
Requirement source	RQ-B-050, RQ-B-051 Sources: A:213, C:116, C:054, C:028, C:136
Hardening, still open	Open: no corpus method, threshold or false-alert rate. Budget alerts at 75, 90 and 100 percent are sourced [A:213, C:116] and live in the alert component. The detector needs a pilot to set thresholds.
Not in client docs	**DevX proposal:** The whole method is a DevX proposal. The corpus has no anomaly-detection method beyond budget alerts at 75, 90 and 100 percent. The outlier method is chosen by Token Economics. No source supplies it.

WHAT IT IS FOR

Catch sudden or slow cost jumps that fixed budget thresholds miss, such as a cron that loses its cache or a loop that runs all night.

EXPECTED

- Runs on daily and hourly series per caller and per model.
- Uses a trailing median and median absolute deviation (MAD) baseline and flags a score above a set limit (DevX proposal).
- Attaches the likely driver: input, output, cache read, cache write, or request count.
- Suppresses repeat alerts for one series for a set period.
- Publishes a heartbeat so a silent detector is itself an alert.

NOT EXPECTED

- No traffic blocking. Budgets and the loop guard block.
- No content use.
- No claim of a corpus-backed method.

ALGORITHM AND MATHS

Median and MAD outlier score (DevX proposal)

```
z = 0.6745 x (x − median) / MAD over a trailing window
```

STEPS

1. Aggregate the series per caller and model at hourly and daily grain.
2. Keep a trailing window (14 days proposed) with at least 7 points.
3. Compute median and MAD. Score the newest point.
4. If the score exceeds the limit (proposed 5) and the absolute rise exceeds a floor in USD, raise an alert.
5. Break the rise into bucket and request-count drivers.
6. Write the alert and start a suppression window.

PARAMETERS

window_days	14 (DevX proposal)
min_points	7 (DevX proposal)
score_limit	5 (DevX proposal)
absolute_floor_usd	set per tenant (DevX proposal)
budget_alert_levels	75, 90, 100 percent, handled by the alert component [A:213, C:116]

SECURITY

- Alerts carry series ids and numbers, no content.
- Alert routing goes through the configured channels with tenant scope.
- Thresholds changes are audited.

PERFORMANCE

Latency and throughput	Hourly alert within 15 minutes of the hour closing (DevX proposal).
Complexity	O(w) per series for window w.
At 10x load	Series count equals callers times models. Evaluate in batch.

EDGE CASES AND HOW EACH IS HANDLED

- New caller with under 7 points: no alert, status warming up.
- Planned load test looks like an anomaly. Allow a silence window by operators.
- Price change shifts cost for every caller at once. Alert once at the model level.
- Weekend dips cause false alarms in a day-based window.
- Failed runs are 32 percent of spend on average [C:028], so a spike in failures is a driver, not an anomaly by itself.

TEST CASES

TEST	GIVEN	EXPECT
spike	series 4,4,5,4,4,5,4 then 31	alert raised, driver input
warming	series with 3 points	no alert, status warming_up
suppress	two spikes in one hour on one series	one alert
heartbeat	detector silent for 2 hours	heartbeat alert fires
price_change	all callers +20 percent after a price row change	one model-level alert

WHERE IT SAVES COST

- Early alerts catch cost leaks such as lost cache or loops before the invoice.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- anom.alert
- anom.heartbeat_missed

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.anom.alert`	series · score · value · baseline · driver	A series crosses the limit.
`token_economics.anom.heartbeat`	last_run · series_count	The detector finishes a run.

REFERENCES

- A:213 Overview of request-based billing (legacy)
- C:116 GitHub Copilot premium requests (billing concepts)
- C:054 The Coding Agent Economy
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:136 Coding agents ignore spend (Ramp Labs)

RECEIVES FROM

Metadata store
series

HANDS TO

None in this stage view.

← Back to group: Views and alerts ← Stage 6

LEVEL 2

7. Eval and rollout

Cost per successful task, paired statistics, shadow runs, canaries, kill-switches and rollback triggers.

Evaluation

Eval harness, private task set, paired statistics and cost per successful task.

4 Eval and rollout

Eval harness · Private eval set · Paired bootstrap stats · CPS calculator

Receives from: Metering (Accounting), Rollout safety
Hands to: Rollout safety

Open group (4 components) →

Rollout safety

Shadow runs, canaries, kill-switches and rollback triggers.

4 Eval and rollout

Shadow runner · Canary controller · Kill-switch · Rollback triggers

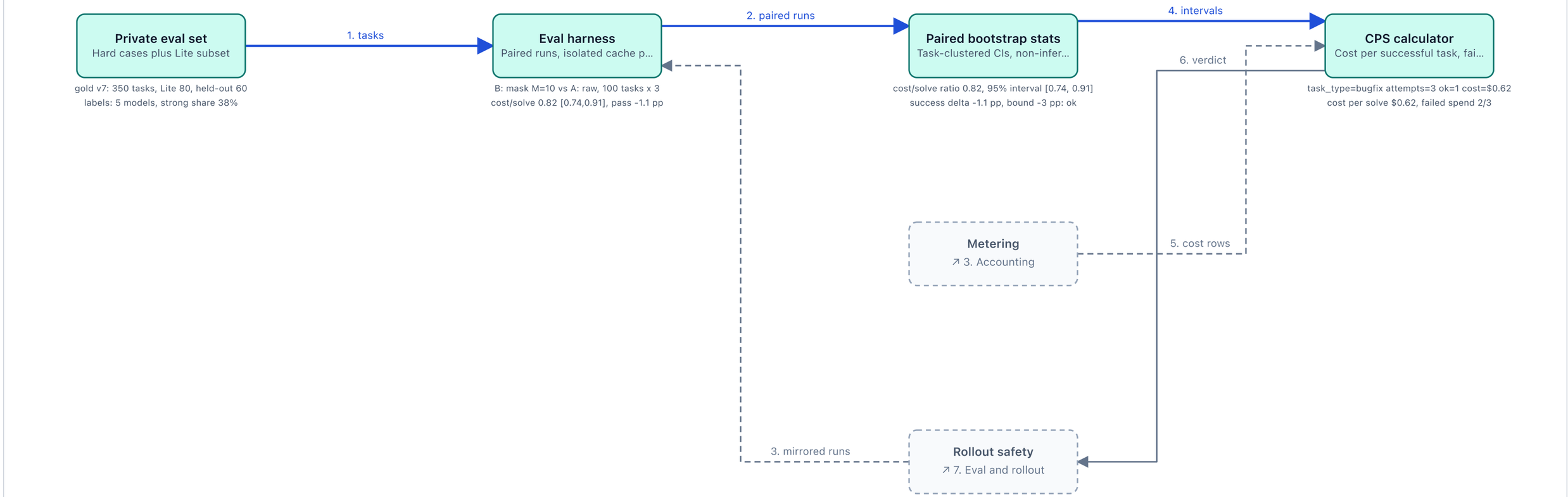
Receives from: Views and alerts (Insight), Evaluation
Hands to: Evaluation, Admin and config (Control plane)

Open group (4 components) →

2 groups, 8 components. Hover a group for the plain-English story; click it to see its components and the arrows between them. ← Insight Control plane →

LEVEL 3 Evaluation in Eval and rollout

Eval harness, private task set, paired statistics and cost per successful task.



Eval and rollout — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Private eval set → Eval harness
Private tasks with hard cases.

2. Eval harness → Paired bootstrap stats
Per-task cost and success per arm.

3. Rollout safety (group) → Eval harness
Shadow results scored by the harness.

4. Paired bootstrap stats → CPS calculator
Bootstrap intervals and non-inferiority result.

5. Metering (group) → CPS calculator
Cost including failed attempts.

6. CPS calculator → Rollout safety (group)
Canary advances only on a passing verdict.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Eval and rollout Rollout safety →

LEVEL 4

Eval harness

Eval and rollout | Stage 7: Eval and rollout

Eval harness

Paired runs, isolated cache per arm

B: mask M=10 vs A: raw, 100 tasks x 3
cost/solve 0.82 [0.74,0.91], pass -1.1 pp

IN PLAIN ENGLISH

> Before any saving goes live, this runs the old way and the new way on the same tasks. It only says yes if cost per solved task drops and the solve rate holds.

Goes in	Arm definitions (baseline, candidate), task set, repeat count, success flags, billed cost per attempt.
Comes out	Run plan, per-arm result table, and a ship, hold or revert verdict with the numbers behind it.
Reads / writes	Reads the gold set and the ledger. Writes run manifests, arm results and verdicts.
If it fails	Fail closed for shipping. With no verdict, an optimisation stays off. Fail open for traffic: production requests are never blocked by the harness.
Requirement source	RQ-D-007, RQ-D-027, RQ-D-028, RQ-D-029, RQ-D-032, RQ-D-033, RQ-D-034, RQ-D-035, RQ-D-036, RQ-D-037, RQ-D-038, RQ-D-045, RQ-D-048, RQ-D-042 Sources: C:024, C:026, C:028, C:025, C:053, A:006, A:122, C:132, C:081, A:002, A:001, C:030, A:052
Hardening, still open	Not in corpus: formal power formulas and minimum test duration (Perplexity ran 2 weeks [A:272]). Also missing: an acceptable quality margin. C:025 accepted about -4 points for -23.6 percent cost, a business call. Whether public scores predict our tasks is unknown. Optimisations do not transfer between harnesses (Headroom +48 percent vs -12.49 percent [C:024]).
Not in client docs	**DevX proposal:** Run manifest format and the log field list are DevX proposals. Test design rules are sourced.

WHAT IT IS FOR

Run paired A/B tests of every optimisation. Decide with cost per successful task (CPS), which is the cost of all attempts divided by successes. Never decide on token savings.

EXPECTED

- Pairs arms by task, with repeats, and a baseline arm on the same tasks.
- Uses a separate cache namespace per arm, or spaces the runs [C:024].
- Charges the optimisation own overhead (a router call, a summary call) to the task [C:026, C:028].
- Logs prompt variant, effort, harness version, tool surface, retries, compaction and whether the optimisation fired.

NOT EXPECTED

- No verdict from token counts alone.
- No voting, ensembles or agent teams without a measured CPS gain [A:122, C:132].
- No tuning on the held-out tasks.

ALGORITHM AND MATHS

Preregistered paired arm test

```
ship iff CI_upper(CPS_B/CPS_A) < 1 and success_lower_bound(B − A) >= −delta
```

STEPS

1. Freeze the plan: tasks, repeats, primary metric CPS, success non-inferiority bound delta, stop rules.
2. Run both arms on the same tasks with isolated cache state and a verifiable success flag.
3. Hand attempt rows to stats for the clustered bootstrap CIs.
4. Check guard metrics: patch-apply, empty-patch, format validity, fix-commit ratio [C:053].
5. Emit verdict ship, hold or revert, with reasons and the manifest.

PARAMETERS

baseline_arm	required [C:024]
repeats	several per cell; 3 runs per cell in the Lite study [A:060]
sample_sizes_seen	100 tasks [C:025]; about 38 to 45 effective tasks per arm [C:024]; 60 tasks over 6 repos [C:028]
delta	set before the test; value is a business call, no corpus default
turn_cap_start	P75 of own per-model baseline, reject P25 [C:025]

SECURITY

- Eval prompts and repos may contain customer code; keep them in the tenant-scoped store and never export content.
- Only the evaluation role can change a run manifest after freeze; changes create a new run id.

PERFORMANCE

Latency and throughput	Verdict within 1 hour of the last attempt row (DevX proposal).
Complexity	Linear in attempts; bootstrap is delegated to stats.
At 10x load	Runs are batch jobs; attempt count is tasks x repeats x arms.

EDGE CASES AND HOW EACH IS HANDLED

- An arm hits the cache of the other arm: invalidate the run and rerun with spacing.
- A task has no verifiable success flag: exclude it and log the count.
- Candidate fires on fewer than 10 percent of tasks: report per-fired subset and overall.
- Public benchmark baseline is 96 to 98 percent: use hard private tasks instead [C:024].

TEST CASES

TEST	GIVEN	EXPECT
paired	100 tasks, 3 repeats, 2 arms	600 attempt rows, each task has both arms
overhead	router call costs $0.004 on a $0.050 attempt	attempt cost recorded as $0.054
ni_breach	success delta -5 pp with delta bound -3 pp	verdict revert, reason ni_breach
token_only	tokens -30 percent but CPS ratio 1.04	verdict hold, reason cps_not_lower
cache_carryover	arm B read tokens from arm A namespace	run invalid

WHERE IT SAVES COST

- Stops harmful optimisations before rollout.
- Finds which optimisation is worth its overhead.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Verdict reversals after rollout.
- Gap between test CPS and production CPS.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.evalh.run_started`	run_id · arms · tasks · repeats	plan frozen and run begins
`token_economics.evalh.attempt`	run_id · arm · task · success · cost_usd · fired	each attempt
`token_economics.evalh.verdict`	run_id · verdict · cps_ratio · ni_ok	run completes

RESEARCH BEHIND IT

C:024 Token Reduction Is Not Cost Reduction
Result we rely on: 2,848-run study: compressor effects flip sign by harness; always use a baseline arm.
Used here: Arm and cache isolation rules.

C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
Result we rely on: 4,644 preregistered runs, paired, task-clustered bootstrap.
Used here: Design template.

C:025 An Empirical Study of Turn-Control Strategies for Efficient Coding Agents
Result we rely on: Turn caps at P75 of own baseline.
Used here: Turn-cap test arm.

HOW OTHERS BUILD IT

Gateways offer A/B routing labels only as navigation items (Portkey [A:241, A:243]). Teams usually compare tokens. Token Economics compares billed cost per success.

REFERENCES

- C:024 Token Reduction Is Not Cost Reduction
- C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:025 An Empirical Study of Turn-Control Strategies for Efficient Coding Agents
- C:053 Inference Economics of Enterprise Coding Agents: A Case Study of Cloud vs. On-Premise LLMs
- A:006 LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing
- A:122 Cost-of-Pass: An Economic Framework for Evaluating Language Models
- C:132 Credit-Budgeted ICPC-Style Coding: When Agents Must Pay for Every Decision
- C:081 Orchestrate teams of Claude Code sessions
- A:002 RouterBench: A Benchmark for Multi-LLM Routing System
- A:001 RouteLLM: Learning to Route LLMs with Preference Data
- C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
- A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels

RECEIVES FROM

Private eval set
tasks

Shadow runner
mirrored runs

HANDS TO

Paired bootstrap stats
paired runs

← Back to group: Evaluation ← Stage 7

LEVEL 4

Private eval set

Eval and rollout | Stage 7: Eval and rollout

Private eval set

Hard cases plus Lite subset

gold v7: 350 tasks, Lite 80, held-out 60
labels: 5 models, strong share 38%

IN PLAIN ENGLISH

> A private set of real tasks with known right answers, so tests use our work and not public puzzles. Some tasks stay hidden from tuning so we can catch overfitting.

Goes in	Labelled tasks from our repos, hard cases, success checkers, held-out flag, router labels.
Comes out	Versioned gold set with a Lite subset and a held-out split.
Reads / writes	Reads task repos and labels. Writes gold set versions, split manifests and label audit rows.
If it fails	Fail closed for tuning. If the held-out split cannot be proven untouched, tuning jobs refuse to read it. Production traffic is not affected.
Requirement source	RQ-D-039, RQ-D-040, RQ-D-041, RQ-D-042 Sources: A:060, C:024, A:052, C:030, A:019, A:013, A:001, A:018, C:032
Hardening, still open	Label count for routers: about 1,500 gold samples [A:001] vs 100 to 1,000 [A:013]. Chosen: 1,000 as a first target, to be checked against a learning curve. Public-score validity for our tasks: not in corpus. Router labels must not come from one dominant model [A:019].
Not in client docs	**DevX proposal:** Split ratios, versioning and the refresh cadence are DevX proposals.

WHAT IT IS FOR

Provide the task set that stats and evalh need. Keep it private and hard, because public sets saturate (baseline 96 to 98 percent [C:024]).

EXPECTED

- Holds our own repo tasks with hard cases and a verifiable success check each.
- Keeps a Lite subset of 80 tasks. It tracked the full 350 within 0.81 pp at 25.3 percent of the cost [A:060].
- Reserves a held-out split that no tuning job reads [A:052].
- Labels router data from several models so no single model dominates [A:019].

NOT EXPECTED

- No public benchmark as the only set.
- No tuning on held-out tasks.
- No labels from a single dominant model.

ALGORITHM AND MATHS

Versioned split with Lite subset

```
gold = tune + heldout; lite = stratified sample(gold, 80)
```

STEPS

1. Collect tasks with a checker and a repo snapshot.
2. Stratify by repo, language and difficulty; draw the Lite subset of 80.
3. Assign the held-out split by hash of task id so it is stable across versions.
4. Record the version and the checksum of each task.
5. Audit labels: count per model and block a set where one model holds the strong-label majority.

PARAMETERS

lite_size	80 tasks of 350 [A:060]
lite_gap	0.81 pp mean abs diff, held-out max 1.70 pp [A:060]
lite_cost	25.3 percent [A:060]
heldout_penalty_seen	regex router lost 23.2 points held out [A:052]
router_labels	about 1,500 [A:001] or 100 to 1,000 [A:013]

SECURITY

- Task repos can hold customer code and secrets; store snapshots per tenant with no cross-tenant read.
- Checkers run in a sandbox with no network.

PERFORMANCE

Latency and throughput	Gold version build under 10 minutes for 350 tasks (DevX proposal).
Complexity	Linear in task count.
At 10x load	Batch only; storage grows with repo snapshots.

EDGE CASES AND HOW EACH IS HANDLED

- A task becomes flaky: remove it and bump the version.
- A new repo enters: add it to both Lite and held-out strata.
- All labels from one model: block the set.
- Held-out task leaks into a prompt: retire it and replace.

TEST CASES

TEST	GIVEN	EXPECT
lite_size	350 tasks	lite has 80 tasks, stratified
holdout_stable	task id 1042 in v6 and v7	same split in both versions
label_block	5 labels, 4 from model X	set blocked, reason dominant_model
tuning_guard	tuning job requests held-out ids	request refused
lite_gap	full pass 0.6610, lite 0.6607	gap 0.03 pp within the 0.81 pp mean seen

WHERE IT SAVES COST

- Lite subset lowers eval cost to 25.3 percent.
- Held-out split prevents wasted rollouts.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Lite vs full gap per version.
- Held-out vs tune gap.
- Flaky task rate.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.gold.version	version, tasks, lite, heldout	new gold version
token_economics.gold.leak	task, job	held-out access attempt
token_economics.gold.label_audit	version, models, max_share	label audit

RESEARCH BEHIND IT

A:060 Claw-SWE-Bench: A Benchmark for Evaluating OpenClaw-style Agent Harnesses on Coding Tasks
Result we rely on: 80-task Lite tracked 350 tasks within 0.81 pp at 25.3 percent cost.
Used here: Lite subset size.

A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels
Result we rely on: Regex router fell 23.2 points on held-out tasks.
Used here: Reason for a held-out split.

A:019 Zen Router 0.6B
Result we rely on: Routers trained on one dominant model do not generalise.
Used here: Label audit rule.

HOW OTHERS BUILD IT

Public coding sets saturate. Teams that track cost per task keep a private set and a cheap subset for confidence interval (CI).

REFERENCES

- A:060 Claw-SWE-Bench: A Benchmark for Evaluating OpenClaw-style Agent Harnesses on Coding Tasks
- C:024 Token Reduction Is Not Cost Reduction
- A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels
- C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
- A:019 Zen Router 0.6B
- A:013 Meta-Router: Bridging Gold-standard and Preference-based Evaluations in Large Language Model Routing
- A:001 RouteLLM: Learning to Route LLMs with Preference Data
- A:018 s1-llm-auto-router
- C:032 Agentic Model Routing for Coding Tasks

RECEIVES FROM

None in this stage view.

HANDS TO

Eval harness
tasks

← Back to group: Evaluation ← Stage 7

LEVEL 4

Paired bootstrap stats

Eval and rollout | Stage 7: Eval and rollout

Paired bootstrap stats

Task-clustered CIs, non-inferiority

cost/solve ratio 0.82, 95% interval [0.74, 0.91]
success delta -1.1 pp, bound -3 pp: ok

IN PLAIN ENGLISH

> This works out whether a new setting really saves money or just got lucky. It resamples whole tasks, so one easy task cannot fake a win.

Goes in	Attempt rows per task and arm: cost, success, repeat id.
Comes out	Paired cost per successful task (CPS) ratio with a task-clustered bootstrap interval, success difference interval, and a test verdict.
Reads / writes	Reads the ledger and eval rows. Writes interval results with random seed.
If it fails	Fail closed for decisions. If the interval cannot be computed, the verdict is hold. No effect on live traffic.
Requirement source	RQ-D-029, RQ-D-030, RQ-D-031, RQ-D-026, RQ-D-037 Sources: C:027, C:023, C:026, C:025, C:024, A:052, A:006, A:002, A:001
Hardening, still open	Not in corpus: power formulas, minimum duration, acceptable margin. Derived from R6: log-SD ln(1.34) = 0.29 from the cost dispersion baseline [C:027]; fit it again on our pilot.
Not in client docs	**DevX proposal:** Bootstrap count (10,000), the one-sided level and the stopping rule are DevX proposals. Power formulas are not in corpus.

WHAT IT IS FOR

Turn paired attempt rows into a trustworthy CPS ratio and a success non-inferiority check. Task-clustered resampling is the method used in C:026.

EXPECTED

- Resamples tasks, not attempts, with all repeats of a task kept together.
- Computes CPS = sum of all attempt costs / number of successes in each arm.
- Reports the paired ratio CPS(arm)/CPS(baseline) with a confidence interval.
- Tests success against a preset non-inferiority bound.

NOT EXPECTED

- No unpaired comparison when pairs exist.
- No bound chosen after seeing the data.
- No claim of power beyond the pilot fit.

ALGORITHM AND MATHS

Task-clustered paired bootstrap

```
for b in 1..B: resample tasks with replacement; ratio_b = CPS_B^b / CPS_A^b; confidence interval (CI) = quantiles of ratio_b
```

STEPS

1. Group rows by task id; each group holds both arms and all repeats.
2. Draw B task samples with replacement using a fixed seed.
3. In each sample compute CPS per arm and the ratio; also the success rate difference.
4. Take the 2.5 and 97.5 percentiles for the ratio and the 5th percentile for the success difference.
5. Ship only if the ratio upper bound is below 1 and the success lower bound is at least the preset bound.

PARAMETERS

B	10,000 resamples (DevX proposal)
ci_level	95 percent (DevX proposal)
dispersion_baseline	geometric SD x1.34 per task and arm [C:027]
pilot_sample_sizes	about 38 to 45 effective tasks per arm [C:024]; 100 tasks [C:025]
ni_bound	preset per optimisation; not in corpus

THE MATHS IN FULL

SYMBOLS

- CPS_arm = sum over attempts of cost, divided by count of successes.
- sigma = SD of log cost ratio per task pair; ln(1.34) = 0.29 is the corpus dispersion baseline [C:027].
- n = number of task pairs; delta = relative cost difference to detect.

FORMULA

```
ratio = CPS_B / CPS_A;  CI via B task-clustered resamples;  rough pair count n ~= ((z_a + z_b) * sigma / delta_log)^2 where delta_log = ln(1 - delta)
```

WHY IT IS RIGHT

For paired log-ratios with SD sigma, the standard error of the mean is sigma / sqrt(n). At a two-sided 5 percent level and 80 percent power, n is about ((1.96 + 0.84) * sigma / delta_log)^2. The corpus gives sigma but not power targets, so z values are DevX proposals.

WORKED EXAMPLE

```
sigma = 0.29 [C:027]. Detect a 10 percent paired cost difference: delta_log = ln(0.9) = -0.105. n = ((1.96 + 0.84) * 0.29 / 0.105)^2 = (7.73)^2 = about 60 task pairs. Compare: 100 tasks in C:025, about 38 to 45 effective tasks in C:024. Check against a pilot SD before use.
```

COST OF COMPUTING IT

O(B * N) for N attempts; B = 10,000 gives about 10^4 passes over the rows.

HOW IT CAN GO WRONG, AND THE GUARD

- Too few tasks: wide interval, verdict hold.
- Cache carryover makes arms dependent.
- Heavy tail from a few runaway tasks inflates CPS variance.
- Success rate near 0 or 1: percentile bootstrap becomes unstable.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Fit sigma on a pilot of 20 tasks; re-run the sample size formula; check coverage by simulation on past runs.

SECURITY

- Rows are aggregated and carry task ids only; no prompts or outputs.
- The random seed is stored so any third party can reproduce an interval.

PERFORMANCE

Latency and throughput	Interval for 100 tasks x 3 repeats x 2 arms in under 5 s (DevX proposal).
Complexity	O(B * N).
At 10x load	Vectorised; parallel across resamples.

EDGE CASES AND HOW EACH IS HANDLED

- A task has zero successes in one arm: CPS undefined for that arm, count as censored and report.
- Repeats differ between arms: use the minimum common repeats.
- Only 1 task pair: refuse an interval.
- All costs identical: interval collapses; flag as degenerate.

TEST CASES

TEST	GIVEN	EXPECT
paired_ratio	arm A cost $10.00 for 20 successes, arm B $8.20 for 20	ratio 0.82
cluster	3 repeats of task T1	all 3 rows resample together
ni_pass	success delta -1.1 pp, bound -3 pp, lower 5th pct -2.4 pp	ni_ok true
ni_fail	lower 5th pct -4.0 pp, bound -3 pp	ni_ok false, verdict revert
sample_size	sigma 0.29, delta 10 percent	n about 60 pairs
seed	same rows and seed twice	same interval

WHERE IT SAVES COST

- Right-sized tests avoid paying for extra runs.
- Catches a fake win before rollout.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Interval width vs plan.
- Coverage on back-tests.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.stats.interval`	run_id · ratio · lo · hi · n_tasks · seed	interval computed
`token_economics.stats.ni_check`	run_id · delta · bound · ok	non-inferiority checked
`token_economics.stats.degenerate`	run_id · reason	cannot compute

RESEARCH BEHIND IT

C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
Result we rely on: Preregistered paired task-clustered bootstrap with a success non-inferiority floor.
Used here: Core method.

C:027 Can your AI agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks
Result we rely on: Geometric SD x1.34 per task and arm; one probe plus 1.9x headroom.
Used here: Sigma input.

HOW OTHERS BUILD IT

Bootstrap on task clusters is common in agent evaluation. Vendors rarely publish an interval or a bound.

REFERENCES

- C:027 Can your AI agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks
- C:023 How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks
- C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
- C:025 An Empirical Study of Turn-Control Strategies for Efficient Coding Agents
- C:024 Token Reduction Is Not Cost Reduction
- A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels
- A:006 LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing
- A:002 RouterBench: A Benchmark for Multi-LLM Routing System
- A:001 RouteLLM: Learning to Route LLMs with Preference Data

RECEIVES FROM

Eval harness
paired runs

HANDS TO

CPS calculator
intervals

← Back to group: Evaluation ← Stage 7

LEVEL 4

CPS calculator

Eval and rollout | Stage 7: Eval and rollout

CPS calculator

Cost per successful task, failures in

task_type=bugfix attempts=3 ok=1 cost=$0.62
cost per solve $0.62, failed spend 2/3

IN PLAIN ENGLISH

> A cheap run that fails is not cheap. This counts the money for every try, including the failed ones, and divides by how many tasks truly got solved.

Goes in	Ledger rows tagged with task id, success flag, attempt cost, caller and model.
Comes out	cost per successful task (CPS) per task type, caller and model; failed-run waste share; spend after first repeated failure.
Reads / writes	Reads the ledger and success flags. Writes CPS aggregates and waste reports.
If it fails	Fail open. A missing success flag leaves the attempt unscored and counted as unknown, never as a success.
Requirement source	RQ-D-023, RQ-D-024, RQ-D-025, RQ-D-026, RQ-D-027, RQ-D-028, RQ-D-034, RQ-D-046 Sources: A:122, A:069, C:026, C:024, C:028, C:027, C:023, C:025, C:053
Hardening, still open	Cost-driver conflict: input [C:023], output [C:027], or harness base 74.7 percent [C:024]. Output is 2.7 percent of tokens and 51.1 percent of dollars at 96.3 percent hit. Chosen: report all three buckets and let data decide. Failure-share baselines (32 and 51 percent) come from one 60-task study [C:028].
Not in client docs	**DevX proposal:** Dashboard layout and category list are DevX proposals. Metric definitions are sourced.

WHAT IT IS FOR

Compute the headline metric. Cost per successful task counts all attempts including failures [A:122, C:026].

EXPECTED

- Sums cost over all attempts including failed ones, divided by successes.
- Reports per task category, since optimisations do not transfer across harnesses [C:024].
- Tracks failed-run waste share (baseline 32 percent) and spend after first repeated failure (baseline 51 percent) [C:028].
- Tracks quality guards: patch-apply, empty patch, format validity, fix-commit ratio [C:025, C:053].

NOT EXPECTED

- Never reports token savings alone.
- Never counts an unknown-outcome run as a success.
- No mixing of arms in one number.

ALGORITHM AND MATHS

Failure-inclusive CPS

```
CPS = sum_i cost_i / count(success_i)
```

STEPS

1. Join ledger rows to task ids and the success flag.
2. Group by category, caller, model, arm.
3. Add the optimisation overhead (router call, summary call) to the task total [C:026].
4. Compute CPS, waste share = failed cost / total cost, and cost after the first repeated failure signature.
5. Publish with median and P95 per developer [C:054].

PARAMETERS

failed_run_waste_baseline	32 percent [C:028]
spend_after_repeat_failure_baseline	51 percent [C:028]
dispersion	geometric SD x1.34 [C:027]
harness_base_share	74.7 percent of cost [C:024]
early_stop_rule	cut runs after the first repeated failure signature [C:028]

THE MATHS IN FULL

SYMBOLS

- c_ij = billed cost of attempt j on task i, including overhead.
- s_i = number of successful attempts for task i (0 or 1 per task, repeats counted).
- S = sum of s_i over tasks.
- W = failed cost / total cost.

FORMULA

```
CPS = (sum_i sum_j c_ij) / S;  W = (sum over failed attempts c_ij) / (sum_i sum_j c_ij)
```

WHY IT IS RIGHT

Dividing total spend by successes charges every failed try to the tasks that did succeed. Token averages hide this because a cheap failure still adds to the numerator without raising S.

WORKED EXAMPLE

```
Example inputs are illustrative. 10 tasks, total spend $5.00, 4 successes: CPS = 5.00 / 4 = $1.25. If failed attempts cost $1.60 of the $5.00, W = 0.32 (the same as the C:028 baseline). Say an optimisation cuts spend to $4.00 but successes drop to 3. Then CPS = 4.00 / 3 = $1.33, so cost per success rose.
```

COST OF COMPUTING IT

O(N) over attempt rows.

HOW IT CAN GO WRONG, AND THE GUARD

- Success flag not verifiable: CPS undefined.
- Long tail of unfinished tasks removes costly failures from the denominator.
- Attempt rows missing overhead of the optimisation under test.

HOW THE NUMBERS ARE CHOSEN AND RE-TUNED

Reconcile the sum of attempt costs against the invoice for the same period. Compare W with the 32 percent baseline on the first 30 days.

SECURITY

- Aggregates drop prompt text and keep ids and numbers only.
- Per-developer views follow tenant roles so one developer cannot read another cost profile without a grant.

PERFORMANCE

Latency and throughput	Dashboard query under 2 s p95 for 30 days of data (DevX proposal).
Complexity	Linear in rows; pre-aggregate daily.
At 10x load	Partition by tenant and day.

EDGE CASES AND HOW EACH IS HANDLED

- Task has several repeats: count each repeat as an attempt, each success separately.
- Success flag arrives late: restate the aggregate with a version tag.
- Zero successes in a category: CPS undefined, show spend only.
- Shared tool-call cost across two tasks: split by turn share.

TEST CASES

TEST	GIVEN	EXPECT
cps_basic	spend $5.00, 4 successes	CPS $1.25
waste_share	failed attempts $1.60 of $5.00	W 0.32
no_success	spend $2.00, 0 successes	CPS undefined, spend shown
overhead	attempt $0.050 plus router $0.004	task cost $0.054
unknown_flag	attempt with no success flag	counted unknown, not success

WHERE IT SAVES COST

- Shows where failed runs burn money.
- Targets early stop after repeated failure.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- CPS trend per category.
- Waste share trend.
- Unknown-flag rate.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.cpsc.computed`	window · category · cps · waste_share · n_success	each aggregate refresh
`token_economics.cpsc.unknown`	task · reason	unscored attempt
`token_economics.cpsc.repeat_fail`	task · signature · spend_after	repeat failure detected

RESEARCH BEHIND IT

A:122 Cost-of-Pass: An Economic Framework for Evaluating Language Models
Result we rely on: Cost-of-pass, including failed attempts, is the right metric.
Used here: Definition of CPS.

C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
Result we rely on: 60 tasks over 6 repos: 32 percent failed-run waste, 51 percent spend after first repeated failure.
Used here: Baselines.

HOW OTHERS BUILD IT

Gateways report spend and tokens. Success-adjusted cost needs a task and outcome join that few products ship.

REFERENCES

- A:122 Cost-of-Pass: An Economic Framework for Evaluating Language Models
- A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
- C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
- C:024 Token Reduction Is Not Cost Reduction
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:027 Can your AI agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks
- C:023 How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks
- C:025 An Empirical Study of Turn-Control Strategies for Efficient Coding Agents
- C:053 Inference Economics of Enterprise Coding Agents: A Case Study of Cloud vs. On-Premise LLMs

RECEIVES FROM

- Append-only ledger
  cost rows, cost rows
- Paired bootstrap stats
  intervals

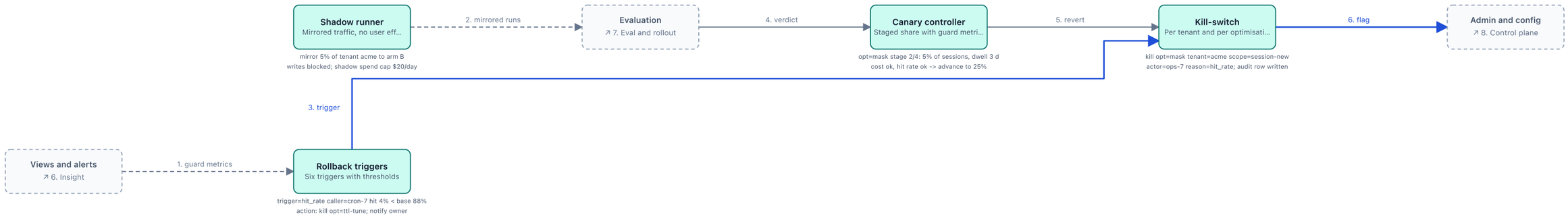
HANDS TO

- Canary controller
  verdict

← Back to group: Evaluation ← Stage 7

LEVEL 3 **Rollout safety** in Eval and rollout

Shadow runs, canaries, kill-switches and rollback triggers.



Eval and rollout — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Views and alerts (group) → Rollback triggers
Success floor, CPS, hit rate, empty patches, p95, divergence.

2. Shadow runner → Evaluation (group)
Shadow results scored by the harness.

3. Rollback triggers → Kill-switch
Rollback triggers flip the switch.

4. Evaluation (group) → Canary controller
Canary advances only on a passing verdict.

5. Canary controller → Kill-switch
A failing canary flips the kill-switch.

6. Kill-switch → Admin and config (group)
Kill-switch state stored as policy.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Eval and rollout ← Evaluation

LEVEL 4

Shadow runner

Eval and rollout | Stage 7: Eval and rollout

Shadow runner

Mirrored traffic, no user effect

mirror 5% of tenant acme to arm B
writes blocked; shadow spend cap $20/day

IN PLAIN ENGLISH

> Token Economics quietly copies a small share of real requests to the new setting and watches what would have happened. Nothing the copy does reaches the user or the repo.

Goes in	Mirrored production requests, candidate arm config, write-isolation setting.
Comes out	Shadow arm outputs, costs and scores, with no effect on the user response.
Reads / writes	Reads mirrored requests. Writes shadow results to the eval store, tagged with the shadow flag.
If it fails	Fail open for production. If the shadow path errors or lags, it is dropped. The user request is never delayed.
Requirement source	RQ-D-043, RQ-D-044, RQ-D-045 Sources: C:063, C:030, A:241, A:243
Hardening, still open	Measured production shadow results: not in corpus. Shadow doubles spend [R6]. Static labelled traces can score routers without live traffic [C:030]; chosen as the default for P2, with live shadow in P3.
Not in client docs	**DevX proposal:** Sampling rate, the spend cap and the queue design are DevX proposals. The shadow idea comes from one survey with placeholder values [C:063].

WHAT IT IS FOR

Compare a candidate on real traffic before any user sees its output.

EXPECTED

- Mirrors a configured share of requests after the primary call is sent.
- Blocks all side effects: file writes, shell commands, external calls [C:063].
- Scores the shadow arm by cost per successful task (CPS) and valid output rate against the live arm.
- Stops at a daily spend cap.

NOT EXPECTED

- No effect on the user response or its latency.
- No shadow arm that writes to a real repo.
- No shadow on tenants that opt out.

ALGORITHM AND MATHS

Mirror and isolate

```
shadow_spend <= cap; live latency delta = 0
```

STEPS

1. After the live request is dispatched, copy the request to the shadow queue with the sampling rate.
2. Run the shadow arm in a write-isolated sandbox with tool stubs.
3. Score outputs with the checkers from gold or a judge.
4. Add shadow cost to the eval ledger under the shadow tag.
5. Stop the arm when spend reaches the cap or the sample reaches the plan size.

PARAMETERS

sample_rate	5 percent (DevX proposal)
daily_spend_cap	per tenant policy (DevX proposal)
revert_triggers_in_survey	schema validity -2 pp, ExecRate -3 pp, p95 +20 percent (placeholder values [C:063])

SECURITY

- Shadow copies hold customer prompts; apply the same retention and tenant isolation as live data.
- Tool calls in shadow are stubbed; credentials are never passed into the sandbox.

PERFORMANCE

Latency and throughput	Added live latency 0 ms; shadow lag under 10 minutes (DevX proposal).
Complexity	One extra call per sampled request.
At 10x load	Separate queue and worker pool.

EDGE CASES AND HOW EACH IS HANDLED

- Shadow arm cost exceeds the live arm by 3x: stop at the cap.
- Request contains a tool with side effects: stub it and tag the result partial.
- Tenant policy forbids copying data: skip.
- Primary call fails: do not mirror.

TEST CASES

TEST	GIVEN	EXPECT
no_effect	shadow arm tries a file write	write blocked, live response unchanged
cap	spend cap $20, shadow spend $20.01	arm stops
sample	rate 5 percent, 1,000 requests	about 50 mirrored, within sampling noise
optout	tenant opted out	0 mirrored
latency	live p95 before and after enabling	no change

WHERE IT SAVES COST

- Finds a regression without user harm.
- Replaces guesswork about a new router.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Shadow CPS vs live CPS.
- Shadow valid-output rate.
- Shadow spend vs cap.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.shadow.mirrored`	tenant · arm · req_id	request copied
`token_economics.shadow.blocked`	tenant · arm · tool	side effect blocked
`token_economics.shadow.stopped`	tenant · arm · reason	cap or plan reached

RESEARCH BEHIND IT

C:063 A Survey of Small Language Models: Architectures, Capabilities, and Deployment Trade-offs
Result we rely on: Survey describes shadow evals, blue/green and auto-rollback with placeholder values.
Used here: Design pattern.

C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
Result we rely on: Static labelled traces score routers without live traffic.
Used here: Offline alternative.

HOW OTHERS BUILD IT

Portkey lists canary and A/B only as navigation labels [A:241, A:243]. No measured production shadow results are in the corpus.

REFERENCES

- C:063 A Survey of Small Language Models: Architectures, Capabilities, and Deployment Trade-offs
- C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
- A:241 AI Gateway (Portkey)
- A:243 Optimizing LLM costs: operational best practices (Portkey)

RECEIVES FROM

None in this stage view.

HANDS TO

Eval harness
mirrored runs

← Back to group: Rollout safety ← Stage 7

LEVEL 4

Canary controller

Eval and rollout | Stage 7: Eval and rollout

Canary controller

Staged share with guard metrics

opt=mask stage 2/4: 5% of sessions, dwell 3 d
cost ok, hit rate ok -> advance to 25%

IN PLAIN ENGLISH

> A new saving is switched on for a few sessions first, then more, as long as the numbers stay good. If any number slips, it stops and goes back.

Goes in	Passing eval verdict, rollout plan stages, tenant list, per-session pin state.
Comes out	Stage transitions: shadow, small share, larger share, full, or hold.
Reads / writes	Reads verdicts and live metrics. Writes stage state per tenant and per optimisation.
If it fails	Fail closed for expansion. If metrics are missing, hold at the current stage. Existing traffic is untouched.
Requirement source	RQ-D-046, RQ-D-047, RQ-D-048, RQ-D-030, RQ-D-042 Sources: C:063, C:024, A:204, C:055, A:129, C:050
Hardening, still open	Not in corpus: production canary or staged-rollout results, minimum duration. Chosen: stage sizes are placeholders, to be set from the first pilot. Sticky pinning for cache is sourced [A:204, C:055].
Not in client docs	**DevX proposal:** Stage sizes (1, 5, 25, 100 percent) and dwell times are DevX proposals. Stage idea is from one survey [C:063].

WHAT IT IS FOR

Roll out an optimisation in stages with per-session stickiness, so the cache and results stay comparable.

EXPECTED

- Assigns by session, not by request, so a session never changes arm mid-run [A:204].
- Advances only when cost per successful task (CPS), success floor and cache hit rate hold.
- Compares to the baseline arm that runs at the same time.
- Reports the rollout state to the dashboard.

NOT EXPECTED

- No mid-session arm change.
- No jump over a stage.
- No expansion without a passing verdict.

ALGORITHM AND MATHS

Staged session-sticky rollout

```
arm = hash(tenant, session, opt) mod 100 < share_stage
```

STEPS

1. Load the plan: stages, share, dwell time, gates.
2. Assign arm by hash of tenant, session and optimisation id.
3. At the end of dwell, read stats gates from the rollout window.
4. If all gates pass, move to the next stage; if any fail, call rollb.
5. Persist state and emit events.

PARAMETERS

stage_shares	1, 5, 25, 100 percent (DevX proposal)
dwell	3 days per stage (DevX proposal)
gate_metrics	CPS vs baseline confidence interval (CI), success floor, per-caller hit rate [C:063, C:024]
session_pin	required because model, tool set or compaction change breaks the cache [C:055, A:204]

SECURITY

- Only the rollout role may advance a stage, and each change is audit logged.
- A tenant can opt out; opt-out is honoured at every stage.

PERFORMANCE

Latency and throughput	Arm assignment under 0.2 ms (DevX proposal).
Complexity	Constant per request (hash).
At 10x load	Stateless assignment; state per tenant and optimisation.

EDGE CASES AND HOW EACH IS HANDLED

- Tenant has too little traffic to pass a gate: stay at the stage.
- Session starts at stage 2 and stage 3 begins: session keeps its arm.
- Two optimisations rolling out together: assign independent hashes, mark interactions.
- Gate metric missing: hold.

TEST CASES

TEST	GIVEN	EXPECT
sticky	same session id at 5 percent stage, 100 requests	same arm every time
advance	all gates pass at dwell end	stage 2 to 3
hold_missing	hit rate metric missing	hold, reason metric_missing
no_skip	plan stage 2 passing	next stage is 3, never 4
optout	tenant opted out	always baseline arm

WHERE IT SAVES COST

- Limits the damage of a bad optimisation.
- Keeps cache hit rate stable inside sessions.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Stage durations.
- Reverts per stage.
- Gate failures by metric.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.canr.assign`	tenant · session · opt · arm · stage	first request of a session
`token_economics.canr.advance`	tenant · opt · from · to	stage advanced
`token_economics.canr.hold`	tenant · opt · reason	stage held

RESEARCH BEHIND IT

C:063 A Survey of Small Language Models: Architectures, Capabilities, and Deployment Trade-offs
Result we rely on: Survey lists staged shadow, blue/green and auto-rollback keyed to CPS.
Used here: Stage design.

HOW OTHERS BUILD IT

Portkey lists canary only as a navigation label [A:241, A:243]. Most gateways have no staged rollout for cost optimisations.

REFERENCES

- C:063 A Survey of Small Language Models: Architectures, Capabilities, and Deployment Trade-offs
- C:024 Token Reduction Is Not Cost Reduction
- A:204 Prompt cache diagnostics
- C:055 How Claude Code uses prompt caching
- A:129 Investing in Performance: Fine-tune small models with LLM insights
- C:050 Evaluation and Optimization of Small Language Models for Agentic Tasks on Edge Devices

RECEIVES FROM

CPS calculator
verdict

HANDS TO

Kill-switch
revert

← Back to group: Rollout safety ← Stage 7

LEVEL 4

Kill-switch

Eval and rollout | Stage 7: Eval and rollout

Kill-switch

Per tenant and per optimisation

kill opt=mask tenant=acme scope=session-new
actor=ops-7 reason=hit_rate; audit row written

IN PLAIN ENGLISH

> A stop button for each saving feature and each customer. Pressing it sends traffic back down the plain, unchanged path.

Goes in	Kill command (tenant, optimisation, scope), actor, reason.
Comes out	Immediate revert to the unchanged default path for the scope.
Reads / writes	Reads switch state on every request. Writes switch state and an audit row.
If it fails	Fail open for optimisations: if the switch store is unreachable, all optimisations stay off and requests pass unchanged. Fail closed for budgets: budget enforcement is never behind this switch.
Requirement source	RQ-D-049, RQ-D-050, RQ-D-051, RQ-D-073 Sources: C:063, A:204, A:234, C:074
Hardening, still open	Not in corpus: kill-switch results. Phase conflict: the brief puts kill in P3, but P2 optimisations ship earlier. Chosen: P2 optimisations ship behind a static off flag until kill is built. Mid-run switches break caches [A:204]. So a kill takes effect at the next session by default. An emergency flag applies it at the next request.
Not in client docs	**DevX proposal:** Propagation target (under 5 s) and the cache of switch state are DevX proposals. Per-tenant and per-optimisation scope comes from C:063.

WHAT IT IS FOR

Give operators and rollb a fast way to turn off an optimisation without a deploy. Enforcement stays at the gateway [A:234, C:074].

EXPECTED

- Scopes: tenant, optimisation, or both.
- Kills at the gateway, not in a client hook [A:234, C:074].
- Takes effect for new sessions by default; an emergency flag applies at the next request.
- Never disables budget enforcement.

NOT EXPECTED

- No change to a session pinned arm unless emergency.
- No dependence on a client hook.
- No silent failure: every switch change is audited.

ALGORITHM AND MATHS

Scoped switch lookup

```
optimisation runs iff not killed(tenant, opt) and not global_kill(opt)
```

STEPS

1. Gateway reads a local cache of switch state, refreshed by push and by a poll every 5 s (DevX proposal).
2. On each request, check the tenant and optimisation scope before applying a stage.
3. If the cache is stale beyond 30 s, treat all optimisations as off.
4. On a kill command, write state, write an audit row and push to gateways.
5. Emergency flag forces the check at the next request.

PARAMETERS

poll_interval	5 s (DevX proposal)
stale_limit	30 s (DevX proposal)
default_effect	new sessions only [A:204]

SECURITY

- Only roles with the kill permission may change a switch; each change is signed and audited.
- The switch store is a separate service from the optimisation code so a bad deploy cannot disable it.

PERFORMANCE

Latency and throughput	Propagation to all gateways under 5 s p95 (DevX proposal).
Complexity	Constant per request (cache lookup).
At 10x load	Small state: tenants x optimisations.

EDGE CASES AND HOW EACH IS HANDLED

- Kill arrives while a session is pinned: default waits for the next session.
- Gateway cannot reach the switch store: optimisations off.
- Two operators send conflicting commands: last write wins, both audited.
- Kill on an optimisation the tenant never enabled: no-op, logged.

TEST CASES

TEST	GIVEN	EXPECT
scope_tenant	kill mask for acme	acme requests skip mask; other tenants unaffected
stale	cache age 31 s	all optimisations off
budget_safe	kill all optimisations	budget checks still run
emergency	emergency flag set	next request, not next session, is unmodified
audit	any kill	audit row with actor and reason

WHERE IT SAVES COST

- Stops a loss in minutes.
- Avoids a deploy for a rollback.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Kill count per optimisation.
- Time from trigger to effect.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.kill.set	tenant · opt · actor · reason · scope	switch changed
token_economics.kill.stale	gateway · age_s	cache stale
token_economics.kill.applied	tenant · opt · req_id	request skipped an optimisation

RESEARCH BEHIND IT

C:063 A Survey of Small Language Models: Architectures, Capabilities, and Deployment Trade-offs
Result we rely on: Per-tenant kill-switch plus auto-rollback in a survey.
Used here: Scope design.

HOW OTHERS BUILD IT

Feature-flag services give the same pattern. Gateways rarely tie it to cost optimisations.

REFERENCES

- C:063 A Survey of Small Language Models: Architectures, Capabilities, and Deployment Trade-offs
- A:204 Prompt cache diagnostics
- A:234 Track and optimize coding agent usage (Langfuse)
- C:074 Hooks reference

RECEIVES FROM

- Canary controller
  revert
- Rollback triggers
  trigger

HANDS TO

- Policy-as-code store
  flag, flag

← Back to group: Rollout safety ← Stage 7

LEVEL 4

Rollback triggers

Eval and rollout | Stage 7: Eval and rollout

Rollback triggers

Six triggers with thresholds

trigger=hit_rate caller=cron-7 hit 4% < base 88%
action: kill opt=ttl-tune; notify owner

IN PLAIN ENGLISH

> Watches six warning signs once a saving is on. If any one goes bad, it switches the saving off by itself.

Goes in	Live rollout metrics per tenant and optimisation, baseline intervals, estimated and billed cost.
Comes out	A rollback decision that calls kill for the scope and notifies the owner.
Reads / writes	Reads metrics and baselines. Writes rollback events with the trigger name and values.
If it fails	Fail closed toward the safe path. If a metric is missing, treat it as breached for the hit-rate and success triggers and roll back.
Requirement source	RQ-D-052, RQ-D-053, RQ-D-054, RQ-D-030, RQ-D-048 Sources: C:063, C:025, C:052, C:076, A:216, C:002, A:052
Hardening, still open	Reconciliation tolerance: not in corpus. Anomaly methods: not in corpus. Chosen: a spike rule using a count against the baseline 95 percent interval, to be calibrated. Per-caller hit rate is a trigger because account-level hit rate hid cron runs near 0 percent behind 77 percent [C:052].
Not in client docs	**DevX proposal:** The divergence tolerance, the empty-patch spike rule and the evaluation window are DevX proposals. The p95 +20 percent value is a placeholder from a survey [C:063].

WHAT IT IS FOR

Auto-revert on any of six breaches so a bad optimisation cannot run unseen.

EXPECTED

- Triggers on success below the floor.
- Triggers on cost per successful task (CPS) above the baseline interval.
- Triggers on per-caller cache hit rate below baseline, not account level [C:052, C:076].
- Triggers on an empty-patch or apply-failure spike, p95 latency +20 percent (placeholder), and billed vs estimate divergence.

NOT EXPECTED

- No rollback on a single noisy minute; use the evaluation window.
- No account-level hit rate as a trigger.
- No silent rollback; the owner is notified.

ALGORITHM AND MATHS

Six-trigger evaluator

```
rollback if any trigger_k(window) is true
```

STEPS

1. Every evaluation window, compute the six metrics per scope.
2. Compare each to its baseline interval or bound.
3. If any breach holds for the required window count, call kill for the scope.
4. Record the trigger name, value and baseline.
5. Notify the rollout owner and open an incident row.

PARAMETERS

success_floor	preset non-inferiority bound [C:025, A:052]
cps_trigger	above baseline confidence interval (CI) upper bound [C:024]
p95_latency	+20 percent (placeholder [C:063])
hit_rate	per caller below baseline [C:052]
cache_miss_event	more than 5 percent and at least 2,000 tokens reprocessed [C:076]
divergence_tolerance	not in corpus (DevX proposal)
schema_validity_drop_in_survey	-2 pp [C:063]
execrate_drop_in_survey	-3 pp [C:063]

SECURITY

- Rollback decisions are audit logged with the metric values and no content.
- Only the evaluator service identity can call kill with source rollb.

PERFORMANCE

Latency and throughput	Rollback issued within 2 evaluation windows of a breach (DevX proposal).
Complexity	Six comparisons per scope per window.
At 10x load	Linear in tenants x optimisations.

EDGE CASES AND HOW EACH IS HANDLED

- Low traffic leaves a metric undefined: treat as missing and roll back for hit rate, hold for latency.
- Baseline moves during a long rollout: re-fit baseline weekly.
- Two triggers fire together: record both.
- Scheduled cron run has near 0 percent hit by design: allow-list the caller.

TEST CASES

TEST	GIVEN	EXPECT
hit_rate	caller cron-7 hit 4 percent, baseline 88 percent	rollback, trigger hit_rate
account_hide	account hit 77 percent but one caller at 0 percent	rollback for that caller [C:052]
p95	p95 up 21 percent	rollback, trigger p95
cps	CPS 1.05 above CI upper 1.02	rollback, trigger cps
divergence	billed vs estimate differs beyond tolerance	rollback, trigger divergence

WHERE IT SAVES COST

- Limits the cost of a bad rollout.
- Protects the quality floor.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Rollbacks per trigger.
- False rollback rate on later reruns.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.rollb.triggered	tenant, opt, trigger, value, baseline	any trigger
token_economics.rollb.skipped	tenant, opt, reason	metric missing for non-fatal trigger
token_economics.rollb.notified	tenant, opt, owner	owner notified

RESEARCH BEHIND IT

C:063 A Survey of Small Language Models: Architectures, Capabilities, and Deployment Trade-offs
Result we rely on: Survey lists auto-rollback on schema validity, ExecRate and p95; placeholders.
Used here: Trigger list.

C:052 Your cache hit rate is lying to you. It's the caller mix.
Result we rely on: Account hit 77 percent hid cron runs near 0 percent.
Used here: Per-caller trigger.

HOW OTHERS BUILD IT

Deployment tools roll back on error rate. Cost-aware rollback on CPS and hit rate is a DevX design.

REFERENCES

- C:063 A Survey of Small Language Models: Architectures, Capabilities, and Deployment Trade-offs
- C:025 An Empirical Study of Turn-Control Strategies for Efficient Coding Agents
- C:052 Your cache hit rate is lying to you. It's the caller mix.
- C:076 Customize your status line
- A:216 Track cost and usage (Claude Agent SDK)
- C:002 Claude Code OpenTelemetry: Track Cost and Tool Use
- A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels

RECEIVES FROM

Dashboards and reports
guard metrics, guard metrics

HANDS TO

Kill-switch
trigger

← Back to group: Rollout safety ← Stage 7

LEVEL 2

8. Control plane

Admin API, policy as code, tenants and keys, audit log, secrets, retention.

Admin and config

Admin API, versioned policy, tenant keys and audit log.

4 Control plane

Admin API · Policy-as-code store · Tenant and key manager · Audit log

Receives from: Context controls (Context guard), Rollout safety (Eval and rollout), Security and data
Hands to: Gateway edge (Ingress and adapters), Budgets (Policy engine)

Open group (4 components) →

Security and data

Secrets vault, admin authorisation and retention rules.

3 Control plane

Secrets vault · Admin authorisation · Retention and redaction

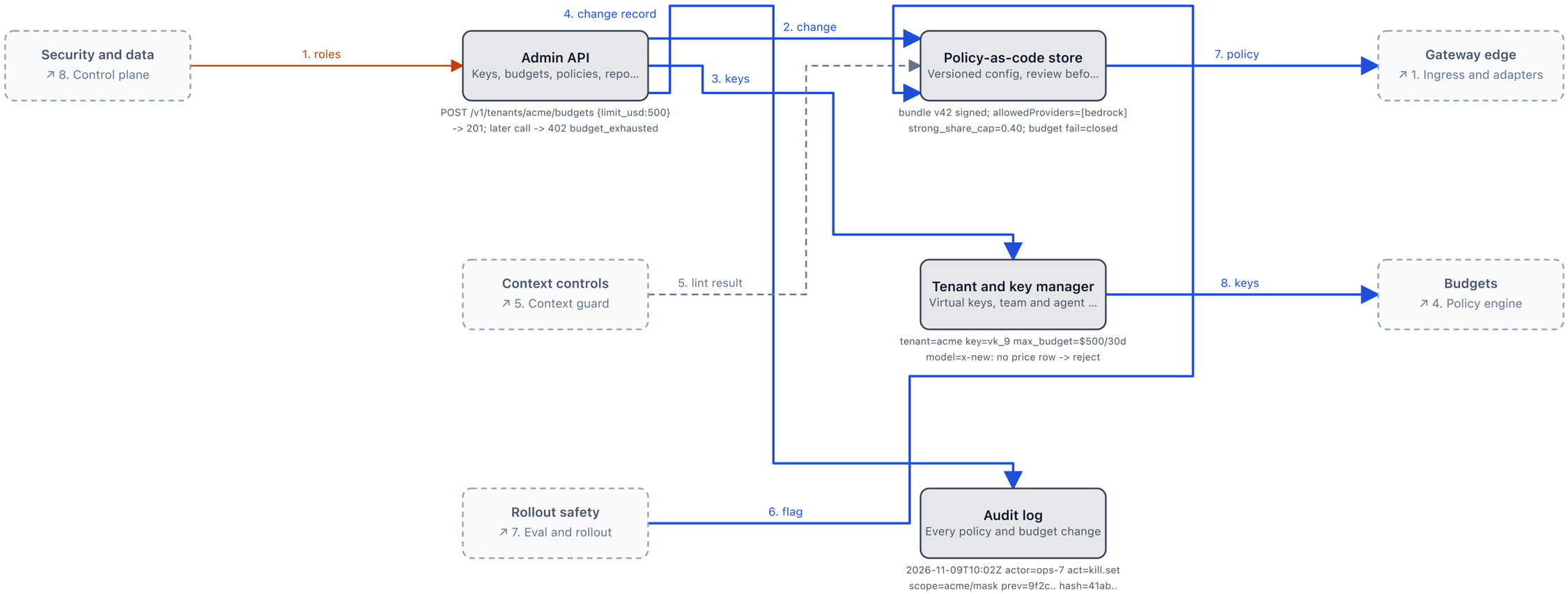
Hands to: API providers (Backends), Stores and attribution (Insight), Admin and config

Open group (3 components) →

2 groups, 7 components. Hover a group for the plain-English story; click it to see its components and the arrows between them. ← Eval and rollout

LEVEL 3 Admin and config in Control plane

Admin API, versioned policy, tenant keys and audit log.



Control plane — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Security and data (group) → Admin API
Scoped admin tokens.

2. Admin API → Policy-as-code store
Admin changes written as versioned policy.

3. Admin API → Tenant and key manager
Create, scope and revoke virtual keys.

4. Admin API → Audit log
Who changed what, when, before and after.

5. Context controls (group) → Policy-as-code store
Instruction-file size findings.

6. Rollout safety (group) → Policy-as-code store
Kill-switch state stored as policy.

7. Policy-as-code store → Gateway edge (group)
Signed policy bundle pulled by the gateway.

8. Tenant and key manager → Budgets (group)
Key records with budget and duration.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Control plane Security and data →

LEVEL 4

Admin API

Control plane | Stage 8: Control plane

Admin API

Keys, budgets, policies, reports

POST /v1/tenants/acme/budgets {limit_usd:500}
-> 201; later call -> 402 budget_exhausted

IN PLAIN ENGLISH

> The control panel for admins and the door for budget decisions. Once a budget runs out, the door returns a clear "payment required" answer and stops the call.

Goes in	Admin requests: create tenant, key, budget, policy; reads of reports; kill commands.
Comes out	JSON responses with stable error codes; HTTP 402 when a budget is exhausted on the data path.
Reads / writes	Reads and writes tenants, keys, budgets and policies in the control store. Writes an audit row per mutation.
If it fails	Fail closed for budgets: no budget state means 503 and no dispatch. Fail open for observation: reads of reports may be stale or empty.
Requirement source	RQ-D-055, RQ-D-056, RQ-D-057, RQ-D-058, RQ-D-061, RQ-D-062 Sources: C:136, A:234, A:191, C:001, C:074
Hardening, still open	Gateway soft caps overshoot [A:234]. Chosen: reserve-before-dispatch with a hard 402, as in Bifrost [C:001], and no reliance on agent-honoured budgets [C:136]. Hosted-provider rate-limit headers are not enough for tenant budgets [A:191].
Not in client docs	**DevX proposal:** Endpoint shapes, versioning and rate limits are DevX proposals. The 402 behaviour and hard external caps are sourced.

WHAT IT IS FOR

Expose the admin surface and enforce budgets at the gateway. Never rely on a client hook, because developers can disable hooks [A:234, C:074].

EXPECTED

- Returns HTTP 402 with code budget_exhausted when a budget cannot cover the reserve [C:001].
- Rejects dispatch with 503 policy_unavailable when budget state cannot be read.
- Writes an audit row for every mutating call.
- Versions the API path and keeps error codes stable.

NOT EXPECTED

- No enforcement by a client hook alone.
- No budget check that trusts an agent-stated estimate [C:136].
- No prompt content in logs.

ALGORITHM AND MATHS

Reserve then dispatch

```
allow iff budget_remaining >= reserve(request); then dispatch
```

STEPS

1. Authenticate the key and resolve tenant and virtual key.
2. Check authorisation for the operation (authz).
3. For data-path calls, compute the worst-case reserve and compare with remaining budget; if short, return 402.
4. If budget or policy state is unavailable, return 503 and do not dispatch.
5. On mutation, commit the change with an audit row in one transaction.

PARAMETERS

budget_exhausted_status	402 [C:001]
state_unavailable_status	503 (DevX proposal)
enforcement_location	gateway, never client hook [A:234, C:074]

SECURITY

- Each key is scoped to a tenant; cross-tenant ids return 404, not 403, to avoid leaks.
- Admin operations require a role (authz) and write audit rows; there are no anonymous mutations.
- Secrets never appear in responses or logs.

PERFORMANCE

Latency and throughput	Admin read p95 under 200 ms; data-path budget check under 2 ms (DevX proposal).
Complexity	Constant per request for the reserve check.
At 10x load	Stateless API tier; budget counters in a low-latency store.

EDGE CASES AND HOW EACH IS HANDLED

- Two concurrent calls race for the last dollar: atomic reserve, one gets 402.
- Budget state store is down: 503, no dispatch.
- Unpriced model id: reject (unpriced models escape budgets [A:242]).
- Client retries a mutating call: idempotency key returns the first result.

TEST CASES

TEST	GIVEN	EXPECT
exhausted	remaining $0.02, reserve $0.05	402 budget_exhausted, no dispatch
store_down	budget store unreachable	503 policy_unavailable, no dispatch
race	2 calls, $0.06 left, reserve $0.05 each	one 200, one 402
audit	create budget	one audit row with actor and diff
cross_tenant	key of acme reads tenant beta	404

WHERE IT SAVES COST

- Hard caps stop runaway spend.
- Atomic reserves remove soft-cap overshoot.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- 402 rate per tenant.
- 503 rate.
- Mutation count by role.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.api.request`	route · tenant · status · latency_ms	each admin call
`token_economics.api.402`	tenant · key · reserve · remaining	budget exhausted
`token_economics.api.503`	tenant · reason	state unavailable

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- Bifrost: HTTP 402 when a virtual-key budget is exhausted.
- LiteLLM: Virtual keys with max_budget and a duration.

HOW OTHERS BUILD IT

Gateways expose budgets on virtual keys. Many allow soft overshoot [A:234].

REFERENCES

- C:136 Coding agents ignore spend (Ramp Labs)
- A:234 Track and optimize coding agent usage (Langfuse)
- A:191 Spend limits
- C:001 Claude Code Token Usage: Monitor It with an AI Gateway
- C:074 Hooks reference

RECEIVES FROM

Admin authorisation
roles

HANDS TO

Policy-as-code store
change

Tenant and key manager
keys

Audit log
change record

← Back to group: Admin and config ← Stage 8

LEVEL 4

Policy-as-code store

Control plane | Stage 8: Control plane

Policy-as-code store

Versioned config, review before apply

bundle v42 signed; allowedProviders=[bedrock]
strong_share_cap=0.40; budget fail=closed

IN PLAIN ENGLISH

> Rules as code: who may use which model, how much they may spend, which savings are on. A bundle must be signed and reviewed before it takes effect.

Goes in	Policy bundle (budgets, allowed providers, caps, optimisation flags), signature, tenant id.
Comes out	Resolved policy for a request: allow-list, caps, stages, kill state.
Reads / writes	Reads the signed bundle. Writes bundle versions and decision logs (metadata).
If it fails	Fail closed. If no valid bundle exists, use a safe default: no dispatch to unlisted providers and no optimisation. Budgets keep their last good state.
Requirement source	RQ-D-059, RQ-D-060, RQ-D-061, RQ-D-062, RQ-D-050, RQ-D-051, RQ-D-056, RQ-D-065, RQ-D-041 Sources: C:001, A:193, C:104, A:234, C:074, A:010, A:019
Hardening, still open	Codex pins routes with model_provider in user config, so managed pinning is weaker there [C:104]. Chosen: the gateway is the enforcement point in all cases. Router labels must not rely on one dominant model [A:019]; strong-share ceiling [A:010] is a policy parameter.
Not in client docs	**DevX proposal:** Bundle format, review flow and cold-start rules are DevX proposals. Route pinning with allowedProviders is sourced for Claude Code [C:104].

WHAT IT IS FOR

Hold the policy-as-code that the gateway evaluates, and pin the route so the client cannot bypass Token Economics.

EXPECTED

- Loads only signed bundles and verifies the signature at the gateway.
- Pins the route with allowedProviders in managed settings (Claude Code) [C:104].
- Keeps enforcement at the gateway, never in a client hook [A:234, C:074].
- Applies a strong-share ceiling and refuses a router trained on one dominant model [A:010, A:019].

NOT EXPECTED

- No unsigned or unreviewed bundle in production.
- No policy that depends on a client hook for enforcement.
- No default that allows unlisted providers.

ALGORITHM AND MATHS

Signed bundle evaluation

```
decision = eval(bundle_vN, request) with default deny
```

STEPS

1. Fetch the bundle for the tenant, verify the signature and version.
2. Evaluate in order: provider allow-list, model price known, caps, budget, optimisation stage, kill state.
3. Cache the decision for the session when the arm is pinned.
4. If the bundle is invalid or missing, apply the cold-start default (deny unlisted, no optimisation).
5. Log metadata only.

PARAMETERS

default	deny (DevX proposal)
pin_claude_code	allowedProviders in managed settings [C:104]
pin_codex	model_provider in user config [C:104]
strong_share_ceiling	policy parameter, see A:010
review	two-person approval (DevX proposal)

SECURITY

- Bundles are signed; keys live in the vault; a failed signature check raises an alert.
- Policy evaluation has no network side effect and logs no prompt text.

PERFORMANCE

Latency and throughput	Evaluation under 1 ms p99 with a cached bundle (DevX proposal).
Complexity	Linear in rule count, small.
At 10x load	Bundles are cached per gateway; invalidated by version push.

EDGE CASES AND HOW EACH IS HANDLED

- Bundle signature fails: keep last good bundle and alert.
- First start with no bundle: cold-start default.
- Two versions during a push: each request evaluates against one version.
- Client sets a different provider: pinned route wins; gateway rejects.

TEST CASES

TEST	GIVEN	EXPECT
unsigned	bundle without a valid signature	rejected, last good bundle kept
unlisted_provider	request to provider not in allowedProviders	403 provider_not_allowed
cold_start	no bundle at start	default deny, optimisations off
strong_share	strong-share ceiling 0.40, current 0.45	router steers down or flags
hook_off	client hook disabled	gateway still enforces

WHERE IT SAVES COST

- Prevents routing around the gateway.
- Caps strong-model spend.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Denials per rule.
- Bundle change lead time.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.pol.loaded`	tenant · version · signed	bundle loaded
`token_economics.pol.denied`	tenant · rule · req_id	request denied
`token_economics.pol.cold_start`	gateway	default applied

RESEARCH BEHIND IT

A:019 Zen Router 0.6B
Result we rely on: Routers trained on one dominant model do not generalise.
Used here: Router label rule.

HOW OTHERS BUILD IT

Policy-as-code with signed bundles is common in cloud access control. In gateways, allow-lists per key are the usual form [C:060].

REFERENCES

- C:001 Claude Code Token Usage: Monitor It with an AI Gateway
- A:193 LLM gateway configuration
- C:104 Codex configuration reference
- A:234 Track and optimize coding agent usage (Langfuse)
- C:074 Hooks reference
- A:010 Rerouting LLM Routers
- A:019 Zen Router 0.6B

RECEIVES FROM

Instruction-file linter
lint result, lint result

Prompt-wording lint
lint result

Kill-switch
flag, flag

Admin API
change

HANDS TO

Edge gateway
policy, policy

← Back to group: Admin and config ← Stage 8

LEVEL 4

Tenant and key manager

Control plane | Stage 8: Control plane

Tenant and key manager

Virtual keys, team and agent scope

tenant=acme key=vk_9 max_budget=$500/30d
model=x-new: no price row -> reject

IN PLAIN ENGLISH

> Each customer gets their own keys and their own spending limit. A model without a known price is blocked, so nothing slips through unbilled.

Goes in	Tenant id, virtual key, price table version, budget, strong-share ceiling.
Comes out	Per-tenant key set, budget counters and price bindings.
Reads / writes	Reads and writes tenant records, keys and budget counters.
If it fails	Fail closed for budgets: an unreadable budget counter blocks dispatch. Unpriced models are rejected, not defaulted.
Requirement source	RQ-D-063, RQ-D-064, RQ-D-065, RQ-D-066, RQ-D-055, RQ-D-056 Sources: C:060, C:061, C:001, A:242, A:245, A:191, A:010
Hardening, still open	Unpriced models and bring-your-own-key (BYOK) routes can escape budgets [A:242, A:245, A:191]. Chosen: reject any model with no price row, and bill BYOK usage in shadow accounting only. Hosted limits are not tenant budgets.
Not in client docs	**DevX proposal:** Key format, revocation latency and the budget period are DevX proposals. Virtual keys with budgets are sourced.

WHAT IT IS FOR

Isolate tenants and attach budgets to virtual keys (LiteLLM max_budget with a duration [C:060, C:061]; Bifrost 402 [C:001]).

EXPECTED

- Issues virtual keys with budget and duration.
- Returns 402 when a key budget is exhausted [C:001].
- Rejects unpriced models and unpriced BYOK routes [A:242, A:245].
- Revokes a key within the revocation service level objective (SLO).

NOT EXPECTED

- No shared budget counter across tenants.
- No dispatch to a model with no price row.
- No key text in logs.

ALGORITHM AND MATHS

Virtual key budget

```
spent + reserve <= max_budget within window
```

STEPS

1. Resolve tenant from the key.
2. Look up the price row of the requested model; reject if absent.
3. Reserve the worst-case cost atomically against the key and tenant counters.
4. On completion, settle with the billed cost and release the unused reserve.
5. At window end, reset or roll over by policy.

PARAMETERS

max_budget	per key (LiteLLM max_budget [C:060])
duration	per key (LiteLLM budget_duration [C:061])
exhausted_status	402 [C:001]
revocation_target	under 60 s (DevX proposal)
strong_share_ceiling	see A:010

SECURITY

- Keys are shown once, stored hashed; revocation is immediate on the gateway cache.
- Cross-tenant reads are impossible by construction: every query includes tenant id.

PERFORMANCE

Latency and throughput	Key lookup under 1 ms p99 (DevX proposal).
Complexity	Constant (hash lookup).
At 10x load	Shard counters by tenant.

EDGE CASES AND HOW EACH IS HANDLED

- Key revoked mid-request: let the in-flight call finish, block the next.
- Budget window rolls over during a long task.
- BYOK call on a tenant with no BYOK policy: reject.
- Price table updated mid-day: bind the price version at reserve time.

TEST CASES

TEST	GIVEN	EXPECT
exhausted	max_budget $500, spent $499.97, reserve $0.05	402
unpriced	model with no price row	400 model_unpriced
revoke	revoke key then call	401 within 60 s
isolation	key of acme reads beta counters	404
settle	reserve $0.05, billed $0.03	counter reduced by $0.03 net

WHERE IT SAVES COST

- Hard per-key limits.
- Blocks unbilled routes.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Budget exhaustion events.
- Unpriced reject count.
- Revocation latency.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.tenant.key_created	tenant · key_id · max_budget · duration	key issued
token_economics.tenant.key_revoked	tenant · key_id · actor	key revoked
token_economics.tenant.unpriced	tenant · model	model rejected

RESEARCH BEHIND IT

OPEN-SOURCE IMPLEMENTATIONS STUDIED

- LiteLLM: Virtual keys, max_budget and budget_duration.
- Bifrost: 402 on exhausted virtual-key budget.

HOW OTHERS BUILD IT

Virtual keys with budgets are the standard gateway pattern [C:060, C:001]. Unpriced and BYOK escapes are known gaps [A:242, A:245].

REFERENCES

- C:060 5 ways to cut Claude Code costs with LiteLLM
- C:061 5 Open-Source Tools to Control Your AI API Costs at the Code Level
- C:001 Claude Code Token Usage: Monitor It with an AI Gateway
- A:242 Budget limits (Portkey)
- A:245 Cost-aware model routing with Vercel AI Gateway
- A:191 Spend limits
- A:010 Rerouting LLM Routers

RECEIVES FROM

Admin API
keys

HANDS TO

Budget store
keys, keys

← Back to group: Admin and config ← Stage 8

LEVEL 4

Audit log

Control plane | Stage 8: Control plane

Audit log

Every policy and budget change

2026-11-09T10:02Z actor=ops-7 act=kill.set
scope=acme/mask prev=9f2c.. hash=41ab..

IN PLAIN ENGLISH

> A permanent record of who changed what and when. It has no prompts and no code, only facts about the actions.

Goes in	Events from api, pol, kill, secrets, authz and the data path: actor, action, scope, outcome.
Comes out	Append-only metadata log with a hash chain and export.
Reads / writes	Writes audit rows. Reads for export and verification.
If it fails	Fail closed for mutations. If an audit row cannot be written, the admin change is rejected. Fail open for data-path reads: a request is not blocked by a failed audit write, it is buffered.
Requirement source	RQ-D-067, RQ-D-068, RQ-D-057 Sources: A:217, C:104
Hardening, still open	Retention period is not in corpus; see the ret card. Whether a hash chain is needed or object-lock storage is enough: open.
Not in client docs	**DevX proposal:** Hash chain, field list and export formats are DevX proposals. Metadata-only logging is sourced: content logging is opt-in in vendor telemetry [A:217, C:104].

WHAT IT IS FOR

Record every control-plane action so changes can be traced. Log metadata only; content stays off unless a tenant opts in (OTEL_LOG_* style opt-in [A:217, C:104]).

EXPECTED

- Writes one row per admin mutation and per policy decision of class deny.
- Stores actor, action, scope, outcome, time, request id and diff of settings.
- Chains each row to the previous with a hash.
- Exports to the tenant store on a schedule.

NOT EXPECTED

- No prompt or output text.
- No edit or delete of a row.
- No secrets in rows.

ALGORITHM AND MATHS

Hash-chained append log

```
hash_n = H(hash_{n-1} || canonical(row_n))
```

STEPS

1. Build the row from the event with a fixed field set.
2. Redact any field not on the allow-list.
3. Compute the hash from the previous hash and the row.
4. Append in one transaction with the mutation it records.
5. Verify the chain daily and alert on a break.

PARAMETERS

hash	SHA-256 (DevX proposal)
content_logging	off by default; opt-in only [A:217, C:104]
verify_interval	24 hours (DevX proposal)

SECURITY

- Rows are append-only; the writer role has no update or delete rights.
- Field allow-list is enforced in code so a new field cannot leak content by default.

PERFORMANCE

Latency and throughput	Append under 5 ms p99 (DevX proposal).
Complexity	Constant per row.
At 10x load	Partition by tenant and month.

EDGE CASES AND HOW EACH IS HANDLED

- Two writers append at once: serialise per tenant chain.
- Clock skew: order by sequence number, not time.
- Row contains a forbidden field: drop the field and log the drop.
- Storage full: reject mutations, buffer reads.

TEST CASES

TEST	GIVEN	EXPECT
chain	append 3 rows	row 3 hash = H(hash2, row3)
tamper	edit row 2	chain verification fails at row 2
no_content	event with prompt text field	field dropped, row kept
atomic	mutation fails after audit write	both rolled back
export	daily export	tenant bucket holds the new rows

WHERE IT SAVES COST

- Fast incident review.
- No cost from content storage.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Chain verification failures.
- Dropped-field count.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.audit.append	tenant · action · actor · seq	row written
token_economics.audit.chain_break	tenant · seq	verification failed
token_economics.audit.field_dropped	field	forbidden field removed

REFERENCES

- A:217 Monitoring usage (Claude Code OpenTelemetry)
- C:104 Codex configuration reference

RECEIVES FROM

Admin API
change record

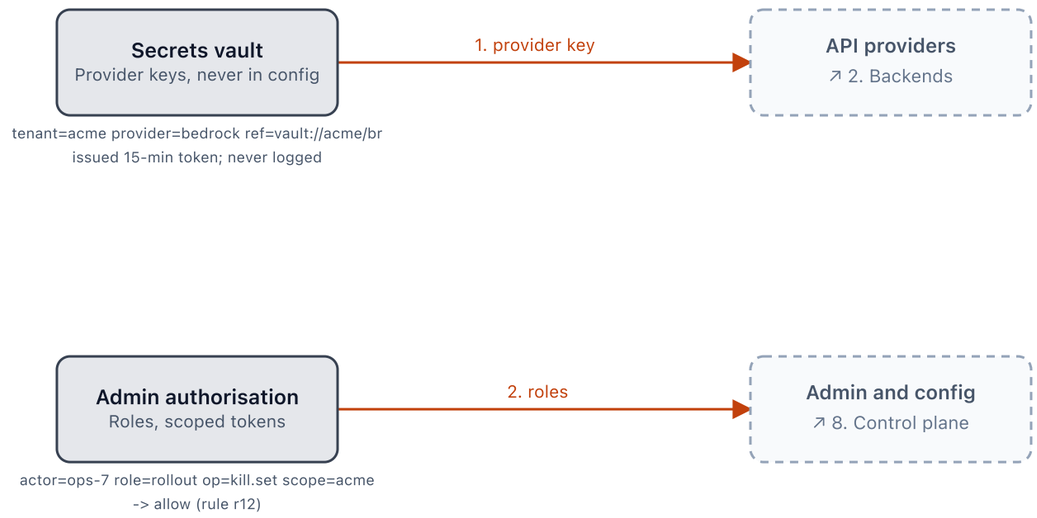
HANDS TO

None in this stage view.

← Back to group: Admin and config ← Stage 8

LEVEL 3 Security and data in Control plane

Secrets vault, admin authorisation and retention rules.



Control plane — main flow — read / call - - - scheduled / async — security ⬚ outside this group (click to go)

WHAT TRAVELS ON EACH NUMBERED LINE

1. Secrets vault → API providers (group)
Fetched at call time, never in config.

2. Admin authorisation → Admin and config (group)
Scoped admin tokens.

3. Retention and redaction → Stores and attribution (group)
Expiry and redaction on stored rows.

Hover a box for the plain-English story, hover a line for what travels on it, click a box for its full spec card. ← All groups in Control plane ← Admin and config

LEVEL 4

Secrets vault

Control plane | Stage 8: Control plane

Secrets vault

Provider keys, never in config

tenant=acme provider=bedrock ref=vault://acme/br
issued 15-min token; never logged

IN PLAIN ENGLISH

> A safe for the keys that let Token Economics talk to model providers. Keys are only handed out briefly and are never written to logs.

Goes in	Provider keys, OAuth tokens, signing keys, per-tenant secret references.
Comes out	Short-lived credentials handed to the gateway at call time.
Reads / writes	Reads and writes encrypted secrets in the vault. Writes access audit rows.
If it fails	Fail closed. If a secret cannot be fetched, the call is rejected with 503 and no request goes out with a wrong key.
Requirement source	RQ-D-069, RQ-D-070, RQ-D-071 Sources: C:104, A:193, C:088
Hardening, still open	How subscription (OAuth) credentials are handled by a gateway is not settled in the corpus. Chosen: do not store end-user subscription tokens; accept only API keys and cloud-role credentials.
Not in client docs	**DevX proposal:** Whole card is a DevX proposal except the point that credentials differ by harness (OAuth and key modes) [A:193, C:088].

WHAT IT IS FOR

Store and deliver provider credentials with the least exposure.

EXPECTED

- Encrypts secrets at rest and per tenant.
- Issues short-lived tokens where the provider supports it.
- Logs each access as a metadata audit row.
- Blocks a secret from appearing in any log, trace or error.

NOT EXPECTED

- No secret in config files, env dumps or logs.
- No sharing of a secret across tenants.
- No storage of end-user subscription tokens.

ALGORITHM AND MATHS

Per-tenant vault fetch

```
token = vault.fetch(tenant, ref, ttl)
```

STEPS

1. Gateway requests a secret by tenant and reference.
2. Vault checks the caller identity and tenant scope.
3. Vault returns a token with a time to live, or a wrapped key.
4. Gateway caches it in memory for the ttl and zeroes it after.
5. Vault writes an access audit row.

PARAMETERS

ttl	15 minutes (DevX proposal)
encryption	per-tenant data key (DevX proposal)
harness_credential_modes	OAuth and API key differ by harness [A:193, C:088]

SECURITY

- Redaction filter on all log sinks drops any string matching key patterns.
- Vault access needs workload identity; no static admin credentials.
- Rotation: keys are rotated on a schedule and on any suspected leak.

PERFORMANCE

Latency and throughput	Fetch under 20 ms p99 with cache hit under 1 ms (DevX proposal).
Complexity	Constant.
At 10x load	Cache in gateway memory, one fetch per ttl per key.

EDGE CASES AND HOW EACH IS HANDLED

- Vault unreachable: 503; no fallback key.
- Key rotated during a request: retry once with the new key.
- Tenant deleted: secrets are destroyed after the retention grace.
- Provider returns 401: mark the secret suspect and alert.

TEST CASES

TEST	GIVEN	EXPECT
fail_closed	vault down	503 secret_unavailable, no outbound call
scope	tenant beta asks for acme ref	denied and audited
ttl	token ttl 15 min	refetched at minute 15
redact	log line contains key string	string replaced by redacted marker
rotation	rotate key	old key 401, new key works

WHERE IT SAVES COST

- Avoids incident cost from leaked keys.
- Short ttl limits blast radius.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Fetch failures.
- Redaction hits.
- Rotation age.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.secrets.fetch	tenant · ref · ok	secret fetched
token_economics.secrets.denied	tenant · ref · reason	access denied
token_economics.secrets.rotated	tenant · ref	rotation done

REFERENCES

- C:104 Codex configuration reference
- A:193 LLM gateway configuration
- C:088 Run Claude Code through a gateway

RECEIVES FROM

None in this stage view.

HANDS TO

Anthropic connector
provider key, provider key

OpenAI connector
provider key

← Back to group: Security and data ← Stage 8

LEVEL 4

Admin authorisation

Control plane | Stage 8: Control plane

Admin authorisation

Roles, scoped tokens

actor=ops-7 role=rollout op=kill.set scope=acme
-> allow (rule r12)

IN PLAIN ENGLISH

> Decides who may press which button. A developer can read their own spend, an operator can stop a saving, and an admin can change budgets.

Goes in	Actor identity, role, requested operation, scope (tenant, optimisation).
Comes out	Allow or deny with the matched rule.
Reads / writes	Reads role bindings. Writes decision rows (deny) to the audit log.
If it fails	Fail closed. If role data is unreadable, deny admin actions. Data-path calls use the cached key scope.
Requirement source	RQ-D-072, RQ-D-073 Sources: C:088
Hardening, still open	Role set is a proposal and needs review with the first customer. Whether to use external identity providers is open.
Not in client docs	**DevX proposal:** Whole card is a DevX proposal. Only the point that harnesses differ in credential and permission models has a source [C:088].

WHAT IT IS FOR

Give each admin action the least role that can do it, so kill and policy changes are controlled.

EXPECTED

- Defines roles: viewer, developer, rollout, finance, admin.
- Allows kill only to rollout and admin.
- Checks scope on every call; tenant roles do not cross tenants.
- Audits every deny.

NOT EXPECTED

- No wildcard role in production.
- No role inherited across tenants.
- No permission through a client hook.

ALGORITHM AND MATHS

Role-scope check

```
allow iff exists binding(actor, role, scope) and op in perms(role)
```

STEPS

1. Resolve the actor from the token.
2. Load bindings for the tenant scope.
3. Check the operation against the role permission table.
4. Return allow or deny with the rule id.
5. Write a deny audit row.

PARAMETERS

roles	viewer, developer, rollout, finance, admin (DevX proposal)
kill_roles	rollout, admin (DevX proposal)
cache_ttl	30 s (DevX proposal)

SECURITY

- Role changes require admin and a second approval for admin grants.
- Bindings are stored per tenant; queries always filter by tenant id.

PERFORMANCE

Latency and throughput	Check under 1 ms p99 with cache (DevX proposal).
Complexity	Constant (table lookup).
At 10x load	Cache bindings in the API tier.

EDGE CASES AND HOW EACH IS HANDLED

- Actor holds two roles: union of permissions.
- Role revoked: cache expires within 30 s.
- Service account without a role: deny.
- Cross-tenant operator: needs an explicit binding per tenant.

TEST CASES

TEST	GIVEN	EXPECT
kill_allowed	rollout role sends kill.set	allow
kill_denied	developer role sends kill.set	deny, audit row
cross_tenant	admin of acme edits beta	deny
no_role	service account without binding	deny
revoke	role revoked at t0	deny after 30 s at most

WHERE IT SAVES COST

- Prevents accidental kills and budget edits.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Deny rate by role.
- Role change count.

EVENTS EMITTED

EVENT	FIELDS	WHEN
`token_economics.authz.allow`	actor · op · scope · rule	allowed
`token_economics.authz.deny`	actor · op · scope · reason	denied

REFERENCES

- C:088 Run Claude Code through a gateway

RECEIVES FROM

None in this stage view.

HANDS TO

Admin API
roles

← Back to group: Security and data ← Stage 8

LEVEL 4

Retention and redaction

Control plane | Stage 8: Control plane

Retention and redaction

Metadata retention, content off

class=metadata keep=400d; class=content keep=0d
hold=none; deleted 12,004 rows

IN PLAIN ENGLISH

> Rules for how long records are kept. By default Token Economics keeps only facts and numbers, and throws away anything with content.

Goes in	Retention policy per tenant and data class, legal hold flags.
Comes out	Deletion jobs and a retention report.
Reads / writes	Reads policy and data inventory. Writes deletion manifests and audit rows.
If it fails	Fail closed for deletion of held data: a hold blocks deletion. Fail open for ordinary jobs: a missed run is retried next cycle.
Requirement source	RQ-D-074, RQ-D-075, RQ-D-076, RQ-D-077, RQ-D-067 Sources: A:217, C:104, C:003, A:232
Hardening, still open	Retention periods are not in corpus. Chosen: metadata 400 days (covers one billing year and a reconciliation window; a DevX proposal). Content 0 days unless a tenant opts in. Frozen price snapshots are kept with the data they priced [A:232].
Not in client docs	**DevX proposal:** Retention windows and the redaction job are DevX proposals. Metadata-only default and opt-in content logging are sourced [A:217, C:104].

WHAT IT IS FOR

Keep only what is needed for billing and audit, for as long as it is needed.

EXPECTED

- Defaults to metadata only; content logging is opt-in per tenant [A:217, C:104].
- Deletes data past its window on a schedule.
- Honours legal holds before any delete.
- Keeps price snapshots with the records they priced [A:232].

NOT EXPECTED

- No content stored without an opt-in.
- No deletion of held data.
- No deletion of audit rows before their window.

ALGORITHM AND MATHS

Window-based purge

```
delete rows where age > window(class) and not hold
```

STEPS

1. Load policy per tenant and data class.
2. Select rows older than the window, excluding held ids.
3. Delete in batches and write a manifest of counts.
4. Redact opted-in content older than the content window.
5. Report counts to the tenant.

PARAMETERS

metadata_window	400 days (DevX proposal)
content_window	0 days unless opt-in (DevX proposal)
content_default	off [A:217, C:104]
price_snapshots	kept with records [A:232]

SECURITY

- Deletion jobs run with a role that cannot read content.
- Manifests hold counts only.

PERFORMANCE

Latency and throughput	Daily job finishes in under 1 hour per 100 million rows (DevX proposal).
Complexity	Linear in expired rows.
At 10x load	Partitioned by day, drop whole partitions.

EDGE CASES AND HOW EACH IS HANDLED

- Legal hold added mid-job: stop deleting held ids.
- Tenant lowers its window: apply at next run, never extend deleted data.
- Tenant deleted: purge after the grace period.
- Clock skew at the window edge: use a one-day safety margin.

TEST CASES

TEST	GIVEN	EXPECT
default_off	new tenant	no content rows stored
purge	rows older than 400 days	deleted, manifest count matches
hold	row on hold older than window	kept
optin	content opt-in, window 30 days	content older than 30 days redacted
snapshot	price snapshot referenced by a kept row	snapshot kept

WHERE IT SAVES COST

- Lower storage cost.
- Smaller breach exposure.

FEEDBACK AND ANALYTICS EVENTS

FEEDBACK CAPTURED OR USED

- Rows deleted per day.
- Hold count.

EVENTS EMITTED

EVENT	FIELDS	WHEN
token_economics.ret.purged	tenant · class · rows	purge batch done
token_economics.ret.held	tenant · hold_id	hold applied
token_economics.ret.redacted	tenant · rows	content redaction

REFERENCES

- A:217 Monitoring usage (Claude Code OpenTelemetry)
- C:104 Codex configuration reference
- C:003 How to Track Claude Code Usage in 2026: Built-in Commands, ccusage, and Desktop Dashboards Compared
- A:232 Token & Cost Tracking (Langfuse)

RECEIVES FROM

None in this stage view.

HANDS TO

Metadata store
retention, retention

← Back to group: Security and data ← Stage 8

CROSS-CUTTING

Security

The gateway holds real credentials, so it is the highest-value target in the system. Controls below apply to every stage.

RULES

- Enforce budgets at the gateway. A client hook can be disabled, so it never carries enforcement (research note R4, adapter rules).
- Fail closed for budget decisions and open for observation. A dead metering path must not block work, a dead budget path must not allow spend.
- Log metadata only. Prompt and tool content logging stays off unless a tenant opts in (OTEL_LOG_* flags).
- Never store OAuth tokens. Forward the auth header and the anthropic-beta header unchanged.
- Provider keys live in the vault and are fetched at call time. Project-local Codex config ignores provider keys, so keys go in user config.
- Reject usage objects with unknown extra keys, so a spoofed upstream cannot inject fields.
- Cap the strong-model share per tenant to blunt router attacks.
- Every policy, budget and key change writes an audit row before it applies.

STILL OPEN

- Claude Code behaviour on an HTTP 402 is not in the corpus. Test it before relying on 402 for hard caps.
- Soft-budget overshoot size is not in the corpus. Pick a margin in the shadow phase.

CROSS-CUTTING

Reliability

Reliability here means the gateway never becomes the reason a developer cannot work, and never lets spend run unmetered.

FAILURE BEHAVIOUR

FAILURE	OBSERVATION PATH	BUDGET PATH
Ledger write fails	Fail open, queue and retry	Fail closed after the hold expires
Price row stale	Estimate flagged low confidence	Billing decisions blocked
Provider 429	Priced fallback hop	Priced fallback hop
Policy bundle unreadable	Keep last good bundle	Keep last good bundle, deny if none
Router unavailable	Use best-single baseline	Use best-single baseline
Kill-switch flipped	Optimisations off, default path	Budgets stay on

NOT IN CORPUS

- Disaster-recovery targets, multi-region design and gateway latency budgets. Each is a DevX proposal to be set in phase P3.

CROSS-CUTTING

Performance

The corpus has no gateway latency measurements. Targets below are DevX proposals to be measured in phase P0.

LATENCY BUDGET (DEVX PROPOSAL)

STEP	TARGET	NOTE
Key resolve and policy read	under 5 ms	In-memory, refreshed from the signed bundle
Reservation	under 10 ms	Single atomic counter
Stream tee	0 ms added	Copy, never buffer the stream
Ledger write	Async	Off the request path

RULES

- Never buffer a streaming response before forwarding it.
- Keep the cache key stable per session. A change in tool set or model invalidates the cache.
- Cap about 15 requests per minute per prefix on OpenAI to keep cache routing stable.

CROSS-CUTTING

Cost

Every lever is gated by the eval harness. Savings ranges are vendor or author claims in the corpus, not Token Economics results.

LEVERS IN SHIP ORDER

LEVER	PHASE	EVIDENCE STATUS
Per-key hard budgets, turn caps, loop guard	P1	Sourced; 51 percent of spend falls after the first repeated failure (research note R2)
Cache planner and stickiness	P2	Sourced; blanket 1h TTL made one bill 8.6 percent worse
Context guard	P2	Sourced; output compressors flip sign by harness, so off by default
Static router	P2	Sourced; savings range from ties to 53 percent, measure first
Cascade, bandits	P3 or later	Only with measured acceptance

COST OF TOKEN ECONOMICS ITSELF

- Charge the optimisation overhead to the task when measuring cost per successful task.
- Probe-based prediction adds one probe call plus a 1.9x multiplier.

CROSS-CUTTING

Quality

A cheaper run that fails more often is not a saving. The headline metric is cost per successful task, failures included.

RULES

- Declare the success non-inferiority bound before the test.
- Use paired, task-clustered bootstrap intervals.
- Isolate cache state per arm.
- Keep a private eval set with hard cases plus a Lite subset (80 tasks tracked 350 within 0.81 percentage points).
- Report median and P95 per user.
- Track failed-run waste share (baseline 32 percent), spend after first repeated failure (51 percent), geometric SD (x1.34), probe error and reconciliation ratio.

CROSS-CUTTING

What this plan does not know

The plan rests on a research set that was summarised by models. Read these limits before trusting any number.

EVIDENCE

- All 423 sources and 411 notes are model-written. Six agents extracted the requirements from notes plus corpus text. The plan does not rest on a first-hand read of every paper.
- Codex stopped early and contributed none of the notes used.
- Many figures are vendor or author claims. Model names such as Fable 5.1 are quoted as the corpus gives them and some cannot be checked.

NOT IN CORPUS

- Measured cache loss on a model switch.
- Claude Code behaviour on HTTP 402.
- Soft-budget overshoot size.
- A measured subagent multiplier. The 7x agent-team figure is quoted, not measured.
- Production shadow results.
- Power formulas, anomaly detection method and reconciliation tolerance.
- Default GPU utilisation for self-hosting.

UNRESOLVED CONFLICTS, LABELLED DEVX PROPOSAL

- Gemini cache discount 75 percent against 90 percent, and minimum prefix sizes: take the larger until tested.
- Auto-compact thresholds: 50 percent, about 90 percent, or window minus 33k.
- MCP schema size 8k to 55k.
- Budget visibility: some sources say it helps, one says agents ignore it. Enforce outside the agent.
- Error-price break-even $0.01 against $0.20.
- Together H100 $3 against $5.49 per hour.
- Batch plus cache stacking differs by vendor, so it is a price-book flag.

SOURCES

Research and references

API-SIDE RESEARCH

- A:193 LLM gateway configuration
- A:234 Track and optimize coding agent usage (Langfuse)
- A:245 Cost-aware model routing with Vercel AI Gateway
- A:233 LLM Cost Management: Track and Control Spend
- A:199 Claude prompt caching docs
- A:216 Track cost and usage (Claude Agent SDK)
- A:232 Token & Cost Tracking (Langfuse)
- A:217 Monitoring usage (Claude Code OpenTelemetry)
- A:056 Gain per-developer visibility into Kiro Enterprise credit usage
- A:263 Usage-based billing for organizations and enterprises (Copilot)
- A:283 What's new in Claude Opus 5
- A:219 Batch processing (Claude)
- A:201 Prompt caching
- A:202 Prompt Caching 201
- A:203 Prompt Caching 101
- A:204 Prompt cache diagnostics
- A:148 Prompt caching with Azure OpenAI in Azure AI Foundry Models
- A:197 Reasoning models
- A:195 Flex processing
- A:150 Use Flex processing with Azure OpenAI in Foundry Models
- A:221 Usage & Pricing (xAI prompt caching)
- A:223 How Prompt Caching Works (xAI)
- A:224 Prompt Caching: Cut the Cost of Long Chats with the Grok API
- A:225 xAI pricing
- A:121 Gemini 2.5 Models now support implicit caching
- A:143 Save costs and decrease latency while using Gemini with Vertex AI context caching
- A:231 Prompt caching best practices (OpenRouter)
- A:237 Prompt caching (Helicone gateway)
- A:280 Context caching overview (Vertex AI)
- A:281 Batch inference (Gemini Enterprise)
- A:157 Prompt caching for faster model inference
- A:159 Capacity and Performance
- A:160 Process multiple prompts with batch inference
- A:227 Provider routing (OpenRouter)
- A:229 How OpenRouter rebuilt provider routing for better, faster inference
- A:247 AI Gateway Automatic Prompt Caching (Vercel)
- A:248 Provider Options (Vercel AI Gateway)
- A:158 Understanding intelligent prompt routing in Amazon Bedrock
- A:192 Auto Router
- A:008 MTRouter: Cost-Aware Multi-Turn LLM Routing with History–Model Joint Embeddings
- A:270 Claude Opus 4.6
- A:214 Models and pricing for GitHub Copilot
- A:069 The Price Reversal Phenomenon: When Cheaper Reasoning Models End Up Costing More
- A:176 Priority Tier

- A:258 GLM 5.2 Fast: Faster Serverless Inference, Same Quality
- A:252 Groq Batch API
- A:255 Batch inference (Fireworks)
- A:177 How Does Cohere's Pricing Work?
- A:182 Context Caching
- A:161 Billing
- A:146 Calculate Provisioned Throughput requirements
- A:190 Custom costs
- A:251 Reasoning models (OpenAI platform)
- A:242 Budget limits (Portkey)
- A:055 Introducing upgrades to Codex
- A:191 Spend limits
- A:213 Overview of request-based billing (legacy)
- A:059 Analyzing and Mitigating Cost-Inefficient Behaviors in Coding Agents
- A:246 AI Gateway vs. OpenAI API (Vercel)
- A:149 Model router for Microsoft Foundry concepts
- A:282 Effort parameter (Claude)
- A:228 Auto Router: Smart AI Model Selection
- A:052 Cost per task: Using cheap classifiers to route across Gemini models and thinking levels
- A:018 s1-llm-auto-router
- A:006 LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing
- A:002 RouterBench: A Benchmark for Multi-LLM Routing System
- A:001 RouteLLM: Learning to Route LLMs with Preference Data
- A:019 Zen Router 0.6B
- A:013 Meta-Router: Bridging Gold-standard and Preference-based Evaluations in Large Language Model Routing
- A:046 RouterArena: An Open Platform for Comprehensive Comparison of LLM Routers
- A:010 Rerouting LLM Routers
- A:007 Efficient Training-Free Online Routing for High-Volume Multi-LLM Serving
- A:028 Conformal Cascade: Distribution-Free Accuracy Guarantees for Multi-Tier LLM Inference
- A:123 Economic Evaluation of LLMs
- A:021 FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance
- A:022 Language Model Cascades: Token-level uncertainty and beyond
- A:111 Batch API guide
- A:152 How to use global batch processing with Azure OpenAI
- A:174 Batch Processing
- A:209 Fast mode for API Customers
- A:129 Investing in Performance: Fine-tune small models with LLM insights
- A:272 Introducing pplx-api
- A:041 Small Language Models are the Future of Agentic AI
- A:186 The new economics of AI cost management
- A:284 Writing effective tools for AI agents
- A:271 Context engineering: memory, compaction, and tool clearing (Claude cookbook)
- A:268 Managing context on the Claude Developer Platform
- A:104 The Benefits of a Concise Chain of Thought on Problem-Solving in Large Language Models
- A:269 The components of agent memory
- A:207 The builder's guide to GPT-5.6

- A:122 Cost-of-Pass: An Economic Framework for Evaluating Language Models
- A:060 Claw-SWE-Bench: A Benchmark for Evaluating OpenClaw-style Agent Harnesses on Coding Tasks
- A:241 AI Gateway (Portkey)
- A:243 Optimizing LLM costs: operational best practices (Portkey)

CLI-AGENT RESEARCH

- C:088 Run Claude Code through a gateway
- C:001 Claude Code Token Usage: Monitor It with an AI Gateway
- C:060 5 ways to cut Claude Code costs with LiteLLM
- C:052 Your cache hit rate is lying to you. It's the caller mix.
- C:054 The Coding Agent Economy
- C:074 Hooks reference
- C:136 Coding agents ignore spend (Ramp Labs)
- C:104 Codex configuration reference
- C:094 Run Claude Code programmatically
- C:037 How I Cut Aider's Token Bill 80%: Prompt Caching, MCP Code Mode, and Tier Routing
- C:057 Gemini CLI: Quotas and pricing
- C:076 Customize your status line
- C:073 Model configuration and effort levels
- C:003 How to Track Claude Code Usage in 2026: Built-in Commands, ccusage, and Desktop Dashboards Compared
- C:097 Codex local config (config.toml)
- C:035 Context Management Across Codex Surfaces
- C:040 How to Manage the Aider Context Window and Token Budgets
- C:127 How to Reduce Aider Token Usage
- C:117 Understanding and managing requests in Copilot
- C:119 Cursor Models & Pricing
- C:121 Token caching and cost optimization (Gemini CLI)
- C:090 Automate actions with hooks (guide)
- C:002 Claude Code OpenTelemetry: Track Cost and Tool Use
- C:010 Claude Code Billing & Usage Limits Explained (Oct 2026)
- C:080 Best practices for Claude Code
- C:095 Choose a permission mode
- C:051 How to Add Prompt Caching to an Anthropic SDK App and Measure the Hit Rate
- C:030 TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing
- C:055 How Claude Code uses prompt caching
- C:011 Claude Code Limits: 5-Hour Caps Doubled in May 2026
- C:115 Codex CLI usage rate limits
- C:009 Claude Code Rate Limits & Usage Quotas Explained (2026)
- C:114 Codex pricing: current costs, usage limits, and realistic budgets (2026)
- C:120 Cursor account pricing
- C:015 Clarifying our pricing
- C:116 GitHub Copilot premium requests (billing concepts)
- C:028 Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
- C:004 Monitoring Claude Code Usage with OpenObserve and MCP
- C:061 5 Open-Source Tools to Control Your AI API Costs at the Code Level
- C:027 Can your AI agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks

- C:110 Where Do Your Tokens Go: Empirical Token Consumption in Coding Agents
- C:101 ChatGPT Work: usage and cost
- C:025 An Empirical Study of Turn-Control Strategies for Efficient Coding Agents
- C:131 Budget-Aware Tool-Use Enables Effective Agent Scaling
- C:132 Credit-Budgeted ICPC-Style Coding: When Agents Must Pay for Every Decision
- C:062 AI Agent Cost Optimization: Token FinOps for the Age of Runaway Agent Spend
- C:059 Gemini API Pricing and Quotas: Complete 2026 Guide
- C:086 Speed up responses with fast mode
- C:031 Response-Level Speculative Decoding for Cost-Efficient LLM API Serving
- C:026 Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost
- C:032 Agentic Model Routing for Coding Tasks
- C:029 A Unified Approach to Routing and Cascading for LLMs
- C:023 How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks
- C:050 Evaluation and Optimization of Small Language Models for Agentic Tasks on Edge Devices
- C:019 How MCP Tool Definitions Inflate Your AI Agent Token Costs
- C:021 Dynamic Tool Gating and Lazy Schema Loading for Eliminating the MCP/Tools Tax in Scalable Agentic Workflows
- C:022 MCP Compression: Preventing tool bloat in AI agents
- C:111 The Context Window Gap: Codex CLI caps GPT-5.6 at 272K
- C:020 MCP Tool Overload: Why More Tools Make Your Agent Worse
- C:106 Mastering Context Compaction in Codex CLI
- C:036 Codex Context Management: Auto-Compact and /context
- C:024 Token Reduction Is Not Cost Reduction
- C:128 How to Measure AI Agent Token Usage
- C:130 The Complexity Trap: Observation Masking vs LLM Summarization
- C:133 Evaluating Memory Condensation Strategies for Coding Agents in Data-Driven Scientific Discovery
- C:122 OpenHands Context Condensation for More Efficient AI Agents
- C:066 Memory & context management with Claude (cookbook)
- C:065 Context editing on the Claude Developer Platform
- C:067 Context Editing Looks Like a Feature. It's Actually a Garbage Collector Without Write Barriers
- C:006 Context engineering: memory, compaction, and tool clearing
- C:112 Context Compaction Showdown: Codex CLI, Claude Code and 5 Other Agents
- C:007 Shedding Heavy Memories: Context Compaction in Codex, Claude Code, and OpenCode
- C:089 How Claude Code works
- C:093 Choosing a Claude model and effort level in Claude Code
- C:077 How Claude remembers your project
- C:075 Extend Claude with skills
- C:079 Explore the context window
- C:053 Inference Economics of Enterprise Coding Agents: A Case Study of Cloud vs. On-Premise LLMs
- C:081 Orchestrate teams of Claude Code sessions
- C:063 A Survey of Small Language Models: Architectures, Capabilities, and Deployment Trade-offs